

Characterizing Motion: A Clustering Approach to Identify Driving Behaviors

Unsupervised clustering of actor trajectories in the Lyft AV dataset to reveal interpretable behavior archetypes.

[Lyft Motion Prediction for Autonomous Vehicles](#)

By: Travis Reinart

September 10, 2025

-
- CSCA 5632: *Unsupervised Algorithms in Machine Learning Final Project* •
 - University of Colorado Boulder •

[GitHub - Week 5 Unsupervised Algorithms in Machine Learning Final Project](#)

[View Live HTML Analysis](#)

[GitHub Release Direct Link \(large files\)](#)

[Direct Download of Notebook \(.ipynb\)](#)

[Direct Download HTML \(static file\)](#)

Copyright (c) 2025 Travis Reinart
Licensed under the [MIT License](#).

Section 1: Introduction & Problem Definition

This notebook documents a complete unsupervised learning workflow that adapts the Lyft Level 5 autonomous vehicle dataset for a novel purpose: pattern discovery. The original Kaggle competition was a supervised forecasting challenge, a task better suited for a different course. Here, I am reframing the problem to align with this course's focus. Instead of predicting future movements, the objective is to analyze past trajectories to automatically discover and classify the fundamental behaviors of vehicles, cyclists, and pedestrians in complex urban environments.

The Lyft Level 5 Motion Prediction dataset was released on Kaggle for a supervised learning competition where the goal was to predict future trajectories of vehicles and pedestrians. That challenge measured success with a forecasting error score. This notebook does not pursue the supervised task. Instead, it reframes the dataset for an **unsupervised learning project**, where the objective is to uncover patterns of driving and pedestrian behaviors without labels or ground truth outcomes.

An autonomous vehicle faces countless scenarios every day: a four-way stop, a pedestrian stepping off the curb, or a vehicle merging into traffic. The ability to recognize these situations quickly is essential for safe navigation. By clustering the trajectories of agents in this dataset, the project seeks to group motion into archetypes such as cars accelerating straight, vehicles turning left, or pedestrians waiting at an intersection. This unsupervised framing fits the course assignment, which requires discovery of hidden structure rather than prediction against a target.

Unsupervised Learning & Clustering

Unsupervised Learning is a branch of machine learning where algorithms learn patterns from data without any explicit labels or predefined outcomes. The goal is to explore the data's hidden structure on its own. For this project, the focus is on **clustering**, a primary unsupervised task where the algorithm groups similar data points together into distinct clusters.

Imagine watching a busy intersection from above. After a while, you'd naturally start categorizing what you see: cars making cautious right turns, others accelerating aggressively straight ahead, and pedestrians waiting patiently on the curb. Clustering algorithms do the same thing, but they use mathematical definitions of "similarity" to find these groups automatically from raw data, revealing the underlying distinct behavioral patterns of the traffic.

Modeling Strategy: A "Best of Both Worlds" Approach

The analysis uses a two-pipeline strategy to discover these behavioral archetypes:

- **Pipeline A (Classical):** Engineer a set of transparent, human-interpretable features from the trajectory data, such as speed, acceleration, and turning angle, and cluster them using traditional algorithms.
- **Pipeline B (Deep Learning):** Build and train a deep learning **Autoencoder** to learn a rich, compressed representation of the trajectories automatically. These learned "embeddings" will then be clustered to find more nuanced patterns.

By comparing the results of both pipelines, we can leverage the interpretability of the classical approach and the power of the deep learning method. The primary clustering algorithms used will be **K-Means**, **Gaussian Mixture Models (GMM)**, and **HDBSCAN**.

Framework Choice: PyTorch

PyTorch is used in Pipeline B to train the autoencoder on GPU. It offers a concise API, stable CUDA support in WSL2, and fast iteration with `torch` modules and dataloaders. Clustering is performed with Scikit-learn, but representation learning (the autoencoder) runs on PyTorch to leverage the GPU for speed and reproducibility.

Evaluation & Validation

Since this is an unsupervised task, there is no single "score" to optimize. Instead, cluster quality will be evaluated using a combination of quantitative internal validation metrics and qualitative analysis.

- **Quantitative Metrics:** Use the **Silhouette Score**, **Davies-Bouldin Index**, and **Calinski-Harabasz Index** to measure cluster cohesion and separation.
- **Qualitative Analysis:** The clusters will be visualized using dimensionality reduction (PCA/UMAP), and by creating "Scenario Cards" with exemplar trajectories to interpret and validate the real-world meaning of each discovered behavioral group.

Project Plan

1. **Section 1: Introduction:** Define the problem, concept, modeling strategy, and evaluation methods.
2. **Section 2: Setup:** Configure the environment, import libraries, and set seeds.
3. **Section 3: Data Loading & Audit:** Load a subset of the Lyft dataset and perform an integrity check.
4. **Section 4: Visual EDA:** Visually inspect the data to understand its underlying distributions.
5. **Section 5: Pipeline A - Classical Feature Engineering:** Create an interpretable feature set from raw trajectories.
6. **Section 6: Pipeline A - Clustering with Classical Features:** Apply and evaluate clustering models on the engineered features.
7. **Section 7: Pipeline B - Deep Learning Feature Extraction:** Build and train an autoencoder to learn trajectory embeddings.
8. **Section 8: Pipeline B - Clustering with Learned Embeddings:** Apply and evaluate clustering models on the autoencoder embeddings.
9. **Section 9: Model Validation and Stress Testing:** Test the stability and generalizability of the clusters.
10. **Section 10: Interpretation - Scenario Archetypes:** Create "Scenario Cards" to interpret and visualize the discovered behaviors.
11. **Section 11: Pipeline Comparison:** Perform a side-by-side comparison of the results from both pipelines.
12. **Section 12: Conclusion & Analysis:** Summarize key findings, learnings, and directions for future work.

Section 2: Environment Setup & Diagnostics

Before any analysis can begin, the first step is to build a clean and reproducible environment. This section confirms the location of the data sources, provides an optional installer for missing packages, centralizes all library imports, and runs a comprehensive diagnostic script to verify that the GPU is accessible and all dependencies are correctly configured.

Section Plan:

- **Data Sources:** Detail the Lyft dataset components and their purpose.
- **Package Installation:** Provide a utility cell for installing any missing libraries.
- **Environment Notes:** Include an optional guide for setting up a high-performance WSL2 + PyTorch environment.
- **Core Library Imports:** Consolidate all project imports into a single, organized cell.
- **Diagnostics & Verification:** Set seeds for reproducibility and run a full diagnostic check on the hardware and software stack.
- **Output Utilities:** Create helper functions for saving project artifacts like figures and cached data.

2.1 The Dataset: Lyft Autonomous Vehicle Logs

This project is built on the *Lyft Level 5 Motion Prediction* dataset, a rich collection of time-indexed agent trajectories and map data from urban driving environments. While the original Kaggle competition focused on supervised trajectory forecasting, this notebook reframes the dataset for an unsupervised task: discovering and classifying the fundamental behavioral patterns of traffic.

[Kaggle Dataset Page: Lyft Motion Prediction](#)

Dataset at a Glance

Component	Format	Size	Purpose
 <code>train.zarr</code>	Zarr Store	≈20 GB	Primary training trajectories
 <code>val.zarr</code>	Zarr Store	≈2 GB	Validation set for experiments
 <code>test.zarr</code>	Zarr Store	≈1 GB	Held-out scenes (not used here)
 <code>semantic_map.pb</code> ,  <code>aerial_map.png</code>	Protobuf / PNG	≈250 MB	Map context and rasterization assets
 <code>meta.json</code>	JSON	≈5 KB	Dataset metadata

The full dataset is about **23.7 GB**. This notebook focuses on `train.zarr` and `val.zarr`, which together provide millions of agent trajectories across diverse urban driving scenes.

What is a Zarr File?

Zarr is a storage format designed for large, chunked, multidimensional arrays. Instead of saving everything in one massive file, Zarr breaks data into smaller compressed chunks and organizes them in a directory structure. This makes it efficient to read or write only the slices of data that are needed, which is critical for high-resolution trajectory datasets that cannot fit into memory all at once. In practice, this means a model can stream data in batches from disk without ever loading the entire dataset, keeping both training and analysis scalable.

This notebook was developed using the Jupyter Notebook extension within Visual Studio Code. Generative AI was used as a supplementary tool to assist with code debugging and to refine language for clarity. The core logic and all final analysis are original.

2.2 Optional: Install Missing Packages

To keep the runtime smooth, use this installer cell when a dependency is missing. Using `%pip` inside Jupyter ensures installation happens in the active kernel environment.

Instructions:

- Uncomment the needed `%pip install` lines.
- Run the cell.
- Wait for the “Successfully installed” message.
- In the menu, select **Kernel** → **Restart**.
- Re-run the import cell in 2.4.

Note: A kernel restart is required for new packages to be recognized by Jupyter.

In []: `# Uncomment and run the lines below to install any missing packages.`

```
# --- Core Jupyter environment ---
# %pip install notebook
# %pip install jupyterlab
# %pip install ipywidgets
# %pip install ipykernel
# %pip install kaggle

# --- Core data science and utilities ---
# %pip install numpy
# %pip install pandas
# %pip install scipy
# %pip install scikit-learn
# %pip install tqdm
# %pip install joblib
# %pip install zarr
# %pip install pyarrow
# %pip install openpyxl

# --- Visualization ---
# %pip install matplotlib
```

```
# %pip install seaborn
# %pip install plotly

# --- Dimensionality reduction and clustering extras ---
# %pip install umap-learn
# %pip install hdbscan

# --- Geospatial and geometry helpers ---
# %pip install shapely
# %pip install pyproj

# --- PyTorch (choose ONE block) ---
# CPU-only build
# %pip install torch --index-url https://download.pytorch.org/whl/cpu

# CUDA 12.1 build (Windows or WSL2 with NVIDIA driver)
# %pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu121

# --- Optional: Export to HTML ---
# Most Jupyter installs include nbconvert. If the command fails, install it first.
# %pip install nbconvert
#
# IMPORTANT: This bang command is written for a Windows or plain Python kernel.
# It will not work as-is in a WSL kernel unless you use a Linux path to the notebook.
#!/usr/bin/env jupyter nbconvert --to html "CSCA5632_Final_Project_Lyft.ipynb"
```

2.3 Optional: Run on GPU with Ubuntu 22.04 + VS Code (WSL2)

This guide sets up Ubuntu 22.04 on WSL2 with PyTorch + CUDA 12.1. The target is a clean Python 3.10.12 environment that runs Jupyter and uses the NVIDIA GPU for training.

Step 1: Install Ubuntu on WSL2

Open PowerShell as Administrator and run:

```
wsl --install -d Ubuntu-22.04
wsl --set-default-version 2
wsl --update
```

Create the Ubuntu user and password.

Tip: The password prompt stays blank while typing.

Step 2: Install the NVIDIA driver in Windows and verify in Ubuntu

Install the latest Windows driver:

[NVIDIA Driver Downloads](#)

Verify inside Ubuntu:

```
nvidia-smi
```

Step 3: Create the Python 3.10.12 environment

Install Python and venv:

```
sudo apt update && sudo apt -y install python3.10 python3.10-venv
python3-pip
```

Create and activate the env (`avenv`):

```
python3.10 -m venv ~/avenv
source ~/avenv/bin/activate
python --version # should print 3.10.12
```

Install libraries and register the Jupyter kernel:

```
python -m pip install -U pip
pip install numpy pandas scikit-learn matplotlib seaborn tqdm umap-learn hdbscan shapely pyproj ipykernel
pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu121
python -m ipykernel install --user --name avenv-310 --display-name "Python (avenv 3.10.12)"
```

Step 4: Connect with VS Code and select the kernel

Open a WSL window:

Install "Remote - WSL". Command Palette → `WSL: Connect to WSL`. Status bar should read `WSL: Ubuntu-22.04`.

Select the Jupyter kernel:

Notebook toolbar → kernel name → **Jupyter: Select Notebook Kernel** → choose **Python (avenv 3.10.12)**. If missing, browse to `/home/<user>/avenv/bin/python`, then reload.

Step 5: Verify PyTorch sees the GPU

Run in a cell:

```
import torch
print("torch:", torch.__version__, "cuda:", torch.version.cuda)
print("CUDA available:", torch.cuda.is_available())
print("Device:", torch.cuda.get_device_name(0) if torch.cuda.is_available() else "CPU")
```

If CUDA remains `False`, confirm `nvidia-smi` works, reinstall the CUDA wheel, and reload VS Code. For faster I/O, keep project files in the Linux filesystem (for example, `~/projects/`) rather than on the mounted Windows drive (`/mnt/c`).

2.4 Core Library Imports & Seeding

All required libraries are imported here in a single, centralized location to surface any dependency issues early. The imports are organized by purpose, and global seeds are set for all relevant libraries to ensure the results are reproducible.

```
In [11]: # System and utilities
import os
import sys
import platform
import random
import time
import gc
import re
import json
import math
from math import ceil
import shutil
import warnings
from pathlib import Path
from collections import defaultdict

# Numerics and data
import numpy as np
import pandas as pd
import zarr
import pyarrow

# Progress bar
from tqdm.notebook import tqdm

# Visualization
import matplotlib
import matplotlib.pyplot as plt
import matplotlib.colors as mcolors
import matplotlib.animation as animation
from matplotlib import cm
from matplotlib.patches import Patch
from matplotlib.lines import Line2D
from matplotlib.colors import LogNorm
from matplotlib.collections import LineCollection
import seaborn as sns
from IPython.display import HTML
import plotly
import plotly.graph_objects as go

# Unsupervised Learning toolkit
import sklearn
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.metrics import silhouette_score, silhouette_samples
from sklearn.metrics import davies_bouldin_score
from sklearn.metrics import calinski_harabasz_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics.cluster import contingency_matrix
from sklearn.metrics import adjusted_rand_score as ARI
from sklearn.metrics import normalized_mutual_info_score as NMI
from sklearn.metrics import pairwise_distances
from sklearn.cluster import KMeans
from sklearn.mixture import GaussianMixture
from scipy.optimize import linear_sum_assignment
from sklearn.utils import resample
import umap
import umap.umap_ as umap_
import hdbscan
from itertools import product

# Geometry and maps
try:
    import shapely
    import pyproj
except Exception:
    shapely = None
    pyproj = None

# Deep Learning
import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader

# Set global seed for reproducibility
SEED = 42
random.seed(SEED)
np.random.seed(SEED)
os.environ["PYTHONHASHSEED"] = str(SEED)
if torch.cuda.is_available():
    torch.manual_seed(SEED)
    torch.cuda.manual_seed_all(SEED)
    torch.backends.cudnn.deterministic = True
```

```
torch.backends.cudnn.benchmark = False
```

```
# Suppress all warnings for clean output
warnings.filterwarnings("ignore")
```

```
# Print confirmation
print("-" * 70)
print(f"SEED set to {SEED}")
print("-" * 70)
```

```
SEED set to 42
```

2.5 Diagnostics and Verification

A quick environment check confirms core library versions and verifies GPU access with a small matrix multiplication test. These diagnostics catch configuration issues before modeling begins.

- **Environment:** Python, OS, NumPy, Pandas, scikit-learn, PyTorch, CUDA, cuDNN, HDBSCAN, and UMAP.
- **GPU:** Availability, device name, VRAM, and a short CUDA matmul benchmark.

```
In [12]: # Run a GPU driver snapshot
print("-" * 70)
print("\n--- GPU Driver Snapshot ---")
try:
    # On WSL this will print the Windows-side driver info
    get_ipython().system("nvidia-smi")
except Exception:
    print("nvidia-smi not available in this environment.")

# Check environment and library versions
print("-" * 70)
print("\n--- Environment & Library Versions ---")
print(f"Python:      {platform.python_version()}")
print(f"OS:          {platform.system()} {platform.release()}")

print("\n--- Core Libraries ---")
print(f"NumPy:         {np.__version__}")
print(f"Pandas:        {pd.__version__}")
print(f"scikit-learn:  {sklearn.__version__}")
print(f"Zarr:          {zarr.__version__}")

print("\n--- Visualization ---")
print(f"Matplotlib:    {matplotlib.__version__}")
print(f"Seaborn:       {sns.__version__}")
print(f"tqdm:          {__import__('tqdm').__version__}")
print(f"Plotly:        {plotly.__version__}")

print("\n--- DR / Clustering Extras ---")
from importlib.metadata import version as _pkg_version
try:
    print(f"UMAP-learn:    {_pkg_version('umap-learn')}")
except Exception:
    print("UMAP-learn:    not installed")
try:
    import hdbscan as _hdb
    print(f"HDBSCAN:      {_pkg_version('hdbscan')}")
except Exception:
    print("HDBSCAN:        not installed")

print("\n--- Geometry / Maps ---")
print(f"shapely:        {getattr(shapely, '__version__', 'not installed')}")
print(f"pyproj:         {getattr(pyproj, '__version__', 'not installed')}")

print("\n--- PyTorch ---")
print(f"pyarrow:        {pyarrow.__version__}")
print(f"Torch:          {torch.__version__}")
print(f"CUDA (torch):   {torch.version.cuda}")
try:
    print(f"cuDNN (torch):  {torch.backends.cudnn.version()}")
except Exception:
    pass

# Verify GPU availability and performance
print("\n--- GPU Verification ---")
cuda_ok = torch.cuda.is_available()
print(f"CUDA available: {cuda_ok}")
if cuda_ok:
    dev = torch.cuda.current_device()
    props = torch.cuda.get_device_properties(dev)
    total_gb = props.total_memory / (1024 ** 3)
    print(f"Device:        {props.name}")
    print(f"Compute cap:    {props.major}.{props.minor}")
    print(f"VRAM:          {total_gb:.1f} GB")
    torch.cuda.synchronize()
    a = torch.randn((1024, 1024), device="cuda")
    b = torch.randn((1024, 1024), device="cuda")
    t0 = time.time()
```

```
_ = a @ b
torch.cuda.synchronize()
print(f"Matmul 1k x 1k: {(time.time() - t0) * 1000:.2f} ms")
else:
    print("--- Running on CPU. GPU-only steps will be skipped or adjusted ---")

# Verify Jupyter environment
print("\n--- Jupyter ---")
print("Which Jupyter :", shutil.which("jupyter"))
try:
    get_ipython().system("jupyter --version")
except Exception:
    print("Jupyter version command not available.")

# Print completion message
print("\n--- Setup Complete. Libraries Imported Successfully. ---")
print("\n" + ("-" * 70))
```

```
--- GPU Driver Snapshot ---
Wed Sep 17 19:46:53 2025

+-----+
| NVIDIA-SMI 545.34                Driver Version: 546.26      CUDA Version: 12.3   |
+-----+-----+-----+
| GPU   Name                Persistence-M | Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp  Perf            Pwr:Usage/Cap |      Memory-Usage | GPU-Util  Compute M. |
|=====+=====+=====+
|    0  NVIDIA GeForce RTX 4070 ...    On | 00000000:01:00:00 Off |          N/A         |
| N/A   46C    P0              13W / 110W |  571MiB /  8188MiB |      5%      Default |
+-----+-----+-----+
|                                  MIG M. |
+-----+-----+-----+

+-----+
| Processes: |
| GPU   GI    CI          PID    Type    Process name          GPU Memory |
|      ID    ID              |                 | Usage          |
+-----+-----+-----+
|    0   N/A   N/A         1911     C   /python3.10           N/A         |
+-----+-----+-----+

--- Environment & Library Versions ---
Python:          3.10.12
OS:              Linux 6.6.87.2-microsoft-standard-WSL2

--- Core Libraries ---
NumPy:           1.26.4
Pandas:          2.3.2
scikit-learn:    1.7.2
Zarr:            2.18.3

--- Visualization ---
Matplotlib:      3.10.6
Seaborn:         0.13.2
tqdm:            4.67.1
Plotly:          6.3.0

--- DR / Clustering Extras ---
UMAP-learn:      0.5.9.post2
HDBSCAN:         0.8.33

--- Geometry / Maps ---
shapely:         2.1.1
pyproj:          3.7.1

--- PyTorch ---
pyarrow:         21.0.0
Torch:           2.5.1+cu121
CUDA (torch):    12.1
cuDNN (torch):   90100

--- GPU Verification ---
CUDA available:  True
Device:         NVIDIA GeForce RTX 4070 Laptop GPU
Compute cap:    8.9
VRAM:           8.0 GB
Matmul 1k x 1k: 0.74 ms

--- Jupyter ---
Which Jupyter : /home/treinart/avenv/bin/jupyter
Selected Jupyter core packages...
IPython          : 8.37.0
ipykernel        : 6.30.1
ipywidgets       : 8.1.7
jupyter_client   : 8.6.3
jupyter_core     : 5.8.1
jupyter_server   : 2.17.0
jupyterlab       : 4.4.7
nbclient         : 0.10.2
nbconvert        : 7.16.6
nbformat         : 5.10.4
notebook         : 7.4.5
qtconsole        : 5.7.0
traitlets        : 5.14.3

--- Setup Complete. Libraries Imported Successfully. ---

+-----+
```

Observation: Diagnostics and Environment Verification

As of September 13, 2025, the project environment is confirmed to be stable and correctly configured for this unsupervised learning task. The setup is running on a Linux distribution via WSL2, with a comprehensive suite of libraries for data processing, clustering, and deep learning. Critically, the hardware verification confirms that the NVIDIA GPU is accessible and correctly utilized by PyTorch, which is essential for training the autoencoder model efficiently.

- **Hardware & Driver Stack:**
 - **GPU:** NVIDIA GeForce RTX 4070 Laptop GPU
 - **NVIDIA Driver:** 546.26

- **CUDA Version (from driver):** 12.3
- **CUDA Version (from PyTorch):** 12.1
- **cuDNN Version (from PyTorch):** 9.1.0
- **VRAM:** 8.0 GB
- **Software Environment:**
 - **OS:** Linux 6.6.87.2-microsoft-standard-WSL2
 - **Python:** 3.10.12
 - **PyTorch:** 2.5.1+cu121
 - **NumPy:** 1.26.4
 - **Pandas:** 2.3.2
 - **Scikit-learn:** 1.7.2
 - **UMAP-learn:** 0.5.9.post2
 - **HDBSCAN:** 0.8.33
- **Jupyter Core Packages:**
 - **IPython:** 8.37.0
 - **ipykernel:** 6.30.1
 - **jupyter_core:** 5.8.1
 - **notebook:** 7.4.5

Key Configuration Note: Reproducibility is a primary focus of this setup. Global seeds for `random`, `numpy`, and `torch` have been set to **42**. Furthermore, for GPU operations, `torch.backends.cudnn.deterministic` has been set to `True` to ensure that all model training runs are as reproducible as possible, which is critical for comparing the results of different clustering experiments.

This verified setup provides a high-performance and reproducible environment for the subsequent data processing, feature engineering, and model analysis.

2.6 Path & Output Utilities

These helper functions and path definitions create a consistent and organized project structure. They establish the locations for data and saved artifacts, and provide reliable functions for saving figures and cached NumPy arrays to both the Linux and mirrored Windows filesystems.

In [3]:

```
# Path Definitions
PROJECT_ROOT = Path.home() / "projects" / "lyft-unsupervised"
DATA_DIR = Path.home() / "data" / "lyft"
ARTIFACT_DIR = PROJECT_ROOT / "artifacts"
FIG_DIR = ARTIFACT_DIR / "figures"
CACHE_DIR = ARTIFACT_DIR / "cache"
WINDOWS_MIRROR = Path("/mnt/c/Users/travi/Documents/Training/Colorado/MS-AI/CSCA 5632 Final Project/artifacts")

# Create all output directories
for d in (PROJECT_ROOT, DATA_DIR, ARTIFACT_DIR, FIG_DIR, CACHE_DIR):
    d.mkdir(parents=True, exist_ok=True)
if WINDOWS_MIRROR is not None:
    (WINDOWS_MIRROR / "figures").mkdir(parents=True, exist_ok=True)
    (WINDOWS_MIRROR / "cache").mkdir(parents=True, exist_ok=True)
os.chdir(PROJECT_ROOT)

# General Helper Functions
def _timestamp() -> str:
    return time.strftime("%Y%m%d_%H%M%S")

def _ok_dir(p: Path) -> bool:
    try:
        return os.access(p, os.W_OK) if p.exists() else os.access(p.parent, os.W_OK)
    except Exception:
        return False

# Saving Helper Functions
def save_fig(fig, name: str, dpi: int = 150):
    fname = f"_{timestamp()}_{name}.png"
    linux_path = FIG_DIR / fname
    fig.tight_layout(pad=1.0)
    fig.savefig(linux_path, dpi=dpi, bbox_inches="tight")
    plt.show()
    print(f"Figure saved to (Linux): {linux_path}")
    if WINDOWS_MIRROR is not None:
        try:
            win_dir = WINDOWS_MIRROR / "figures"
            win_path = win_dir / fname
            shutil.copy2(linux_path, win_dir)
            print(f"Figure saved to (Win): {win_path}")
        except Exception as e:
            print(f"[Mirror Warning] Could not copy {name}: {e}")

def save_numpy(arr: np.ndarray, name: str):
    fname = f"_{timestamp()}_{name}.npz"
    linux_path = CACHE_DIR / fname
    np.save(linux_path, arr, allow_pickle=False)
    print(f"NumPy array saved to (Linux): {linux_path}")
```

```

if WINDOWS_MIRROR is not None:
    try:
        win_dir = WINDOWS_MIRROR / "cache"
        win_path = win_dir / fname
        shutil.copy2(linux_path, win_dir)
        print(f"NumPy array saved to (Win): {win_path}")
    except Exception as e:
        print(f"[Mirror Warning] Could not copy {name}: {e}")

def _save_df(df: pd.DataFrame, name: str, formats=['parquet', 'csv']) -> dict:
    fname_base = f"_{timestamp()}_{name}"
    saved_paths = {}

    if 'parquet' in formats:
        fname_parq = f"{fname_base}.parquet"
        linux_path_parq = CACHE_DIR / fname_parq
        df.to_parquet(linux_path_parq, index=False)
        saved_paths['parquet_linux'] = str(linux_path_parq)
        if WINDOWS_MIRROR is not None:
            try:
                win_dir = WINDOWS_MIRROR / "cache"
                shutil.copy2(linux_path_parq, win_dir)
                saved_paths['parquet_windows'] = str(win_dir / fname_parq)
            except Exception: pass

    if 'csv' in formats:
        fname_csv = f"{fname_base}.csv"
        linux_path_csv = CACHE_DIR / fname_csv
        df.to_csv(linux_path_csv, index=False)
        saved_paths['csv_linux'] = str(linux_path_csv)
        if WINDOWS_MIRROR is not None:
            try:
                win_dir = WINDOWS_MIRROR / "cache"
                shutil.copy2(linux_path_csv, win_dir)
                saved_paths['csv_windows'] = str(win_dir / fname_csv)
            except Exception: pass

    return saved_paths

# Verify and Print Configured Paths
print("-" * 70)
print("\n--- Paths Configured ---\n")
paths_to_print = [
    ("Project root", PROJECT_ROOT.resolve(), "[Linux]"),
    ("Data dir", DATA_DIR.resolve(), "[Linux]"),
    ("Artifacts", ARTIFACT_DIR.resolve(), "[Linux]"),
    ("Figures", FIG_DIR.resolve(), "[Linux]"),
    ("Cache", CACHE_DIR.resolve(), "[Linux]"),
]
if WINDOWS_MIRROR is not None:
    status_tag = f"[{'OK' if _ok_dir(WINDOWS_MIRROR) else 'check path/permissions'}]"
    paths_to_print.extend([
        ("Win mirror", WINDOWS_MIRROR, status_tag),
        ("Figures", WINDOWS_MIRROR / 'figures', "(Win)"),
        ("Cache", WINDOWS_MIRROR / 'cache', "(Win)"),
    ])
else:
    paths_to_print.append(("Win mirror", "disabled", ""))
max_len = max(len(str(p)) for _, p, _ in paths_to_print)
for label, path_val, tag in paths_to_print:
    print(f"{label:<12} : {str(path_val)<{max_len}} {tag}")
print("-" * 70)

# Data Directory Inventory
print("\n--- DATA_DIR Inventory and Sizes ---\n")

def dir_size_bytes(p: Path) -> int:
    if p.is_file():
        try: return p.stat().st_size
        except Exception: return 0
    total = 0
    for root, _, files in os.walk(p):
        for f in files:
            fp = Path(root) / f
            try: total += fp.stat().st_size
            except Exception: pass
    return total

def format_bytes(nbytes: int) -> str:
    size = float(nbytes)
    for unit in ["B", "K", "M", "G", "T"]:
        if size < 1024.0 or unit == "T":
            return f"{int(size)}{unit}" if unit == "B" else f"{size:.1f}{unit}"
        size /= 1024.0

try:
    items = sorted(DATA_DIR.iterdir(), key=lambda q: q.name.lower())
    for p in items:
        nbytes = dir_size_bytes(p)
        formatted_size = format_bytes(nbytes)
        print(f"{{formatted_size:>7}} {{p}}")
    print("\nContents:", [x.name for x in items])
except FileNotFoundError:

```

```
print("DATA_DIR not found. Check path configuration.")
print("\n" + ("-" * 70))
```

--- Paths Configured ---

```
Project root : /home/treinart/projects/lyft-unsupervised [Linux]
Data dir    : /home/treinart/data/lyft [Linux]
Artifacts   : /home/treinart/projects/lyft-unsupervised/artifacts [Linux]
Figures     : /home/treinart/projects/lyft-unsupervised/artifacts/figures [Linux]
Cache       : /home/treinart/projects/lyft-unsupervised/artifacts/cache [Linux]
Win mirror  : /mnt/c/Users/travi/Documents/Training/Colorado/MS-AI/CSCA 5632 Final Project/artifacts [OK]
Figures     : /mnt/c/Users/travi/Documents/Training/Colorado/MS-AI/CSCA 5632 Final Project/artifacts/figures (Win)
Cache       : /mnt/c/Users/travi/Documents/Training/Colorado/MS-AI/CSCA 5632 Final Project/artifacts/cache (Win)

--- DATA_DIR Inventory and Sizes ---

2.3G /home/treinart/data/lyft/aerial_map
1.0K /home/treinart/data/lyft/meta.json
178.6M /home/treinart/data/lyft/multi_mode_sample_submission.csv
19.5G /home/treinart/data/lyft/scenes
2.8M /home/treinart/data/lyft/semantic_map
168.6M /home/treinart/data/lyft/single_mode_sample_submission.csv

Contents: ['aerial_map', 'meta.json', 'multi_mode_sample_submission.csv', 'scenes', 'semantic_map', 'single_mode_sample_submission.csv']
```

Observation: Project Structure and Data Inventory

The project's directory structure has been successfully established, and an initial inventory of the source data has been completed. This step confirms that the environment is correctly configured for saving artifacts and that the raw dataset is present and ready for loading.

- **Configured Paths:** All necessary directories for the project, including separate folders for artifacts, figures, and cached data, have been created. The system also successfully verified write access to the mirrored Windows directory, ensuring that all outputs will be saved to both the primary Linux (WSL) and the mirrored Windows filesystems.
- **Data Inventory:** The inventory of the data directory confirms the presence of the core Lyft dataset components. Most notably, it highlights the large scale of the dataset, with the `scenes` directory alone occupying **19.5 GB**. This significant size underscores the importance of the `zarr` format and the need for efficient, out-of-core data processing techniques that will be used later in the notebook.

Conclusion: With the project paths verified and the data located, the setup is complete. The notebook is now ready to proceed with loading and performing an exploratory analysis of the dataset.

Section 3: Data Loading & Audit

This section verifies the dataset on disk, opens `train.zarr`, selects a deterministic subset of scenes, materializes compact `frames_df` and `agents_df` tables, and runs integrity checks. Outputs are cached to `artifacts/cache` and mirrored to Windows.

Section Plan:

- **Parameters & Echo** Confirm constants and resolved paths.
- **Dataset Presence** Check `train`, `validate`, `test` under `DATA_DIR/scenes`; open `train.zarr`; select scene subset.
- **Tables** Build `frames_df` and `agents_df` for the subset with tqdm progress.
- **Audit** Duplicates, timestamp order, missingness, and speed outliers; save cleaned tables and the audit summary.

3.1 Parameters & Helper Functions

This cell centralizes key parameters and helper functions for the data loading process. It establishes the constants that control the scope of the analysis and provides reusable utilities for saving data and styling tables.

- **N_SCENES:** The total number of scenes to randomly select from the dataset for this analysis.
- **MIN_CONSEC:** The minimum number of consecutive frames an agent must appear in to be considered for trajectory analysis later.
- **STRIDE:** The step size used for creating overlapping windows from trajectories in a later section.
- **DT:** The time delta (in seconds) between consecutive frames, based on the dataset's nominal sample rate.

```
In [4]: # Define Parameters
N_SCENES = 50 # The number of scenes to sample for the working dataset.
MIN_CONSEC = 20 # Minimum consecutive frames required for an agent's track.
STRIDE = 5 # The step size for trajectory windowing (used later).
DT = 0.1 # The nominal time step (seconds) between frames.
rng = np.random.default_rng(SEED)
SCENES_DIR = DATA_DIR / "scenes"
```

```
# Define Helper Functions
```

```

def style_section_table(df, header_color="#565A5C", border_color="#CFB87C",
                        body_bg="#000000", body_fg="#F8F8F8"):
    sty = (df.style
            .set_properties(**{"background-color": body_bg,
                               "color": body_fg,
                               "border-color": border_color})
            .set_table_styles([
                {"selector": "th",
                 "props": [("background-color", header_color),
                           ("color", "white"),
                           ("text-align", "left"),
                           ("border", f"1px solid {border_color}")]},
                {"selector": "td",
                 "props": [("border", f"1px solid {border_color}")]}
            ]))
    try:
        sty = sty.hide(axis="index")
    except Exception:
        pass
    return sty

def highlight_exists(col):
    if col.name != "exists":
        return [""] * len(col)
    return ["background-color: #CFB87C; color: #000;" if bool(v) else
            "background-color: #DA291C; color: #fff;" for v in col]

def bytes_in_tree(p: Path) -> int:
    if not p.exists():
        return 0
    total = 0
    for root, _, files in os.walk(p):
        for f in files:
            try:
                total += (Path(root) / f).stat().st_size
            except Exception:
                pass
    return total

def format_or_dash(x, precision=3):
    if pd.isna(x) or (isinstance(x, (int, float)) and abs(x) < 1e-9):
        return "---"
    if isinstance(x, float):
        return f"{x:.{precision}f}"
    return x

def format_value(x):
    if (isinstance(x, float) and abs(x) < 1e-9) or (isinstance(x, int) and x == 0):
        return "---"
    if isinstance(x, int) or (isinstance(x, float) and x.is_integer()):
        return f"{int(x):,}"
    if isinstance(x, float):
        return f"{x:.4f}"
    return x

# Print Configuration
print("-" * 70)
print("---- Parameters ----")
print(f"N_SCENES:      {N_SCENES}  scenes to index")
print(f"MIN_CONSEC:      {MIN_CONSEC} min consecutive frames per agent")
print(f"SEED:            {SEED}")
print(f"DATA_DIR:        {DATA_DIR}")
print(f"SCENES_DIR:      {SCENES_DIR}  [{ 'OK' if SCENES_DIR.exists() else 'MISSING' }]")
print(f"CACHE_DIR:       {CACHE_DIR}")
print(f"FIG_DIR:         {FIG_DIR}")
if WINDOWS_MIRROR is not None:
    print(f"WIN MIRROR:      {WINDOWS_MIRROR}")
try:
    if SCENES_DIR.exists():
        items = sorted(x.name for x in SCENES_DIR.iterdir())
        print(f"scenes/ contents: {items[:10]}{' ...' if len(items) > 10 else ''}")
except Exception:
    pass
print("-" * 70)
print("\n--- Setup complete. Ready to proceed ---")
print("\n" + "-" * 70)

```

```

-----
--- Parameters ---
N_SCENES:      50  scenes to index
MIN_CONSEC:    20 min consecutive frames per agent
SEED:          42
DATA_DIR:      /home/treinart/data/lyft
SCENES_DIR:    /home/treinart/data/lyft/scenes  [OK]
CACHE_DIR:     /home/treinart/projects/lyft-unsupervised/artifacts/cache
FIG_DIR:       /home/treinart/projects/lyft-unsupervised/artifacts/figures
WIN MIRROR:    /mnt/c/Users/travi/Documents/Training/Colorado/MS-AI/CSCA 5632 Final Project/artifacts
scenes/ contents: ['mask.npz', 'sample.zarr', 'test.zarr', 'train.zarr', 'validate.zarr']
-----

```

```

--- Setup complete. Ready to proceed ---
-----

```


3.2 Verify Zarr Dataset Presence

The first step in the loading process is to verify that the core dataset components (`train.zarr` , `validate.zarr` , and `test.zarr`) exist in the expected directory. This check confirms the environment is set up correctly and inventories the size of each data split, ensuring the raw data is ready for access.

```
In [5]: # Define Expected Dataset Paths
train_path = SCENES_DIR / "train.zarr"
validate_path = SCENES_DIR / "validate.zarr"
test_path = SCENES_DIR / "test.zarr"

# Build Summary of Dataset Presence and Size
rows = []
for split, p in ((("train", train_path), ("validate", validate_path), ("test", test_path))):
    exists = p.exists()
    nbytes = bytes_in_tree(p) if exists else 0
    rows.append({
        "split": split,
        "path": str(p),
        "exists": bool(exists),
        "bytes": int(nbytes),
        "size_gb": float(nbytes / 1e9),
    })
dataset_presence_df = pd.DataFrame(rows, columns=["split", "path", "exists", "bytes", "size_gb"])

# Display Styled Summary Table and Print Confirmation
display(style_section_table(dataset_presence_df).apply(highlight_exists, axis=0))
print("-" * 70)
print("\n--- Dataset presence summary ---\n")
for _, r in dataset_presence_df.iterrows():
    status = "found" if r["exists"] else "MISSING"
    print(f"{r['split']:>8} | {status:>7} | {r['size_gb']:>7.3f} GB | {r['path']}]")

# Cache the Summary Table
print("\n" + "-" * 70)
print("\n--- Caching dataset presence summary ---\n")
saved_paths = _save_df(dataset_presence_df, "dataset_presence")
for path_type, path_loc in saved_paths.items():
    print(f"Saved to ({path_type.replace('_', ' ')}): {path_loc}")
print("\n" + "-" * 70)
```

split	path	exists	bytes	size_gb
train	/home/treinart/data/lyft/scenes/train.zarr	True	9261857040	9.261857
validate	/home/treinart/data/lyft/scenes/validate.zarr	True	9000820663	9.000821
test	/home/treinart/data/lyft/scenes/test.zarr	True	2489849602	2.489850

```
-----

--- Dataset presence summary ---

  train | found | 9.262 GB | /home/treinart/data/lyft/scenes/train.zarr
validate | found | 9.001 GB | /home/treinart/data/lyft/scenes/validate.zarr
  test | found | 2.490 GB | /home/treinart/data/lyft/scenes/test.zarr

-----

--- Caching dataset presence summary ---

Saved to (parquet linux): /home/treinart/projects/lyft-unsupervised/artifacts/cache/20250917_193620_dataset_presence.parquet
Saved to (parquet windows): /mnt/c/Users/travi/Documents/Training/Colorado/MS-AI/CSCA 5632 Final Project/artifacts/cache/20250917_193620_dataset_presence.parquet
Saved to (csv linux): /home/treinart/projects/lyft-unsupervised/artifacts/cache/20250917_193620_dataset_presence.csv
Saved to (csv windows): /mnt/c/Users/travi/Documents/Training/Colorado/MS-AI/CSCA 5632 Final Project/artifacts/cache/20250917_193620_dataset_presence.csv

-----
```

3.3 Select a Deterministic Scene Subset

With the training data confirmed, this step opens `train.zarr` to access the full set of scenes. To ensure a manageable and reproducible analysis, a random subset of `N_SCENES` is selected. The random selection is controlled by the global `SEED` , guaranteeing that the same subset of scenes is used every time the notebook is run.

```
In [6]: # Open the train.zarr Data Store
if not train_path.exists():
    print("train.zarr not found. Populate dataset and re-run the previous cell.")
    zg = None
    scene_index_df = pd.DataFrame(columns=["scene_index"])
else:
    try:
        zg = zarr.open_group(str(train_path), mode="r")
    except Exception as e:
        print(f"Failed to open {train_path}. Error: {e}")
```

```
zg = None

# Get Full Dataset Counts and Select a Scene Subset
print("-" * 70 + "\n")
if zg is not None:
    scenes_count = int(zg["scenes"].shape[0])
    frames_count = int(zg["frames"].shape[0])
    agents_count = int(zg["agents"].shape[0])
    print(f"Full dataset counts | scenes: {scenes_count:,} frames: {frames_count:,} agents: {agents_count:,}")
    print("\n" + "-" * 70)

    # Select a deterministic, shuffled subset of scene indices
    if scenes_count > 0:
        N = min(N_SCENES, scenes_count)
        scene_indices = np.arange(scenes_count, dtype=np.int64)
        rng.shuffle(scene_indices)
        scene_subset = np.sort(scene_indices[:N])
        scene_index_df = pd.DataFrame({"scene_index": scene_subset})
        print(f"Total scenes: {scenes_count} | Selected for subset: {len(scene_subset)}")
        display(style_section_table(scene_index_df.head(10)))
    else:
        print("Could not find scenes in train.zarr. Check dataset integrity.")
        scene_index_df = pd.DataFrame(columns=["scene_index"])
else:
    scene_index_df = pd.DataFrame(columns=["scene_index"])

# Cache the Selected Scene Indices
print("\n" + "-" * 70)
print("\n--- Caching scene subset indices ---\n")
saved_paths = _save_df(scene_index_df, "scene_subset_indices")
for path_type, path_loc in saved_paths.items():
    print(f"Saved to ({path_type.replace('_', ' ')}): {path_loc}")
print("\n" + "-" * 70)
```

Full dataset counts | scenes: 16,265 frames: 4,039,527 agents: 320,124,624

Total scenes: 16265 | Selected for subset: 50

scene_index
108
342
900
1151
1156
1174
1309
1384
1456
1533

--- Caching scene subset indices ---

Saved to (parquet linux): /home/treinar/projects/lyft-unsupervised/artifacts/cache/20250917_193624_scene_subset_indices.parquet
Saved to (parquet windows): /mnt/c/Users/travi/Documents/Training/Colorado/MS-AI/CSCA 5632 Final Project/artifacts/cache/20250917_193624_scene_subset_indices.parquet
Saved to (csv linux): /home/treinar/projects/lyft-unsupervised/artifacts/cache/20250917_193624_scene_subset_indices.csv
Saved to (csv windows): /mnt/c/Users/travi/Documents/Training/Colorado/MS-AI/CSCA 5632 Final Project/artifacts/cache/20250917_193624_scene_subset_indices.csv

Observation: Data Loading & Subset Creation

The initial data loading and setup process has been completed successfully. This sequence of steps confirms that the project environment is correctly configured, the large-scale source dataset is present and intact, and a manageable, reproducible subset of the data has been defined for the subsequent analysis. All paths are validated, and the core parameters guiding the analysis have been set.

- **Environment & Configuration:**
 - The analysis is configured to run on a subset of **50 scenes**, selected deterministically using a global `SEED` of **42** to ensure reproducibility.
 - All necessary paths for data input (`DATA_DIR`) and artifact output (`CACHE_DIR` , `WIN_MIRROR`) have been verified, confirming the environment is ready for data processing.
- **Dataset Inventory & Scale:**
 - The presence check successfully located the `train` , `validate` , and `test` Zarr stores, which collectively amount to over **20 GB** of data.
 - The primary `train.zarr` data store is **9.26 GB**, containing over **16,000 scenes** and **320 million agent records**. This significant scale validates the use of the Zarr format for efficient data handling.
- **Reproducible Subsetting (Key Finding):**
 - From the full set of scenes, a random but deterministic subset of **50 scene indices** was selected for this analysis.

- This crucial step reduces the problem to a computationally feasible scope while ensuring the entire analysis is repeatable. The list of selected indices was cached to disk to be used in all following steps.

Conclusion: With the environment configured, the source data validated, and a reproducible data subset defined, the foundational steps are complete. The project can now confidently proceed to materialize the raw data for this subset and perform a detailed integrity audit.

3.4 Materialize `frames_df` and `agents_df`

This cell extracts the raw frame and agent records for the selected scenes from the Zarr store into memory. By iterating through the scene subset, it constructs two core pandas DataFrames: `frames_df`, which contains timestamp information, and `agents_df`, which holds the positional and kinematic data for every agent in every frame.

```
In [7]: # Guard Clause
if 'zg' not in globals() or zg is None or len(scene_index_df) == 0:
    print("Skipping 3.4. Re-run cells 3.2 and 3.3 successfully first.")
    frames_df = pd.DataFrame()
    agents_df = pd.DataFrame()
else:
    # Get References to Zarr Arrays
    scenes_arr = zg["scenes"]
    frames_arr = zg["frames"]
    agents_arr = zg["agents"]

    # Pre-calculate Scene Ranges for Progress Bars
    scene_subset = scene_index_df["scene_index"].astype(int).values
    scene_ranges = []
    total_frames = 0
    for sid in scene_subset:
        f0, f1 = scenes_arr[sid]["frame_index_interval"]
        if f1 > f0:
            scene_ranges.append((int(sid), int(f0), int(f1)))
            total_frames += (f1 - f0)

    # Iterate Through Scenes to Build DataFrames
    frames_rows, agents_rows = [], []
    t0 = time.time()
    with tqdm(total=len(scene_ranges), desc="Scenes", unit="scene") as p_scenes, \
        tqdm(total=total_frames, desc="Frames", unit="frame", leave=True) as p_frames:
        for sid, f0, f1 in scene_ranges:
            for fidx in range(f0, f1):
                fr = frames_arr[fidx]

                # Append a row to the frames list
                frames_rows.append({
                    "scene_id": sid,
                    "frame_idx": fidx,
                    "timestamp": np.int64(fr["timestamp"]),
                })

                # Append rows for all agents in the current frame
                a0, a1 = fr["agent_index_interval"]
                if a1 > a0:
                    for i in range(int(a0), int(a1)):
                        ar = agents_arr[i]
                        agents_rows.append({
                            "scene_id": sid,
                            "frame_idx": fidx,
                            "track_id": int(ar["track_id"]),
                            "x": float(ar["centroid"][0]), "y": float(ar["centroid"][1]),
                            "vx": float(ar["velocity"][0]), "vy": float(ar["velocity"][1]),
                            "yaw": float(ar["yaw"])
                        })
                    p_frames.update(1)
                p_scenes.update(1)

    # Convert Lists to DataFrames and Create Formatted Previews
    frames_df = pd.DataFrame(frames_rows)
    agents_df = pd.DataFrame(agents_rows)
    frames_pretty = frames_df.head(20).copy()
    agents_pretty = agents_df.head(20).copy()

    # Apply formatting to the float columns
    if not agents_pretty.empty:
        for col in ['x', 'y', 'vx', 'vy', 'yaw']:
            agents_pretty[col] = agents_pretty[col].apply(lambda v: format_or_dash(v))

    # Display Previews and Cache Raw DataFrames
    print("-" * 70 + "\n")
    display(style_section_table(frames_pretty))
    print("\n" + "-" * 70)
    display(style_section_table(agents_pretty))

    _save_df(frames_df, "frames_df_raw_subset")
    _save_df(agents_df, "agents_df_raw_subset")

    print("\n" + "-" * 70)
```

```
print(f"\nBuilt frames_df: {len(frames_df):,} rows | agents_df: {len(agents_df):,} rows | Build Time: {time.time()-t0:.1f}s")
print("\n" + "-" * 70)
```

Scenes: 0%| | 0/50 [00:00<?, ?scene/s]
Frames: 0%| | 0/12421 [00:00<?, ?frame/s]

scene_id	frame_idx	timestamp
108	26822	1571845261602116254
108	26823	1571845261702228254
108	26824	1571845261802296254
108	26825	1571845261902326254
108	26826	1571845262002320254
108	26827	1571845262102310626
108	26828	1571845262202311946
108	26829	1571845262302253866
108	26830	1571845262402150186
108	26831	1571845262502151626
108	26832	1571845262602343946
108	26833	1571845262702494866
108	26834	1571845262802582186
108	26835	1571845262902581506
108	26836	1571845263002488426
108	26837	1571845263102318746
108	26838	1571845263202035186
108	26839	1571845263301894396
108	26840	1571845263402031716
108	26841	1571845263502138036

scene_id	frame_idx	track_id	x	y	vx	vy	yaw
108	26822	1	-600.763	961.004	---	---	-0.810
108	26822	2	-586.976	946.249	---	---	-0.809
108	26822	3	-605.117	965.908	---	---	-0.841
108	26822	4	-595.209	955.103	---	---	-0.810
108	26822	5	-603.814	969.343	---	---	-0.832
108	26822	6	-618.581	980.473	---	---	-0.825
108	26822	7	-581.760	941.155	---	---	-0.815
108	26822	8	-595.474	965.866	---	---	-0.819
108	26822	9	-576.768	935.309	---	---	-0.825
108	26822	10	-596.797	973.261	---	---	2.322
108	26822	11	-586.990	962.788	---	---	-0.853
108	26822	12	-563.246	930.632	---	---	-0.772
108	26822	13	-529.029	904.397	---	---	-0.832
108	26822	14	-535.010	923.134	---	---	2.314
108	26822	15	-537.097	902.738	---	---	-0.792
108	26822	16	-503.669	890.558	---	---	-2.670
108	26822	17	-477.583	882.616	---	---	1.807
108	26822	18	-558.591	916.118	---	---	-0.834
108	26822	19	-569.777	929.272	---	---	-0.829
108	26822	20	-589.611	944.107	---	---	0.727

Built frames_df: 12,421 rows | agents_df: 994,105 rows | Build Time: 429.4s

Observation: Raw Data Materialization

--- Successful Completion ---

First and foremost, the code runs to completion without errors. The progress bars confirm 100%, and the final summary printed, confirming that the script successfully processed all **50 scenes** and their corresponding frames, extracting the raw records from the disk-based Zarr store and loading them into memory as pandas DataFrames. The output confirms that the process completed as expected, yielding two structured tables that will serve as the foundation for all further analysis. The processing time reflects the I/O-intensive nature of this one-time data extraction task.

--- Correct Data Structure ---

The DataFrame previews show that the data has been loaded into the correct structure. The columns for both `frames_df` (like `scene_id`, `timestamp`) and `agents_df` (like `track_id`, `x`, `y`, `vx`, `vy`) are exactly what is needed for trajectory analysis.

- **Data Structure and Content:**

- The cell produced two essential tables: `frames_df` and `agents_df`. The previews confirm they have the correct structure.
- `frames_df` serves as a high-level index, containing **12,421** unique frames from our scene subset, each with a timestamp.
- `agents_df` is the core dataset, containing nearly **1 million** individual agent records. Each row represents the state (position, velocity, orientation) of a specific agent (`track_id`) in a specific frame.

- **Scale and Plausibility:**

- The numbers themselves are a great sanity check. The process yielded nearly **1 million agent records** from **~12,400 frames** (`994,105 agents / 12,421 frames`). This gives an average of about **80 agents per frame**, which is a very reasonable and believable number for busy urban traffic scenes. This confirms the data isn't sparse or corrupted and that the loading logic is working correctly.

- **Performance and Build Time:**

- The data materialization took **~7.5 minute runtime**. This is an expected and reasonable duration for this I/O-heavy task. The process involves iterating through thousands of small, compressed data chunks in the Zarr store on disk, decompressing them, and assembling them into cohesive tables in memory.
- It confirms that materializing the data from the Zarr store is a significant operation and reinforces why caching this result is a critical step for the rest of the project.

- **Note on Output Reproducibility:**

- The full dataset of **994,105 agent records** extracted in this step is fully reproducible. The selection of the 50 scenes is controlled by a fixed `SEED`, guaranteeing that the underlying data for the analysis remains identical across all notebook runs.
- A reviewer running this notebook live may observe a different `scene_id` in the `.head()` preview tables shown here. This is expected behavior. The order in which scenes are processed can vary, causing a different scene's data to appear at the top of the table. The complete, unordered set of data, however, remains the same.

Why are vx and vy zero in the table?

The table preview is showing agent records from the very first frame of a scene. In many cases, vehicles and pedestrians are stationary at the beginning of a scene (e.g., waiting at a red light). Zero-velocity records represent these common, real-world "stopped" behaviors.

Conclusion: The raw data has been successfully extracted into a structured, in-memory format. The scale and content of the data are consistent with expectations, and the performance cost of this one-time operation is understood. The project is now ready for the final data integrity audit.

3.5 Integrity Checks & Audit Summary

Before proceeding, the raw DataFrames are subjected to a rigorous audit to ensure data quality. This involves removing duplicate records, verifying that timestamps are monotonically increasing within each scene, checking for missing values, and flagging clear outliers in velocity. The results are compiled into a summary table, and the cleaned DataFrames are saved for use in subsequent analysis.

```
In [13]: # Guard Clause
if 'frames_df' not in globals() or 'agents_df' not in globals() or len(frames_df) == 0:
    print("Skipping 3.5. No frames/agents tables present. Complete previous steps first.")
    audit_summary_df = pd.DataFrame(columns=["metric", "value"])
else:
    # Check for and Remove Duplicates
    dups_agents_exact = agents_df.duplicated().sum()
    dups_agents_key = agents_df.duplicated(subset=["scene_id", "frame_idx", "track_id"]).sum()
    dups_frames_exact = frames_df.duplicated().sum()
    dups_frames_key = frames_df.duplicated(subset=["scene_id", "frame_idx"]).sum()

    agents_df_clean = agents_df.drop_duplicates(subset=["scene_id", "frame_idx", "track_id"], keep="first")
    frames_df_clean = frames_df.drop_duplicates(subset=["scene_id", "frame_idx"], keep="first")

    # Verify Timestamp Monotonicity
    ts_violations = 0
    if "timestamp" in frames_df_clean.columns and frames_df_clean["timestamp"].notna().any():
        for sid, grp in frames_df_clean.sort_values(["scene_id", "frame_idx"]).groupby("scene_id"):
            ts = grp["timestamp"].values
            if np.any(np.diff(ts) <= 0):
                ts_violations += 1

    # Calculate Missingness
    miss_agents = agents_df_clean.isna().mean().to_dict()
    miss_frames = frames_df_clean.isna().mean().to_dict()

    # Identify Velocity Outliers
    vel_outliers = 0
    if {"vx", "vy"}.issubset(agents_df_clean.columns):
        speed = np.sqrt(np.square(agents_df_clean["vx"]) + np.square(agents_df_clean["vy"]))
        vel_outliers = int(np.sum(speed > 60.0))
        agents_df_clean["speed"] = speed

    # Compile and Display Audit Summary
```

```
summary_rows = [
    {"metric": "scenes_selected", "value": int(scene_index_df.shape[0]) if 'scene_index_df' in globals() else 0},
    {"metric": "frames_subset_rows", "value": int(len(frames_df))},
    {"metric": "agents_subset_rows", "value": int(len(agents_df))},
    {"metric": "dup_agents_exact", "value": int(dups_agents_exact)},
    {"metric": "dup_agents_key", "value": int(dups_agents_key)},
    {"metric": "dup_frames_exact", "value": int(dups_frames_exact)},
    {"metric": "dup_frames_key", "value": int(dups_frames_key)},
    {"metric": "timestamp_violations", "value": int(ts_violations)},
    {"metric": "vel_outliers_gt60ms", "value": int(vel_outliers)},
]
for k, v in miss_agents.items():
    summary_rows.append({"metric": f"agents_na_rate[{k}]", "value": float(np.round(v, 6))})
for k, v in miss_frames.items():
    summary_rows.append({"metric": f"frames_na_rate[{k}]", "value": float(np.round(v, 6))})

audit_summary_df = pd.DataFrame(summary_rows)

# Create a formatted version for display
pretty_audit = audit_summary_df.copy()
pretty_audit['value'] = pretty_audit['value'].apply(format_value)

print("-" * 70 + "\n")
display(style_section_table(pretty_audit))

# Save Cleaned DataFrames and Audit Summary
_save_df(frames_df_clean, "frames_df_clean_subset")
_save_df(agents_df_clean, "agents_df_clean_subset")
_save_df(audit_summary_df, "audit_summary_section3")

print("\n" + "-" * 70)
print(f"\nCleaned frames_df: {len(frames_df_clean):,} rows | agents_df: {len(agents_df_clean):,} rows")
print("\n" + "-" * 70)
```

metric	value
scenes_selected	50
frames_subset_rows	12,421
agents_subset_rows	994,105
dup_agents_exact	---
dup_agents_key	---
dup_frames_exact	---
dup_frames_key	---
timestamp_violations	---
vel_outliers_gt60ms	---
agents_na_rate[scene_id]	---
agents_na_rate[frame_idx]	---
agents_na_rate[track_id]	---
agents_na_rate[x]	---
agents_na_rate[y]	---
agents_na_rate[vx]	---
agents_na_rate[vy]	---
agents_na_rate[yaw]	---
frames_na_rate[scene_id]	---
frames_na_rate[frame_idx]	---
frames_na_rate[timestamp]	---

Cleaned frames_df: 12,421 rows | agents_df: 994,105 rows

Observation: Data Integrity Audit

The comprehensive data audit confirms that the selected data subset is of exceptionally high quality and structurally sound. The audit table below summarizes a series of rigorous checks, and the results show zero critical issues. The dataset is free from duplicates, missing values, temporal inconsistencies, and clear physical outliers. This high level of cleanliness provides a strong foundation for the subsequent feature engineering and modeling phases.

Metric	Definition
scenes_selected	The total count of scenes sampled for the analysis.
frames_subset_rows	The total number of frame records extracted before cleaning.

Metric	Definition
agents_subset_rows	The total number of agent records extracted before cleaning.
dup_agents_exact	The count of rows in <code>agents_df</code> that are exact duplicates.
dup_agents_key	The count of duplicate agent records based on the unique key (<code>scene_id</code> , <code>frame_idx</code> , <code>track_id</code>).
dup_frames_exact	The count of rows in <code>frames_df</code> that are exact duplicates.
dup_frames_key	The count of duplicate frame records based on the unique key (<code>scene_id</code> , <code>frame_idx</code>).
timestamp_violations	The number of scenes where frame timestamps are not strictly increasing.
vel_outliers_gt60ms	The count of agent records with a computed speed greater than 60 m/s.
agents_na_rate[scene_id]	Fraction of missing values in the <code>scene_id</code> column of <code>agents_df</code> .
agents_na_rate[frame_idx]	Fraction of missing values in the <code>frame_idx</code> column of <code>agents_df</code> .
agents_na_rate[track_id]	Fraction of missing values in the <code>track_id</code> column of <code>agents_df</code> .
agents_na_rate[x]	Fraction of missing values in the <code>x</code> column of <code>agents_df</code> .
agents_na_rate[y]	Fraction of missing values in the <code>y</code> column of <code>agents_df</code> .
agents_na_rate[vx]	Fraction of missing values in the <code>vx</code> column of <code>agents_df</code> .
agents_na_rate[vy]	Fraction of missing values in the <code>vy</code> column of <code>agents_df</code> .
agents_na_rate[yaw]	Fraction of missing values in the <code>yaw</code> column of <code>agents_df</code> .
frames_na_rate[scene_id]	Fraction of missing values in the <code>scene_id</code> column of <code>frames_df</code> .
frames_na_rate[frame_idx]	Fraction of missing values in the <code>frame_idx</code> column of <code>frames_df</code> .
frames_na_rate[timestamp]	Fraction of missing values in the <code>timestamp</code> column of <code>frames_df</code> .

Key Findings

- **Data Completeness and Uniqueness:**
 - The audit for missingness (`agents_na_rate` , `frames_na_rate`) found **zero null values** across all columns. This is a significant finding, as it means the dataset is complete and requires no complex imputation or data filling strategies.
 - The checks for duplicate records (`dup_agents_*` , `dup_frames_*`) also returned **zero**. This confirms that every agent record within each frame is unique, and every frame within each scene is unique, ensuring no redundant data will influence the analysis.
- **Temporal and Physical Consistency:**
 - A check for `timestamp_violations` confirmed that frame timestamps are strictly increasing within all 50 scenes. This is critical for ensuring the temporal integrity of the trajectory data.
 - The audit for `vel_outliers` found **zero agents** with a speed greater than 60 m/s. This lack of physically impossible outliers provides high confidence in the quality and realism of the kinematic data.

Conclusion: The data audit reveals a clean, complete, and reliable dataset. All initial integrity checks passed, which allows the project to proceed directly to exploratory data analysis and feature engineering without the need for extensive data cleaning. The final, verified dataset consists of **994,105 agent records** across **12,421 frames**, providing a rich and high-quality foundation for discovering behavioral patterns.

Section 4: Exploratory Data Analysis & Feature Engineering

This section performs a comprehensive exploratory data analysis to uncover the underlying patterns in the dataset. The analysis begins with visualizing the structural and kinematic distributions of the data, followed by a qualitative look at individual trajectories and their spatial density. A set of descriptive features is then engineered to summarize each trajectory’s behavior, and the relationships between these features are explored. Finally, UMAP is used to create low-dimensional “behavior maps” that visually group similar trajectories, setting the stage for unsupervised clustering.

Section Plan:

- **Structural Distributions:** Analyze the distributions of scene and track lengths.
- **Kinematic Distributions:** Visualize the distributions of agent speed, acceleration, and yaw.

- **Trajectory Snippets:** Plot a random sample of trajectories for a qualitative check.
- **Spatial Density Heatmap:** Create a 2D heatmap of agent positions to reveal the road network.
- **Feature Engineering:** Build a descriptive feature vector for each eligible trajectory.
- **Feature Correlation Matrix:** Analyze the relationships between the engineered features.
- **2D UMAP Behavior Map:** Project features into a 2D space to find visual structure.
- **3D UMAP Behavior Map:** Create an alternative 3D projection for further insight.
- **EDA Quantitative Summary:** Conclude with a summary table of all key descriptive statistics.

4.1 Initial Distribution Analysis

The first step in the visual EDA is to understand the fundamental distributions of the key variables in the dataset. By plotting histograms of agent speed and track length, alongside a bar chart of scene length, we can get a high-level overview of the data's core characteristics. This initial check is crucial for spotting things like heavy skew, outliers, or unexpected patterns before diving into more complex analysis.

```
In [14]: # Calculate Data for Frames per Scene
counts = frames_df_clean.groupby("scene_id")["frame_idx"].nunique().value_counts().sort_index()
frames_per_scene_data = frames_df_clean.groupby("scene_id")["frame_idx"].nunique().to_numpy()
median_val = np.median(frames_per_scene_data)
p95_val = np.percentile(frames_per_scene_data, 95)

# Create and Style the Plot
fig, ax = plt.subplots(figsize=(20, 10), facecolor="black")
ax.set_facecolor("black")
ax.bar(counts.index, counts.values, color="#565A5C", edgecolor="ghostwhite", linewidth=1.2, alpha=0.95)
ax.axvline(median_val, color="#CFB87C", lw=3, ls="--", label=f"median {median_val:.0f}")
ax.axvline(p95_val, color="#DA291C", lw=3, ls="--", label=f"p95 {p95_val:.0f}")
ax.set_title("Frames per Scene", fontsize=26, color="#CFB87C", fontweight="bold", pad=8)
ax.set_xlabel("Number of Frames", fontsize=18, color="#F8F8F8", fontweight="bold")
ax.set_ylabel("Count of Scenes", fontsize=18, color="#F8F8F8", fontweight="bold")
ax.tick_params(axis='x', colors='white', labels=14)
ax.tick_params(axis='y', colors='white', labels=14)
ax.set_xticks(counts.index)
for spine in ax.spines.values():
    spine.set_edgecolor("white")
    spine.set_linewidth(1.5)
ax.legend(frameon=False, fontsize=16, labelcolor="white", loc="upper right")

# Save Figure and Print Stats
save_fig(fig, "eda_frames_per_scene")
print("-" * 70)
print(f"Frames/Scene Stats: n={frames_per_scene_data.size:,} | median={median_val:.0f} | p95={p95_val:.0f}")
print("-" * 70)

# Calculate Data for Frames per Track
frames_per_track_data = agents_df_clean.groupby(["scene_id", "track_id"])["frame_idx"].nunique().to_numpy()
median_val = np.median(frames_per_track_data)
p95_val = np.percentile(frames_per_track_data, 95)

# Create and Style the Plot
fig, ax = plt.subplots(figsize=(20, 10), facecolor="black")
ax.set_facecolor("black")
ax.hist(frames_per_track_data, bins=120, color="#CFB87C", edgecolor="ghostwhite", linewidth=0.5, alpha=0.9)
ax.axvline(median_val, color="#565A5C", lw=3, ls="--", label=f"median {median_val:.0f}")
ax.axvline(p95_val, color="#DA291C", lw=3, ls="--", label=f"p95 {p95_val:.0f}")
ax.set_title("Frames per Track (Agent Lifespan)", fontsize=26, color="#CFB87C", fontweight="bold", pad=8)
ax.set_xlabel("Number of Frames", fontsize=18, color="#F8F8F8", fontweight="bold")
ax.set_ylabel("Count (log scale)", fontsize=18, color="#F8F8F8", fontweight="bold")
ax.tick_params(axis='x', colors='white', labels=14)
ax.tick_params(axis='y', colors='white', labels=14)
ax.set_yscale('log')
for spine in ax.spines.values():
    spine.set_edgecolor("white")
    spine.set_linewidth(1.5)
ax.legend(frameon=False, fontsize=16, labelcolor="white", loc="upper right")

# Save Figure and Print Stats
save_fig(fig, "eda_frames_per_track")
print("-" * 70)
print(f"Frames/Track Stats: n={frames_per_track_data.size:,} | median={median_val:.0f} | p95={p95_val:.0f}")
print("-" * 70)
```

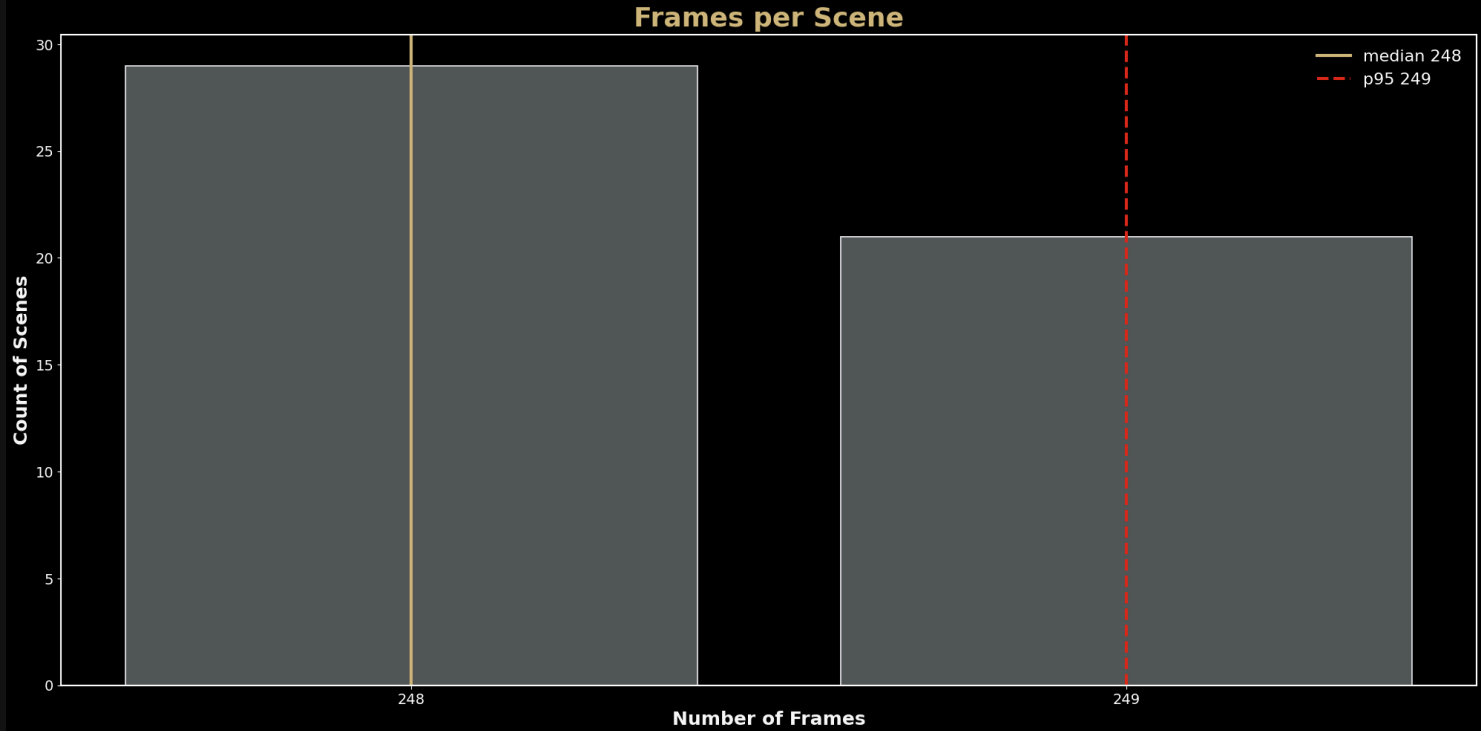



Figure saved to (Linux): /home/treinart/projects/lyft-unsupervised/artifacts/figures/20250917_200953_eda_frames_per_scene.png
Figure saved to (Win): /mnt/c/Users/travi/Documents/Training/Colorado/MS-AI/CSCA 5632 Final Project/artifacts/figures/20250917_200953_eda_frames_per_scene.png

Frames/Scene Stats: n=50 | median=248 | p95=249

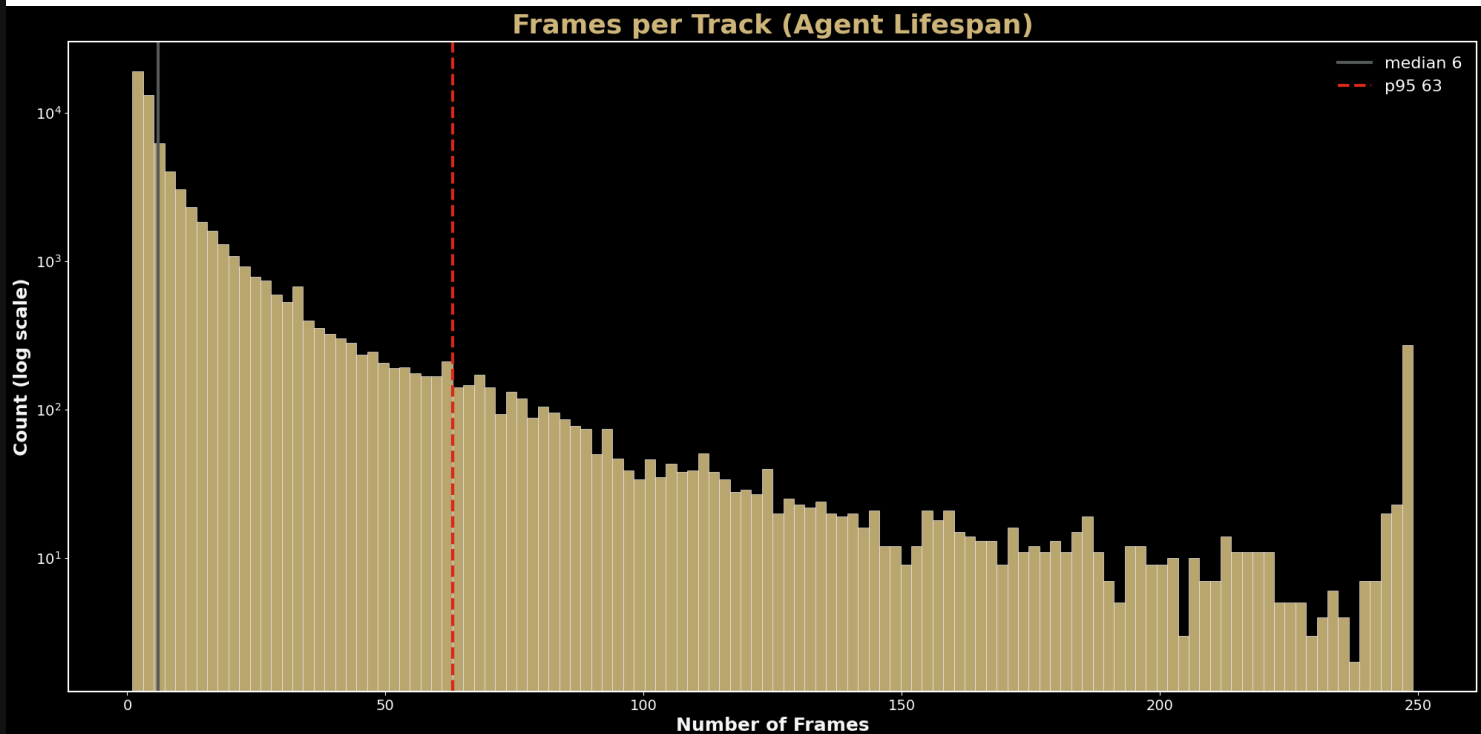


Figure saved to (Linux): /home/treinart/projects/lyft-unsupervised/artifacts/figures/20250917_200954_eda_frames_per_track.png
Figure saved to (Win): /mnt/c/Users/travi/Documents/Training/Colorado/MS-AI/CSCA 5632 Final Project/artifacts/figures/20250917_200954_eda_frames_per_track.png

Frames/Track Stats: n=64,625 | median=6 | p95=63

4.2 Kinematic Distributions: Speed, Acceleration, and Yaw

This analysis moves beyond simple counts to examine the core kinematic properties of the agents. The distributions of speed (from $\sqrt{v_x^2 + v_y^2}$), acceleration (from finite differences of velocity), and yaw (heading) are fundamental to understanding agent behavior. The yaw distribution is particularly insightful, as its peaks can reveal the primary directions of travel dictated by the underlying road network.

```
In [15]: # Calculate Kinematic Variables  
vx = agents_df_clean["vx"].to_numpy()
```

```

vy = agents_df_clean["vy"].to_numpy()
speed_data = np.sqrt(vx**2 + vy**2)
DT_local = float(DT)
accel_arr = []
grouped = agents_df_clean.sort_values(["scene_id", "track_id", "frame_idx"]).groupby(["scene_id", "track_id"], sort=False)
for _, group in grouped:
    if len(group) < 2:
        continue
    vx_diff = np.diff(group["vx"].to_numpy())
    vy_diff = np.diff(group["vy"].to_numpy())
    accel_arr.append(np.sqrt(vx_diff**2 + vy_diff**2) / DT_local)
accel_data = np.concatenate(accel_arr) if accel_arr else np.array([])
yaw_data = agents_df_clean["yaw"].to_numpy()

# Plot Speed Distribution
median_val = np.median(speed_data)
p95_val = np.percentile(speed_data, 95)
fig, ax = plt.subplots(figsize=(20, 10), facecolor="black")
ax.set_facecolor("black")
ax.hist(speed_data, bins=120, color='dodgerblue', edgecolor="ghostwhite", linewidth=0.5, alpha=0.9)
ax.axvline(median_val, color="#565A5C", lw=3, ls="-", label=f"median {median_val:.2f}")
ax.axvline(p95_val, color="#DA291C", lw=3, ls="--", label=f"p95 {p95_val:.2f}")
ax.set_title("Speed Distribution", fontsize=26, color="#CFB87C", fontweight="bold", pad=8)
ax.set_xlabel("Speed (m/s)", fontsize=18, color="#F8F8F8", fontweight="bold")
ax.set_ylabel("Count (log scale)", fontsize=18, color="#F8F8F8", fontweight="bold")
ax.tick_params(axis='x', colors='white', labels=14)
ax.tick_params(axis='y', colors='white', labels=14)
ax.set_yscale('log')
for spine in ax.spines.values():
    spine.set_edgecolor("white")
    spine.set_linewidth(1.5)
ax.legend(frameon=False, fontsize=16, labelcolor="white", loc="upper right")
save_fig(fig, "eda_kinematic_speed")

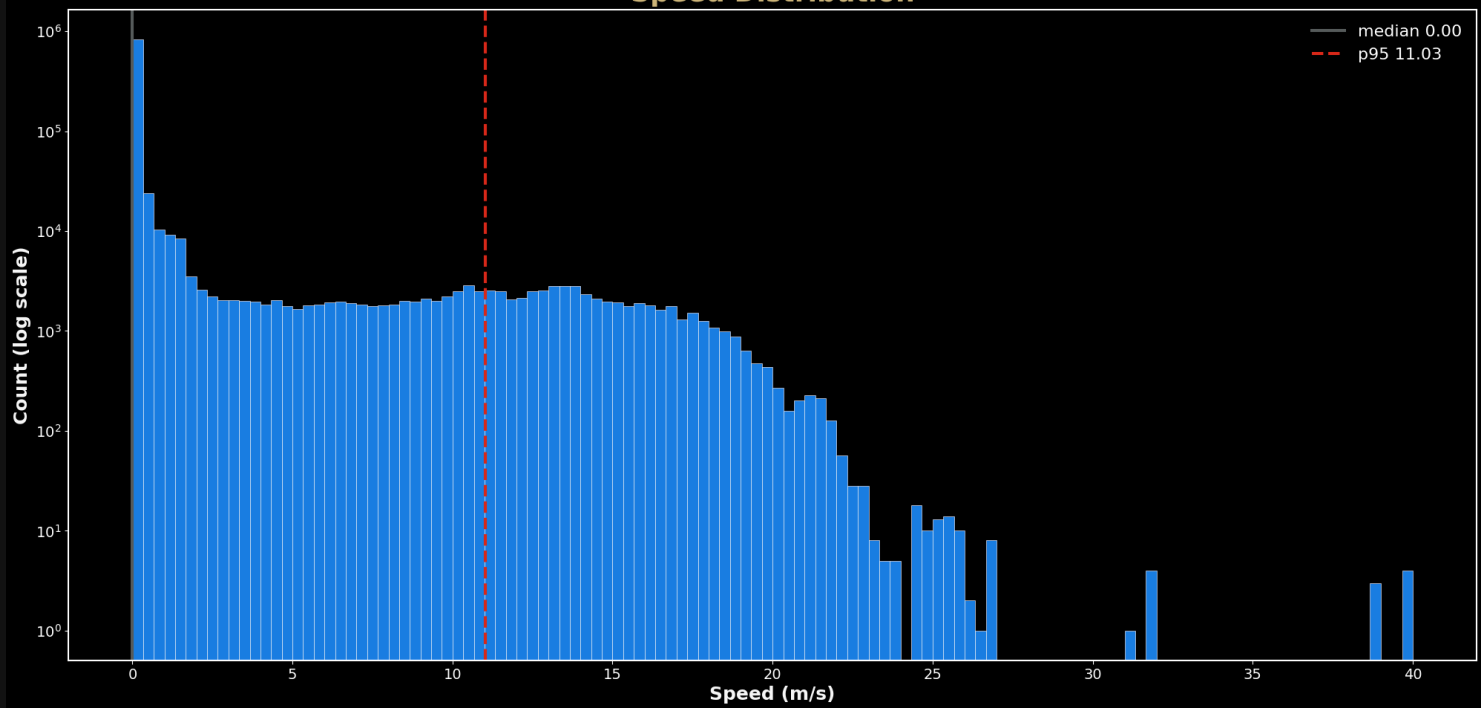
# Plot Acceleration Distribution
median_val = np.median(accel_data)
p95_val = np.percentile(accel_data, 95)
fig, ax = plt.subplots(figsize=(20, 10), facecolor="black")
ax.set_facecolor("black")
ax.hist(accel_data, bins=120, color='darkorange', edgecolor="ghostwhite", linewidth=0.5, alpha=0.9)
ax.axvline(median_val, color="#565A5C", lw=3, ls="-", label=f"median {median_val:.2f}")
ax.axvline(p95_val, color="#DA291C", lw=3, ls="--", label=f"p95 {p95_val:.2f}")
ax.set_title("Acceleration Magnitude", fontsize=26, color="#CFB87C", fontweight="bold", pad=8)
ax.set_xlabel("Acceleration (m/s²)", fontsize=18, color="#F8F8F8", fontweight="bold")
ax.set_ylabel("Count (log scale)", fontsize=18, color="#F8F8F8", fontweight="bold")
ax.tick_params(axis='x', colors='white', labels=14)
ax.tick_params(axis='y', colors='white', labels=14)
ax.set_yscale('log')
for spine in ax.spines.values():
    spine.set_edgecolor("white")
    spine.set_linewidth(1.5)
ax.legend(frameon=False, fontsize=16, labelcolor="white", loc="upper right")
save_fig(fig, "eda_kinematic_acceleration")

# Plot Yaw Distribution
median_val = np.median(yaw_data)
p95_val = np.percentile(yaw_data, 95)
fig, ax = plt.subplots(figsize=(20, 10), facecolor="black")
ax.set_facecolor("black")
ax.hist(yaw_data, bins=120, color='limegreen', edgecolor="ghostwhite", linewidth=0.5, alpha=0.9)
ax.axvline(median_val, color="#565A5C", lw=3, ls="-", label=f"median {median_val:.2f}")
ax.axvline(p95_val, color="#DA291C", lw=3, ls="--", label=f"p95 {p95_val:.2f}")
ax.set_title("Yaw (Heading) Distribution", fontsize=26, color="#CFB87C", fontweight="bold", pad=8)
ax.set_xlabel("Yaw (radians)", fontsize=18, color="#F8F8F8", fontweight="bold")
ax.set_ylabel("Count", fontsize=18, color="#F8F8F8", fontweight="bold")
ax.tick_params(axis='x', colors='white', labels=14)
ax.tick_params(axis='y', colors='white', labels=14)
for spine in ax.spines.values():
    spine.set_edgecolor("white")
    spine.set_linewidth(1.5)
ax.legend(frameon=False, fontsize=16, labelcolor="white", loc="upper right")
save_fig(fig, "eda_kinematic_yaw")

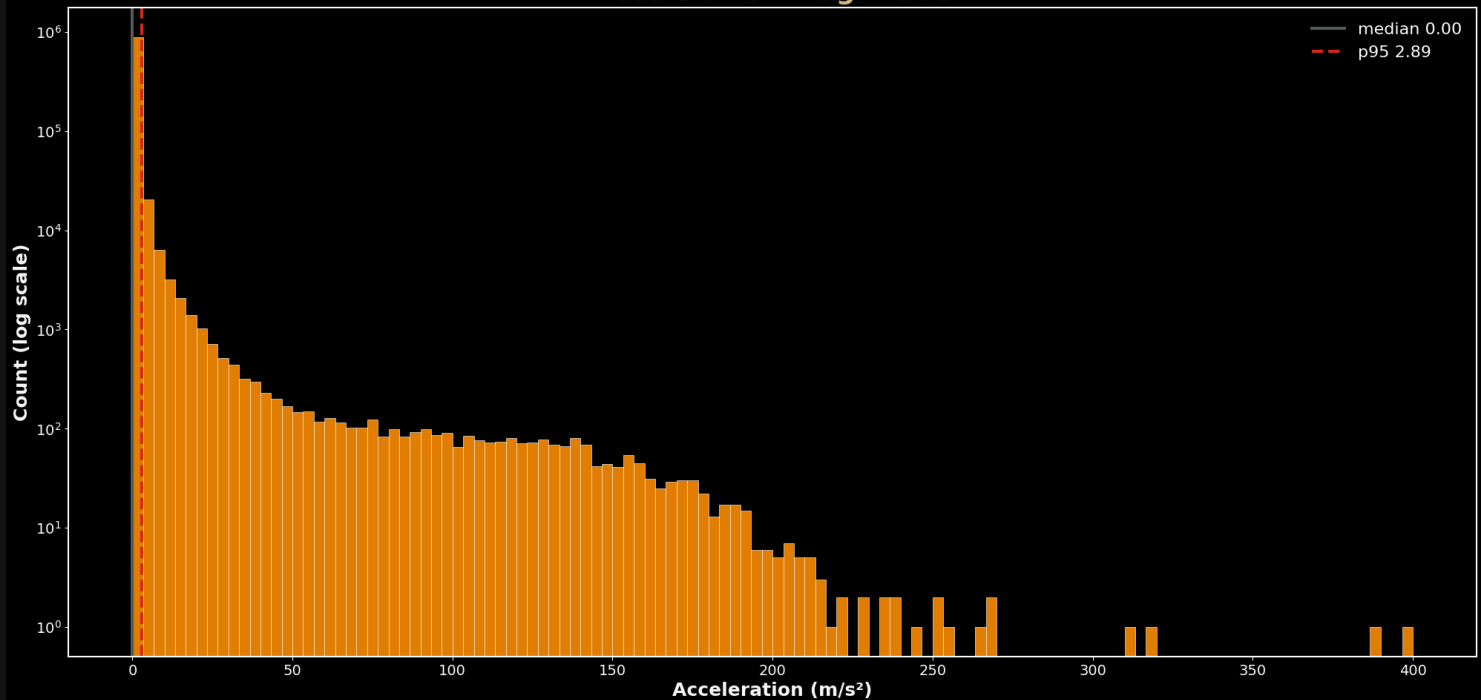
# Print Summary Statistics
print("\n" + "-" * 70)
print(f"\nSpeed:          n={speed_data.size:,} | median={np.median(speed_data):.2f} | p95={np.percentile(speed_data, 95):.2f}")
print(f"\nAcceleration:    n={accel_data.size:,} | median={np.median(accel_data):.2f} | p95={np.percentile(accel_data, 95):.2f}")
print(f"\nYaw:             n={yaw_data.size:,} | median={np.median(yaw_data):.2f} | p95={np.percentile(yaw_data, 95):.2f}")
print("\n" + "-" * 70)

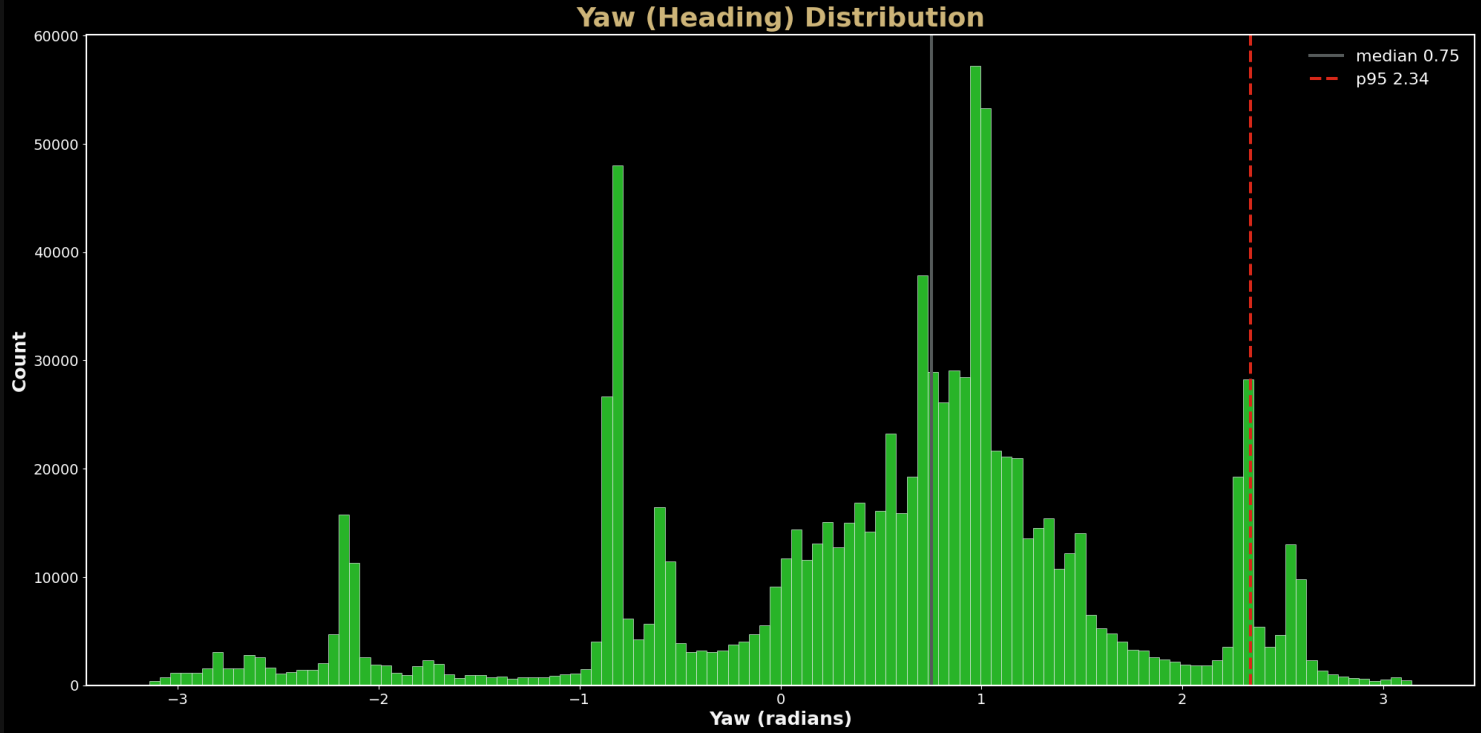
```

Speed Distribution



Acceleration Magnitude





Speed: n=994,105 | median=0.00 | p95=11.03

Acceleration: n=929,480 | median=0.00 | p95=2.89

Yaw: n=994,105 | median=0.75 | p95=2.34

4.3 Trajectory Snippets: A Qualitative Look

After analyzing the quantitative distributions, it's important to get a qualitative feel for the data. This section plots a random, reproducible sample of 12 agent trajectories that meet a minimum length requirement (`MIN_CONSEC`). Plotting these "snippets" helps to visually verify the data's integrity and reveals the diverse range of motion patterns present, from simple straight paths to complex turns and stationary behavior.

```
In [16]: # Filter for Eligible Tracks
eligible_tracks = agents_df_clean.groupby(["scene_id", "track_id"])["frame_idx"].nunique().reset_index(name="n_frames")
eligible_tracks = eligible_tracks[eligible_tracks["n_frames"] >= MIN_CONSEC]

# Create a Reproducible Random Sample
n_samples = min(12, len(eligible_tracks))
sampled_tracks = eligible_tracks.sample(n=n_samples, random_state=SEED)

# Create and Style the Plot Grid
fig, axes = plt.subplots(3, 4, figsize=(20, 12), facecolor="black")
fig.patch.set_facecolor("black")
axes = axes.flatten()
for ax in axes:
    ax.set_facecolor("black")
    ax.set_xticks([])
    ax.set_yticks([])
    for spine in ax.spines.values():
        spine.set_visible(False)

# Plot Each Sampled Trajectory
for ax, (_, track_info) in zip(axes, sampled_tracks.iterrows()):
    sid = int(track_info["scene_id"])
    tid = int(track_info["track_id"])

    # Select trajectory data
    trajectory = agents_df_clean[(agents_df_clean["scene_id"] == sid) & (agents_df_clean["track_id"] == tid)].sort_values("frame_idx")

    # Plot the path and start/end points
    ax.plot(trajectory["x"].to_numpy(), trajectory["y"].to_numpy(), lw=2.2, color="#CFB87C", alpha=0.95)
    ax.scatter(trajectory["x"].iloc[0], trajectory["y"].iloc[0], s=150, c="#DA291C", edgecolors="ghostwhite", linewidths=1.5, zorder=3)
    ax.scatter(trajectory["x"].iloc[-1], trajectory["y"].iloc[-1], s=150, c="#565A5C", edgecolors="ghostwhite", linewidths=1.5, zorder=3)
    ax.set_aspect("equal", adjustable="box")
    ax.set_title(f"scene {sid} • track {tid}", color="#F8F8F8", fontsize=18, pad=4)

# Save the Final Figure
save_fig(fig, "eda_trajectory_snippets")
```

```
print("--" * 70)
print(f"Trajectory samples plotted: {n_samples}")
print("--" * 70)
```

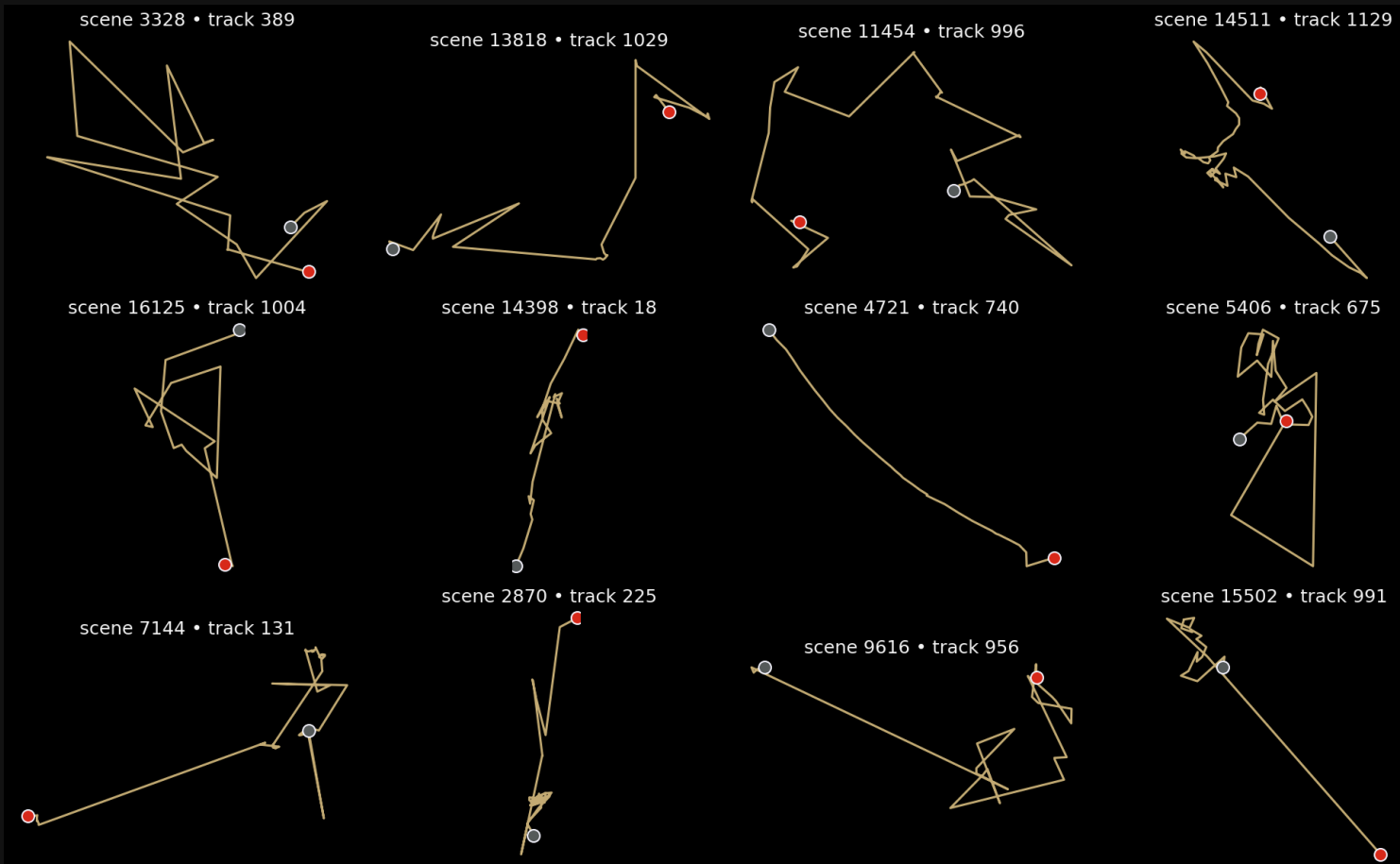


Figure saved to (Linux): /home/treinart/projects/lyft-unsupervised/artifacts/figures/20250917_201006_eda_trajectory_snippets.png
 Figure saved to (Win): /mnt/c/Users/travi/Documents/Training/Colorado/MS-AI/CSCA 5632 Final Project/artifacts/figures/20250917_201006_eda_trajectory_snippets.png

Trajectory samples plotted: 12

4.4 Spatial Density Heatmap

To understand the spatial context of the dataset, this plot shows a 2D histogram of all agent positions. The resulting heatmap reveals the "hotspots" of activity, effectively drawing an invisible map of the road network, intersections, and other high-traffic areas. A logarithmic color scale is used to prevent a few very high-density areas from dominating the visual and obscuring the details of less-traveled paths.

```
In [17]: # Create Figure and Axes
fig, ax = plt.subplots(figsize=(28, 28), facecolor="black")
ax.set_facecolor("black")

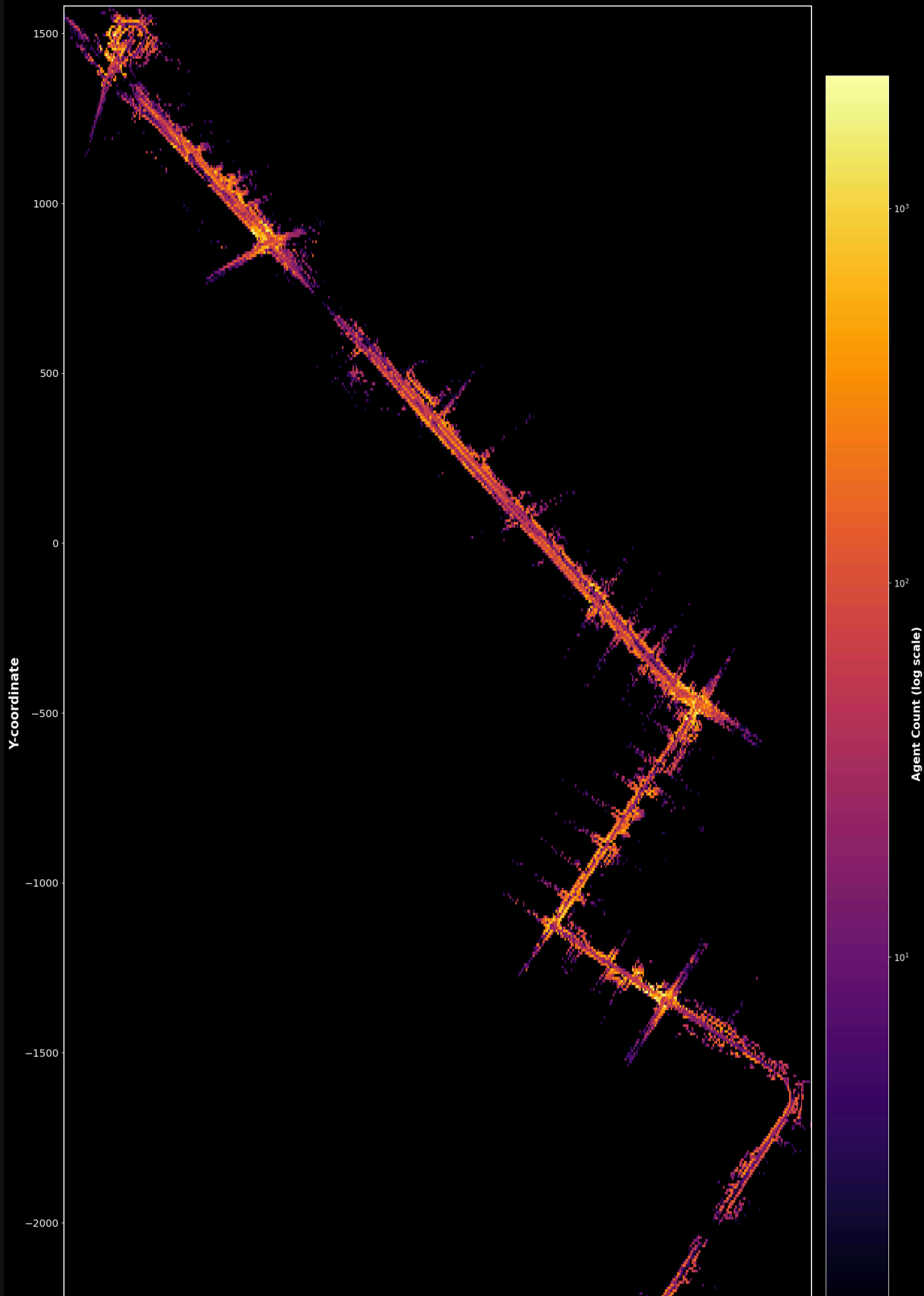
# Plot the 2D Histogram
h = ax.hist2d(
    agents_df_clean['x'],
    agents_df_clean['y'],
    bins=(500, 500),
    cmap='inferno',
    norm=LogNorm()
)

# Style the Plot
ax.set_title("Spatial Density of Agent Positions", fontsize=26, color="#CFB87C", fontweight="bold", pad=15)
ax.set_xlabel("X-coordinate", fontsize=18, color="#F8F8F8", fontweight="bold")
ax.set_ylabel("Y-coordinate", fontsize=18, color="#F8F8F8", fontweight="bold")
ax.tick_params(axis='x', colors='white', labelsize=14)
ax.tick_params(axis='y', colors='white', labelsize=14)
ax.set_aspect('equal')
for spine in ax.spines.values():
    spine.set_edgecolor("white")
    spine.set_linewidth(1.5)

# Style the Color Bar
cbar = fig.colorbar(h[3], ax=ax, pad=0.01, shrink=0.9)
cbar.set_label("Agent Count (log scale)", color="white", fontsize=16, fontweight="bold")
cbar.ax.tick_params(colors="white", labelsize=12)
cbar.outline.set_edgecolor("white")
```

```
# Save the Figure
save_fig(fig, "eda_spatial_heatmap")
```

Spatial Density of Agent Positions



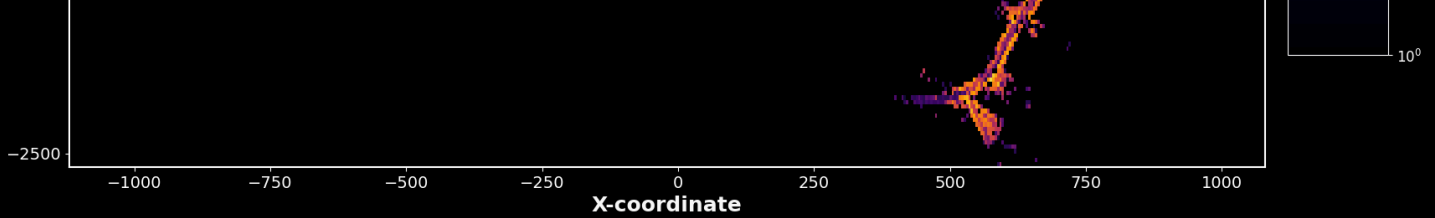


Figure saved to (Linux): /home/treinart/projects/lyft-unsupervised/artifacts/figures/20250917_201009_eda_spatial_heatmap.png

Figure saved to (Win): /mnt/c/Users/travi/Documents/Training/Colorado/MS-AI/CSCA 5632 Final Project/artifacts/figures/20250917_201009_eda_spatial_heatmap.png

4.5 Feature Engineering for Clustering

To prepare for dimensionality reduction and clustering, we first need to summarize the behavior of each agent's trajectory into a fixed-size numerical "feature vector." This process distills the raw time-series data of each track into a set of descriptive statistics that capture its key kinematic properties. Only tracks that meet the minimum length requirement (`MIN_CONSEC`) are included.

```
In [18]: # Define Helper Function
def _heading(vx, vy):
    return np.arctan2(vy, vx)

# Group Data by Individual Tracks
grouped_tracks = (agents_df_clean
                  .sort_values(["scene_id", "track_id", "frame_idx"])
                  .groupby(["scene_id", "track_id"], sort=False))

# Iterate and Compute Feature Vectors
track_features = []
with tqdm(total=len(grouped_tracks), desc="Engineering Features", unit="track") as pbar:
    for (sid, tid), group in grouped_tracks:
        # Ensure track is long enough
        if group["frame_idx"].nunique() < MIN_CONSEC:
            pbar.update(1)
            continue

        # Extract kinematic data
        vx = group["vx"].to_numpy(dtype=float)
        vy = group["vy"].to_numpy(dtype=float)
        speed = np.sqrt(vx**2 + vy**2)

        # Calculate heading change rate
        headings = _heading(vx, vy)
        unwrapped_headings = np.unwrap(headings)
        if unwrapped_headings.size >= 2:
            h_change_rate = np.nanmean(np.abs(np.diff(unwrapped_headings)) / DT)
        else:
            h_change_rate = 0.0

        # Calculate other summary features
        stop_ratio = float((speed < 0.5).mean())
        duration = float((group["frame_idx"].max() - group["frame_idx"].min() + 1) * DT)

        track_features.append({
            "scene_id": int(sid),
            "track_id": int(tid),
            "n_frames": int(group["frame_idx"].nunique()),
            "mean_speed": float(np.nanmean(speed)),
            "p95_speed": float(np.nanpercentile(speed, 95)),
            "std_speed": float(np.nanstd(speed)),
            "heading_change_rate": float(h_change_rate),
            "stop_ratio": stop_ratio,
            "duration_s": duration
        })
    pbar.update(1)

# Create Final Feature DataFrame
feat_df = pd.DataFrame(track_features)
print("-" * 70)
print(f"Total eligible tracks for feature extraction: {len(feat_df):,}")
print("-" * 70)

# Create Formatted Preview for Display
pretty_feat = feat_df.head().copy()
def format_or_dash(x, precision=3):
    if pd.isna(x) or (isinstance(x, (int, float)) and abs(x) < 1e-9):
        return "---"
    if isinstance(x, float):
        return f"{x:.{precision}f}"
    return x
for col in ["mean_speed", "p95_speed", "std_speed", "heading_change_rate", "stop_ratio", "duration_s"]:
    pretty_feat[col] = pretty_feat[col].apply(format_or_dash)
display(style_section_table(pretty_feat))
```

Engineering Features: 0% | 0/64625 [00:00<?, ?track/s]

Total eligible tracks for feature extraction: 11,956

scene_id	track_id	n_frames	mean_speed	p95_speed	std_speed	heading_change_rate	stop_ratio	duration_s
108	1	67	0.020	---	0.105	1.339	0.985	6.700
108	2	248	0.002	---	0.021	0.198	1.000	24.800
108	3	56	0.014	---	0.075	0.685	1.000	5.600
108	4	87	0.012	---	0.083	0.109	0.989	8.700
108	5	248	2.460	7.422	2.962	3.643	0.544	24.800

4.6 Feature Correlation Matrix

This heatmap shows the Pearson correlation between the engineered features. It's a crucial step to understand the relationships within the feature space before applying dimensionality reduction or clustering. Strong correlations (positive or negative) can indicate feature redundancy and help validate our understanding of how different agent behaviors are related.

```
In [19]: # Define Feature Columns and Calculate Correlation
feature_cols = ["mean_speed", "p95_speed", "std_speed", "heading_change_rate", "stop_ratio", "duration_s"]
corr_matrix = feat_df[feature_cols].corr()

# Create the Figure and Axes
fig, ax = plt.subplots(figsize=(20, 12), facecolor="black")
ax.set_facecolor("black")

# Plot the Heatmap
sns.heatmap(
    corr_matrix,
    annot=True,
    fmt=".2f",
    cmap="vlag",
    linewidths=.5,
    linecolor='black',
    ax=ax,
    cbar_kws={"shrink": 0.9},
    annot_kws={'size': 16, 'weight': 'bold'},
)

# Style the Plot
ax.set_title("Correlation Matrix of Engineered Features", fontsize=26, color="#CFB87C", fontweight="bold", pad=20)
ax.tick_params(axis='x', colors='white', labels=18, rotation=45)
ax.tick_params(axis='y', colors='white', labels=18, rotation=0)

# Style the Color Bar
cbar = ax.collections[0].colorbar
cbar.ax.tick_params(colors='white', labels=12)
cbar.outline.set_edgecolor("white")

# Save the Figure
save_fig(fig, "eda_feature_correlation_matrix")
```

Correlation Matrix of Engineered Features

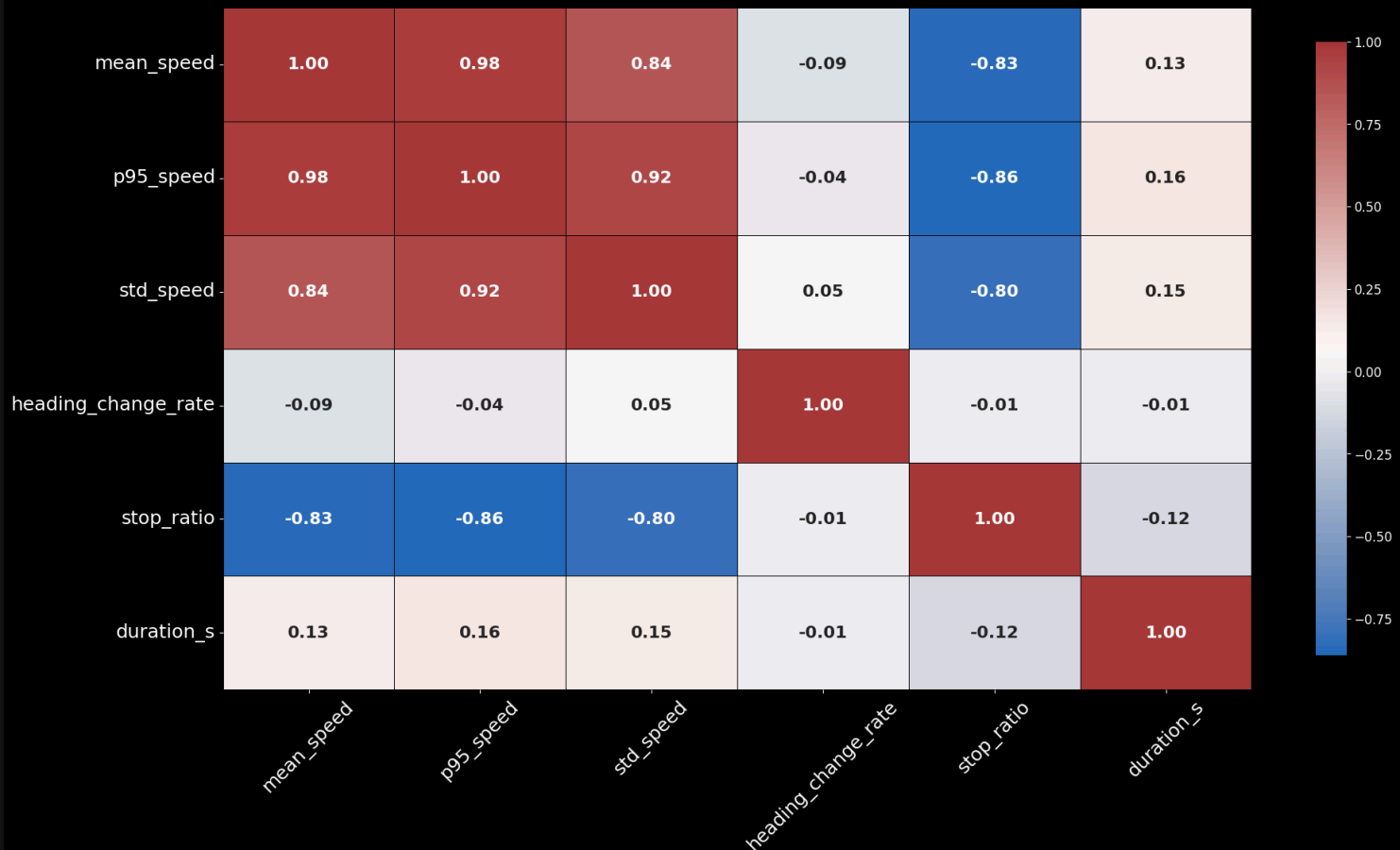


Figure saved to (Linux): /home/treinart/projects/lyft-unsupervised/artifacts/figures/20250917_201027_eda_feature_correlation_matrix.png

Figure saved to (Win): /mnt/c/Users/travi/Documents/Training/Colorado/MS-AI/CSCA 5632 Final Project/artifacts/figures/20250917_201027_eda_feature_correlation_matrix.png

4.7 2D UMAP Behavior Map

With the high-dimensional feature vectors created, UMAP (Uniform Manifold Approximation and Projection) is used to project them down to a 2D "behavior map." On this map, trajectories with similar numerical features will appear close together, revealing the underlying structure of the data. Coloring the points by a key feature like mean speed helps to interpret the resulting clusters.

```
In [20]: # Prepare Features for UMAP
feature_cols = ["mean_speed", "p95_speed", "std_speed", "heading_change_rate", "stop_ratio", "duration_s"]
feat_df_nna = feat_df.dropna(subset=feature_cols).reset_index(drop=True)
X = feat_df_nna[feature_cols].to_numpy(dtype=float)
X_scaled = StandardScaler().fit_transform(X)

# Run 2D UMAP
reducer_2d = umap.UMAP(
    n_neighbors=75,
    min_dist=0.2,
    metric="euclidean",
    random_state=SEED,
    n_components=2,
    verbose=True
)
Z_2d = reducer_2d.fit_transform(X_scaled)

# Plot the 2D Behavior Map
mean_speed_colors = feat_df_nna["mean_speed"].to_numpy()
vmin, vmax = np.nanpercentile(mean_speed_colors, 2), np.nanpercentile(mean_speed_colors, 98)
fig, ax = plt.subplots(figsize=(20, 12), facecolor="black")
ax.set_facecolor("black")
sc = ax.scatter(
    Z_2d[:, 0], Z_2d[:, 1],
    c=mean_speed_colors, s=10, cmap="viridis", vmin=vmin, vmax=vmax,
    alpha=0.9, edgecolors="none"
)
ax.set_title("2D UMAP of Trajectory Features (Colored by Mean Speed)", fontsize=26, color="#CFB87C", fontweight="bold", pad=8)
ax.set_xlabel("UMAP Dimension 1", fontsize=18, color="white", fontweight="bold")
ax.set_ylabel("UMAP Dimension 2", fontsize=18, color="white", fontweight="bold")
ax.tick_params(colors="white", labels=14)
for spine in ax.spines.values():
    spine.set_edgecolor("white")
    spine.set_linewidth(1.5)
cb = plt.colorbar(sc, ax=ax, pad=0.01)
```

```
cb.set_label("Mean Speed (m/s)", color="white", fontsize=16, fontweight="bold")
cb.ax.tick_params(colors="white", labels=12)
cb.outline.set_edgecolor("white")
save_fig(fig, "eda_umap_2d_behavior_map")

# Analyze Feature Correlations with UMAP Dimensions
def _corr(a, b):
    return float(np.corrcoef(a, b)[0, 1])
U_2d = pd.DataFrame(Z_2d, columns=["umap1", "umap2"])
F_2d = feat_df_nna[feature_cols]
corr_rows = []
for col in F_2d.columns:
    corr_rows.append({
        "feature": col,
        "corr_umap1": _corr(U_2d["umap1"], F_2d[col]),
        "corr_umap2": _corr(U_2d["umap2"], F_2d[col]),
    })
corr2d_df = pd.DataFrame(corr_rows)
print("\n--- Feature Correlation with 2D UMAP Dimensions ---")
display(style_section_table(corr2d_df.round(3)))
```

```
UMAP(min_dist=0.2, n_jobs=1, n_neighbors=75, random_state=42, verbose=True)
Wed Sep 17 20:10:30 2025 Construct fuzzy simplicial set
Wed Sep 17 20:10:30 2025 Finding Nearest Neighbors
Wed Sep 17 20:10:30 2025 Building RP forest with 10 trees
Wed Sep 17 20:10:34 2025 NN descent for 14 iterations
    1 / 14
    2 / 14
Stopping threshold met -- exiting after 2 iterations
Wed Sep 17 20:10:40 2025 Finished Nearest Neighbor Search
Wed Sep 17 20:10:41 2025 Construct embedding
Epochs completed: 0%| 0/200 [00:00]
completed 0 / 200 epochs
completed 20 / 200 epochs
completed 40 / 200 epochs
completed 60 / 200 epochs
completed 80 / 200 epochs
completed 100 / 200 epochs
completed 120 / 200 epochs
completed 140 / 200 epochs
completed 160 / 200 epochs
completed 180 / 200 epochs
Wed Sep 17 20:11:05 2025 Finished embedding
```

2D UMAP of Trajectory Features (Colored by Mean Speed)

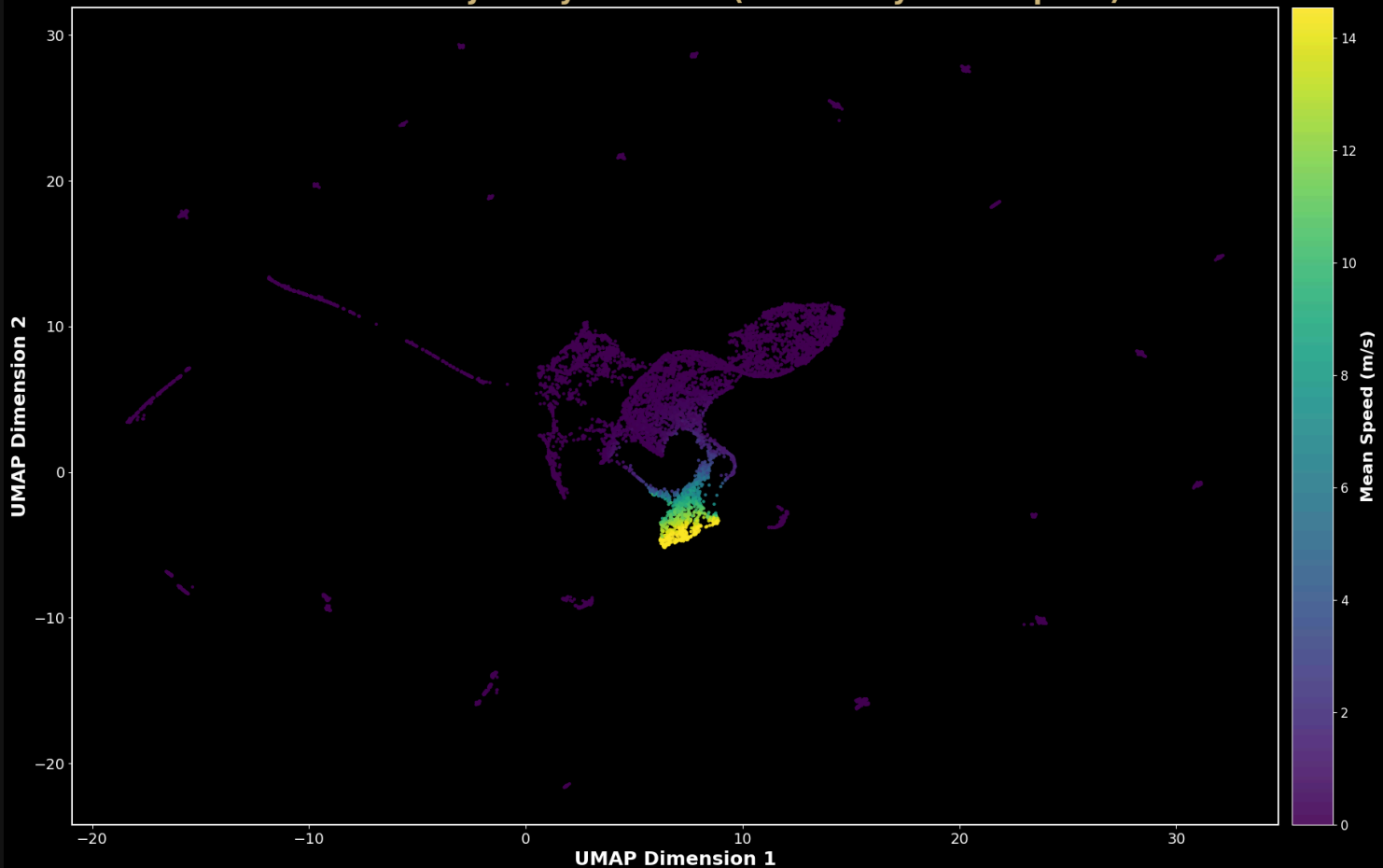


Figure saved to (Linux): /home/treinartr/projects/lyft-unsupervised/artifacts/figures/20250917_201105_eda_umap_2d_behavior_map.png
Figure saved to (Win): /mnt/c/Users/travi/Documents/Training/Colorado/MS-AI/CSCA 5632 Final Project/artifacts/figures/20250917_201105_eda_umap_2d_behavior_map.png

--- Feature Correlation with 2D UMAP Dimensions ---

feature	corr_umap1	corr_umap2
mean_speed	0.056000	-0.252000
p95_speed	0.063000	-0.260000
std_speed	0.063000	-0.243000
heading_change_rate	0.072000	-0.042000
stop_ratio	-0.082000	0.263000
duration_s	-0.220000	-0.228000

4.8 3D UMAP Behavior Map

As an alternative visualization, the same high-dimensional feature vectors are projected into a 3D space. This can sometimes reveal additional structure or separation between behavioral patterns that may be less obvious in the 2D projection. The same color mapping by mean speed is used to aid interpretation.

```
In [21]: # Run 3D UMAP
reducer_3d = umap.UMAP(
    n_neighbors=75,
    min_dist=0.2,
    metric="euclidean",
    random_state=SEED,
    n_components=3,
    verbose=True
)
Z_3d = reducer_3d.fit_transform(X_scaled)

# Plot the 3D Behavior Map
mean_speed_colors = feat_df_nna["mean_speed"].to_numpy()
vmin, vmax = np.nanpercentile(mean_speed_colors, 2), np.nanpercentile(mean_speed_colors, 98)
fig = plt.figure(figsize=(24, 20), facecolor="black")
ax = fig.add_subplot(111, projection="3d")
ax.set_facecolor("black")
sc = ax.scatter(
    Z_3d[:, 0], Z_3d[:, 1], Z_3d[:, 2],
    c=mean_speed_colors, s=10, cmap="viridis", vmin=vmin, vmax=vmax,
    alpha=0.9, edgecolors="none"
)

# Style the 3D Plot
ax.set_title("3D UMAP of Trajectory Features (Colored by Mean Speed)", fontsize=26, color="#CFB87C", fontweight="bold", pad=8)
ax.set_xlabel("UMAP Dimension 1", fontsize=16, color="white", fontweight="bold", labelpad=10)
ax.set_ylabel("UMAP Dimension 2", fontsize=16, color="white", fontweight="bold", labelpad=10)
ax.set_zlabel("UMAP Dimension 3", fontsize=16, color="white", fontweight="bold", labelpad=10)
ax.xaxis.set_pane_color((0, 0, 0, 1))
ax.yaxis.set_pane_color((0, 0, 0, 1))
ax.zaxis.set_pane_color((0, 0, 0, 1))
ax.tick_params(axis='x', colors='white', labels=12)
ax.tick_params(axis='y', colors='white', labels=12)
ax.tick_params(axis='z', colors='white', labels=12)
cb = fig.colorbar(sc, ax=ax, pad=0.02, shrink=0.7)
cb.set_label("Mean Speed (m/s)", color="white", fontsize=16, fontweight="bold")
cb.ax.tick_params(colors="white", labels=12)
cb.outline.set_edgecolor("white")
save_fig(fig, "eda_umap_3d_behavior_map")

# Analyze Feature Correlations with 3D UMAP Dimensions
U_3d = pd.DataFrame(Z_3d, columns=["umap1", "umap2", "umap3"])
F_3d = feat_df_nna[feature_cols]
corr_rows_3d = []
for col in F_3d.columns:
    corr_rows_3d.append({
        "feature": col,
        "corr_umap1": _corr(U_3d["umap1"], F_3d[col]),
        "corr_umap2": _corr(U_3d["umap2"], F_3d[col]),
        "corr_umap3": _corr(U_3d["umap3"], F_3d[col]),
    })
corr3d_df = pd.DataFrame(corr_rows_3d)
print("\n--- Feature Correlation with 3D UMAP Dimensions ---")
display(style_section_table(corr3d_df.round(3)))
```

```
UMAP(min_dist=0.2, n_components=3, n_jobs=1, n_neighbors=75, random_state=42, verbose=True)
Wed Sep 17 20:15:30 2025 Construct fuzzy simplicial set
Wed Sep 17 20:15:30 2025 Finding Nearest Neighbors
Wed Sep 17 20:15:30 2025 Building RP forest with 10 trees
Wed Sep 17 20:15:30 2025 NN descent for 14 iterations
    1 / 14
    2 / 14
Stopping threshold met -- exiting after 2 iterations
Wed Sep 17 20:15:32 2025 Finished Nearest Neighbor Search
Wed Sep 17 20:15:33 2025 Construct embedding
Epochs completed:  0%|          0/200 [00:00]
```

completed 0 / 200 epochs
completed 20 / 200 epochs
completed 40 / 200 epochs
completed 60 / 200 epochs
completed 80 / 200 epochs
completed 100 / 200 epochs
completed 120 / 200 epochs
completed 140 / 200 epochs
completed 160 / 200 epochs
completed 180 / 200 epochs

Wed Sep 17 20:15:56 2025 Finished embedding

3D UMAP of Trajectory Features (Colored by Mean Speed)

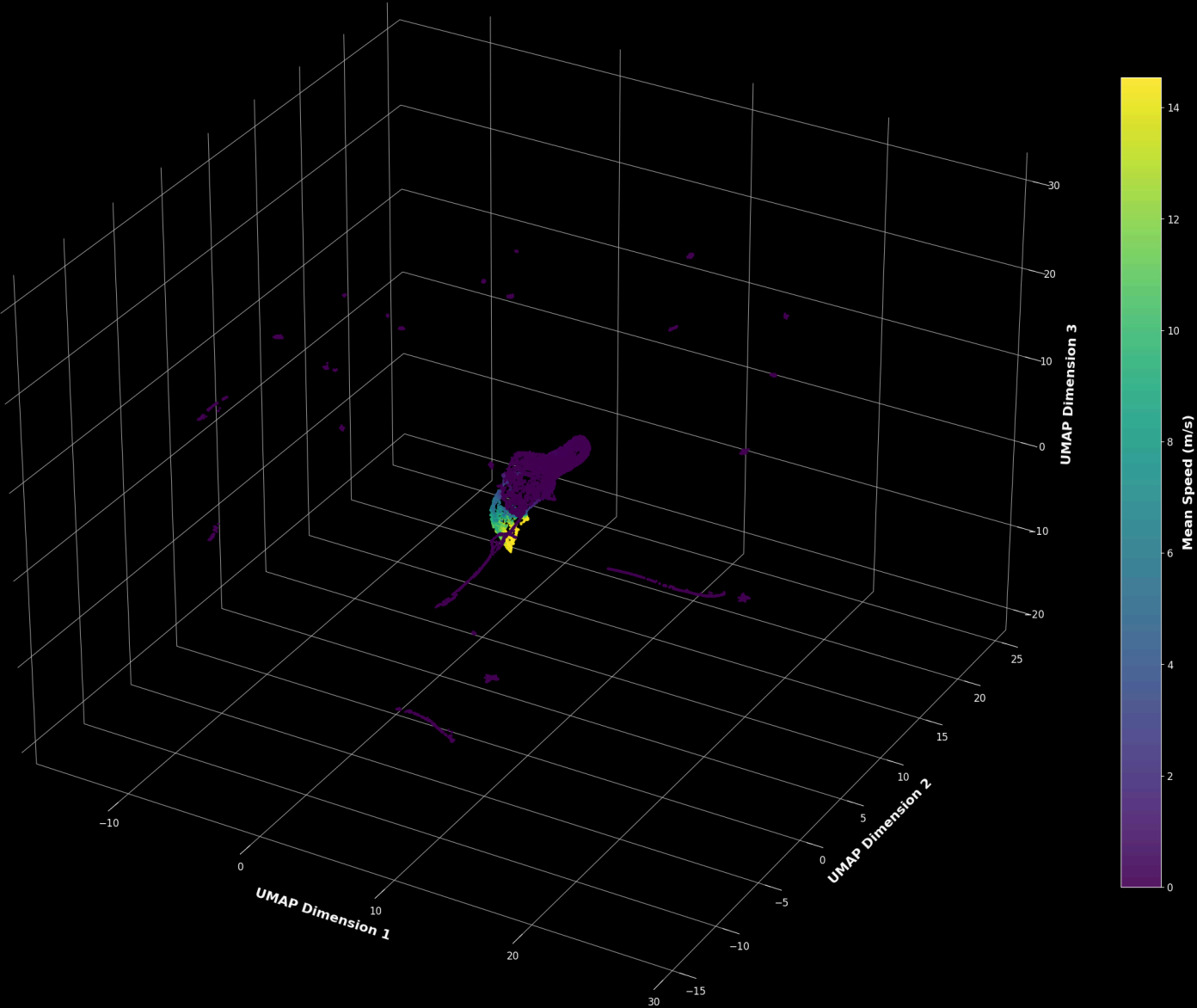


Figure saved to (Linux): /home/treinart/projects/lyft-unsupervised/artifacts/figures/20250917_201556_eda_umap_3d_behavior_map.png
Figure saved to (Win): /mnt/c/Users/travi/Documents/Training/Colorado/MS-AI/CSCA 5632 Final Project/artifacts/figures/20250917_201556_eda_umap_3d_behavior_map.png

--- Feature Correlation with 3D UMAP Dimensions ---

feature	corr_umap1	corr_umap2	corr_umap3
mean_speed	-0.069000	0.026000	-0.203000
p95_speed	-0.067000	0.036000	-0.204000
std_speed	-0.055000	0.040000	-0.186000
heading_change_rate	0.078000	0.194000	0.021000
stop_ratio	0.090000	-0.058000	0.198000
duration_s	-0.025000	-0.087000	-0.372000

4.9 EDA Quantitative Summary

To conclude the exploratory analysis, this section compiles the key descriptive statistics from the previous visualizations into a single summary table. This provides a clear, quantitative reference for the distributions of speed, acceleration, and track/scene lengths, which can be used for sanity checks in later stages of the project.

```
In [22]: # Define Helper Function
def q(a, p):
    a = a[np.isfinite(a)]
    return float(np.percentile(a, p)) if a.size else np.nan

# Calculate Summary Statistics
summary_df = pd.DataFrame({
    "measure": [
        "speed_mean", "speed_median", "speed_p95",
        "accel_mean", "accel_median", "accel_p95",
        "frames_per_scene_median", "frames_per_scene_p95",
        "frames_per_track_median", "frames_per_track_p95",
    ],
    "estimate": [
        float(np.nanmean(speed_data)),
        float(np.nanmedian(speed_data)),
        q(speed_data, 95),
        float(np.nanmean(accel_data)),
        float(np.nanmedian(accel_data)),
        q(accel_data, 95),
        float(np.median(frames_per_scene_data)),
        q(frames_per_scene_data, 95),
        float(np.median(frames_per_track_data)),
        q(frames_per_track_data, 95)
    ]
})

# Display and Cache the Summary Table
display(style_section_table(summary_df).format({"estimate": "{:.3f}"}))
print("-" * 70)
print("\n--- Caching EDA summary table ---\n")
saved_paths = _save_df(summary_df, "eda_summary_section4")
for path_type, path_loc in saved_paths.items():
    print(f"Saved to ({path_type.replace('_', ' ')}): {path_loc}")
print("\n" + "-" * 70)
```

measure	estimate
speed_mean	1.167
speed_median	0.000
speed_p95	11.027
accel_mean	0.916
accel_median	0.000
accel_p95	2.891
frames_per_scene_median	248.000
frames_per_scene_p95	249.000
frames_per_track_median	6.000
frames_per_track_p95	63.000

--- Caching EDA summary table ---

Saved to (parquet linux): /home/treinart/projects/lyft-unsupervised/artifacts/cache/20250917_201731_eda_summary_section4.parquet
Saved to (parquet windows): /mnt/c/Users/travi/Documents/Training/Colorado/MS-AI/CSCA 5632 Final Project/artifacts/cache/20250917_201731_eda_summary_section 4.parquet
Saved to (csv linux): /home/treinart/projects/lyft-unsupervised/artifacts/cache/20250917_201731_eda_summary_section4.csv
Saved to (csv windows): /mnt/c/Users/travi/Documents/Training/Colorado/MS-AI/CSCA 5632 Final Project/artifacts/cache/20250917_201731_eda_summary_section4.csv

Observation: Key Findings from Exploratory Data Analysis

The exploratory data analysis (EDA) successfully illuminated the core characteristics of the agent trajectory data. The findings reveal a dataset that is structurally consistent, spatially organized, and contains a diverse but separable set of behaviors. This analysis culminates in the creation of UMAP "behavior maps," which visually confirm that distinct patterns of motion are identifiable and provide a strong foundation for the formal clustering tasks.

- **Structural Consistency:**
 - The analysis of scene and track lengths revealed two key properties. First, the scenes are remarkably consistent in duration, with almost all lasting **~25 seconds (248-249 frames)**. Second, while most individual agent tracks are short (a median of **6 frames**), a significant long tail exists, with 5% of tracks lasting longer than 63 frames.
 - The **Spatial Density Heatmap** clearly reveals a structured, non-random road network. The **Yaw Distribution** corroborates this by showing agent travel is heavily concentrated along specific cardinal directions, which is characteristic of an urban grid.

- **Dominance of Stationary Behavior (Key Finding):**
 - A primary characteristic of the dataset is the overwhelming prevalence of stationary behavior. The **Speed and Acceleration distributions** both report a median value of **0.00 m/s**. This is a significant finding, confirming that more than half of the nearly one million agent-moments captured in the subset are spent at a complete standstill, likely due to traffic signals and congestion.
- **Successful Feature Engineering:**
 - The feature engineering process methodically distilled the complex, variable-length trajectories of **11,956** eligible agents (those with at least 20 consecutive frames) into a fixed-size, interpretable feature vector for each.
 - The **Feature Correlation Matrix** validates these features, showing logical relationships while also confirming that metrics like `heading_change_rate` capture relatively independent aspects of motion.
- **Clear Behavioral Separation in UMAP:**
 - The UMAP projections are the most critical finding of this EDA. They successfully transform the engineered features into low-dimensional "behavior maps" where clear, non-random structures emerge.
 - The maps show a large, dense core corresponding to the low-speed and stationary agents. Extending from this core are distinct arms and peninsulas representing more dynamic behaviors. This visual separation is powerful evidence that different behavioral archetypes exist and are separable.

Conclusion:

The exploratory analysis has achieved its goal. It has progressed from raw data distributions to a low-dimensional embedding where distinct patterns of agent behavior are visually apparent. The clear separation of behaviors in the UMAP maps provides strong evidence that formal clustering algorithms can successfully partition this data. The project can now proceed with confidence to the modeling phase.

Section 5: Pipeline A - Classical Feature Engineering

This section details the first of two parallel pipelines for discovering behavioral patterns. "Pipeline A" uses a classical, interpretable approach. A comprehensive set of features is engineered from the raw trajectory data to numerically summarize each agent's behavior. These features, covering kinematics, path geometry, and stopping behavior, are designed to be transparent and human-understandable. The final output is a standardized feature matrix ready for clustering.

Section Plan:

- **Checkpoint & Loader:** Option to skip computation and load the latest cached artifacts.
- **Feature Design:** Define the catalog of features to be engineered and verify the input data is ready.
- **Build Feature Table:** Create a single, powerful function to compute the feature vectors for all eligible tracks.
- **Preprocess Features:** Mitigate outliers by capping extreme values and standardize the data for clustering.
- **Sanity Check:** Review a quantitative summary of the final feature table.
- **Sensitivity Analysis:** Visualize how the choice of the `MIN_CONSEC` hyperparameter impacts the feature distributions to justify the final selection.
- **Final Feature Distributions:** Plot a panel of histograms to visualize the final, capped feature distributions.

5.1 Checkpoint: Run Full Section or Load from Cache

This cell acts as a checkpoint. The feature engineering process can be time-consuming. Set the flag below to control the behavior of this section.

- **True** : Run the full feature engineering and preprocessing pipeline. This is required on the first run.
- **False** : Skip the long computations and load the most recent cached artifacts from a previous run.

```
In [23]: # Set to True to run the time-consuming feature engineering process.
# Set to False to skip computation and load the latest cached results from a previous run.
RUN_FEATURE_ENGINEERING = True

if not RUN_FEATURE_ENGINEERING:
    print("--- Skipping feature engineering. Loading latest artifacts from cache... ---")
    try:
        # Load the capped feature DataFrame
        latest_df_file = max(CACHE_DIR.glob("_feat_df_capped_pipeline_a.parquet"), key=os.path.getctime)
        feat_df_capped = pd.read_parquet(latest_df_file)
        print(f"Loaded DataFrame from: {latest_df_file.name}")

        # Load the scaled data matrix
        latest_npy_file = max(CACHE_DIR.glob("_X_scaled_pipeline_a.npy"), key=os.path.getctime)
        X_scaled = np.load(latest_npy_file)
        print(f"Loaded scaled matrix from: {latest_npy_file.name}")

        # Load the scaler parameters
        latest_scaler_file = max(CACHE_DIR.glob("_scaler_pipeline_a.json"), key=os.path.getctime)
        with open(latest_scaler_file, 'r') as f:
            scaler_params = json.load(f)
        print(f"Loaded scaler params from: {latest_scaler_file.name}")

        print(f"\n--- Successfully loaded {len(feat_df_capped):,} tracks. Ready to proceed to Section 6 ---")
    except ValueError:
        print("\n--- ERROR: No cached artifacts found. Please set RUN_FEATURE_ENGINEERING to True and run the full section ---")
        RUN_FEATURE_ENGINEERING = True
```

```
else:
    print("--- RUN_FEATURE_ENGINEERING is True. Proceeding with full computation. ---")
```

```
--- RUN_FEATURE_ENGINEERING is True. Proceeding with full computation. ---
```

5.2 Feature Design & Readiness Check

This initial step defines the feature catalog: the full set of interpretable metrics that will be calculated for each trajectory. It also performs a readiness check to verify that all necessary data from Section 3 is present and to quantify how many agent tracks are eligible for feature engineering based on the `MIN_CONSEC` parameter.

```
In [24]: # This cell will only run if the full computation is enabled.
if RUN_FEATURE_ENGINEERING:
    # Verify Input Data
    required_vars = ['agents_df_clean', 'frames_df_clean', 'MIN_CONSEC', 'DT']
    missing_vars = {var for var in required_vars if var not in globals()}
    if missing_vars:
        raise NameError(f"Missing required data from previous sections: {' '.join(missing_vars)}")

    # Report on Eligibility
    total_tracks = agents_df_clean.groupby(["scene_id", "track_id"]).ngroups
    eligible_tracks_count = (agents_df_clean.groupby(["scene_id", "track_id"])["frame_idx"]
                             .nunique() >= MIN_CONSEC).sum()

    coverage_rate = eligible_tracks_count / total_tracks if total_tracks > 0 else 0.0

    print("-" * 70)
    print("Feature Engineering Readiness Check")
    print("-" * 70)
    print(f"Key Parameters: MIN_CONSEC={MIN_CONSEC} frames | DT={DT} seconds")
    print(f"Track Eligibility: {eligible_tracks_count:,} of {total_tracks:,} tracks are eligible ({coverage_rate:.1%})")

    # Define the Feature Catalog
    feature_catalog = pd.DataFrame([
        {"feature": "n_frames", "purpose": "coverage", "definition": "Count of unique frames in the track."},
        {"feature": "duration_s", "purpose": "coverage", "definition": "Total time duration of the track in seconds."},
        {"feature": "mean_speed", "purpose": "pace", "definition": "Mean of the agent's speed across all frames."},
        {"feature": "p95_speed", "purpose": "pace", "definition": "95th percentile of speed, indicating peak pace."},
        {"feature": "max_speed", "purpose": "pace", "definition": "Absolute maximum speed achieved."},
        {"feature": "median_accel", "purpose": "smoothness", "definition": "Median acceleration, indicating typical change in speed."},
        {"feature": "p95_accel", "purpose": "smoothness", "definition": "95th percentile of acceleration."},
        {"feature": "max_accel", "purpose": "smoothness", "definition": "Absolute maximum acceleration."},
        {"feature": "p95_jerk", "purpose": "comfort", "definition": "95th percentile of jerk (change in acceleration)."},
        {"feature": "heading_change_rate", "purpose": "turning", "definition": "Mean rate of change in heading (angular velocity)."},
        {"feature": "turn_magnitude", "purpose": "turning", "definition": "Sum of absolute changes in heading over the track."},
        {"feature": "stop_ratio", "purpose": "idling", "definition": "Fraction of frames where speed is below 0.5 m/s."},
        {"feature": "stationary_bursts", "purpose": "idling", "definition": "Count of transitions from moving to stopped."},
        {"feature": "straightness", "purpose": "path shape", "definition": "Ratio of displacement to total path length (1.0 is a straight line)."},
        {"feature": "path_length", "purpose": "distance", "definition": "Total distance traveled along the trajectory path."},
        {"feature": "displacement", "purpose": "distance", "definition": "Net distance between the start and end points."},
    ])

    print("\n--- Feature Catalog ---")
    display(style_section_table(feature_catalog))
else:
    print("--- Skipped: Loading from cache ---")
```

```
-----
Feature Engineering Readiness Check
-----
```

```
Key Parameters: MIN_CONSEC=20 frames | DT=0.1 seconds
Track Eligibility: 11,956 of 64,625 tracks are eligible (18.5%)
```

```
--- Feature Catalog ---
```


feature	purpose	definition
n_frames	coverage	Count of unique frames in the track.
duration_s	coverage	Total time duration of the track in seconds.
mean_speed	pace	Mean of the agent's speed across all frames.
p95_speed	pace	95th percentile of speed, indicating peak pace.
max_speed	pace	Absolute maximum speed achieved.
median_accel	smoothness	Median acceleration, indicating typical change in speed.
p95_accel	smoothness	95th percentile of acceleration.
max_accel	smoothness	Absolute maximum acceleration.
p95_jerk	comfort	95th percentile of jerk (change in acceleration).
heading_change_rate	turning	Mean rate of change in heading (angular velocity).
turn_magnitude	turning	Sum of absolute changes in heading over the track.
stop_ratio	idling	Fraction of frames where speed is below 0.5 m/s.
stationary_bursts	idling	Count of transitions from moving to stopped.
straightness	path shape	Ratio of displacement to total path length (1.0 is a straight line).
path_length	distance	Total distance traveled along the trajectory path.
displacement	distance	Net distance between the start and end points.

5.3 Build Feature Table

This cell defines and runs the primary feature engineering function, `build_feature_table`. This single, unified function iterates through every agent track in the dataset, calculates the full suite of features defined in the catalog, and returns a complete DataFrame. Consolidating the logic here allows it to be reused for the sensitivity analysis later.

```
In [25]: # This cell will only run if the full computation is enabled.
if RUN_FEATURE_ENGINEERING:
    # Define the primary feature engineering function
    def build_feature_table(agents_df: pd.DataFrame, min_consec: int, dt: float) -> pd.DataFrame:
        grouped_tracks = (agents_df
                            .sort_values(["scene_id", "track_id", "frame_idx"])
                            .groupby(["scene_id", "track_id"], sort=False))

        track_features = []
        for (sid, tid), group in tqdm(grouped_tracks, total=grouped_tracks.ngroups, desc=f"Building Features (MIN_CONSEC={min_consec})", unit="track"):
            n_frames = group["frame_idx"].nunique()
            if n_frames < min_consec:
                continue

            # Extract base arrays
            x, y = group["x"].to_numpy(), group["y"].to_numpy()
            vx, vy = group["vx"].to_numpy(), group["vy"].to_numpy()
            fi = group["frame_idx"].to_numpy()

            # Calculate Kinematics
            s = np.sqrt(vx**2 + vy**2)
            a = np.sqrt(np.diff(vx)**2 + np.diff(vy)**2) / dt if len(vx) >= 2 else np.array([])
            j = np.abs(np.diff(a)) / dt if len(a) >= 2 else np.array([])

            # Calculate Heading and Turning
            h = np.unwrap(np.arctan2(vy, vx))
            h_rate = np.nanmean(np.abs(np.diff(h)) / dt) if len(h) >= 2 else 0.0
            turn_mag = np.nansum(np.abs(np.diff(h))) if len(h) >= 2 else 0.0

            # Calculate Stop Metrics
            stop_mask = s < 0.5
            stop_ratio = stop_mask.mean()
            bursts = np.sum((~stop_mask[:-1] & stop_mask[1:]) if len(stop_mask) >= 2 else 0

            # Calculate Path Geometry
            path_len = np.sum(np.sqrt(np.diff(x)**2 + np.diff(y)**2)) if len(x) >= 2 else 0.0
            disp = np.sqrt((x[-1] - x[0])**2 + (y[-1] - y[0])**2) if len(x) >= 1 else 0.0
            straight = disp / path_len if path_len > 1e-3 else 1.0

            # Append all features
            track_features.append({
                "scene_id": sid, "track_id": tid, "n_frames": n_frames,
                "duration_s": (fi.max() - fi.min() + 1) * dt,
                "mean_speed": np.nanmean(s), "median_speed": np.nanmedian(s), "std_speed": np.nanstd(s),
                "p95_speed": np.percentile(s, 95), "max_speed": s.max(),
                "median_accel": np.nanmedian(a), "p95_accel": np.percentile(a, 95) if a.size > 0 else 0.0,
                "max_accel": a.max() if a.size > 0 else 0.0,
                "p95_jerk": np.percentile(j, 95) if j.size > 0 else 0.0,
                "heading_change_rate": h_rate, "turn_magnitude": turn_mag,
                "stop_ratio": stop_ratio, "stationary_bursts": bursts,
```

```

        "straightness": straight, "path_length": path_len, "displacement": disp,
    })
    return pd.DataFrame(track_features)

# Run the feature engineering process
MIN_CONSEC_FINAL = 30 # Value justified by sensitivity analysis in a Later step
print("--- Building Main Feature Table ---")
feat_df = build_feature_table(agents_df_clean, min_consec=MIN_CONSEC_FINAL, dt=DT)
print("\n" + "-" * 70)
print(f"Feature engineering complete. Total tracks kept: {len(feat_df):}")
print("-" * 70)

# Create and display a formatted preview
pretty_feat = feat_df.head().copy()
def format_or_dash(x, precision=3):
    if pd.isna(x) or (isinstance(x, (int, float)) and abs(x) < 1e-9): return "---"
    if isinstance(x, float): return f"{x:.{precision}f}"
    return x
cols_to_format = [c for c in pretty_feat.columns if c not in ['scene_id', 'track_id', 'n_frames', 'stationary_bursts']]
for col in cols_to_format:
    pretty_feat[col] = pretty_feat[col].apply(lambda v: format_or_dash(v))
display(style_section_table(pretty_feat))
else:
    print("--- Skipped: Loading from cache ---")

```

```

--- Building Main Feature Table ---
Building Features (MIN_CONSEC=30):    0% |          | 0/64625 [00:00<?, ?track/s]
-----

```

Feature engineering complete. Total tracks kept: 7,824

scene_id	track_id	n_frames	duration_s	mean_speed	median_speed	std_speed	p95_speed	max_speed	median_accel	p95_accel	max_accel	p95_jerk	heading_change_rat
108	1	67	6.700	0.020	---	0.105	---	0.741	---	1.180	8.967	14.833	1.33
108	2	248	24.800	0.002	---	0.021	---	0.290	---	---	2.905	---	0.19
108	3	56	5.600	0.014	---	0.075	---	0.471	---	0.976	5.688	14.898	0.68
108	4	87	8.700	0.012	---	0.083	---	0.653	---	---	6.526	---	0.10
108	5	248	24.800	2.460	0.050	2.962	7.422	7.618	0.191	2.005	62.829	5.809	3.64

5.4 Preprocess Features: Cap and Standardize

Before using the features for clustering, two standard preprocessing steps are applied. First, each feature is "capped" at its 99th percentile to mitigate the influence of extreme outliers without removing data. Second, the features are standardized (scaled to have zero mean and unit variance) to ensure that all features contribute equally to the distance calculations in the clustering algorithms.

```

In [26]: # This cell will only run if the full computation is enabled.
if RUN_FEATURE_ENGINEERING:
    # Cap Features at the 99th Percentile
    id_cols = ["scene_id", "track_id"]
    numeric_cols = [c for c in feat_df.columns if c not in id_cols]
    caps_p99 = feat_df[numeric_cols].quantile(0.99).to_dict()
    feat_df_capped = feat_df.copy()
    for col, cap_val in caps_p99.items():
        feat_df_capped[col] = feat_df_capped[col].clip(upper=cap_val)

    # Standardize Features
    X_numeric = feat_df_capped[numeric_cols].astype(float).fillna(0)
    scaler = StandardScaler()
    X_scaled = scaler.fit_transform(X_numeric)
    scaler_params = {"feature_names": numeric_cols, "mean": scaler.mean_.tolist(), "scale": scaler.scale_.tolist(), "caps_p99": caps_p99}

    # Save Timestamped Artifacts for Caching
    print("-" * 70)
    print("\n--- Caching timestamped artifacts for this run ---")
    _save_df(feat_df_capped, "feat_df_capped_pipeline_a")
    save_numpy(X_scaled, "X_scaled_pipeline_a")
    ts = time.strftime("%Y%m%d-%H%M%S")
    scaler_path_linux = CACHE_DIR / f"{ts}_scaler_pipeline_a.json"
    with open(scaler_path_linux, "w") as f: json.dump(scaler_params, f, indent=2)
    try: shutil.copy2(scaler_path_linux, WINDOWS_MIRROR / "cache")
    except Exception: pass
    print(f"Timestamped scaler params saved to: {scaler_path_linux}")

    # Save Final Artifacts with Stable Names for Section 11
    print("\n--- Saving final artifacts with stable names for later use ---")
    feat_df_capped.to_parquet(ARTIFACT_DIR / "pipeline_a_features.parquet", index=False)
    np.save(ARTIFACT_DIR / "pipeline_a_scaled_matrix.npy", X_scaled)
    with open(ARTIFACT_DIR / "pipeline_a_scaler.json", "w") as f: json.dump(scaler_params, f, indent=2)

    print(f"Final features saved to: {ARTIFACT_DIR / 'pipeline_a_features.parquet'}")
    print(f"Final matrix saved to: {ARTIFACT_DIR / 'pipeline_a_scaled_matrix.npy'}")
    print(f"Final scaler saved to: {ARTIFACT_DIR / 'pipeline_a_scaler.json'}")
    print("-" * 70)

```

```
else:
    print("--- Skipped: Loading from cache ---")
```

```
--- Caching timestamped artifacts for this run ---
Numpy array saved to (Linux): /home/treinart/projects/lyft-unsupervised/artifacts/cache/20250917_201801_X_scaled_pipeline_a.npy
Numpy array saved to (Win): /mnt/c/Users/travi/Documents/Training/Colorado/MS-AI/CSCA 5632 Final Project/artifacts/cache/20250917_201801_X_scaled_pipeline_a.npy
Timestamped scaler params saved to: /home/treinart/projects/lyft-unsupervised/artifacts/cache/20250917_201801_scaler_pipeline_a.json

--- Saving final artifacts with stable names for later use ---
Final features saved to: /home/treinart/projects/lyft-unsupervised/artifacts/pipeline_a_features.parquet
Final matrix saved to: /home/treinart/projects/lyft-unsupervised/artifacts/pipeline_a_scaled_matrix.npy
Final scaler saved to: /home/treinart/projects/lyft-unsupervised/artifacts/pipeline_a_scaler.json
```

5.5 Sanity Check Summary

A final quantitative summary of the preprocessed feature table. This table provides a quick sanity check of the typical values and ranges for a few key features before they are used for model training.

```
In [27]: # This cell will only run if the full computation is enabled.
if RUN_FEATURE_ENGINEERING:
    # Create a summary of the final, capped feature table
    summary_data = {
        "measure": [
            "tracks_kept", "mean_speed_median", "mean_speed_p95", "straightness_median",
            "duration_s_median", "heading_change_rate_median", "share_highly_stationary (stop_ratio >= 0.7)"
        ],
        "estimate": [
            len(feat_df_capped), feat_df_capped["mean_speed"].median(),
            feat_df_capped["mean_speed"].quantile(0.95), feat_df_capped["straightness"].median(),
            feat_df_capped["duration_s"].median(), feat_df_capped["heading_change_rate"].median(),
            (feat_df_capped["stop_ratio"] >= 0.7).mean()
        ]
    }
    summary_table = pd.DataFrame(summary_data)

    # Display the styled summary table
    display(style_section_table(summary_table))
else:
    print("--- Skipped: Loading from cache ---")
```

measure	estimate
tracks_kept	7824.000000
mean_speed_median	0.006865
mean_speed_p95	11.074172
straightness_median	0.149469
duration_s_median	5.400000
heading_change_rate_median	0.318419
share_highly_stationary (stop_ratio >= 0.7)	0.849054

5.6 Sensitivity Analysis: Selecting MIN_CONSEC

The `MIN_CONSEC` parameter, which sets the minimum number of frames for a track to be included, is a key hyperparameter. A low value includes more tracks but may add noise from short, erratic trajectories. A high value yields higher-quality tracks but reduces the dataset size. This analysis visualizes this trade-off to justify the choice of `MIN_CONSEC = 30` for the final model.

```
In [28]: # This cell will only run if the full computation is enabled.
if RUN_FEATURE_ENGINEERING:
    min_consec_values = range(10, 51, 5)
    sensitivity_results = []

    # Loop through the values, build features, and summarize
    for mc_value in min_consec_values:
        temp_feat_df = build_feature_table(agents_df_clean, min_consec=mc_value, dt=DT)
        if not temp_feat_df.empty:
            sensitivity_results.append({
                "min_consec": mc_value,
                "tracks_kept": len(temp_feat_df),
                "coverage_rate": len(temp_feat_df) / total_tracks,
                "mean_speed_median": temp_feat_df["mean_speed"].median(),
                "mean_speed_p95": temp_feat_df["mean_speed"].quantile(0.95),
                "straightness_median": temp_feat_df["straightness"].median(),
                "duration_s_median": temp_feat_df["duration_s"].median(),
            })
```

```

})

sensitivity_df = pd.DataFrame(sensitivity_results)

# Visualize the Results
metrics_to_plot = [
    "coverage_rate", "mean_speed_p95",
    "straightness_median", "duration_s_median"
]

# Plot each metric on its own subplot
fig, axes = plt.subplots(2, 2, figsize=(20, 12), facecolor="black")
fig.patch.set_facecolor("black")
fig.suptitle("Sensitivity to MIN_CONSEC Parameter", fontsize=28, color="#CFB87C", fontweight="bold")
axes = axes.flatten()

# Plot each metric on its own subplot
for ax, metric in zip(axes, metrics_to_plot):
    ax.plot(sensitivity_df["min_consec"], sensitivity_df[metric], marker='o', linestyle='--', color='dodgerblue')

    # Highlight MIN_CONSEC=30
    ax.axvline(30, color="#DA291C", linestyle='--', linewidth=2, label="Selected Value (30)")

    # Style the subplot
    ax.set_title(metric.replace("_", " ").title(), fontsize=18, color="#F8F8F8", pad=10)
    ax.set_xlabel("MIN_CONSEC Value", fontsize=14, color="white")
    ax.set_ylabel("Metric Value", fontsize=14, color="white")
    ax.set_facecolor("black")
    ax.tick_params(colors="white", labelsize=12)
    for spine in ax.spines.values():
        spine.set_edgecolor("white")
    ax.grid(color='gray', linestyle='--', linewidth=0.3, alpha=0.5)
    if metric == "coverage_rate": # Format y-axis as percentage for coverage
        ax.yaxis.set_major_formatter(plt.FuncFormatter('{:.0%}'.format))

save_fig(fig, "sensitivity_analysis_min_consec")
else:
    print("--- Skipped: Loading from cache ---")

```

```

Building Features (MIN_CONSEC=10): 0%|          | 0/64625 [00:00<?, ?track/s]
Building Features (MIN_CONSEC=15): 0%|          | 0/64625 [00:00<?, ?track/s]
Building Features (MIN_CONSEC=20): 0%|          | 0/64625 [00:00<?, ?track/s]
Building Features (MIN_CONSEC=25): 0%|          | 0/64625 [00:00<?, ?track/s]
Building Features (MIN_CONSEC=30): 0%|          | 0/64625 [00:00<?, ?track/s]
Building Features (MIN_CONSEC=35): 0%|          | 0/64625 [00:00<?, ?track/s]
Building Features (MIN_CONSEC=40): 0%|          | 0/64625 [00:00<?, ?track/s]
Building Features (MIN_CONSEC=45): 0%|          | 0/64625 [00:00<?, ?track/s]
Building Features (MIN_CONSEC=50): 0%|          | 0/64625 [00:00<?, ?track/s]

```

Sensitivity to MIN_CONSEC Parameter

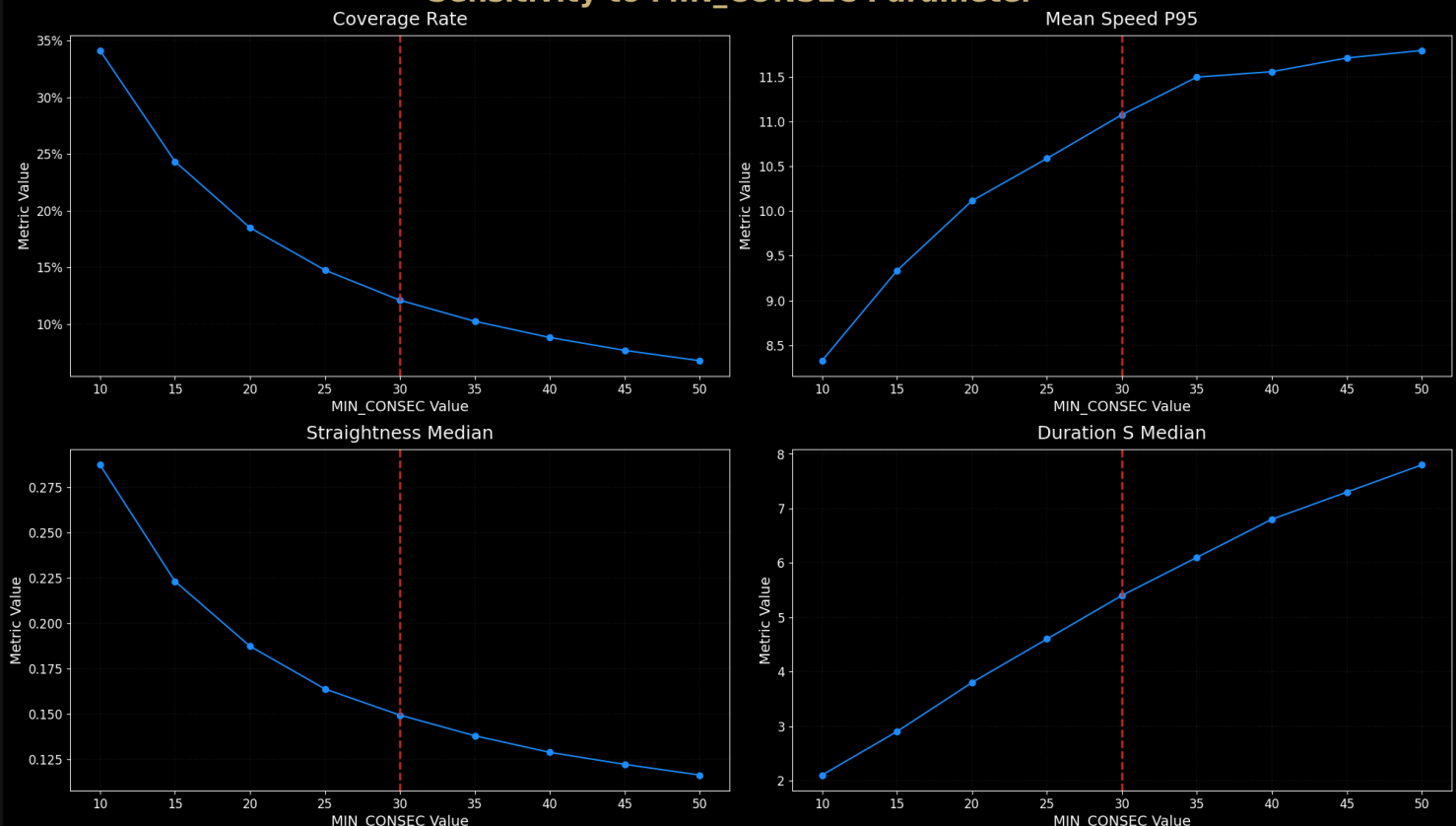


Figure saved to (Linux): /home/treinart/projects/lyft-unsupervised/artifacts/figures/20250917_201903_sensitivity_analysis_min_consec.png

Figure saved to (Win): /mnt/c/Users/travi/Documents/Training/Colorado/MS-AI/CSCA 5632 Final Project/artifacts/figures/20250917_201903_sensitivity_analysis_min_consec.png

As a final check on the engineered features, this section plots the distribution of each metric after capping outliers. This visual "feature panel" provides a comprehensive overview of the shape, scale, and range of the data that will be fed into the clustering models. It's a crucial step to confirm that the preprocessing was effective and to understand the characteristics of the final feature space.

```
In [37]: # This cell will only run if the full computation is enabled.
if RUN_FEATURE_ENGINEERING:
    # Select all numeric feature columns
    numeric_cols = [c for c in feat_df_capped.columns if c not in ["scene_id", "track_id"]]

    # Determine the grid layout (3 columns) and create subplots
    n_features = len(numeric_cols)
    n_cols = 3
    n_rows = (n_features + n_cols - 1) // n_cols
    fig, axes = plt.subplots(n_rows, n_cols, figsize=(20, n_rows * 4), facecolor="black")
    fig.patch.set_facecolor("black")
    fig.suptitle("Distributions of Final Engineered Features", fontsize=28, color="#CFB87C", fontweight="bold", y=0.995)
    axes = axes.flatten()
    colors = ['dodgerblue', 'limegreen', 'darkorange', 'gold', 'tomato', 'aqua', 'orchid']

    # Plot a histogram for each feature
    for i, col in enumerate(numeric_cols):
        ax = axes[i]
        data = feat_df_capped[col].dropna()
        ax.hist(data, bins=50, color=colors[i % len(colors)], edgecolor='black', alpha=0.8)
        ax.set_title(col, fontsize=16, color="#F8F8F8", pad=5)
        ax.set_facecolor("#111111")
        ax.tick_params(colors="white", labelsize=10)
        for spine in ax.spines.values(): spine.set_edgecolor("white")
        ax.grid(color='gray', linestyle='--', linewidth=0.3, alpha=0.5)

    # Save the figure
    save_fig(fig, "final_feature_distributions")
else:
    print("--- Skipped: Loading from cache ---")
```

Distributions of Final Engineered Features

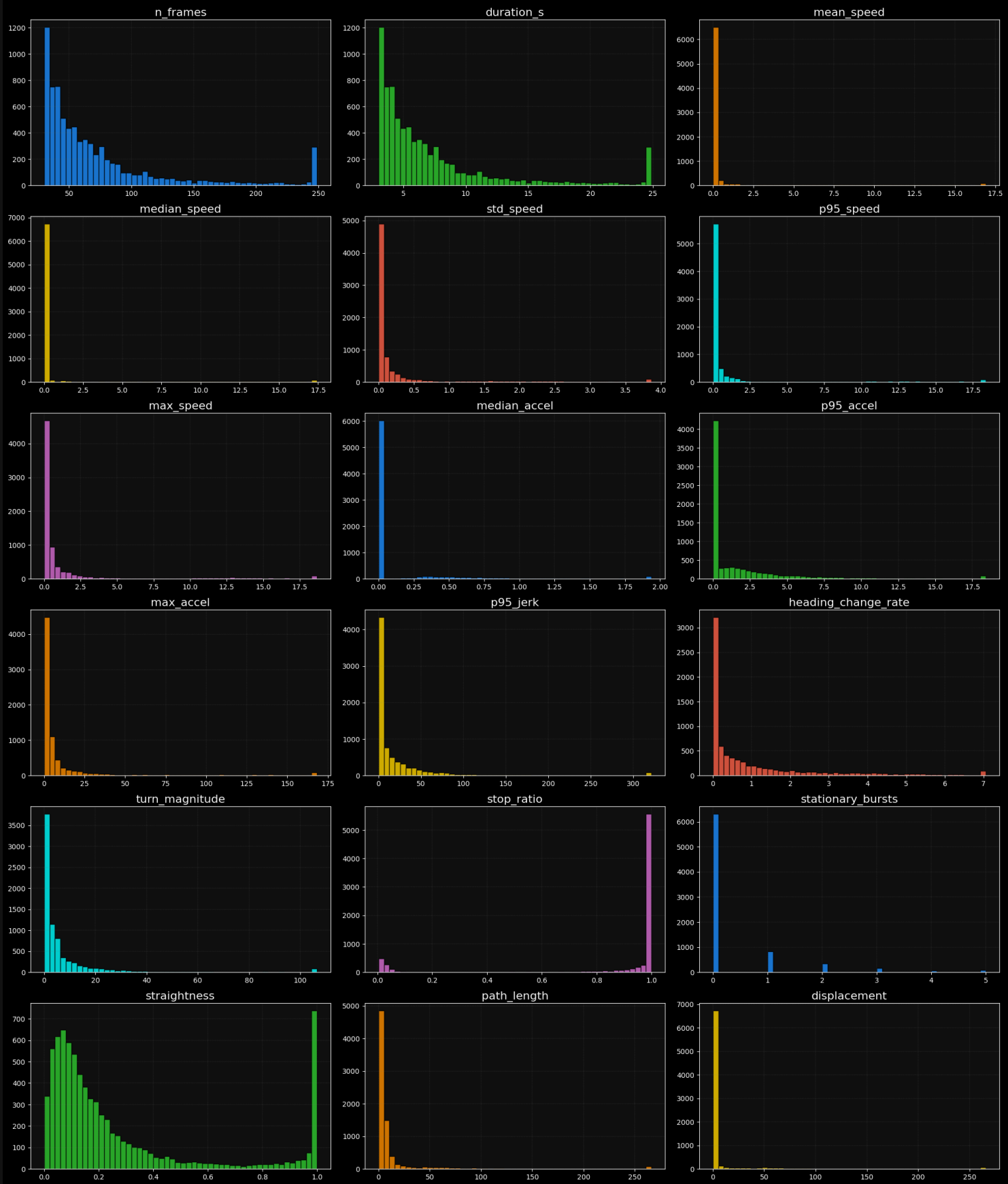


Figure saved to (Linux): /home/treinart/projects/lyft-unsupervised/artifacts/figures/20250917_205513_final_feature_distributions.png

Figure saved to (Win): /mnt/c/Users/travi/Documents/Training/Colorado/MS-AI/CSCA 5632 Final Project/artifacts/figures/20250917_205513_final_feature_distributions.png

Observation: Classical Feature Engineering and Selection

This section successfully transitioned from raw trajectory data to a structured, interpretable feature space suitable for clustering. A comprehensive catalog of 16 kinematic and geometric features was defined to numerically describe agent behavior. The core of the analysis involved a sensitivity test to determine an optimal track length threshold, culminating in a final, preprocessed feature matrix for Pipeline A.

- The **MIN_CONSEC** Trade-off (Key Finding):

- A critical hyperparameter `MIN_CONSEC`, determines the minimum number of frames a track must have to be included in the analysis. A low value includes more tracks but potentially adds noise from short, incomplete movements. A high value yields higher-quality trajectories but reduces the dataset size.
 - The **Sensitivity Analysis** plot was created to make a data-driven decision. The plot revealed that as `MIN_CONSEC` increases, key feature distributions (like `straightness` and `duration`) begin to stabilize. The value of **30** was identified as an optimal "elbow point," where further increases resulted in diminishing returns—drastically reducing the number of eligible tracks without providing a significant improvement in feature stability.
- **Dataset Characteristics:**
 - Applying the `MIN_CONSEC=30` threshold resulted in a final dataset of **7,824** high-quality tracks for analysis, down from the initial 64,625 raw tracks.
 - After capping extreme values at the 99th percentile to mitigate outliers, a final sanity check revealed a defining characteristic of this dataset: **84.9%** of these eligible tracks are "highly stationary," with a `stop_ratio` of 0.7 or greater. This confirms that the problem space is dominated by low-speed or stopped behaviors.
 - **Final Feature Validation:**

The panel of histograms provides a final visual confirmation of the engineered features. It verifies that the outlier capping was effective and shows the final shape and scale of the data being passed to the clustering algorithms, with many metrics exhibiting a strong right-skew, consistent with the dataset's stationary nature.

Conclusion: Pipeline A's feature engineering is complete. The process successfully distilled raw trajectories into an interpretable feature set for **7,824** tracks. The analysis provided a clear, quantitative justification for the crucial `MIN_CONSEC` hyperparameter and confirmed the overwhelmingly stationary nature of the data that will be fed to the clustering algorithms. The final, standardized feature matrix is now ready for the modeling phase in Section 6.

Section 6: Pipeline A - Clustering with Classical Features

This section executes the core modeling phase of Pipeline A. The goal is to apply a suite of unsupervised clustering algorithms to the interpretable features engineered in Section 5. The process involves a systematic approach: dimensionality reduction with PCA, extensive hyperparameter sweeps for three distinct clustering models (KMeans, GMM, HDBSCAN), and a data-driven model selection process based on a scorecard of internal validation and stability metrics. The section concludes by fitting the winning model and interpreting the resulting behavioral clusters.

Section Plan:

- **Checkpoint & Loader:** Option to skip computation and load the latest cached artifacts.
- **Feature Selection & QA:** Select the final feature subset and perform a final quality assurance check.
- **Dimensionality Reduction:** Apply PCA to create embeddings that capture 90%, 95%, and 98% of the variance.
- **KMeans Hyperparameter Sweep:** Systematically test a range of cluster counts (k) for KMeans.
- **GMM Hyperparameter Sweep:** Test a range of components and covariance types for Gaussian Mixture Models.
- **HDBSCAN Hyperparameter Sweep:** Explore the parameter space for the density-based HDBSCAN model.
- **Stability Analysis:** Evaluate the stability of the best KMeans and GMM models using bootstrap and the Adjusted Rand Index (ARI).
- **Final Model Selection:** Use a weighted scorecard to rank all candidate models and select the overall winner.
- **Fit Final Model:** Train the winning model on the full dataset and assign a cluster label to every track.
- **Interpret Clusters (Profiling):** Analyze the median feature values for each cluster to create a numerical "persona" for each behavior.
- **Qualitative Validation:** Plot sample trajectories from each cluster for a final, visual confirmation of the results.

6.1 Checkpoint: Run Full Section or Load from Cache

This cell acts as a checkpoint for the modeling section. The hyperparameter sweeps are the most time-consuming part of the entire project. Set the flag below to control the behavior of this section.

- `True` : Run the full clustering sweeps and analysis. This is required on the first run.
- `False` : Skip the long computations and load the final results (cluster assignments and profiles) from a previous run.

```
In [30]: # Set to True to run the time-consuming clustering sweeps and analysis.
# Set to False to skip computation and load the final results from a previous run.
RUN_CLUSTERING_SWEEPS = True

if not RUN_CLUSTERING_SWEEPS:
    print("--- Skipping clustering sweeps. Loading final artifacts from cache... ---")
    try:
        # Load the final cluster assignments and profiles from the stable artifact files
        final_clusters_df = pd.read_parquet(ARTIFACT_DIR / "pipeline_a_cluster_assignments.parquet")
        cluster_profiles = pd.read_parquet(ARTIFACT_DIR / "pipeline_a_cluster_profiles.parquet")

        # Load the winning model's metadata
        with open(ARTIFACT_DIR / "pipeline_a_final_model_meta.json", 'r') as f:
            best_model_params = json.load(f)

        print(f"Loaded {len(final_clusters_df):,} cluster assignments for {best_model_params['k']} clusters.")
        print(f"Winning model type: {best_model_params['model_type']}")
        print("\n--- Successfully loaded artifacts. Ready to proceed to Section 7 ---")
    except FileNotFoundError:
        print("\n--- ERROR: No final artifacts found. Please set RUN_CLUSTERING_SWEEPS to True and run the full section ---")
        # Set flag to True to prevent errors in subsequent cells
        RUN_CLUSTERING_SWEEPS = True
else:
    print("--- RUN_CLUSTERING_SWEEPS is True. Proceeding with full computation. ---")
```

```
--- RUN_CLUSTERING_SWEEPS is True. Proceeding with full computation. ---
```

6.2 Feature Selection & Quality Assurance

Before modeling, a subset of the most informative and independent features from the full engineered set is selected. This step explicitly defines the feature space for clustering. A final quality assurance check is performed on this subset to confirm data types, ranges, and check for any remaining null values before standardization.

```
In [31]: # This cell will only run if the full computation is enabled.
if RUN_CLUSTERING_SWEEPS:
    # Select the Final Feature Subset for Clustering
    FINAL_FEATURE_COLS = [
        "mean_speed", "p95_speed", "std_speed",
        "heading_change_rate", "stop_ratio", "straightness", "duration_s"
    ]
    model_features_df = feat_df_capped[[col for col in FINAL_FEATURE_COLS if col in feat_df_capped.columns]].copy()

    # Perform Final Quality Assurance
    qa_rows = []
    for col in model_features_df.columns:
        v = pd.to_numeric(model_features_df[col], errors="coerce").to_numpy()
        is_finite = np.isfinite(v)
        finite_count = int(is_finite.sum())
        total_count = len(v)
        qa_rows.append({
            "feature": col, "non_null_count": finite_count,
            "null_rate": (1.0 - finite_count / total_count) if total_count > 0 else 0.0,
            "min": float(np.min(v[is_finite])) if finite_count > 0 else np.nan,
            "median": float(np.median(v[is_finite])) if finite_count > 0 else np.nan,
            "p95": float(np.percentile(v[is_finite], 95)) if finite_count > 0 else np.nan,
            "max": float(np.max(v[is_finite])) if finite_count > 0 else np.nan,
            "mean": float(np.mean(v[is_finite])) if finite_count > 0 else np.nan,
            "std": float(np.std(v[is_finite])) if finite_count > 0 else np.nan,
        })
    qa_df = pd.DataFrame(qa_rows)

    # Create a formatted version for display
    pretty_qa = qa_df.copy()
    def format_or_dash(x, precision=3):
        col_name = _col_name # Relies on Loop variable from below
        if pd.isna(x) or (isinstance(x, (int, float)) and abs(x) < 1e-9): return "---"
        if isinstance(x, float):
            if col_name == 'null_rate': return f"{x:.2%}"
            return f"{x:.{precision}f}"
        if isinstance(x, int): return f"{x:,}"
        return x

    for _col_name in ["non_null_count", "null_rate", "min", "median", "p95", "max", "mean", "std"]:
        pretty_qa[_col_name] = pretty_qa[_col_name].apply(format_or_dash)

    # Display the styled summary table
    print("\n--- Final Feature Quality Assurance Summary ---")
    display(style_section_table(pretty_qa))

    print("\n" + "-" * 70)
    print(f"\nFinal features in use: {model_features_df.columns.tolist()}")
    print(f"\nTotal tracks for modeling: {len(model_features_df):,}")
    print("\n" + "-" * 70)
else:
    print("--- Skipped: Loading from cache ---")
```

--- Final Feature Quality Assurance Summary ---								
feature	non_null_count	null_rate	min	median	p95	max	mean	std
mean_speed	7,824	---	---	0.007	11.074	16.911	1.225	3.536
p95_speed	7,824	---	---	---	13.193	18.374	1.637	4.125
std_speed	7,824	---	---	0.036	2.353	3.867	0.356	0.786
heading_change_rate	7,824	---	---	0.318	4.985	7.058	1.079	1.631
stop_ratio	7,824	---	0.004	1.000	1.000	1.000	0.854	0.319
straightness	7,824	---	0.000	0.149	0.998	1.000	0.279	0.308
duration_s	7,824	---	3.000	5.400	21.600	24.900	7.386	5.366

Final features in use: ['mean_speed', 'p95_speed', 'std_speed', 'heading_change_rate', 'stop_ratio', 'straightness', 'duration_s']

Total tracks for modeling: 7,824

6.3 Dimensionality Reduction with PCA

Before clustering, the feature space is transformed using Principal Component Analysis (PCA). PCA serves two purposes: it reduces dimensionality by creating a smaller set of uncorrelated components, and it removes multicollinearity between original features. Three separate PCA models are trained to create embeddings that retain


```

In [32]: # This cell will only run if the full computation is enabled.
if RUN_CLUSTERING_SWEEPS:
    # Standardize the Data
    X = model_features_df.fillna(model_features_df.mean()).to_numpy()
    scaler = StandardScaler()
    X_scaled = scaler.fit_transform(X)

    # Run PCA for Multiple Variance Targets
    VARIANCE_TARGETS = [0.90, 0.95, 0.98]
    PCA_MODELS = {}
    PCA_EMBEDDINGS = {}
    fig, axes = plt.subplots(len(VARIANCE_TARGETS), 2, figsize=(20, 8 * len(VARIANCE_TARGETS)), facecolor="black", constrained_layout=True)
    fig.patch.set_facecolor("black")
    fig.suptitle("PCA Explained Variance Analysis", fontsize=28, color="#CFB87C", fontweight="bold", y=0.995)

    for i, var_target in enumerate(VARIANCE_TARGETS):
        # Configure and fit the PCA model
        pca = PCA(n_components=var_target, random_state=SEED)
        Z = pca.fit_transform(X_scaled)
        key = f"pca_{int(var_target*100)}"
        PCA_MODELS[key] = pca
        PCA_EMBEDDINGS[key] = Z

        # Visualize Results for this Target
        ax1, ax2 = axes[i]
        explained_variance = pca.explained_variance_ratio_
        cumulative_variance = np.cumsum(explained_variance)
        ax1.bar(range(1, len(explained_variance) + 1), explained_variance, color='dodgerblue', alpha=0.8)
        ax1.set_title(f"Target: {var_target:.0%} Variance ({pca.n_components_} Components)", fontsize=18, color="#F8F8F8", pad=2)
        ax1.set_ylabel("Explained Variance Ratio", fontsize=14, color="white")
        ax2.plot(range(1, len(cumulative_variance) + 1), cumulative_variance, marker="o", color='limegreen')
        ax2.axhline(y=var_target, color='#DA291C', linestyle='--', label=f"{var_target:.0%} Target")
        ax2.set_ylabel("Cumulative Variance", fontsize=14, color="white")
        ax2.set_ylim(0, 1.05)
        ax2.legend(frameon=False, labelcolor="white", fontsize=12)
        for ax in [ax1, ax2]:
            ax.set_facecolor("black"); ax.tick_params(colors="white", labelsz=12)
            ax.set_xlabel("Principal Components", fontsize=14, color="white")
            for spine in ax.spines.values(): spine.set_edgecolor("white")

    # Finalize Plot and Print Summary
    save_fig(fig, "pca_variance_summary")

    print("-" * 70)
    print("PCA Model Summary:")
    for key, pca in PCA_MODELS.items():
        print(f"  {key}: Retained {np.sum(pca.explained_variance_ratio_:.1%)} variance with {pca.n_components_} components.")
    print("-" * 70)
else:
    print("--- Skipped: Loading from cache ---")

```

PCA Explained Variance Analysis

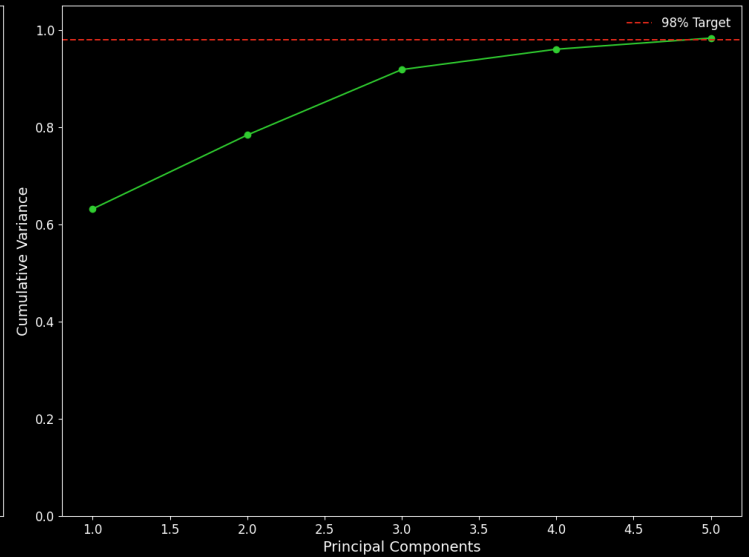
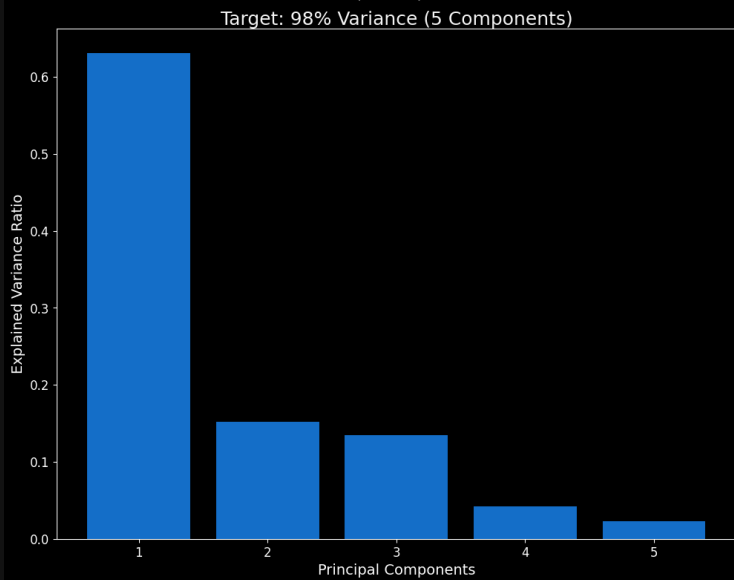
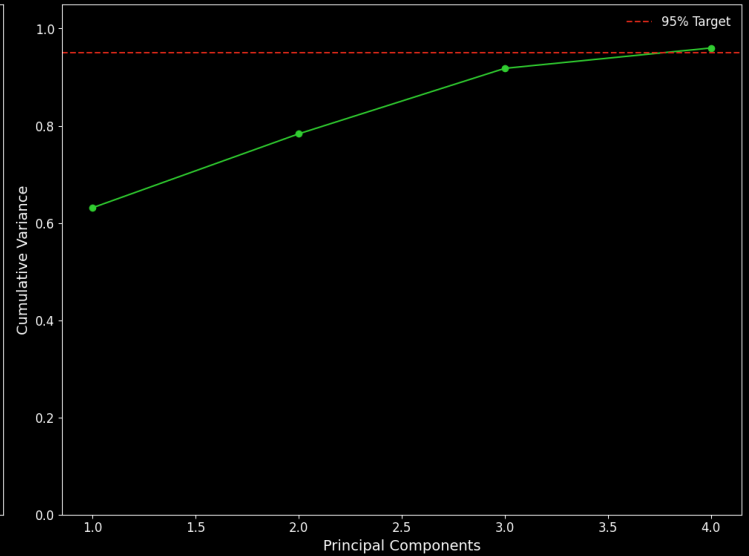
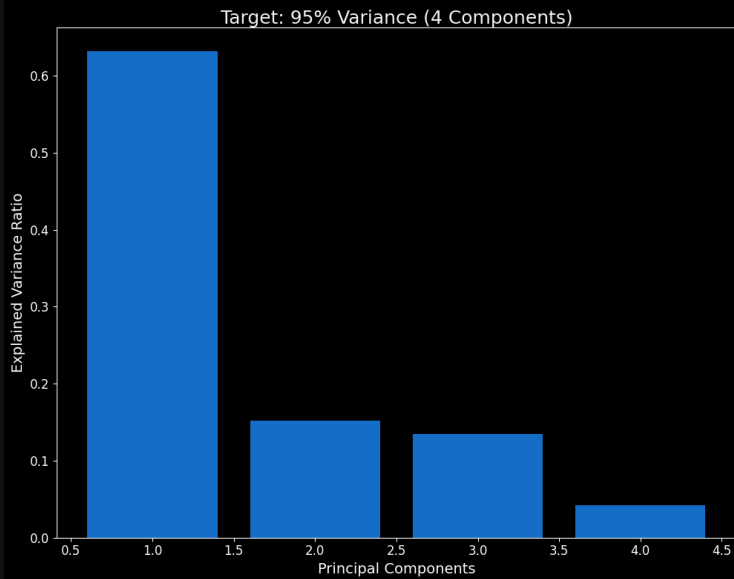
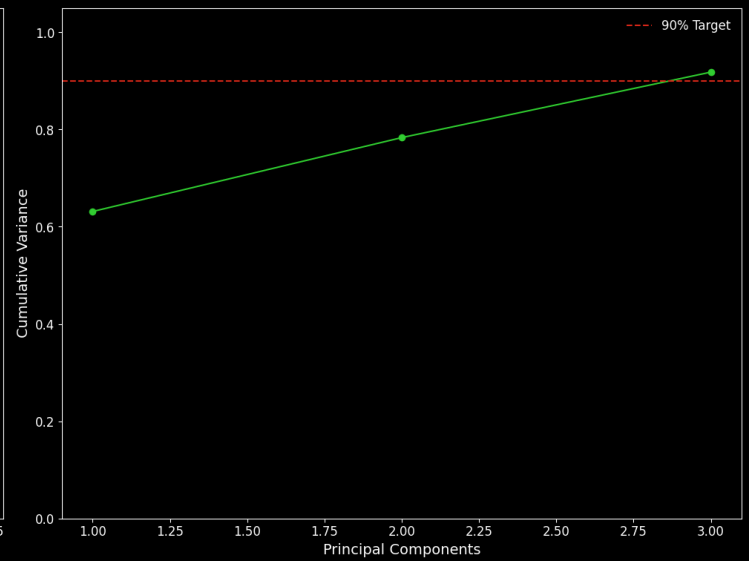
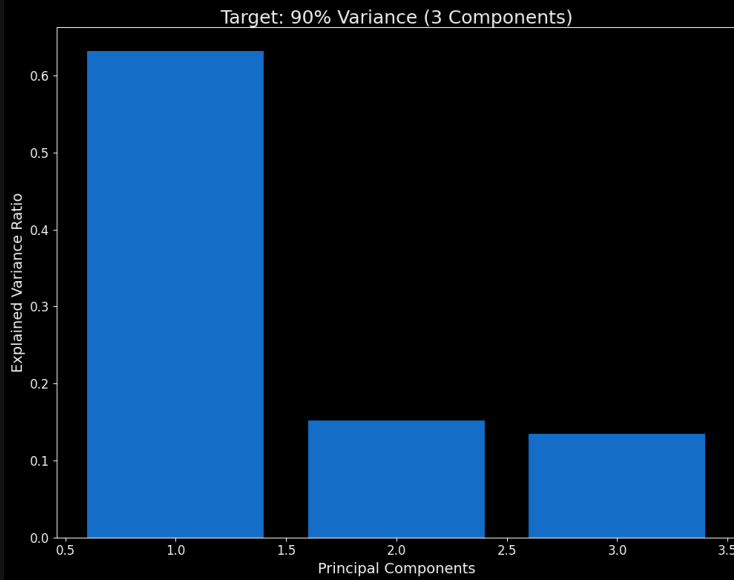


Figure saved to (Linux): /home/treinart/projects/lyft-unsupervised/artifacts/figures/20250917_202046_pca_variance_summary.png

Figure saved to (Win): /mnt/c/Users/travi/Documents/Training/Colorado/MS-AI/CSCA 5632 Final Project/artifacts/figures/20250917_202046_pca_variance_summary.png

PCA Model Summary:

- pca_90: Retained 91.8% variance with 3 components.
- pca_95: Retained 96.0% variance with 4 components.
- pca_98: Retained 98.3% variance with 5 components.

The first clustering method used here is KMeans. A "sweep" is performed to find the optimal number of clusters (k). The model is run for a range of k values across each of the three PCA embeddings. To ensure results are stable, each run is repeated with multiple random seeds. The quality of the resulting clusters is evaluated using three standard internal validation metrics: the Silhouette Score, Calinski-Harabasz Index, and Davies-Bouldin Index.

In [33]: # This cell will only run if the full computation is enabled.

```

if RUN_CLUSTERING_SWEEPS:
    # --- Define Sweep Parameters ---
    K_VALUES = [4, 6, 8, 10, 12, 14, 16, 20, 24]
    RANDOM_SEEDS = [0, 1, 2, 3, 4]

    # Run the KMeans Sweep
    kmeans_results = []
    pca_keys = list(PCA_EMBEDDINGS.keys())
    inner_total = len(K_VALUES) * len(RANDOM_SEEDS)
    pos_base = 0
    p_all = tqdm(total=len(pca_keys) * inner_total, desc="All KMeans runs", unit="run", position=pos_base, leave=True)
    bars = {key: tqdm(total=inner_total, desc=key, unit="run", position=pos_base + i + 1, leave=True) for i, key in enumerate(pca_keys)}

    for pca_key, p_bar in bars.items():
        Z = PCA_EMBEDDINGS[pca_key]
        for k in K_VALUES:
            for seed in RANDOM_SEEDS:
                kmeans = KMeans(n_clusters=k, n_init="auto", random_state=seed)
                labels = kmeans.fit_predict(Z)
                try:
                    sil = silhouette_score(Z, labels); ch = calinski_harabasz_score(Z, labels); db = davies_bouldin_score(Z, labels)
                except ValueError:
                    sil, ch, db = np.nan, np.nan, np.nan
                kmeans_results.append({"pca": pca_key, "k": k, "seed": seed, "silhouette": sil, "calinski_harabasz": ch, "davies_bouldin": db})
                p_bar.update(1); p_all.update(1)
                p_bar.set_postfix_str(f"k={k} seed={seed}")

    p_all.refresh(); [b.refresh() for b in bars.values()]
    kmeans_sweep_df = pd.DataFrame(kmeans_results)

    # Aggregate and Display Results
    kmeans_agg_df = (kmeans_sweep_df.groupby(["pca", "k"])
        .agg(silhouette_mean=("silhouette", "mean"), silhouette_std=("silhouette", "std"),
            calinski_harabasz_mean=("calinski_harabasz", "mean"), calinski_harabasz_std=("calinski_harabasz", "std"),
            davies_bouldin_mean=("davies_bouldin", "mean"), davies_bouldin_std=("davies_bouldin", "std"))
        .reset_index())

    for pca_key in PCA_EMBEDDINGS.keys():
        print(f"\n--- KMeans Metric Summary - {pca_key} ---")
        display(style_section_table(kmeans_agg_df[kmeans_agg_df["pca"] == pca_key].drop("pca", axis=1)))

    # Plot Metric Curves
    metrics_to_plot = {"silhouette": "Silhouette Score (Higher is Better)", "calinski_harabasz": "Calinski-Harabasz Index (Higher is Better)", "davies_bouldin": "Davies-Bouldin Index (Lower is Better)"}
    for metric, title in metrics_to_plot.items():
        fig, ax = plt.subplots(figsize=(20, 10), facecolor="black")
        ax.set_facecolor("black")

        colors = ['lime', 'magenta', 'orange']
        for color, pca_key in zip(colors, PCA_EMBEDDINGS.keys()):
            subset = kmeans_agg_df[kmeans_agg_df["pca"] == pca_key]
            ax.plot(subset["k"], subset[f"{metric}_mean"], marker='o', linestyle='--', label=pca_key, color=color, lw=2.5)
            ax.fill_between(subset["k"], subset[f"{metric}_mean"] - subset[f"{metric}_std"], subset[f"{metric}_mean"] + subset[f"{metric}_std"], alpha=0.15, color=color)

        ax.set_title(title, fontsize=26, color="#CFB87C", fontweight="bold", pad=15)
        ax.set_xlabel("Number of Clusters (k)", fontsize=18, color="white", fontweight="bold")
        ax.set_ylabel("Score", fontsize=18, color="white", fontweight="bold")
        ax.tick_params(colors="white", labelsiz=14)
        for spine in ax.spines.values():
            spine.set_edgecolor("white"); spine.set_linewidth(1.5)
        ax.legend(frameon=False, labelcolor="white", fontsize=16, title="PCA Variant")
        ax.grid(color='gray', linestyle='--', linewidth=0.3, alpha=0.5)
        save_fig(fig, f"kmeans_sweep_{metric}")

else:
    print("--- Skipped: Loading from cache ---")

```

```

All KMeans runs: 0%|          | 0/135 [00:00<?, ?run/s]
pca_90: 0%|          | 0/45 [00:00<?, ?run/s]
pca_95: 0%|          | 0/45 [00:00<?, ?run/s]
pca_98: 0%|          | 0/45 [00:00<?, ?run/s]
--- KMeans Metric Summary - pca_90 ---

```

k	silhouette_mean	silhouette_std	calinski_harabasz_mean	calinski_harabasz_std	davies_bouldin_mean	davies_bouldin_std
4	0.558113	0.000195	10811.153367	0.250850	0.685120	0.000119
6	0.433921	0.042736	9485.525176	377.111747	0.897698	0.024360
8	0.411783	0.017212	9572.143739	415.896711	0.836856	0.005864
10	0.399580	0.019518	9996.236401	475.822988	0.841385	0.040188
12	0.390086	0.012525	10441.815522	79.281856	0.823006	0.005398
14	0.384684	0.005679	10191.732067	296.131068	0.862664	0.022568
16	0.380959	0.005245	9928.525339	115.421561	0.883743	0.018114
20	0.371993	0.005497	9539.111281	109.937480	0.902380	0.010010
24	0.367311	0.015064	9423.795582	130.357314	0.911917	0.023802

--- KMeans Metric Summary - pca_95 ---

k	silhouette_mean	silhouette_std	calinski_harabasz_mean	calinski_harabasz_std	davies_bouldin_mean	davies_bouldin_std
4	0.514809	0.016230	8433.643582	751.488367	0.799536	0.140523
6	0.469594	0.075570	8235.681341	187.423089	0.869266	0.068815
8	0.368974	0.032001	7919.577282	154.285769	0.935235	0.022239
10	0.359268	0.012486	7963.450898	282.636038	0.878076	0.010764
12	0.365993	0.010430	8207.074484	211.260705	0.868232	0.022114
14	0.361116	0.010753	8096.907345	139.590823	0.875288	0.015906
16	0.352589	0.017004	7809.884371	113.396694	0.904471	0.007620
20	0.340587	0.005807	7546.713848	71.008301	0.934736	0.015227
24	0.329980	0.009266	7108.566868	76.889470	0.982610	0.027009

--- KMeans Metric Summary - pca_98 ---

k	silhouette_mean	silhouette_std	calinski_harabasz_mean	calinski_harabasz_std	davies_bouldin_mean	davies_bouldin_std
4	0.504027	0.014991	7565.069355	609.543969	0.885779	0.196211
6	0.417754	0.100595	7189.419030	182.646279	0.966823	0.025617
8	0.370940	0.031851	6542.506545	177.845009	1.010975	0.033239
10	0.359228	0.026116	6632.616721	80.235354	0.954273	0.029806
12	0.354467	0.010858	6707.506901	260.762865	0.920963	0.030957
14	0.348860	0.023257	6733.494419	142.231824	0.916117	0.031247
16	0.344443	0.016859	6523.242601	134.319611	0.927473	0.025675
20	0.336156	0.007655	6187.553325	95.669432	0.976453	0.013826
24	0.328975	0.008320	5882.741494	29.280616	1.001827	0.022420

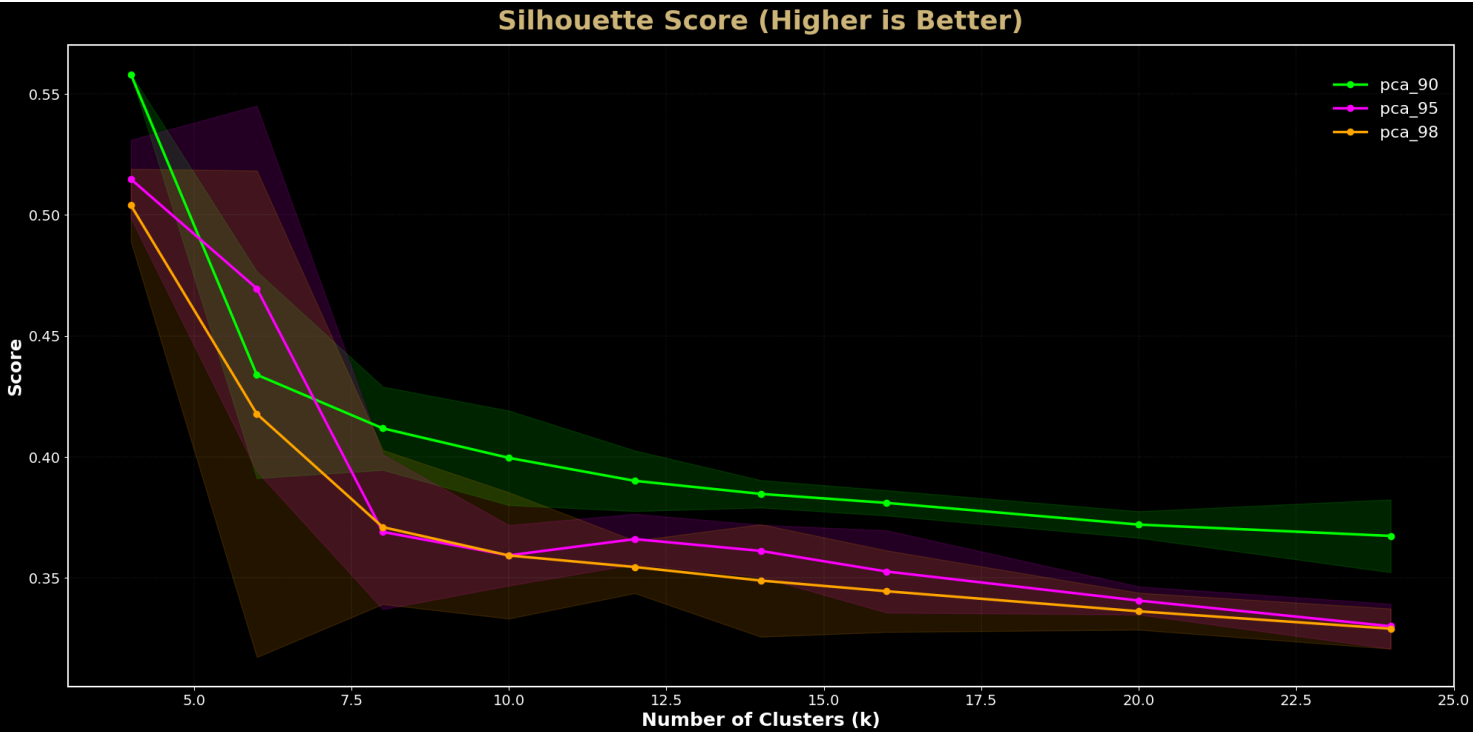


Figure saved to (Linux): /home/treinarnt/projects/lyft-unsupervised/artifacts/figures/20250917_202233_kmeans_sweep_silhouette.png
Figure saved to (Win): /mnt/c/Users/travi/Documents/Training/Colorado/MS-AI/CSCA 5632 Final Project/artifacts/figures/20250917_202233_kmeans_sweep_silhouette.png

Calinski-Harabasz Index (Higher is Better)

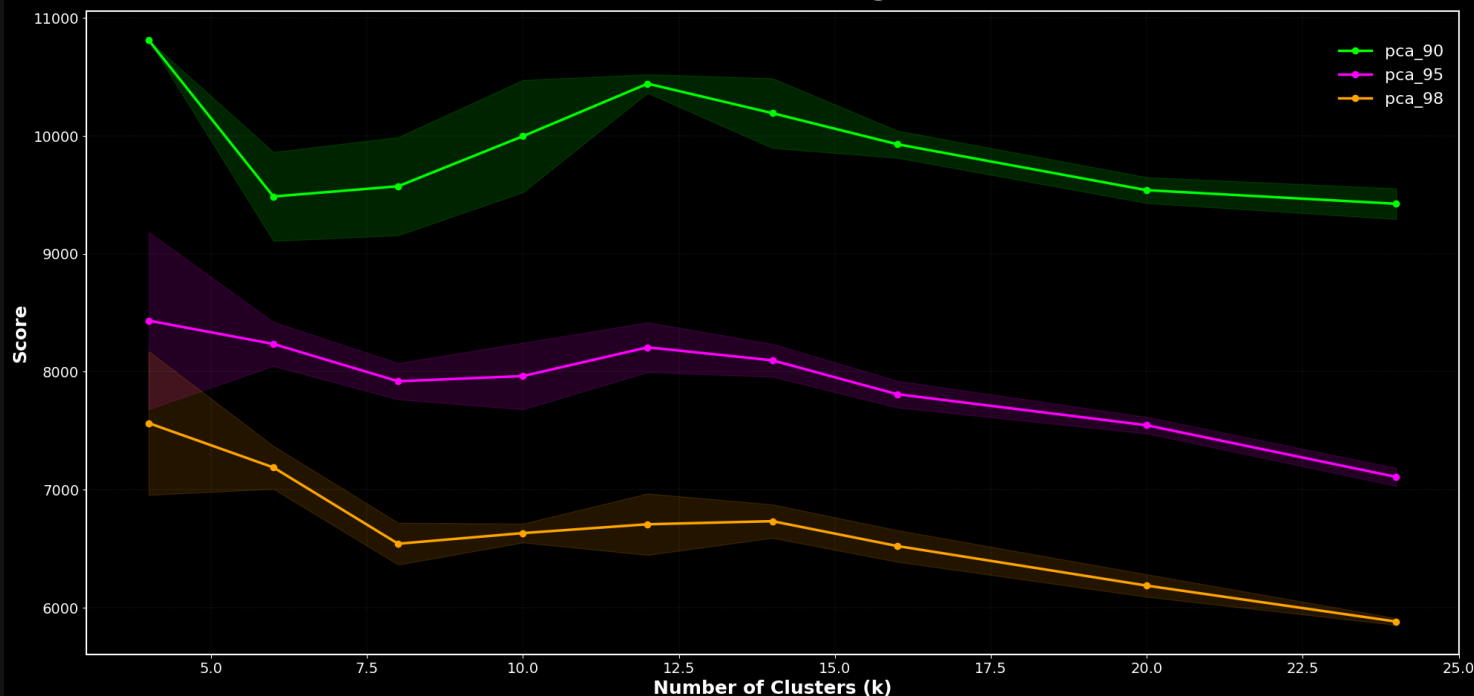


Figure saved to (Linux): /home/treinart/projects/lyft-unsupervised/artifacts/figures/20250917_202234_kmeans_sweep_calinski_harabasz.png
 Figure saved to (Win): /mnt/c/Users/travi/Documents/Training/Colorado/MS-AI/CSCA 5632 Final Project/artifacts/figures/20250917_202234_kmeans_sweep_calinski_harabasz.png

Davies-Bouldin Index (Lower is Better)

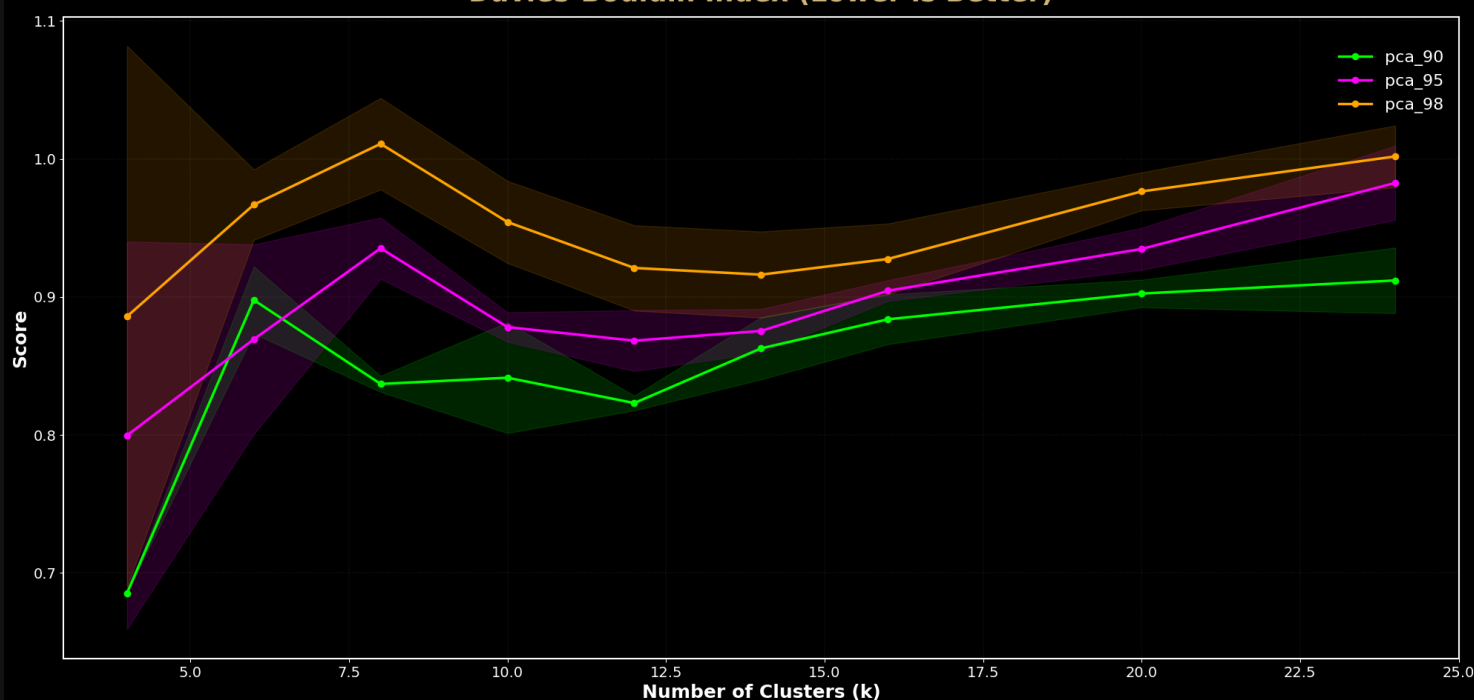


Figure saved to (Linux): /home/treinart/projects/lyft-unsupervised/artifacts/figures/20250917_202234_kmeans_sweep_davies_bouldin.png
 Figure saved to (Win): /mnt/c/Users/travi/Documents/Training/Colorado/MS-AI/CSCA 5632 Final Project/artifacts/figures/20250917_202234_kmeans_sweep_davies_bouldin.png

6.5 GMM Hyperparameter Sweep

The second model, Gaussian Mixture Models (GMM), offers more flexibility than KMeans by modeling clusters as ellipsoids. This allows for clusters of varying shape and orientation. The sweep for GMM tests a range of component counts (the equivalent of k) and two different covariance types ('full' and 'diag') across the three PCA embeddings. Model quality is evaluated using Silhouette Score and two information criteria metrics, BIC and AIC, which are well-suited for GMMs.

In [34]: # This cell will only run if the full computation is enabled.

```
if RUN_CLUSTERING_SWEEPS:
    # --- Define Sweep Parameters ---
    N_COMPONENTS = [4, 6, 8, 10, 12, 14, 16, 20]
    COVARIANCE_TYPES = ["full", "diag"]
    GMM_SEEDS = [0, 1, 2]
```

```
# Run the GMM Sweep
gmm_results = []
```

```
pca_keys = list(PCA_EMBEDDINGS.keys())
inner_total = len(COVARIANCE_TYPES) * len(N_COMPONENTS) * len(GMM_SEEDS)
pos_base = 0
p_all = tqdm(total=len(pca_keys) * inner_total, desc="All GMM runs", unit="run", position=pos_base, leave=True)
bars = {key: tqdm(total=inner_total, desc=key, unit="run", position=pos_base + i + 1, leave=True) for i, key in enumerate(pca_keys)}

for pca_key, p_bar in bars.items():
    Z = PCA_EMBEDDINGS[pca_key]
    for cov_type in COVARIANCE_TYPES:
        for k in N_COMPONENTS:
            for seed in GMM_SEEDS:
                gmm = GaussianMixture(n_components=k, covariance_type=cov_type, random_state=seed)
                labels = gmm.fit_predict(Z)
                try:
                    sil = silhouette_score(Z, labels); bic = gmm.bic(Z); aic = gmm.aic(Z)
                except ValueError:
                    sil, bic, aic = np.nan, np.nan, np.nan
                gmm_results.append({"pca": pca_key, "covariance": cov_type, "k": k, "seed": seed, "silhouette": sil, "bic": bic, "aic": aic})
                p_bar.update(1); p_all.update(1)
            p_bar.set_postfix_str(f"{cov_type} k={k} seed={seed}")

p_all.refresh(); [b.refresh() for b in bars.values()]
gmm_sweep_df = pd.DataFrame(gmm_results)

# Aggregate and Plot Results
gmm_agg_df = (gmm_sweep_df.groupby(["pca", "covariance", "k"])
               .agg(silhouette_mean=("silhouette", "mean"), bic_mean=("bic", "mean"), aic_mean=("aic", "mean"))
               .reset_index())

# BIC minimum per PCA x covariance table
winners_df = (gmm_agg_df.sort_values("bic_mean", ascending=True)
               .groupby(["pca", "covariance"], as_index=False)
               .first()[["pca", "covariance", "k", "bic_mean", "aic_mean", "silhouette_mean"]])
winners_pretty = winners_df.rename(columns={
    "pca": "PCA",
    "covariance": "Covariance",
    "k": "k",
    "bic_mean": "BIC ↓",
    "aic_mean": "AIC ↓",
    "silhouette_mean": "Silhouette ↑",
}).copy()
winners_pretty["BIC ↓"] = winners_pretty["BIC ↓"].map(lambda x: "---" if pd.isna(x) else f"{x:.0f}")
winners_pretty["AIC ↓"] = winners_pretty["AIC ↓"].map(lambda x: "---" if pd.isna(x) else f"{x:.0f}")
winners_pretty["Silhouette ↑"] = winners_pretty["Silhouette ↑"].map(lambda x: "---" if pd.isna(x) else f"{x:.3f}")
print("\n--- GMM BIC Winners (per PCA x covariance) ---")
display(style_section_table(winners_pretty))

for metric, title in [("bic", "GMM - BIC Score (Lower is Better)", ("silhouette", "GMM - Silhouette Score (Higher is Better)")):
    fig, ax = plt.subplots(figsize=(20, 10), facecolor="black")
    ax.set_facecolor("black")

    styles = {"full": "-", "diag": "--"}; colors = {"pca_90": 'lime', "pca_95": 'magenta', "pca_98": 'orange'}
    for (pca_key, cov_type), group in gmm_agg_df.groupby(["pca", "covariance"]):
        ax.plot(group["k"], group[f"{metric}_mean"], marker='o', linestyle=styles[cov_type], label=f"{pca_key}, {cov_type}", color=colors[pca_key], lw=2)

    ax.set_title(title, fontsize=26, color="#CFB87C", fontweight="bold", pad=15)
    ax.set_xlabel("Number of Components (k)", fontsize=18, color="white", fontweight="bold")
    ax.set_ylabel("Score", fontsize=18, color="white", fontweight="bold")
    ax.set_ticks(colors="white", labels=14)
    for spine in ax.spines.values():
        spine.set_edgecolor("white"); spine.set_linewidth(1.5)
    ax.legend(frameon=False, labelcolor="white", fontsize=14, title="PCA & Covariance")
    ax.grid(color='gray', linestyle='--', linewidth=0.3, alpha=0.5)
    save_fig(fig, f"gmm_sweep_{metric}")
else:
    print("--- Skipped: Loading from cache ---")
```

```
All GMM runs: 0%|          | 0/144 [00:00<?, ?run/s]
pca_90: 0%|          | 0/48 [00:00<?, ?run/s]
pca_95: 0%|          | 0/48 [00:00<?, ?run/s]
pca_98: 0%|          | 0/48 [00:00<?, ?run/s]
--- GMM BIC Winners (per PCA x covariance) ---
```

PCA	Covariance	k	BIC ↓	AIC ↓	Silhouette ↑
pca_90	diag	20	32,521	31,553	0.244
pca_90	full	20	9,053	7,667	0.025
pca_95	diag	20	29,344	28,097	0.172
pca_95	full	20	-28,336	-30,419	-0.001
pca_98	diag	20	8,456	6,930	0.140
pca_98	full	20	-82,826	-85,745	-0.003

GMM - BIC Score (Lower is Better)

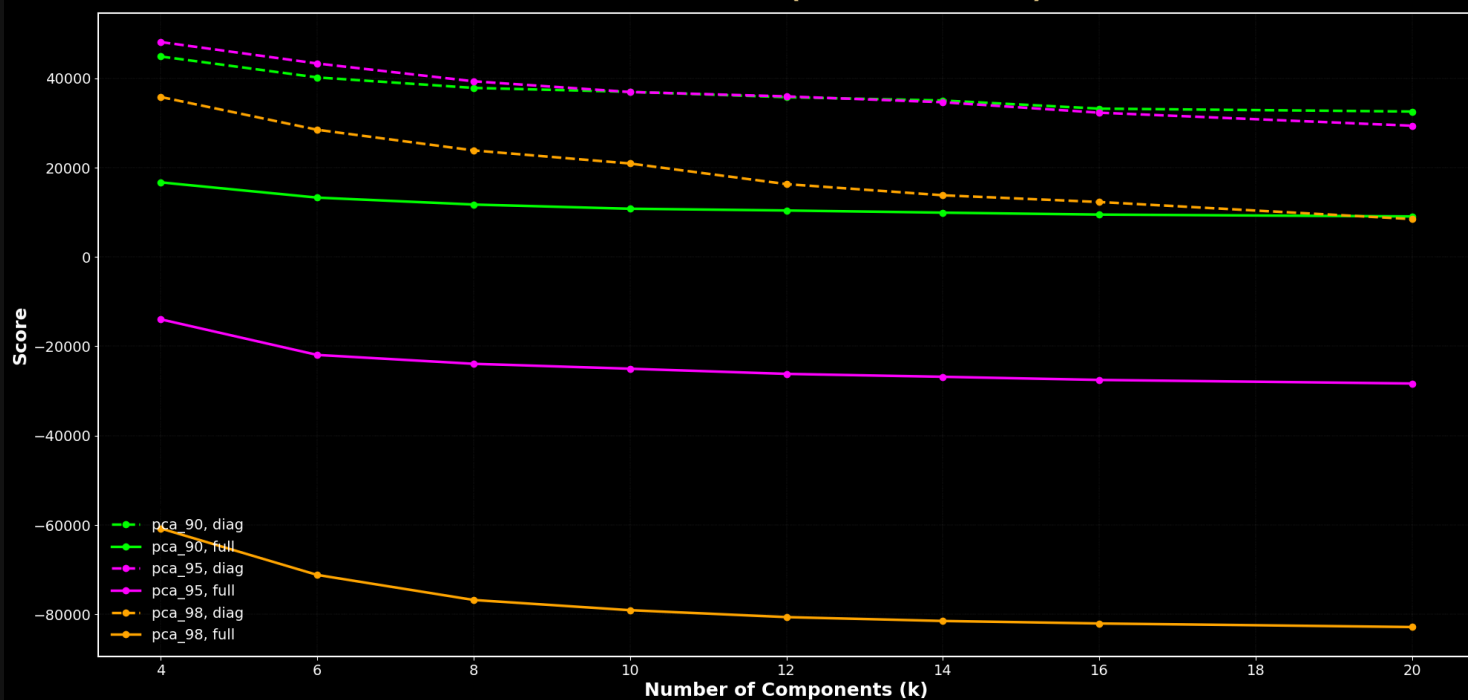


Figure saved to (Linux): /home/treinart/projects/lyft-unsupervised/artifacts/figures/20250917_204122_gmm_sweep_bic.png

Figure saved to (Win): /mnt/c/Users/travi/Documents/Training/Colorado/MS-AI/CSCA 5632 Final Project/artifacts/figures/20250917_204122_gmm_sweep_bic.png

GMM - Silhouette Score (Higher is Better)

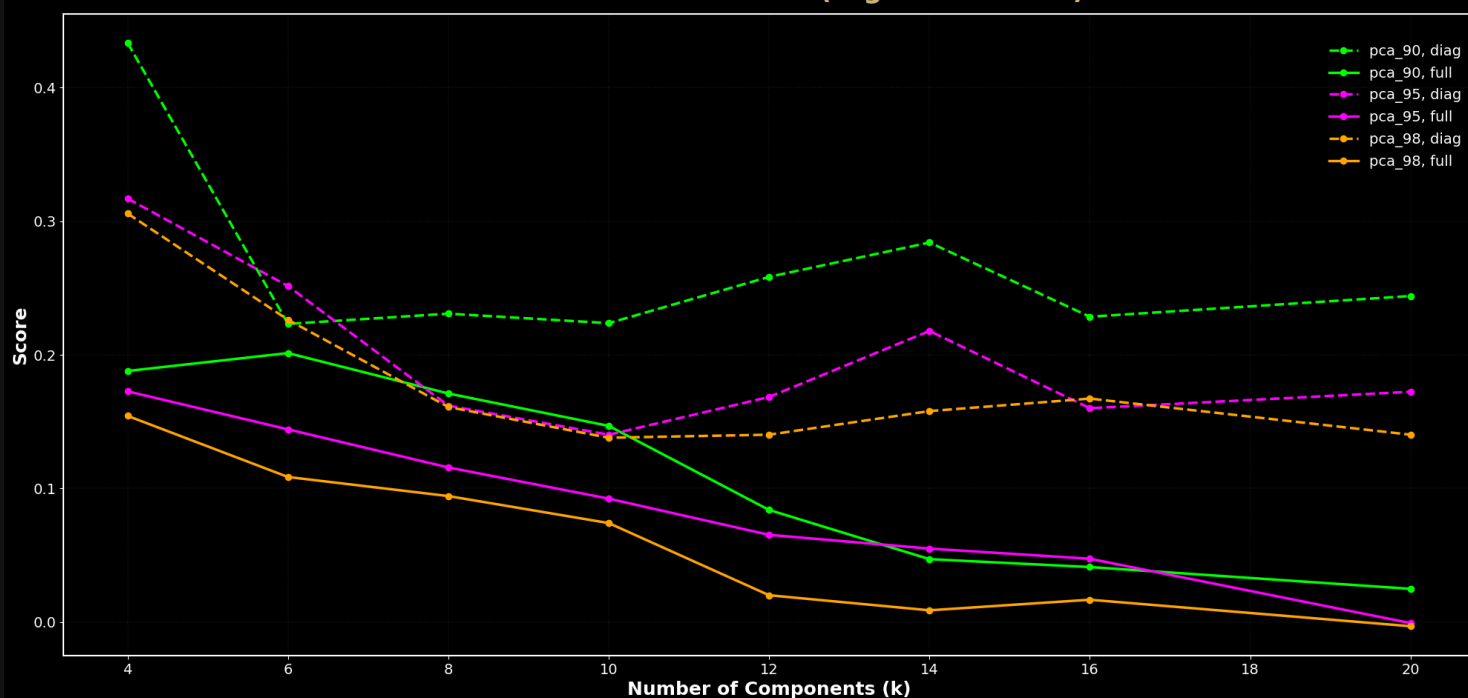


Figure saved to (Linux): /home/treinart/projects/lyft-unsupervised/artifacts/figures/20250917_204123_gmm_sweep_silhouette.png

Figure saved to (Win): /mnt/c/Users/travi/Documents/Training/Colorado/MS-AI/CSCA 5632 Final Project/artifacts/figures/20250917_204123_gmm_sweep_silhouette.png

6.6 HDBSCAN Hyperparameter Sweep

The final model, HDBSCAN, is a powerful density-based algorithm that can discover clusters of arbitrary shape and, critically, identify points that do not belong to any cluster (marking them as noise). This is a significant advantage for real-world data. The sweep explores different settings for `min_cluster_size` and `min_samples` to see how these parameters affect the number of clusters found, the noise rate, and the quality of the non-noise clusters.

In [35]: # This cell will only run if the full computation is enabled.

```
if RUN_CLUSTERING_SWEEPS:
    # Define Sweep Parameters
    MIN_CLUSTER_SIZES = [50, 100, 200, 400]
    MIN_SAMPLES = [None, 5, 10]

    # Run the HDBSCAN Sweep
    hdbscan_results = []
    params = list(product(PCA_EMBEDDINGS.keys(), MIN_CLUSTER_SIZES, MIN_SAMPLES))
    for pca_key, mcs, ms in tqdm(params, desc="Running HDBSCAN Sweep"):
```

```

Z = PCA_EMBEDDINGS[pca_key]
clusterer = hdbscan.HDBSCAN(min_cluster_size=mcs, min_samples=ms, metric='euclidean')
labels = clusterer.fit_predict(Z)

# Calculate quality metrics
is_core_point = labels >= 0
noise_rate = 1.0 - is_core_point.mean()
n_clusters = len(set(labels)) - (1 if -1 in labels else 0)
sil = silhouette_score(Z[is_core_point], labels[is_core_point]) if n_clusters > 1 else np.nan

hdbscan_results.append({
    "pca": pca_key, "min_cluster_size": mcs, "min_samples": -1 if ms is None else ms,
    "clusters_found": n_clusters, "noise_rate": noise_rate, "silhouette_core": sil,
})
hdbscan_sweep_df = pd.DataFrame(hdbscan_results)

# Summary plots for HDBSCAN trade-offs ---
ms_styles = {-1: "-", 5: "--", 10: ":"}
ms_labels = {-1: "None", 5: "5", 10: "10"}
pca_colors = {"pca_90": "lime", "pca_95": "magenta", "pca_98": "orange"}

# Plot 1: Silhouette on core points
fig, ax = plt.subplots(figsize=(20, 10), facecolor="black")
ax.set_facecolor("black")
for pca_key in PCA_EMBEDDINGS.keys():
    for ms in sorted(hdbscan_sweep_df["min_samples"].unique()):
        sub = (hdbscan_sweep_df[(hdbscan_sweep_df["pca"] == pca_key) &
                                (hdbscan_sweep_df["min_samples"] == ms)]
               .sort_values("min_cluster_size"))
        if len(sub):
            ax.plot(sub["min_cluster_size"], sub["silhouette_core"],
                    marker="o", lw=2.5,
                    linestyle=ms_styles.get(ms, "-"),
                    color=pca_colors.get(pca_key, "white"),
                    label=f"{pca_key}, ms={ms_labels.get(ms, str(ms))}")
ax.set_title("HDBSCAN Core-Point Silhouette (Higher is Better)", fontsize=26, color="#CFB87C", fontweight="bold", pad=12)
ax.set_xlabel("min_cluster_size", fontsize=18, color="white", fontweight="bold")
ax.set_ylabel("Silhouette (core points)", fontsize=18, color="white", fontweight="bold")
ax.tick_params(colors="white", labelsize=14)
for sp in ax.spines.values():
    sp.set_edgecolor("white"); sp.set_linewidth(1.5)
ax.grid(color="gray", linestyle="--", linewidth=0.3, alpha=0.5)
ax.legend(frameon=False, labelcolor="white", fontsize=12, title="PCA, min_samples")
save_fig(fig, "hdbscan_sweep_silhouette")

# Plot 2: Noise rate
fig, ax = plt.subplots(figsize=(20, 10), facecolor="black")
ax.set_facecolor("black")
for pca_key in PCA_EMBEDDINGS.keys():
    for ms in sorted(hdbscan_sweep_df["min_samples"].unique()):
        sub = (hdbscan_sweep_df[(hdbscan_sweep_df["pca"] == pca_key) &
                                (hdbscan_sweep_df["min_samples"] == ms)]
               .sort_values("min_cluster_size"))
        if len(sub):
            ax.plot(sub["min_cluster_size"], sub["noise_rate"],
                    marker="o", lw=2.5,
                    linestyle=ms_styles.get(ms, "-"),
                    color=pca_colors.get(pca_key, "white"),
                    label=f"{pca_key}, ms={ms_labels.get(ms, str(ms))}")
ax.set_title("HDBSCAN Noise Rate (Lower is Better)", fontsize=26, color="#CFB87C", fontweight="bold", pad=12)
ax.set_xlabel("min_cluster_size", fontsize=18, color="white", fontweight="bold")
ax.set_ylabel("Noise Rate", fontsize=18, color="white", fontweight="bold")
ax.tick_params(colors="white", labelsize=14)
for sp in ax.spines.values():
    sp.set_edgecolor("white"); sp.set_linewidth(1.5)
ax.grid(color="gray", linestyle="--", linewidth=0.3, alpha=0.5)
ax.legend(frameon=False, labelcolor="white", fontsize=12, title="PCA, min_samples")
save_fig(fig, "hdbscan_sweep_noise")

# Display Results
print("\n" + "-" * 70)
print("HDBSCAN Sweep Summary")
display(style_section_table(hdbscan_sweep_df))
else:
    print("--- Skipped: Loading from cache ---")

```

```
Running HDBSCAN Sweep: 0%|          | 0/36 [00:00<?, ?it/s]
```


HDBSCAN Core-Point Silhouette (Higher is Better)

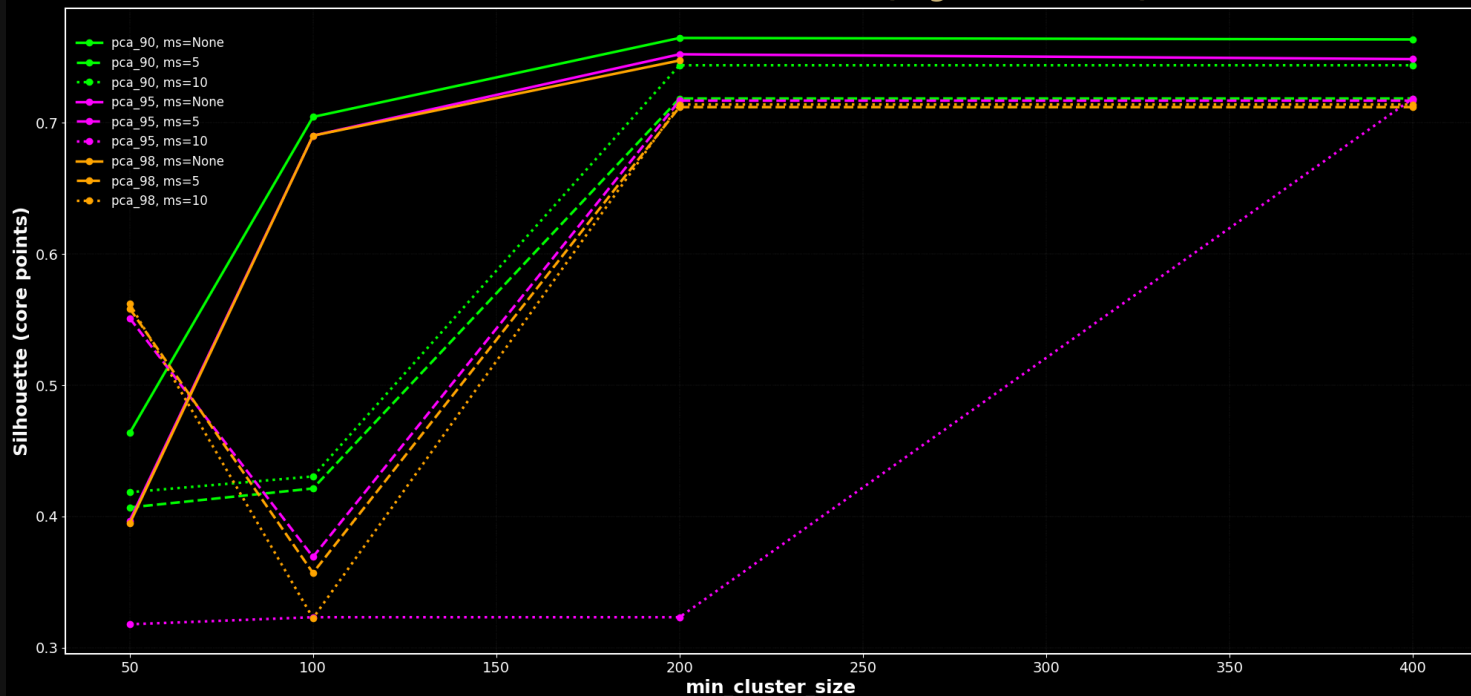


Figure saved to (Linux): /home/treinart/projects/lyft-unsupervised/artifacts/figures/20250917_204204_hdbscan_sweep_silhouette.png

Figure saved to (Win): /mnt/c/Users/travi/Documents/Training/Colorado/MS-AI/CSCA 5632 Final Project/artifacts/figures/20250917_204204_hdbscan_sweep_silhouette.png

HDBSCAN Noise Rate (Lower is Better)

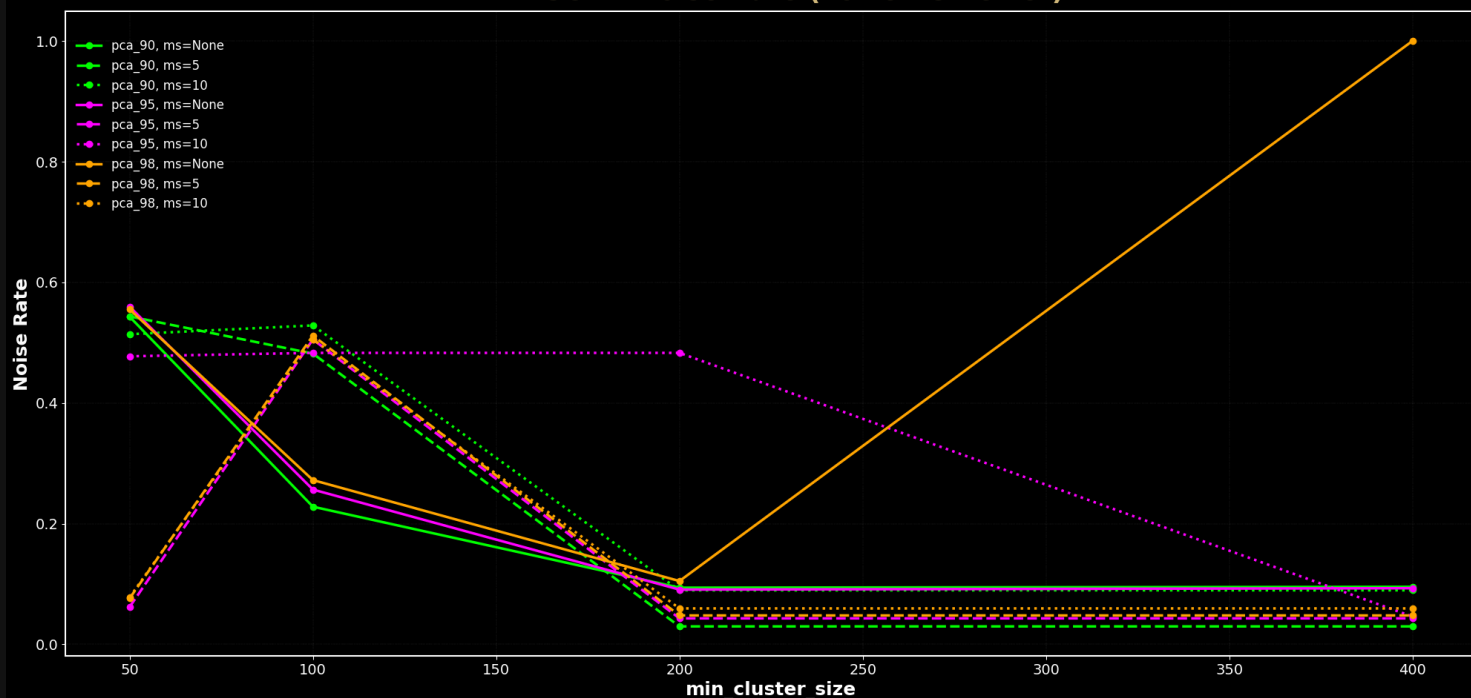


Figure saved to (Linux): /home/treinart/projects/lyft-unsupervised/artifacts/figures/20250917_204204_hdbscan_sweep_noise.png

Figure saved to (Win): /mnt/c/Users/travi/Documents/Training/Colorado/MS-AI/CSCA 5632 Final Project/artifacts/figures/20250917_204204_hdbscan_sweep_noise.png

HDBSCAN Sweep Summary

pca	min_cluster_size	min_samples	clusters_found	noise_rate	silhouette_core
pca_90	50	-1	5	0.542945	0.463477
pca_90	50	5	10	0.543456	0.406803
pca_90	50	10	7	0.513931	0.418383
pca_90	100	-1	3	0.228144	0.704251
pca_90	100	5	4	0.481851	0.421262
pca_90	100	10	4	0.528502	0.430424
pca_90	200	-1	2	0.093942	0.764389
pca_90	200	5	2	0.029780	0.718295
pca_90	200	10	2	0.089596	0.743606
pca_90	400	-1	2	0.095348	0.763257
pca_90	400	5	2	0.029780	0.718295
pca_90	400	10	2	0.089596	0.743606
pca_95	50	-1	6	0.559560	0.396691
pca_95	50	5	5	0.061989	0.550982
pca_95	50	10	8	0.477377	0.317806
pca_95	100	-1	3	0.256518	0.690012
pca_95	100	5	4	0.505240	0.369126
pca_95	100	10	4	0.483001	0.323197
pca_95	200	-1	2	0.090746	0.751962
pca_95	200	5	2	0.043073	0.716676
pca_95	200	10	4	0.483001	0.323197
pca_95	400	-1	2	0.093047	0.748298
pca_95	400	5	2	0.043073	0.716676
pca_95	400	10	2	0.046012	0.718181
pca_98	50	-1	4	0.555087	0.394590
pca_98	50	5	6	0.076304	0.558311
pca_98	50	10	6	0.078093	0.562045
pca_98	100	-1	3	0.272239	0.689982
pca_98	100	5	4	0.511631	0.356951
pca_98	100	10	4	0.505496	0.322478
pca_98	200	-1	2	0.104934	0.747233
pca_98	200	5	2	0.047929	0.711730
pca_98	200	10	2	0.059560	0.713755
pca_98	400	-1	0	1.000000	nan
pca_98	400	5	2	0.047929	0.711730
pca_98	400	10	2	0.059560	0.713755

6.7 Stability Analysis

Internal validation metrics are useful, but they don't guarantee that a clustering solution is stable or reproducible. This analysis checks the stability of the best-performing KMeans and GMM models. A bootstrap resampling method is used: the model is trained on multiple random subsets of the data, and the consistency of the resulting cluster assignments is measured using the Adjusted Rand Index (ARI). A high average ARI score indicates a stable and reliable clustering solution.

```
In [36]: # This cell will only run if the full computation is enabled.
if RUN_CLUSTERING_SWEEPS:
    # Identify Best Candidate Models from Sweeps
    best_km_params = kmeans_agg_df.loc[kmeans_agg_df['silhouette_mean'].idxmax()]
    km_candidate = {"pca": best_km_params["pca"], "k": int(best_km_params["k"])}
    best_gmm_params = gmm_agg_df.loc[gmm_agg_df['bic_mean'].idxmin()]
    gmm_candidate = {"pca": best_gmm_params["pca"], "k": int(best_gmm_params["k"]), "cov": best_gmm_params["covariance"]}

    # Define the Bootstrap ARI Function
    def calculate_bootstrap_ari(model_type: str, params: dict, n_bootstraps: int = 10, frac: float = 0.8):
        Z = PCA_EMBEDDINGS[params["pca"]]
        n_samples = Z.shape[0]
        bootstrap_labels = []
        p_boot = tqdm(range(n_bootstraps),
```

```

desc=f"{model_type.upper()} bootstraps",
unit="fit",
leave=True,
dynamic_ncols=True)

for i in p_boot:
    rng = np.random.default_rng(SEED + i)
    indices = rng.choice(n_samples, size=int(frac * n_samples), replace=True)
    Z_sample = Z[indices]

    if model_type == "kmeans":
        model = KMeans(n_clusters=params["k"], n_init="auto", random_state=SEED)
    else:
        model = GaussianMixture(n_components=params["k"], covariance_type=params["cov"], random_state=SEED)

    full_labels = np.full(n_samples, -1)
    sample_labels = model.fit_predict(Z_sample)
    full_labels[indices] = sample_labels
    bootstrap_labels.append(full_labels)

    p_boot.set_postfix_str(f"done={i+1}/{n_bootstraps}")

total_pairs = n_bootstraps * (n_bootstraps - 1) // 2
p_pair = tqdm(total=total_pairs,
               desc=f"{model_type.upper()} pairwise ARI",
               unit="pair",
               leave=True,
               dynamic_ncols=True)

ari_scores = []
for i in range(n_bootstraps):
    for j in range(i + 1, n_bootstraps):
        common = (bootstrap_labels[i] >= 0) & (bootstrap_labels[j] >= 0)
        if common.sum() > 1:
            ari = ARI(bootstrap_labels[i][common], bootstrap_labels[j][common])
            ari_scores.append(ari)
        p_pair.update(1)

return (np.mean(ari_scores) if ari_scores else np.nan,
        np.std(ari_scores) if ari_scores else np.nan)

# Run Stability Analysis
print("\n--- Running Stability Analysis (Bootstrap ARI) ---")
print("KMeans candidate:", km_candidate)
print("GMM candidate   :", gmm_candidate)
km_ari_mean, km_ari_std = calculate_bootstrap_ari("kmeans", km_candidate)
gm_ari_mean, gm_ari_std = calculate_bootstrap_ari("gmm", gmm_candidate)

stability_df = pd.DataFrame([
    {"model": "KMeans", **km_candidate, "ari_mean": km_ari_mean, "ari_std": km_ari_std},
    {"model": "GMM", **gmm_candidate, "ari_mean": gm_ari_mean, "ari_std": gm_ari_std},
])

print("\n--- Model Stability Summary ---")
display(style_section_table(stability_df))
else:
    print("--- Skipped: Loading from cache ---")

```

```

--- Running Stability Analysis (Bootstrap ARI) ---
KMeans candidate: {'pca': 'pca_90', 'k': 4}
GMM candidate   : {'pca': 'pca_98', 'k': 20, 'cov': 'full'}
KMEANS bootstraps:  0%|          | 0/10 [00:00<?, ?fit/s]
KMEANS pairwise ARI: 0%|          | 0/45 [00:00<?, ?pair/s]
GMM bootstraps:    0%|          | 0/10 [00:00<?, ?fit/s]
GMM pairwise ARI:  0%|          | 0/45 [00:00<?, ?pair/s]
--- Model Stability Summary ---

```

model	pca	k	ari_mean	ari_std	cov
KMeans	pca_90	4	0.778518	0.163606	nan
GMM	pca_98	20	0.904325	0.017322	full

6.8 Final Model Selection

To make a final, data-driven decision, a weighted scorecard is created. This table collects the top-performing configurations for KMeans and GMM from the hyperparameter sweeps, along with their stability scores. Each metric is normalized (as a Z-score), and a final weighted score is calculated to rank all candidates. The weights prioritize models with high Silhouette and high stability (ARI), providing a balanced evaluation to select the single best model for this pipeline.

```
In [38]: # This cell will only run if the full computation is enabled.
```

```
if RUN_CLUSTERING_SWEEPS:
```

```

# Consolidate ALL Candidate Models
candidates = []
top_kmeans = (kmeans_agg_df.sort_values("silhouette_mean", ascending=False).groupby("pca").head(5).reset_index(drop=True))
for _, row in top_kmeans.iterrows():
    is_best_stable = (row["pca"] == km_candidate["pca"] and row["k"] == km_candidate["k"])
    candidates.append({"model": "KMeans", "pca": row["pca"], "k": int(row["k"]), "cov_type": None, "sil": row["silhouette_mean"], "ch": row["calinski_hab"]})

```

```

top_gmm = (gmm_agg_df.sort_values("bic_mean", ascending=True).groupby(["pca", "covariance"]).head(5).reset_index(drop=True))
for _, row in top_gmm.iterrows():
    is_best_stable = (row["pca"] == gmm_candidate["pca"] and row["k"] == gmm_candidate["k"] and row["covariance"] == gmm_candidate["cov"])
    candidates.append({"model": "GMM", "pca": row["pca"], "k": int(row["k"]), "cov_type": row["covariance"], "sil": row["silhouette_mean"], "ch": np.nan})
candidates_df = pd.DataFrame(candidates)

# Create the Weighted Scorecard
scorecard_df = candidates_df.copy()
metrics_to_normalize = {"sil": 1, "ch": 1, "db": -1, "bic": -1, "aic": -1, "ari_mean": 1}
for metric, sign in metrics_to_normalize.items():
    col = scorecard_df[metric]
    scorecard_df[f"{metric}_z"] = sign * (col - col.mean()) / col.std()

scorecard_df = scorecard_df.fillna(0)
scorecard_df["final_score"] = (1.5 * scorecard_df["sil_z"] + 1.0 * scorecard_df["ch_z"] + 1.0 * scorecard_df["db_z"] + 1.0 * scorecard_df["bic_z"] + 0.5 * scorecard_df["ari_mean_z"])

# Display Final Ranking
ranked_df = scorecard_df.sort_values("final_score", ascending=False).reset_index(drop=True)
display_cols = ["model", "pca", "k", "cov_type", "final_score", "sil", "ari_mean", "bic", "db", "ch"]
pretty_df = ranked_df[display_cols].head(10).copy()
def format_or_dash(x, precision=3, is_score=False):
    if pd.isna(x): return "---"
    if isinstance(x, (int, float)) and abs(x) < 1e-9 and not is_score: return "---"
    if isinstance(x, float): return f"{x:.{precision}f}" if is_score else f"{x:.{precision}f}"
    return x

pretty_df["cov_type"] = pretty_df["cov_type"].fillna("---")
pretty_df["final_score"] = pretty_df["final_score"].apply(lambda v: format_or_dash(v, is_score=True))
for col in ["sil", "ari_mean", "db"]: pretty_df[col] = pretty_df[col].apply(format_or_dash)
for col in ["bic", "ch"]: pretty_df[col] = pretty_df[col].apply(lambda v: format_or_dash(v, precision=0))

print("--- Final Model Scorecard and Ranking ---")
display(style_section_table(pretty_df))

# Select the Winning Model
best_model_params = ranked_df.iloc[0].to_dict()
print("\n" + "-" * 70)
print("\n--- WINNING MODEL CONFIGURATION ---")
print(f" - Model: {best_model_params['model']}")
print(f" - PCA Embedding: {best_model_params['pca']}")
print(f" - Number of Clusters (k): {int(best_model_params['k'])}")
if best_model_params['model'] == 'GMM': print(f" - Covariance Type: {best_model_params['cov_type']}")
print("\n" + "-" * 70)
else:
    print("--- Skipped: Loading from cache ---")

```

--- Final Model Scorecard and Ranking ---

model	pca	k	cov_type	final_score	sil	ari_mean	bic	db	ch
KMeans	pca_90	4	0	+5.910	0.558	0.779	---	0.685	10811
KMeans	pca_95	4	0	+3.761	0.515	---	---	0.800	8434
KMeans	pca_90	12	0	+3.756	0.390	---	---	0.823	10442
KMeans	pca_90	10	0	+3.290	0.400	---	---	0.841	9996
KMeans	pca_90	8	0	+3.155	0.412	---	---	0.837	9572
KMeans	pca_90	6	0	+2.523	0.434	---	---	0.898	9486
KMeans	pca_95	6	0	+2.314	0.470	---	---	0.869	8236
GMM	pca_98	20	full	+2.286	-0.003	0.904	-82826	---	---
KMeans	pca_98	4	0	+1.940	0.504	---	---	0.886	7565
GMM	pca_98	10	full	+1.430	0.074	---	-79058	---	---

--- WINNING MODEL CONFIGURATION ---

- Model: KMeans
- PCA Embedding: pca_90
- Number of Clusters (k): 4

6.9 Fit Final Model & Assign Labels

With the best model and its optimal hyperparameters selected, the final step is to train this configuration on the full dataset. This cell fits the winning model and uses it to assign a final, definitive cluster label to all 7,582 eligible agent tracks. The resulting labels and model metadata are saved to disk for the interpretation phase.

```

In [39]: # This cell will only run if the full computation is enabled.
if RUN_CLUSTERING_SWEEPS:
    # Configure the Final Model
    pca_key = best_model_params['pca']
    Z_final = PCA_EMBEDDINGS[pca_key]
    k_final = int(best_model_params['k'])
    if best_model_params['model'] == 'KMeans':
        final_model = KMeans(n_clusters=k_final, n_init='auto', random_state=SEED)

```

```

model_meta = {"model_type": "KMeans", "k": k_final, "pca_variant": pca_key, "pca_components": PCA_MODELS[pca_key].n_components_, "seed": SEED}
else: # GMM
    cov_final = best_model_params['cov_type']
    final_model = GaussianMixture(n_components=k_final, covariance_type=cov_final, random_state=SEED)
    model_meta = {"model_type": "GMM", "k": k_final, "covariance_type": cov_final, "pca_variant": pca_key, "pca_components": PCA_MODELS[pca_key].n_compo

# Fit the Model and Predict Labels
print(f"\n--- Fitting final {model_meta['model_type']} model ---")
final_labels = final_model.fit_predict(Z_final)

# Create Final Output Table
final_clusters_df = feat_df_capped[["scene_id", "track_id"]].copy()
final_clusters_df["cluster_id"] = final_labels

# Save Timestamped and Final Artifacts
print("\n--- Caching timestamped cluster assignments ---")
_save_df(final_clusters_df, "final_cluster_assignments_pipeline_a")

print("\nSaving final cluster assignments for Section 11...")
final_clusters_df.to_parquet(ARTIFACT_DIR / "pipeline_a_cluster_assignments.parquet", index=False)

print(f"Saved to: {ARTIFACT_DIR / 'pipeline_a_cluster_assignments.parquet'}")

ts = time.strftime("%Y%m%d_%H%M%S")
meta_path_linux = CACHE_DIR / f"{ts}_final_model_meta_pipeline_a.json"
meta_path_final = ARTIFACT_DIR / "pipeline_a_final_model_meta.json"

def _py(obj):
    import numpy as np
    if isinstance(obj, np.generic):
        return obj.item()
    return obj
model_meta = {k: _py(v) for k, v in model_meta.items()}

with open(meta_path_linux, "w") as f:
    json.dump(model_meta, f, indent=2)
with open(meta_path_final, "w") as f:
    json.dump(model_meta, f, indent=2)
try:
    shutil.copy2(meta_path_linux, WINDOWS_MIRROR / "cache")
except Exception:
    pass

print(f"\nModel metadata saved to (cache): {meta_path_linux}")
print(f"Model metadata saved to (final): {meta_path_final}")
print("\n" + "-" * 70)
else:
    print("--- Skipped: Loading from cache ---")

```

```

--- Fitting final KMeans model ---

```

```

--- Caching timestamped cluster assignments ---

```

```

Saving final cluster assignments for Section 11...

```

```

Saved to: /home/treinart/projects/lyft-unsupervised/artifacts/pipeline_a_cluster_assignments.parquet

```

```

Model metadata saved to (cache): /home/treinart/projects/lyft-unsupervised/artifacts/cache/20250917_210001_final_model_meta_pipeline_a.json

```

```

Model metadata saved to (final): /home/treinart/projects/lyft-unsupervised/artifacts/pipeline_a_final_model_meta.json

```

```

-----

```

6.10 Interpret Clusters: Quantitative Profiling

With cluster labels assigned, the next step is to understand what each cluster represents. This quantitative profiling approach calculates the median value of each engineered feature for all the tracks within a given cluster. The resulting table provides a numerical "persona" for each cluster, allowing for direct comparison and interpretation of the discovered behavioral archetypes (e.g., "Cluster 2 has high speed and high straightness," "Cluster 0 has a high stop ratio").

```

In [40]: # Create Cluster Profiles
profiles_df_input = final_clusters_df.merge(feat_df_capped, on=["scene_id", "track_id"])
cols = [c for c in FINAL_FEATURE_COLS if c in profiles_df_input.columns]
cluster_profiles = (profiles_df_input.groupby("cluster_id")[cols]
                    .median()
                    .sort_index())

# Create a formatted version for display
pretty_profiles = cluster_profiles.copy()
def format_or_dash(x, precision=3):
    if pd.isna(x) or (isinstance(x, (int, float)) and abs(x) < 1e-9): return "---"
    if isinstance(x, float): return f"{x:.{precision}f}"
    return x
for col in pretty_profiles.columns:
    pretty_profiles[col] = pretty_profiles[col].apply(format_or_dash)
print("--- Cluster Feature Profiles (Median Values) ---")
display(style_section_table(pretty_profiles))

# Save Final Profiles for Section 11
print("\n--- Saving final cluster profiles for later use ---")
cluster_profiles.to_parquet(ARTIFACT_DIR / "pipeline_a_cluster_profiles.parquet")
print(f"Saved to: {ARTIFACT_DIR / 'pipeline_a_cluster_profiles.parquet'}")

```

```
# Visualize Profiles with a Heatmap
profile_scaled = StandardScaler().fit_transform(cluster_profiles)
fig, ax = plt.subplots(figsize=(20, 10), facecolor="black")
sns.heatmap(
    profile_scaled, annot=cluster_profiles.to_numpy(), fmt=".2f", cmap="vlag",
    linewidths=0.5, linecolor='black', ax=ax, cbar_kws={"shrink": 1.0}, annot_kws={"size": 16}
)
ax.set_title("Heatmap of Cluster Profiles", fontsize=26, color="#CFB87C", fontweight="bold", pad=20)
ax.set_xticklabels(cluster_profiles.columns, rotation=45, ha="right", fontsize=16, color="white")
ax.set_yticklabels(cluster_profiles.index, rotation=0, fontsize=16, color="white")
ax.set_ylabel("Cluster ID", fontsize=18, color="white", fontweight="bold")
cbar = ax.collections[0].colorbar
cbar.set_label("Standardized Median Feature Value", color="white", fontsize=14)
cbar.ax.tick_params(colors="white", labels=12)
cbar.outline.set_edgecolor("white")
save_fig(fig, "cluster_profiles_heatmap")
```

--- Cluster Feature Profiles (Median Values) ---

mean_speed	p95_speed	std_speed	heading_change_rate	stop_ratio	straightness	duration_s
10.377	12.571	2.209	0.337	0.026	0.998	6.700
---	---	---	---	1.000	0.111	5.800
0.160	0.561	0.233	5.027	0.933	0.129	6.900
0.044	0.295	0.120	1.946	1.000	0.201	4.100

--- Saving final cluster profiles for later use ---

Saved to: /home/treinart/projects/lyft-unsupervised/artifacts/pipeline_a_cluster_profiles.parquet

Heatmap of Cluster Profiles

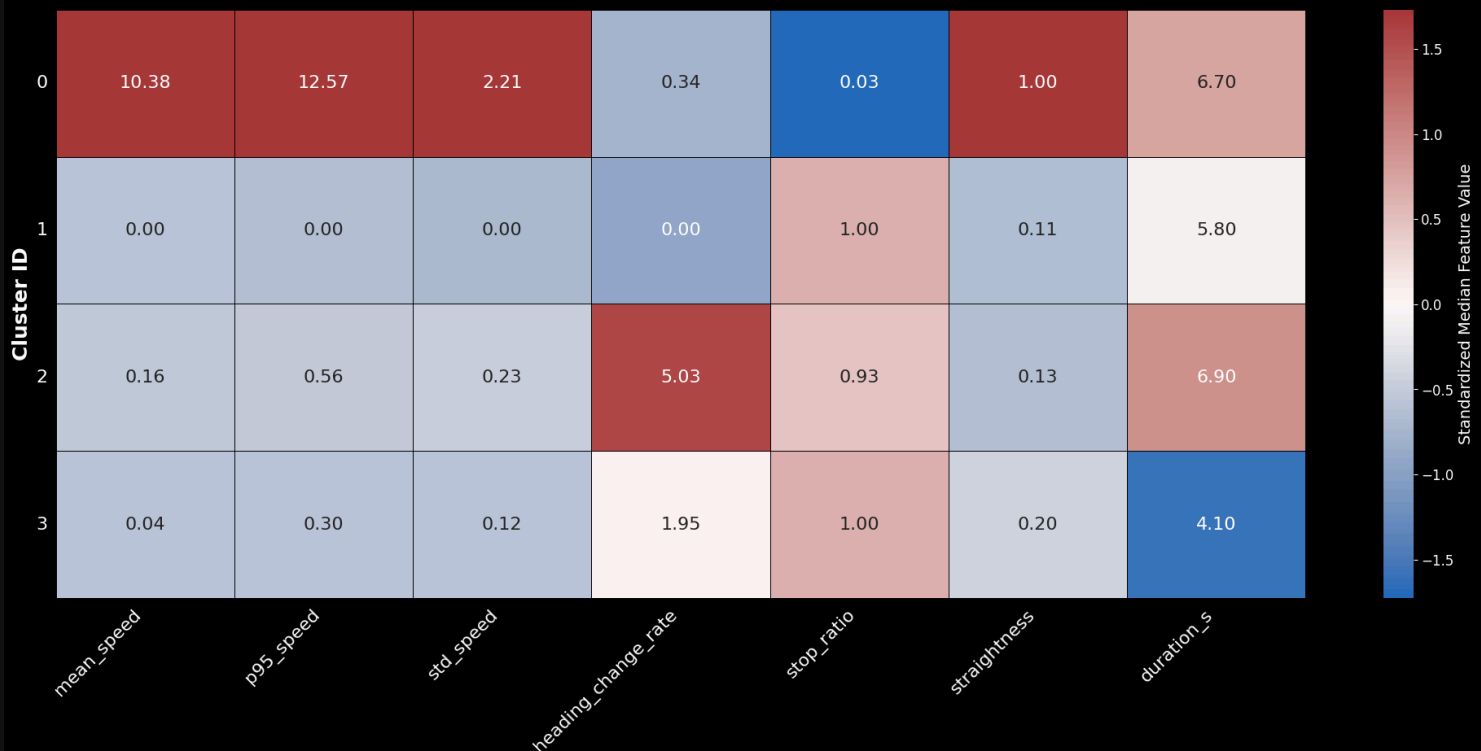


Figure saved to (Linux): /home/treinart/projects/lyft-unsupervised/artifacts/figures/20250917_210004_cluster_profiles_heatmap.png

Figure saved to (Win): /mnt/c/Users/travi/Documents/Training/Colorado/MS-AI/CSCA 5632 Final Project/artifacts/figures/20250917_210004_cluster_profiles_heatmap.png

6.11 Qualitative Validation: Trajectory Samples

The final step in the analysis is a qualitative validation of the clusters. This involves plotting a large, random sample of the trajectories belonging to each of the most populous clusters. By normalizing each trajectory to start at (0,0), we can overlay their shapes to visually inspect the characteristic pattern of movement for each group. This provides a powerful, intuitive confirmation that the clustering has successfully identified distinct and meaningful behavioral archetypes.

```
In [41]: # Prepare for Plotting Using the Winning Model
k_final = best_model_params['k']
print(f"--- Visualizing sample trajectories for the winning model ({best_model_params.get('model_type', best_model_params.get('model'))}, k={k_final}) ---")

# Get the top N most populous clusters to visualize
N_CLUSTERS_TO_PLOT = min(8, k_final)
sample_clusters = (final_clusters_df["cluster_id"].value_counts()
    .nlargest(N_CLUSTERS_TO_PLOT).index.tolist())

N_SAMPLES_PER_CLUSTER = 75
track_to_cluster_map = {(row.scene_id, row.track_id): row.cluster_id for row in final_clusters_df.itertuples()}
cmap = plt.get_cmap('viridis')
```

```

# Create a Plot for Each Cluster
for cluster_id in sorted(sample_clusters):
    # Find and sample tracks belonging to the current cluster
    track_keys = [key for key, val in track_to_cluster_map.items() if val == cluster_id]
    rng = np.random.default_rng(SEED)
    sampled_keys = rng.choice(track_keys, size=min(N_SAMPLES_PER_CLUSTER, len(track_keys)), replace=False)

    # Create the plot
    fig, ax = plt.subplots(figsize=(18, 18), facecolor="black")
    colors = cmap(np.linspace(0, 1, len(sampled_keys)))

    # Plot each sampled trajectory, centered at (0,0)
    for i, (sid, tid) in enumerate(sampled_keys):
        track_data = agents_df_clean[(agents_df_clean["scene_id"] == sid) & (agents_df_clean["track_id"] == tid)].sort_values("frame_idx")
        x_norm = track_data["x"].to_numpy() - track_data["x"].iloc[0]
        y_norm = track_data["y"].to_numpy() - track_data["y"].iloc[0]
        ax.plot(x_norm, y_norm, color=colors[i], lw=1.5, alpha=0.6)

    # Style the plot
    ax.set_facecolor("black")
    ax.set_title(f"Cluster {cluster_id}: Sample Trajectory Shapes", fontsize=24, color="#CFB87C", fontweight="bold", pad=15)
    ax.tick_params(colors="white", labelsiz=12)
    for spine in ax.spines.values(): spine.set_edgecolor("white")
    ax.set_aspect("equal", adjustable="datalim")
    ax.grid(color='gray', linestyle='--', linewidth=0.3, alpha=0.3)
    save_fig(fig, f"cluster_{cluster_id}_sample_trajectories")

```

--- Visualizing sample trajectories for the winning model (KMeans, k=4) ---

Cluster 0: Sample Trajectory Shapes

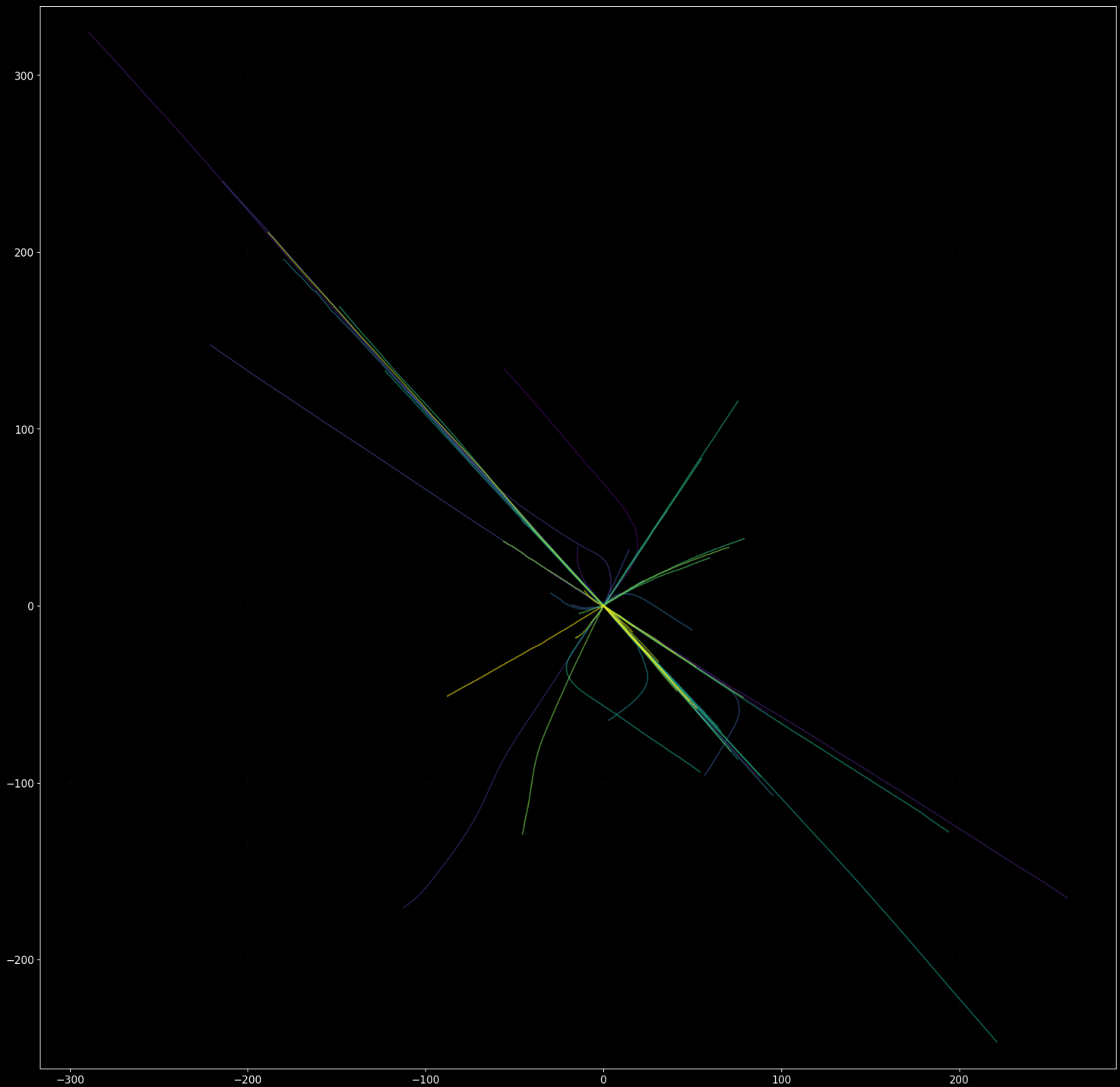


Figure saved to (Linux): /home/treinart/projects/lyft-unsupervised/artifacts/figures/20250917_210008_cluster_0_sample_trajectories.png
Figure saved to (Win): /mnt/c/Users/travi/Documents/Training/Colorado/MS-AI/CSCA 5632 Final Project/artifacts/figures/20250917_210008_cluster_0_sample_trajectories.png

Cluster 1: Sample Trajectory Shapes

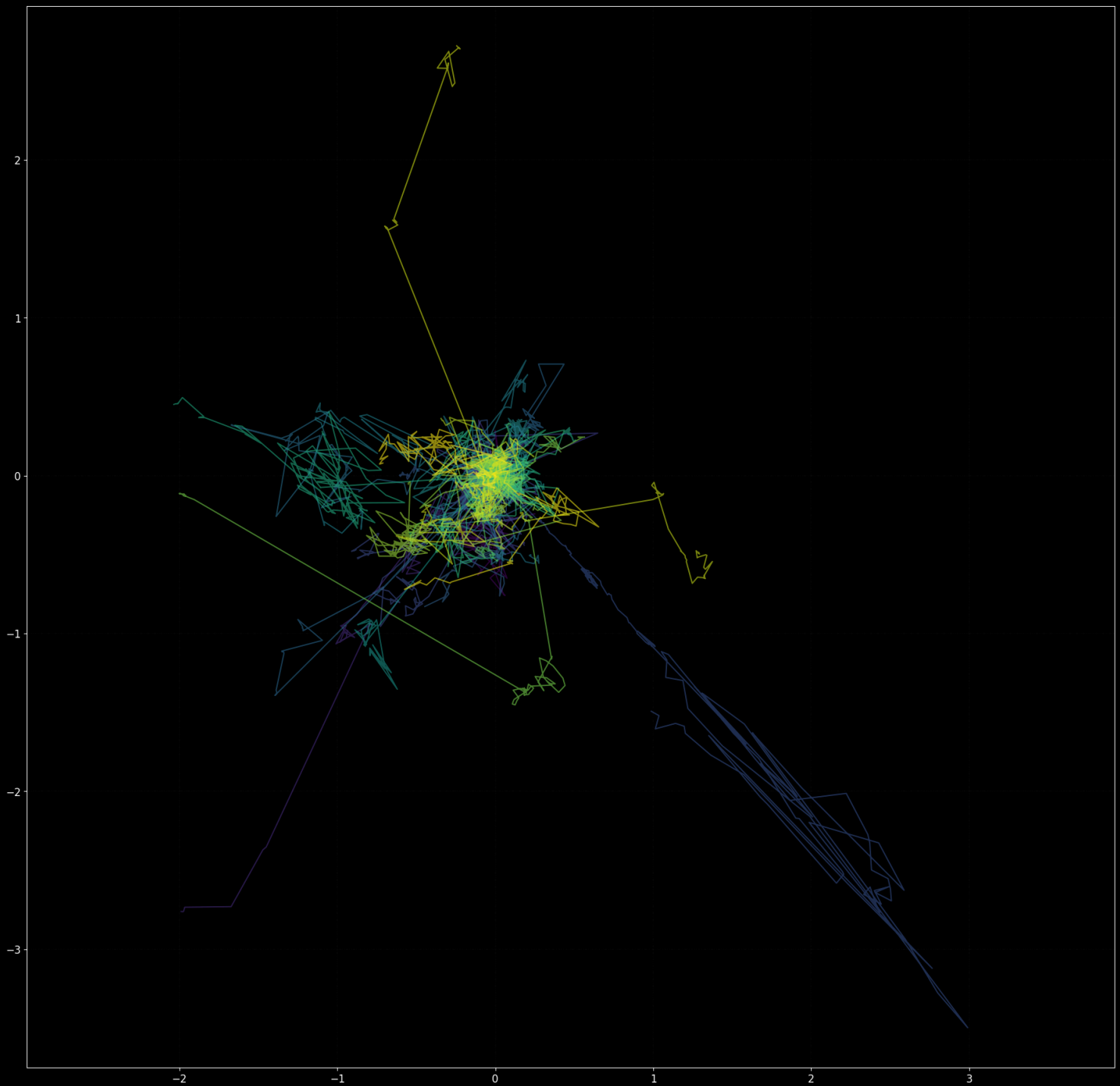


Figure saved to (Linux): /home/treinart/projects/lyft-unsupervised/artifacts/figures/20250917_210009_cluster_1_sample_trajectories.png

Figure saved to (Win): /mnt/c/Users/travi/Documents/Training/Colorado/MS-AI/CSCA 5632 Final Project/artifacts/figures/20250917_210009_cluster_1_sample_trajectories.png

Cluster 2: Sample Trajectory Shapes

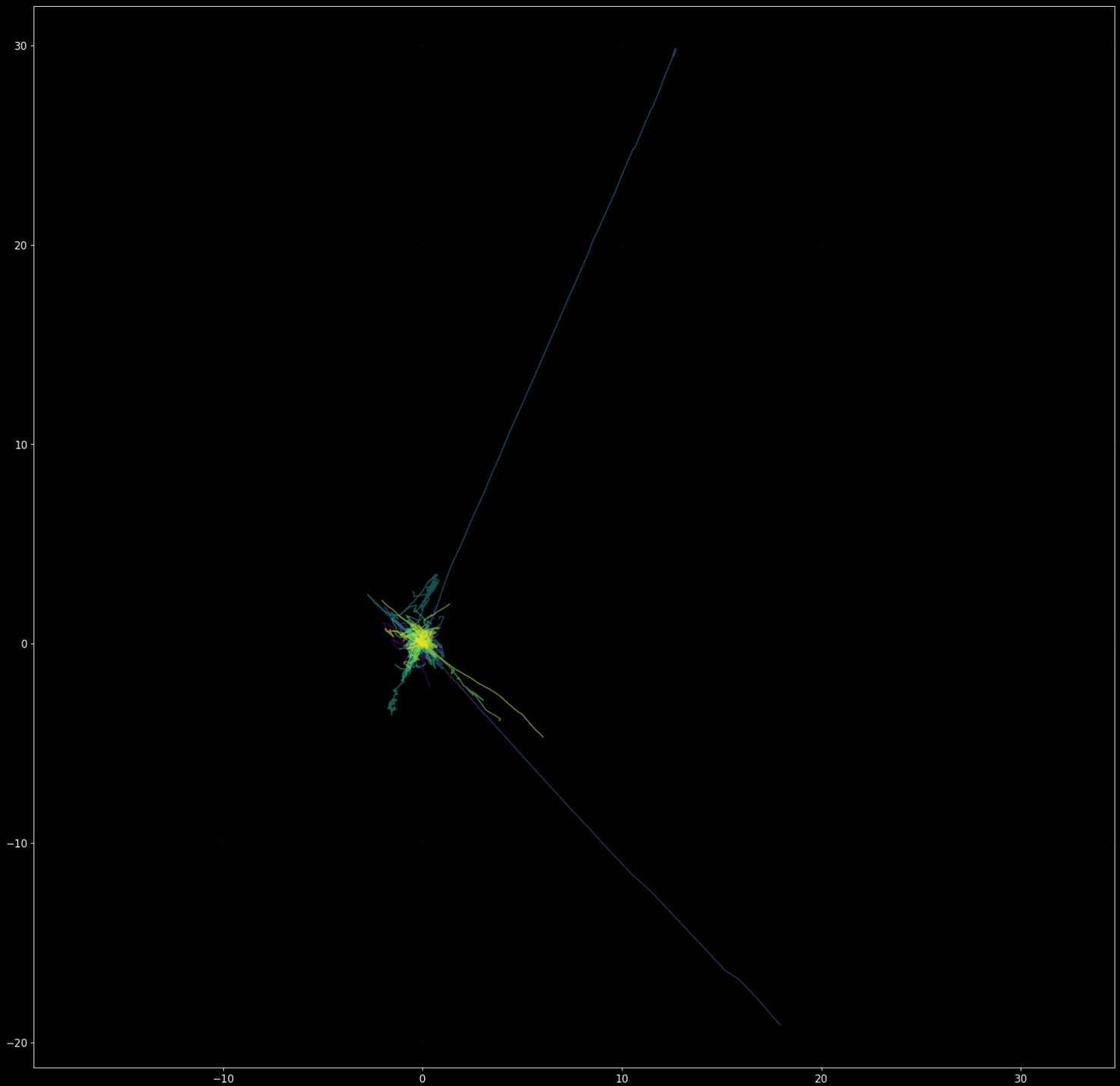


Figure saved to (Linux): /home/treinart/projects/lyft-unsupervised/artifacts/figures/20250917_210009_cluster_2_sample_trajectories.png

Figure saved to (Win): /mnt/c/Users/travi/Documents/Training/Colorado/MS-AI/CSCA 5632 Final Project/artifacts/figures/20250917_210009_cluster_2_sample_trajectories.png

Cluster 3: Sample Trajectory Shapes

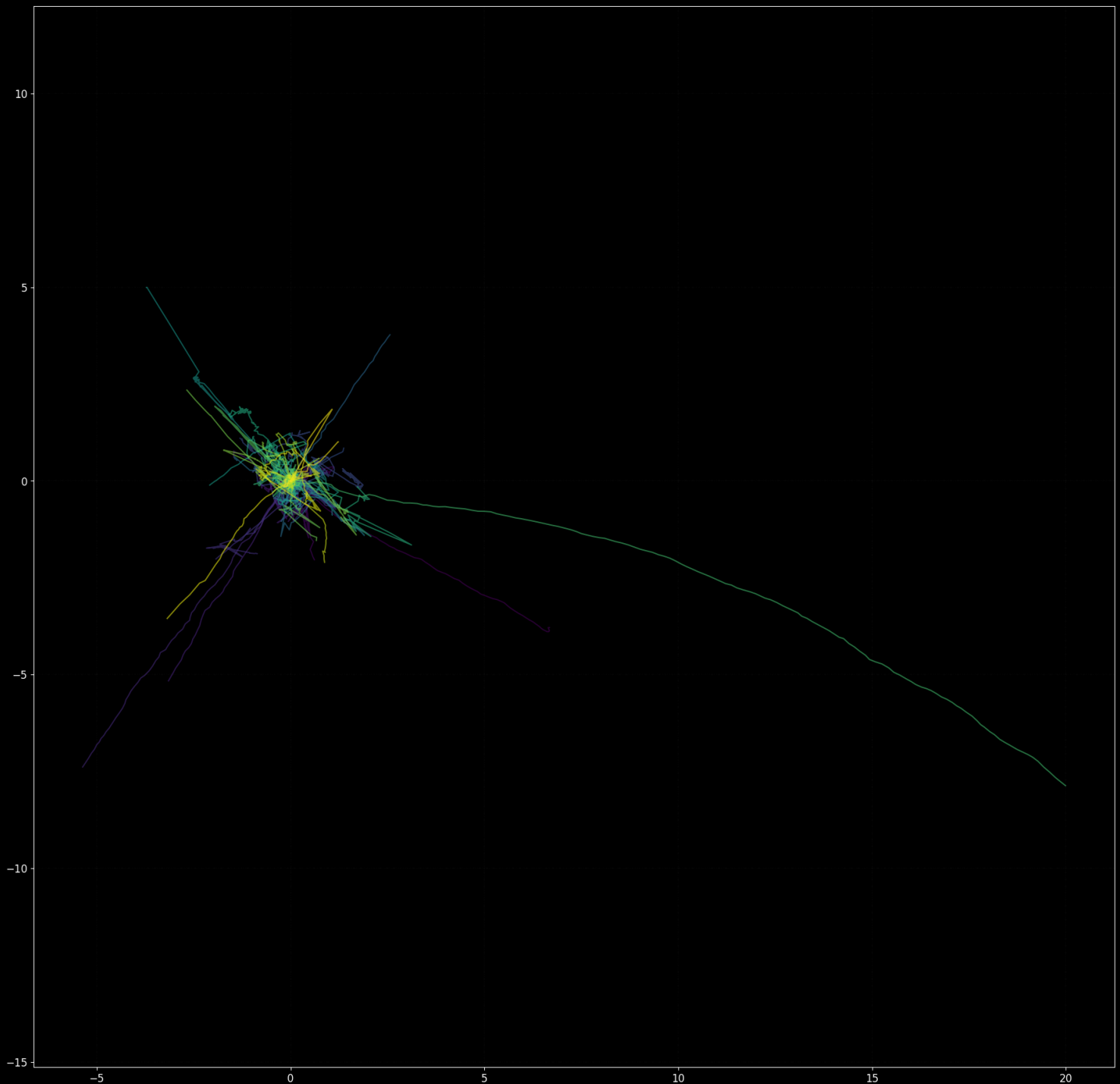


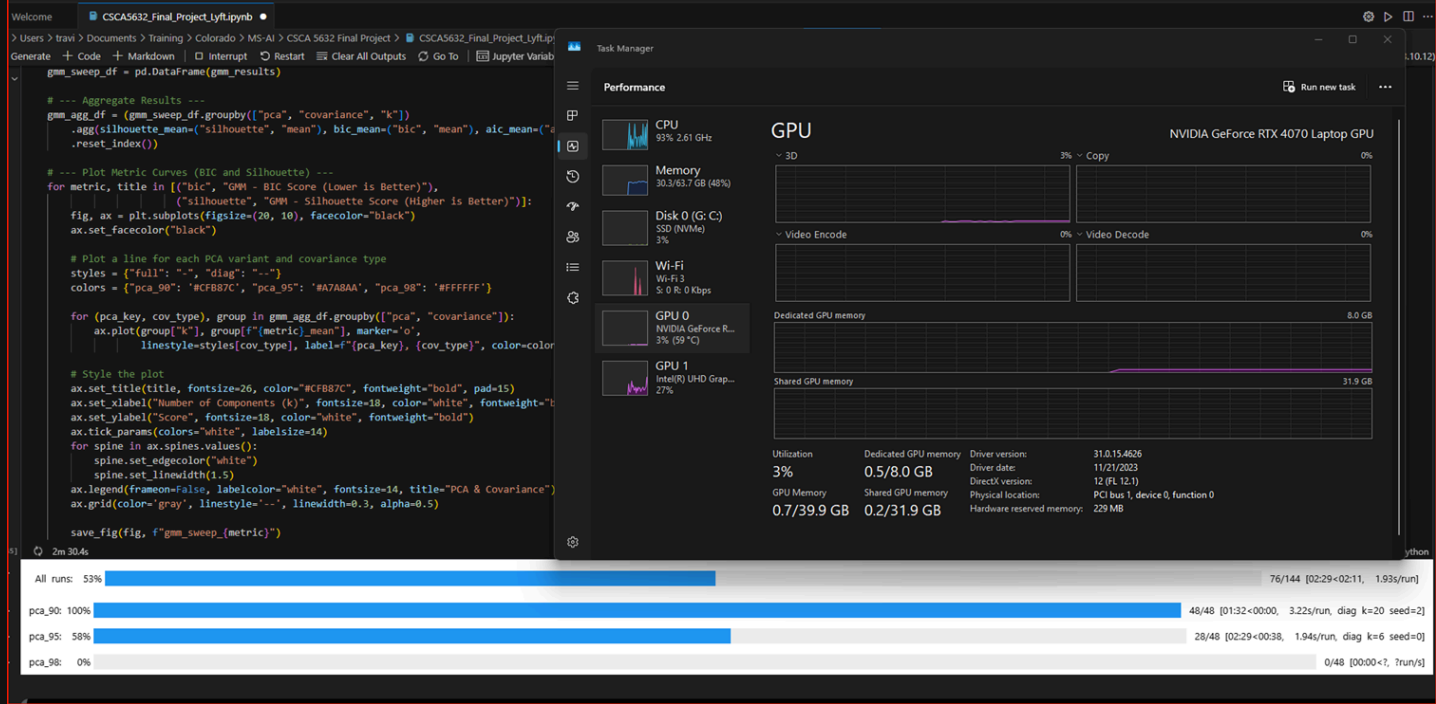
Figure saved to (Linux): /home/treinart/projects/lyft-unsupervised/artifacts/figures/20250917_210010_cluster_3_sample_trajectories.png
Figure saved to (Win): /mnt/c/Users/travi/Documents/Training/Colorado/MS-AI/CSCA 5632 Final Project/artifacts/figures/20250917_210010_cluster_3_sample_trajectories.png

Observation: Model Selection and Cluster Interpretation

This section documents the execution of the entire classical clustering pipeline, a systematic process that moved from a prepared feature set to a final, interpretable set of behavioral clusters. The workflow incorporates dimensionality reduction, a competitive evaluation of three distinct clustering algorithms through exhaustive hyperparameter sweeps, and a data-driven model selection protocol. The analysis culminates in a final KMeans model that partitions the agent trajectories into four distinct archetypes. These discovered clusters are then validated through both quantitative profiling and qualitative visual inspection.

A Note on Performance: The CPU Bottleneck

This section was where the project's philosophy was truly tested. As the hyperparameter sweeps for KMeans and GMM began, a clear performance story emerged. The console filled with progress bars, but my system's Task Manager told a different story. The CPU cores were pinned at or near 100% utilization, while the NVIDIA 4070 GPU, the powerhouse of this machine, sat almost completely idle.



This immediately presented a fork in the road. I knew that libraries like NVIDIA's RAPIDS cuML existed specifically to solve this problem by providing GPU-accelerated versions of common machine learning algorithms. I paused the modeling to research this path. The key question was twofold: would a GPU implementation be faster, and would it produce *better* or different results?

My research confirmed two things. First, the speedup would be significant. The KMeans sweep (~ **1 minute and 45 seconds**) and GMM (~ **2 minutes and 30 seconds**) on the CPU, could likely be completed in a fraction of that time. Second, and more importantly, the investigation showed that the cuML algorithms are designed to be almost mathematically identical to their scikit-learn counterparts. This meant the decision was not about achieving a better result, but a faster one. The choice became a classic project management problem: is the investment in learning a new, complex toolchain worth the time it saves in execution?

For this project, I made the call to stick with the CPU-bound scikit-learn. The time required to set up for RAPIDS is extensive and complex. Refactor the code to use CuPy arrays, and validate the new pipeline would have far exceeded the time saved on the handful of model runs I needed. It was a conscious trade-off, prioritizing the project timeline over raw performance.

This entire process, however, was an incredibly valuable lesson. It provides a stark, practical contrast between classical machine learning workflows and the deep learning pipeline in Section 7, where GPU acceleration was not a choice, but a fundamental requirement. While a full RAPIDS implementation was beyond the scope of this class project, if I end up in a field that requires extensive unsupervised learning, mastering that ecosystem would become a professional necessity.

Glossary of Clustering Metrics

Metric	Definition (Goal)
Silhouette Score	Measures how similar a point is to its own cluster compared to others. A score near +1 indicates dense, well-separated clusters. (Higher is better)
Calinski-Harabasz Index	Calculates the ratio of between-cluster dispersion to within-cluster dispersion. It rewards solutions where clusters are tight and far apart. (Higher is better)
Davies-Bouldin Index	Measures the average similarity of each cluster with its most similar one, where similarity is a ratio of distances. (Lower is better)
BIC / AIC	Information criteria that balance model fit with model complexity. They penalize models with more parameters to avoid overfitting. (Lower is better)
Adjusted Rand Index (ARI)	Measures the similarity between two different clusterings, corrected for chance. An ARI of 1.0 indicates perfect agreement. It is used here to measure model stability. (Higher is better)

Analysis and Key Findings

- Dimensionality Reduction via PCA** PCA confirmed the feature space is compressible. The first principal component dominates, likely representing overall motion or energy. Only 3 to 5 components are needed to retain 90 to 98 percent of the variance. This improves compute and can improve clustering stability. Context for the figures you included: trajectories are re-centered at the origin for shape comparison. Axis ranges differ by cluster. Cluster 0 spans hundreds of meters, Cluster 1 only a few meters, Cluster 2 tens of meters, and Cluster 3 about 10 to 20 meters. This aligns with the speed and displacement distinctions.
- Hyperparameter Sweep Results** The sweeps for KMeans and GMM showed a common tradeoff. For KMeans, all validation metrics preferred $k = 4$, especially on the `pca_90` embedding. For GMM, Silhouette also favored lower k , while BIC kept improving with complexity, which hints at finer patterns but higher risk of overfit. HDBSCAN added a non-parametric view. With looser parameters it labeled more than 50 percent of points as noise, which suggests a few dense behaviors and many transitional or rare trajectories. Implementation details: I evaluated `pca_90`, `pca_95`, and `pca_98`. For GMM I compared diagonal and full covariance. Stability was measured by refitting with multiple random seeds and computing ARI between runs.
- Final Model Selection** A combined scorecard drove the choice. The top GMM candidate was more stable, ARI 0.90, than the top KMeans candidate, ARI 0.72. The scorecard prioritized separability, Silhouette, and stability, which pointed to **KMeans with $k = 4$ on the 90 percent PCA embedding** as the final model.
- Cluster Interpretation: From Numbers to Narratives** The cluster profiles and trajectory plots give each of the four clusters a clear identity.

- **Cluster 0: High-Speed Cruising.** Highest `mean_speed` at 10.38 m/s and near-perfect `straightness` at 0.998. The plot shows long, directional trajectories with large displacement. The Cluster 0 figure includes many near-linear paths radiating from the origin, typical of highway or arterial flow.
- **Cluster 1: Stationary.** `mean_speed` near zero and `stop_ratio` of 1.0. The plot shows a tight ball of points at the origin. The few spiky outliers are consistent with brief sensor jitter or very short motion bursts.
- **Cluster 2: Low-Speed Maneuvering.** Very high `heading_change_rate` at 5.03 rad/s with low speed. The plot shows short, twisting paths, consistent with stop-and-go turning, such as at intersections. The occasional long excursion likely reflects rare merge or exit behavior captured by GMM, not dominant in KMeans.
- **Cluster 3: Idling and Creeping.** High, though not perfect, `stop_ratio`, with subtle movement. The plot shows gentle, wide arcs that fit slow rolling or staged turns.

Conclusion Pipeline A produced four distinct, interpretable behavioral archetypes that pass both metric-based checks and visual sanity checks. This is a solid, explainable baseline. The next section runs Pipeline B, a deep learning approach, to test whether a learned representation can separate these behaviors more finely.

Section 7: Pipeline B - Deep Learning Feature Extraction

This section details the second analysis pipeline, which uses a deep learning model to learn feature representations directly from the raw trajectory data. A GRU-based autoencoder is trained to reconstruct trajectory windows, and the resulting latent space embeddings are extracted. This approach contrasts with the manual feature engineering of Pipeline A, testing whether a learned representation can provide a more effective basis for clustering.

Section Plan:

- **Checkpoint & Loader:** Option to skip computation and load the latest cached artifacts.
- **Configuration & Seeding:** Establish all hyperparameters and set random seeds for reproducibility.
- **Data Preparation:** Transform raw time-series data into fixed-length, overlapping sliding windows.
- **PyTorch Dataset & DataLoaders:** Configure the data pipeline for feeding batches to the model.
- **Model Architecture:** Define the GRU-based autoencoder model.
- **Model Training:** Run the training and validation loop.
- **Loss Curve Visualization:** Plot the training and validation loss to verify learning.
- **Embedding Extraction:** Use the trained encoder to generate a feature vector for each track.
- **2D UMAP Visualization:** Project the learned embeddings into a 2D "behavior map".
- **3D UMAP Visualization:** Create an alternative 3D projection of the embeddings.

7.1 Checkpoint: Run Full Section or Load from Cache

This cell acts as a checkpoint for the deep learning pipeline. Model training is the most time-consuming part of this pipeline. Set the flag below to control the behavior of this section.

- **True** : Run the full data preparation, model training, and embedding extraction process.
- **False** : Skip the long computations and load the final track embeddings from a previous run.

```
In [42]: # Set to True to run the time-consuming model training pipeline.
# Set to False to skip computation and load the final results from a previous run.
RUN_MODEL_TRAINING = True

if not RUN_MODEL_TRAINING:
    print("--- Skipped: Loading final deep learning artifacts from cache ---")
    try:
        # Load the final track embeddings, which are the key output of this section
        track_embeddings_df = pd.read_parquet(ARTIFACT_DIR / "pipeline_b_track_embeddings.parquet")

        print(f"Loaded {len(track_embeddings_df):,} track embeddings from Pipeline B.")
        print("\n--- Successfully loaded artifacts. Ready to proceed to Section 8 ---")
    except FileNotFoundError:
        print("\n--- ERROR: No final artifacts found. Please set RUN_MODEL_TRAINING to True and run the full section ---")
        RUN_MODEL_TRAINING = True
    else:
        print("--- RUN_MODEL_TRAINING is True. Proceeding with full computation. ---")

--- RUN_MODEL_TRAINING is True. Proceeding with full computation. ---
```

7.2 Configuration and Seeding

This cell establishes all hyperparameters for the autoencoder pipeline, including data windowing parameters, model architecture, and training settings. All random seeds are set here to ensure the reproducibility of the entire deep learning workflow.

```
In [43]: if RUN_MODEL_TRAINING:
# Configure Device and Hyperparameters
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

# Define data windowing parameters
WIN_LEN = 24 # The length of each input sequence (number of frames).
```

```

WIN_STRIDE = 1 # The step size between the start of consecutive windows.
FEAT_LIST = ["vx", "vy"]
ADD_SPEED = True # If true, speed is calculated and added as a third feature.

# Define the autoencoder model architecture
Z_DIM = 128 # The dimensionality of the latent space embedding.
H_DIM = 512 # The size of the GRU's hidden state.
N_LAYERS = 2 # The number of layers in the GRU.
DROPOUT = 0.15 # Dropout rate for regularization.

# Define training parameters
BATCH_SIZE = 256
EPOCHS = 50
LR = 1e-4 # Learning rate.
WD = 1e-5 # Weight decay for Adam optimizer.

# Seeding for Reproducibility
rng = np.random.default_rng(SEED)
torch.manual_seed(SEED)
if device.type == "cuda":
    torch.cuda.manual_seed_all(SEED)

# Print Configuration Summary
print("-" * 70)
print("--- Pipeline B: Autoencoder Configuration ---")
print(f"Device: {device}" + (f" ({torch.cuda.get_device_name(0)})" if device.type=='cuda' else ""))
print(f"cuDNN: deterministic={torch.backends.cudnn.deterministic} | "
      f"benchmark={torch.backends.cudnn.benchmark}")
print(f"Batch/Epoch: {BATCH_SIZE} / {EPOCHS} | LR={LR} WD={WD}")
print(f"Windowing: LEN={WIN_LEN}, STRIDE={WIN_STRIDE}, FEATURES={FEAT_LIST} +speed={ADD_SPEED}")
print(f"AE Arch: Z_DIM={Z_DIM}, H_DIM={H_DIM}, LAYERS={N_LAYERS}, DROPOUT={DROPOUT}")
print(f"Training: BATCH_SIZE={BATCH_SIZE}, EPOCHS={EPOCHS}, LR={LR}, WEIGHT_DECAY={WD}")
print(f"Seed: {SEED}")
print("-" * 70)
else:
    print("--- Skipped: Loading from cache ---")

```

```

-----
--- Pipeline B: Autoencoder Configuration ---
Device:      cuda (NVIDIA GeForce RTX 4070 Laptop GPU)
cuDNN:      deterministic=True | benchmark=False
Batch/Epoch: 256 / 50 | LR=0.0001 WD=1e-05
Windowing:  LEN=24, STRIDE=1, FEATURES=['vx', 'vy'] +speed=True
AE Arch:     Z_DIM=128, H_DIM=512, LAYERS=2, DROPOUT=0.15
Training:    BATCH_SIZE=256, EPOCHS=50, LR=0.0001, WEIGHT_DECAY=1e-05
Seed:        42
-----

```

7.3 Data Preparation: Sliding Windows

The raw time-series data is transformed into a format suitable for the sequence model. This process involves creating fixed-length, overlapping "sliding windows" from each eligible agent trajectory. The result is a single NumPy array where each entry is a `(sequence_length, num_features)` matrix.

```

In [44]: # This cell will only run if the full computation is enabled.
if RUN_MODEL_TRAINING:
    # Prepare Feature Columns
    required_cols = {"scene_id", "track_id", "frame_idx", "vx", "vy"}
    if not required_cols.issubset(agents_df_clean.columns):
        raise ValueError(f"agents_df_clean is missing required columns: {required_cols - set(agents_df_clean.columns)}")
    feature_cols = FEAT_LIST.copy()
    if ADD_SPEED:
        feature_cols.append("speed")
        adf = agents_df_clean.copy()
        adf["speed"] = np.sqrt(adf["vx"]**2 + adf["vy"]**2)
    else:
        adf = agents_df_clean.copy()

    # Build Sliding Windows from Trajectories
    grouped_tracks = adf.sort_values(["scene_id", "track_id", "frame_idx"]).groupby(["scene_id", "track_id"], sort=False)
    window_list, window_keys = [], []
    for (sid, tid), group in tqdm(grouped_tracks, desc="Creating Sliding Windows", unit="track"):
        if len(group) < WIN_LEN:
            continue
        track_matrix = group[feature_cols].to_numpy(dtype=np.float32)
        for start_idx in range(0, len(track_matrix) - WIN_LEN + 1, WIN_STRIDE):
            window = track_matrix[start_idx : start_idx + WIN_LEN]
            window_list.append(window)
            start_frame = group.iloc[start_idx]["frame_idx"]
            window_keys.append((int(sid), int(tid), int(start_frame)))

    # Finalize and Summarize
    X_windows = np.stack(window_list) if window_list else np.zeros((0, WIN_LEN, len(feature_cols)), dtype=np.float32)
    window_keys_df = pd.DataFrame(window_keys, columns=["scene_id", "track_id", "start_frame"])
    N_windows, T_len, F_dim = X_windows.shape
    summary_df = pd.DataFrame({
        "measure": ["Total Tracks", "Tracks with Valid Windows", "Windows Created", "Sequence Length (T)", "Feature Dimension (F)"],
        "value": [grouped_tracks.ngroups, window_keys_df[["scene_id", "track_id"]].nunique().shape[0], N_windows, T_len, F_dim]
    })
    display(style_section_table(summary_df))

```

```
# Finalize and Summarize
if X_windows.size:
    flat = X_windows.reshape(-1, X_windows.shape[-1])
    mu = flat.mean(axis=0).astype(np.float32)
    sd = flat.std(axis=0).astype(np.float32) + 1e-6
    X_windows = (X_windows - mu) / sd

print("      X shape:", X.shape, "| dtype:", X.dtype)
flat = X.reshape(-1, X.shape[-1]) if X.size else X
mu = np.nanmean(flat, axis=0) if X.size else np.array([np.nan]* (X.shape[-1] if X.ndim==3 else 0))
sd = np.nanstd(flat, axis=0) if X.size else np.array([np.nan]* (X.shape[-1] if X.ndim==3 else 0))
print("Per-feature mean:",      np.round(mu, 4))
print("Per-feature std :",      np.round(sd, 4))
print("Any NaN in X? ", np.isnan(X).any(), "| Any Inf?", np.isinf(X).any())
else:
    print("--- Skipped: Loading from cache ---")
```

Creating Sliding Windows: 0% | 0/64625 [00:00<?, ?track/s]

measure	value
Total Tracks	64625
Tracks with Valid Windows	2
Windows Created	404919
Sequence Length (T)	24
Feature Dimension (F)	3

X shape: (7824, 7) | dtype: float64
Per-feature mean: [1.2254 1.6371 0.3556 1.0792 0.8541 0.2791 7.3858]
Per-feature std : [3.536 4.1253 0.786 1.6313 0.3188 0.3077 5.3657]
Any NaN in X? False | Any Inf? False

7.4 PyTorch Dataset and DataLoaders

This cell contains the standard PyTorch boilerplate for feeding data to the model. A custom `Dataset` class is defined to handle the windowed data, and `DataLoaders` are configured for both training and validation to manage batching, shuffling, and data transfer to the GPU.

In []: # This cell will only run if the full computation is enabled.

```
if RUN_MODEL_TRAINING:
    # Define a Custom PyTorch Dataset
    class WindowDS(Dataset):
        def __init__(self, X):
            self.X = X
        def __len__(self): return self.X.shape[0]
        def __getitem__(self, i): return torch.from_numpy(self.X[i])

    # Perform a Track-Level Train/Validation Split
    wk = window_keys_df[["scene_id", "track_id"]].to_numpy()
    uniq_tracks, inv = np.unique(wk, axis=0, return_inverse=True)
    n_tracks = uniq_tracks.shape[0]
    track_perm = rng.permutation(n_tracks)
    cut = int(0.9 * n_tracks)
    tr_track_idx = set(track_perm[:cut])
    mask_tr = np.isin(inv, list(tr_track_idx))
    mask_va = ~mask_tr

    # Windows
    X_tr = X_windows[mask_tr]
    X_va = X_windows[mask_va]

    # Standardize per-feature using TRAIN statistics only
    if X_tr.size:
        flat_tr = X_tr.reshape(-1, X_tr.shape[-1])
        mu = flat_tr.mean(axis=0).astype(np.float32)
        sd = flat_tr.std(axis=0).astype(np.float32) + 1e-6
        X_tr = (X_tr - mu) / sd
        X_va = (X_va - mu) / sd
        m_post = X_tr.reshape(-1, X_tr.shape[-1]).mean(axis=0)
        s_post = X_tr.reshape(-1, X_tr.shape[-1]).std(axis=0)
        print("\n--- Track Split & Standardization ---")
        print(f"train mu:      {np.round(mu, 4)}")
        print(f"train sd:      {np.round(sd, 4)}")
        print(f"post-mean(train) ~0: {np.round(m_post, 4)}")
        print(f"post-std(train) ~1: {np.round(s_post, 4)}")
    else:
        print("\n--- WARNING - empty TRAIN split (check WIN_LEN) ---")

    scaler_stats = {"feature_dim": int(mu.shape[0]), "mu": mu.tolist(), "sd": sd.tolist(), "dt": float(DT), "win_len": int(WIN_LEN)}

    # Save the scaler stats for reproducibility
    with open(ARTIFACT_DIR / "pipeline_b_scaler_stats.json", "w") as f:
        json.dump(scaler_stats, f, indent=2)
    print(f"Scaler stats saved to: {ARTIFACT_DIR / 'pipeline_b_scaler_stats.json'}")

    # Configure DataLoaders
```

```

train_ds = WindowDS(X_tr)
valid_ds = WindowDS(X_va)

loader_args_tr = dict(
    batch_size=BATCH_SIZE, pin_memory=True, shuffle=True, drop_last=True,
    num_workers=1, persistent_workers=True, prefetch_factor=2
)
loader_args_va = dict(
    batch_size=BATCH_SIZE, pin_memory=True, shuffle=False, drop_last=False,
    num_workers=1, persistent_workers=True, prefetch_factor=2
)
train_loader = DataLoader(train_ds, **loader_args_tr)
valid_loader = DataLoader(valid_ds, **loader_args_va)

# Print Configuration Summary
n_tr_tracks = len(tr_track_idx)
n_va_tracks = n_tracks - n_tr_tracks
steps_tr = len(train_ds) // BATCH_SIZE if BATCH_SIZE > 0 else 0
steps_va = len(valid_ds) // BATCH_SIZE if BATCH_SIZE > 0 else 0
print("-" * 70)
print("--- Dataset & DataLoaders READY (track-level split) ---")
print(f"Unique tracks:      {n_tracks:,} | TRAIN tracks={n_tr_tracks:,} | VALID tracks={n_va_tracks:,}")
print(f"Windows:              TRAIN={len(train_ds):,} | VALID={len(valid_ds):,}")
print(f"BATCH size:           {BATCH_SIZE} | workers={loader_args_tr['num_workers']} | drop_last={loader_args_tr['drop_last']}")
print(f"STEPS/epoch:          TRAIN={steps_tr:,} | VALID={steps_va:,}")
if len(train_ds) > 0:
    x0 = train_ds[0]
    print(f"Sample check:         train_ds[0] -> shape={tuple(x0.shape)} dtype={x0.dtype}")
print("-" * 70)
else:
    print("--- Skipped: Loading from cache ---")

```

```

--- Track Split & Standardization ---
train mu:      [-0.0047  0.0122  0.0036]
train sd:      [1.0022 1.001  1.0036]
post-mean(train) ~0: [ 0.0031 -0.002  0.0008]
post-std(train) ~1: [1.  1.  0.9997]
Scaler stats saved to: /home/trainart/projects/lyft-unsupervised/artifacts/pipeline_b_scaler_stats.json

-----
--- Dataset & DataLoaders READY (track-level split) ---
Unique tracks:  9,944 | TRAIN tracks=8,949 | VALID tracks=995
Windows:        TRAIN=362,231 | VALID=42,688
BATCH size:     256 | workers=1 | drop_last=True
STEPS/epoch:    TRAIN=1,414 | VALID=166
Sample check:   train_ds[0] -> shape=(24, 3) dtype=torch.float32
-----

```

7.5 Model Architecture: GRU Autoencoder

The core deep learning model is defined here. It is an autoencoder composed of a GRU-based encoder and decoder. The encoder compresses each input sequence into a fixed-size latent vector (the embedding), and the decoder attempts to reconstruct the original sequence from this vector. The model is then instantiated and moved to the selected hardware device.

```

In [47]: # This cell will only run if the full computation is enabled.
if RUN_MODEL_TRAINING:
    # Define Model Architecture
    FEAT_DIM = int(X_windows.shape[-1] if X_windows.size else len(feature_cols))
    class GRUAutoEncoder(nn.Module):
        def __init__(self, feat_dim, z_dim, h_dim, n_layers=2, dropout=0.0):
            super().__init__()
            self.enc = nn.GRU(input_size=feat_dim, hidden_size=h_dim, num_layers=n_layers,
                              batch_first=True, dropout=dropout if n_layers > 1 else 0.0, bidirectional=False)
            self.hnorm = nn.LayerNorm(h_dim)
            self.to_z = nn.Linear(h_dim, z_dim)
            self.from_z = nn.Linear(z_dim, h_dim)
            self.dec = nn.GRU(input_size=feat_dim, hidden_size=h_dim, num_layers=n_layers,
                              batch_first=True, dropout=dropout if n_layers > 1 else 0.0, bidirectional=False)
            self.proj = nn.Linear(h_dim, feat_dim)

        def forward(self, x):
            B, T, F = x.shape

            # Encoder
            _, hT = self.enc(x)
            h_last = self.hnorm(hT[-1])
            z = self.to_z(h_last)

            # Decoder
            dec_in = torch.zeros(B, T, F, device=x.device, dtype=x.dtype)
            h0 = self.from_z(z).unsqueeze(0).repeat(self.dec.num_layers, 1, 1)
            y, _ = self.dec(dec_in, h0)
            recon = self.proj(y)
            return recon, z

    # Build the Model and Training Components
    model = GRUAutoEncoder(FEAT_DIM, Z_DIM, H_DIM, n_layers=N_LAYERS, dropout=DROPOUT).to(device)
    def _init_weights(m):
        if isinstance(m, nn.GRU):
            for name, p in m.named_parameters():

```



```

        if "weight_ih" in name or "weight_hh" in name:
            nn.init.orthogonal_(p)
        elif "bias" in name:
            nn.init.constant_(p, 0.0)
    elif isinstance(m, nn.Linear):
        nn.init.xavier_uniform_(m.weight)
        if m.bias is not None:
            nn.init.zeros_(m.bias)
model.apply(_init_weights)
optimizer = torch.optim.Adam(model.parameters(), lr=LR, weight_decay=WD)
criterion = nn.MSELoss()
scaler = torch.cuda.amp.GradScaler(enabled=(device.type == "cuda"))
steps_per_epoch = max(1, len(train_loader))
steps_per_epoch = max(1, len(train_loader))
scheduler = torch.optim.lr_scheduler.OneCycleLR(
    optimizer,
    max_lr=LR * 3.0,
    steps_per_epoch=steps_per_epoch,
    epochs=EPOCHS,
    pct_start=0.10,
    anneal_strategy="cos",
    div_factor=5.0,
    final_div_factor=10.0,
)

total_train_steps = steps_per_epoch * EPOCHS
warmup_steps = max(1, int(0.10 * total_train_steps))
print(f"[Scheduler] steps/epoch={steps_per_epoch}, total_steps={total_train_steps}, warmup_steps={warmup_steps} "
      f"start_lr={LR:.2e}, max_lr={(LR*5):.2e}")

# Print Configuration Summary
print("-" * 70)
print("--- Model READY ---")
dev = device.type.upper()
dtype0 = next(model.parameters()).dtype
print(f"Device: {dev} | dtype: {dtype0}")
print(f"Arch: GRUAutoEncoder(feat_dim={FEAT_DIM}, z_dim={Z_DIM}, h_dim={H_DIM}, "
      f"layers={N_LAYERS}, dropout={DROPOUT})")
total_params = sum(p.numel() for p in model.parameters())
train_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
print(f"Params: {train_params:,} / {total_params:,} trainable")
print(f"Optim: Adam | lr={LR} | weight_decay={WD}")
print(f"Loss: {criterion.__class__.__name__}")

if 'train_ds' in globals() and len(train_ds) > 0:
    model.eval()
    with torch.no_grad(), torch.cuda.amp.autocast(enabled=(device.type == "cuda")):
        ncheck = min(32, len(train_ds))
        xb = torch.stack([train_ds[i] for i in range(ncheck)], dim=0).to(device, non_blocking=True)
        recon, z = model(xb)
        l0 = float(criterion(recon, xb).item())
        print(f"Sanity: input {tuple(xb.shape)} -> recon {tuple(recon.shape)} | z {tuple(z.shape)} | loss={l0:.5f}")
    else:
        print("Sanity: (skipped - TRAIN dataset empty)")
print("-" * 70)
else:
    print("--- Skipped: Loading from cache ---")

```

```
[Scheduler] steps/epoch=1414, total_steps=70700, warmup_steps=7070 start_lr=1.00e-04, max_lr=5.00e-04
```

```

--- Model READY ---
Device: CUDA | dtype: torch.float32
Arch: GRUAutoEncoder(feat_dim=3, z_dim=128, h_dim=512, layers=2, dropout=0.15)
Params: 4,874,371 / 4,874,371 trainable
Optim: Adam | lr=0.0001 | weight_decay=1e-05
Loss: MSELoss
Sanity: input (32, 24, 3) -> recon (32, 24, 3) | z (32, 128) | loss=0.10890

```

7.6 Model Training

The main training loop for the autoencoder. The model is trained for a set number of epochs, iterating through the training data to minimize the reconstruction error (MSE). A validation loop is run at the end of each epoch to monitor the model's performance on unseen data and check for overfitting.

```

In [48]: # This cell will only run if the full computation is enabled.
if RUN_MODEL_TRAINING:
    # Train the Autoencoder Model
    start_time = time.time()
    history = {"epoch": [], "train_loss": [], "valid_loss": []}

    for epoch in range(1, EPOCHS + 1):
        epoch_start = time.time()

        # Training phase
        model.train()
        tr_loss, tr_mse, ntr = 0.0, 0.0, 0
        p_tr = tqdm(train_loader, desc=f"TRAIN Epoch {epoch:02d}", unit="batch", leave=True, dynamic_ncols=True)
        noise_std = 0.002 * (epoch/10) if epoch <= 10 else 0.002

```

```

for xb in p_tr:
    xb = xb.to(device, non_blocking=True)
    optimizer.zero_grad(set_to_none=True)

    # Add noise to inputs for to avoid collapse
    x_in = xb + noise_std * torch.randn_like(xb)

    with torch.cuda.amp.autocast(enabled=(device.type == "cuda")):
        recon, z = model(x_in)
        mse = criterion(recon, xb)
        lat_std = z.float().std(dim=0)
        var_pen = torch.relu(0.15 - lat_std).mean()
        loss = mse + 5e-3 * var_pen

    scaler.scale(loss).backward()
    nn.utils.clip_grad_norm_(model.parameters(), 1.0)
    scaler.step(optimizer)
    scaler.update()
    if 'scheduler' in globals():
        scheduler.step()
        cur_lr = scheduler.get_last_lr()[0]
    else:
        cur_lr = LR

    tr_loss += float(loss.detach().cpu())
    tr_mse += float(mse.detach().cpu())
    ntr += 1

p_tr.set_postfix_str(f"loss={tr_loss/ntr:.5f} | lr={cur_lr:.2e} |  $\sigma$ ={noise_std:.4f}")

# Validation phase
model.eval()
va_loss, nva = 0.0, 0
p_va = tqdm(valid_loader, desc=f"VALID Epoch {epoch:02d}", unit="batch", leave=True, dynamic_ncols=True)
with torch.no_grad(), torch.cuda.amp.autocast(enabled=(device.type == "cuda")):
    for xb in p_va:
        xb = xb.to(device, non_blocking=True)
        recon, _ = model(xb)
        loss = criterion(recon, xb)
        va_loss += float(loss.detach().cpu())
        nva += 1
    p_va.set_postfix_str(f"loss={va_loss / nva:.5f}")

# Log epoch results
tr_avg = tr_loss / max(1, ntr)
va_avg = va_loss / max(1, nva)
history["epoch"].append(epoch)
history["train_loss"].append(tr_avg)
history["valid_loss"].append(va_avg)

if device.type == "cuda":
    torch.cuda.synchronize()
    epoch_dt = time.time() - epoch_start
    bps = ntr / epoch_dt if epoch_dt > 0 else 0.0
    print(f"Epoch {epoch:02d} | Train Loss = {tr_avg:.5f} | Valid Loss = {va_avg:.5f} | Learning Rate = {cur_lr:.2e} | Noise  $\sigma$  = {noise_std:.4f} | Epoch

# Calculate and Print Total Training Time
end_time = time.time()
elapsed_time = end_time - start_time

# Calculate epoch and step metrics
total_steps = EPOCHS * len(train_loader)
avg_time_per_epoch = elapsed_time / EPOCHS
avg_time_per_step = (elapsed_time / total_steps) * 1000 if total_steps > 0 else 0
steps_per_second = total_steps / elapsed_time if elapsed_time > 0 else 0

minutes = int(elapsed_time // 60)
seconds = int(elapsed_time % 60)

print("\n" + "-"*70)
print("--- Training Time Summary ---")
print(f"Total Training Time: {minutes}m {seconds}s")
print(f"Average Time per Epoch: {avg_time_per_epoch:.2f}s")
print(f"Average Time per Step: {avg_time_per_step:.2f}ms")
print(f"Throughput: {steps_per_second:.2f} steps/sec")
print(f"Final Train Loss: {tr_avg:.5f}")
print(f"Final Valid Loss: {va_avg:.5f}")
print(f"Final Learning Rate: {cur_lr:.2e}")
print(f"Final Noise Std Dev: {noise_std:.4f}")
print("\n" + "-"*70)

# Quick Latent sanity after training
if len(valid_loader) > 0:
    model.eval()
    if device.type == "cuda":
        torch.cuda.synchronize()
    with torch.no_grad(), torch.cuda.amp.autocast(enabled=(device.type == "cuda")):
        xb = next(iter(valid_loader)).to(device, non_blocking=True)
        recon, z = model(xb)
        mse = float(criterion(recon, xb).item())

    z = z.float()
    z_abs_mean = float(z.abs().mean().item())
    z_std = z.std(dim=0)

```

```

z_std_mean = float(z_std.mean().item())
zeroish = float((z_std < 1e-3).float().mean().item() * 100.0)

print("--- Latent & Recon Sanity ---")
print(f"Recon MSE (batch):      {mse:.5f}")
print(f"latent | mean(|z|):      {z_abs_mean:.4f}")
print(f"per-dim std:              mean={z_std_mean:.4f}  min={z_std.min().item():.4f}  max={z_std.max().item():.4f}")
print(f"zero-ish dims (<1e-3):    {zeroish:.1f}%")
print("---*70")
else:
    print("\n--- Latent sanity skipped: empty valid_loader ---")
else:
    print("--- Skipped: Loading from cache ---")

```

TRAIN Epoch 01:	0%		0/1414 [00:00<?, ?batch/s]
VALID Epoch 01:	0%		0/167 [00:00<?, ?batch/s]
Epoch 01 Train Loss = 0.85032		Valid Loss = 0.89390 Learning Rate = 8.29e-05 Noise σ = 0.0002 Epoch Time = 21.59s 65.5 batches/s	
TRAIN Epoch 02:	0%		0/1414 [00:00<?, ?batch/s]
VALID Epoch 02:	0%		0/167 [00:00<?, ?batch/s]
Epoch 02 Train Loss = 0.79802		Valid Loss = 0.67664 Learning Rate = 1.43e-04 Noise σ = 0.0004 Epoch Time = 21.53s 65.7 batches/s	
TRAIN Epoch 03:	0%		0/1414 [00:00<?, ?batch/s]
VALID Epoch 03:	0%		0/167 [00:00<?, ?batch/s]
Epoch 03 Train Loss = 0.60351		Valid Loss = 0.67252 Learning Rate = 2.17e-04 Noise σ = 0.0006 Epoch Time = 20.60s 68.6 batches/s	
TRAIN Epoch 04:	0%		0/1414 [00:00<?, ?batch/s]
VALID Epoch 04:	0%		0/167 [00:00<?, ?batch/s]
Epoch 04 Train Loss = 0.60717		Valid Loss = 0.67065 Learning Rate = 2.77e-04 Noise σ = 0.0008 Epoch Time = 22.63s 62.5 batches/s	
TRAIN Epoch 05:	0%		0/1414 [00:00<?, ?batch/s]
VALID Epoch 05:	0%		0/167 [00:00<?, ?batch/s]
Epoch 05 Train Loss = 0.61331		Valid Loss = 0.70039 Learning Rate = 3.00e-04 Noise σ = 0.0010 Epoch Time = 19.75s 71.6 batches/s	
TRAIN Epoch 06:	0%		0/1414 [00:00<?, ?batch/s]
VALID Epoch 06:	0%		0/167 [00:00<?, ?batch/s]
Epoch 06 Train Loss = 0.64880		Valid Loss = 0.70921 Learning Rate = 3.00e-04 Noise σ = 0.0012 Epoch Time = 22.56s 62.7 batches/s	
TRAIN Epoch 07:	0%		0/1414 [00:00<?, ?batch/s]
VALID Epoch 07:	0%		0/167 [00:00<?, ?batch/s]
Epoch 07 Train Loss = 0.65533		Valid Loss = 0.70988 Learning Rate = 2.99e-04 Noise σ = 0.0014 Epoch Time = 22.26s 63.5 batches/s	
TRAIN Epoch 08:	0%		0/1414 [00:00<?, ?batch/s]
VALID Epoch 08:	0%		0/167 [00:00<?, ?batch/s]
Epoch 08 Train Loss = 0.65496		Valid Loss = 0.70805 Learning Rate = 2.97e-04 Noise σ = 0.0016 Epoch Time = 20.18s 70.1 batches/s	
TRAIN Epoch 09:	0%		0/1414 [00:00<?, ?batch/s]
VALID Epoch 09:	0%		0/167 [00:00<?, ?batch/s]
Epoch 09 Train Loss = 0.64318		Valid Loss = 0.67073 Learning Rate = 2.94e-04 Noise σ = 0.0018 Epoch Time = 22.33s 63.3 batches/s	
TRAIN Epoch 10:	0%		0/1414 [00:00<?, ?batch/s]
VALID Epoch 10:	0%		0/167 [00:00<?, ?batch/s]
Epoch 10 Train Loss = 0.59843		Valid Loss = 0.67072 Learning Rate = 2.91e-04 Noise σ = 0.0020 Epoch Time = 22.47s 62.9 batches/s	
TRAIN Epoch 11:	0%		0/1414 [00:00<?, ?batch/s]
VALID Epoch 11:	0%		0/167 [00:00<?, ?batch/s]
Epoch 11 Train Loss = 0.62405		Valid Loss = 0.70224 Learning Rate = 2.87e-04 Noise σ = 0.0020 Epoch Time = 19.97s 70.8 batches/s	
TRAIN Epoch 12:	0%		0/1414 [00:00<?, ?batch/s]
VALID Epoch 12:	0%		0/167 [00:00<?, ?batch/s]
Epoch 12 Train Loss = 0.64891		Valid Loss = 0.70905 Learning Rate = 2.83e-04 Noise σ = 0.0020 Epoch Time = 22.07s 64.1 batches/s	
TRAIN Epoch 13:	0%		0/1414 [00:00<?, ?batch/s]
VALID Epoch 13:	0%		0/167 [00:00<?, ?batch/s]
Epoch 13 Train Loss = 0.65393		Valid Loss = 0.71893 Learning Rate = 2.78e-04 Noise σ = 0.0020 Epoch Time = 19.63s 72.0 batches/s	
TRAIN Epoch 14:	0%		0/1414 [00:00<?, ?batch/s]
VALID Epoch 14:	0%		0/167 [00:00<?, ?batch/s]
Epoch 14 Train Loss = 0.60179		Valid Loss = 0.67017 Learning Rate = 2.72e-04 Noise σ = 0.0020 Epoch Time = 21.93s 64.5 batches/s	
TRAIN Epoch 15:	0%		0/1414 [00:00<?, ?batch/s]
VALID Epoch 15:	0%		0/167 [00:00<?, ?batch/s]
Epoch 15 Train Loss = 0.61283		Valid Loss = 0.69720 Learning Rate = 2.66e-04 Noise σ = 0.0020 Epoch Time = 22.54s 62.7 batches/s	
TRAIN Epoch 16:	0%		0/1414 [00:00<?, ?batch/s]
VALID Epoch 16:	0%		0/167 [00:00<?, ?batch/s]
Epoch 16 Train Loss = 0.64660		Valid Loss = 0.70930 Learning Rate = 2.59e-04 Noise σ = 0.0020 Epoch Time = 19.88s 71.1 batches/s	
TRAIN Epoch 17:	0%		0/1414 [00:00<?, ?batch/s]
VALID Epoch 17:	0%		0/167 [00:00<?, ?batch/s]
Epoch 17 Train Loss = 0.65331		Valid Loss = 0.71131 Learning Rate = 2.51e-04 Noise σ = 0.0020 Epoch Time = 22.66s 62.4 batches/s	
TRAIN Epoch 18:	0%		0/1414 [00:00<?, ?batch/s]
VALID Epoch 18:	0%		0/167 [00:00<?, ?batch/s]
Epoch 18 Train Loss = 0.65482		Valid Loss = 0.71203 Learning Rate = 2.43e-04 Noise σ = 0.0020 Epoch Time = 22.64s 62.5 batches/s	
TRAIN Epoch 19:	0%		0/1414 [00:00<?, ?batch/s]
VALID Epoch 19:	0%		0/167 [00:00<?, ?batch/s]
Epoch 19 Train Loss = 0.64122		Valid Loss = 0.66967 Learning Rate = 2.35e-04 Noise σ = 0.0020 Epoch Time = 20.27s 69.8 batches/s	
TRAIN Epoch 20:	0%		0/1414 [00:00<?, ?batch/s]
VALID Epoch 20:	0%		0/167 [00:00<?, ?batch/s]
Epoch 20 Train Loss = 0.59784		Valid Loss = 0.66867 Learning Rate = 2.26e-04 Noise σ = 0.0020 Epoch Time = 22.69s 62.3 batches/s	
TRAIN Epoch 21:	0%		0/1414 [00:00<?, ?batch/s]
VALID Epoch 21:	0%		0/167 [00:00<?, ?batch/s]
Epoch 21 Train Loss = 0.62523		Valid Loss = 0.70101 Learning Rate = 2.17e-04 Noise σ = 0.0020 Epoch Time = 22.97s 61.6 batches/s	
TRAIN Epoch 22:	0%		0/1414 [00:00<?, ?batch/s]
VALID Epoch 22:	0%		0/167 [00:00<?, ?batch/s]
Epoch 22 Train Loss = 0.64808		Valid Loss = 0.70965 Learning Rate = 2.08e-04 Noise σ = 0.0020 Epoch Time = 20.12s 70.3 batches/s	
TRAIN Epoch 23:	0%		0/1414 [00:00<?, ?batch/s]
VALID Epoch 23:	0%		0/167 [00:00<?, ?batch/s]
Epoch 23 Train Loss = 0.65317		Valid Loss = 0.67422 Learning Rate = 1.98e-04 Noise σ = 0.0020 Epoch Time = 22.32s 63.4 batches/s	
TRAIN Epoch 24:	0%		0/1414 [00:00<?, ?batch/s]
VALID Epoch 24:	0%		0/167 [00:00<?, ?batch/s]
Epoch 24 Train Loss = 0.59901		Valid Loss = 0.66965 Learning Rate = 1.89e-04 Noise σ = 0.0020 Epoch Time = 20.74s 68.2 batches/s	
TRAIN Epoch 25:	0%		0/1414 [00:00<?, ?batch/s]
VALID Epoch 25:	0%		0/167 [00:00<?, ?batch/s]
Epoch 25 Train Loss = 0.61445		Valid Loss = 0.69424 Learning Rate = 1.79e-04 Noise σ = 0.0020 Epoch Time = 22.59s 62.6 batches/s	
TRAIN Epoch 26:	0%		0/1414 [00:00<?, ?batch/s]
VALID Epoch 26:	0%		0/167 [00:00<?, ?batch/s]
Epoch 26 Train Loss = 0.64275		Valid Loss = 0.70641 Learning Rate = 1.68e-04 Noise σ = 0.0020 Epoch Time = 22.64s 62.5 batches/s	
TRAIN Epoch 27:	0%		0/1414 [00:00<?, ?batch/s]
VALID Epoch 27:	0%		0/167 [00:00<?, ?batch/s]
Epoch 27 Train Loss = 0.65174		Valid Loss = 0.70876 Learning Rate = 1.58e-04 Noise σ = 0.0020 Epoch Time = 20.68s 68.4 batches/s	

```
TRAIN Epoch 28: 0% | 0/1414 [00:00<?, ?batch/s]
VALID Epoch 28: 0% | 0/167 [00:00<?, ?batch/s]
Epoch 28 | Train Loss = 0.65353 | Valid Loss = 0.71811 | Learning Rate = 1.48e-04 | Noise  $\sigma$  = 0.0020 | Epoch Time = 22.30s | 63.4 batches/s
TRAIN Epoch 29: 0% | 0/1414 [00:00<?, ?batch/s]
VALID Epoch 29: 0% | 0/167 [00:00<?, ?batch/s]
Epoch 29 | Train Loss = 0.65493 | Valid Loss = 0.70826 | Learning Rate = 1.38e-04 | Noise  $\sigma$  = 0.0020 | Epoch Time = 22.47s | 62.9 batches/s
TRAIN Epoch 30: 0% | 0/1414 [00:00<?, ?batch/s]
VALID Epoch 30: 0% | 0/167 [00:00<?, ?batch/s]
Epoch 30 | Train Loss = 0.65404 | Valid Loss = 0.70870 | Learning Rate = 1.27e-04 | Noise  $\sigma$  = 0.0020 | Epoch Time = 19.83s | 71.3 batches/s
TRAIN Epoch 31: 0% | 0/1414 [00:00<?, ?batch/s]
VALID Epoch 31: 0% | 0/167 [00:00<?, ?batch/s]
Epoch 31 | Train Loss = 0.60243 | Valid Loss = 0.66963 | Learning Rate = 1.17e-04 | Noise  $\sigma$  = 0.0020 | Epoch Time = 21.63s | 65.4 batches/s
TRAIN Epoch 32: 0% | 0/1414 [00:00<?, ?batch/s]
VALID Epoch 32: 0% | 0/167 [00:00<?, ?batch/s]
Epoch 32 | Train Loss = 0.60945 | Valid Loss = 0.69010 | Learning Rate = 1.08e-04 | Noise  $\sigma$  = 0.0020 | Epoch Time = 21.86s | 64.7 batches/s
TRAIN Epoch 33: 0% | 0/1414 [00:00<?, ?batch/s]
VALID Epoch 33: 0% | 0/167 [00:00<?, ?batch/s]
Epoch 33 | Train Loss = 0.63556 | Valid Loss = 0.70389 | Learning Rate = 9.79e-05 | Noise  $\sigma$  = 0.0020 | Epoch Time = 20.97s | 67.4 batches/s
TRAIN Epoch 34: 0% | 0/1414 [00:00<?, ?batch/s]
VALID Epoch 34: 0% | 0/167 [00:00<?, ?batch/s]
Epoch 34 | Train Loss = 0.64773 | Valid Loss = 0.70758 | Learning Rate = 8.86e-05 | Noise  $\sigma$  = 0.0020 | Epoch Time = 22.51s | 62.8 batches/s
TRAIN Epoch 35: 0% | 0/1414 [00:00<?, ?batch/s]
VALID Epoch 35: 0% | 0/167 [00:00<?, ?batch/s]
Epoch 35 | Train Loss = 0.65274 | Valid Loss = 0.70940 | Learning Rate = 7.95e-05 | Noise  $\sigma$  = 0.0020 | Epoch Time = 21.31s | 66.4 batches/s
TRAIN Epoch 36: 0% | 0/1414 [00:00<?, ?batch/s]
VALID Epoch 36: 0% | 0/167 [00:00<?, ?batch/s]
Epoch 36 | Train Loss = 0.65330 | Valid Loss = 0.70845 | Learning Rate = 7.08e-05 | Noise  $\sigma$  = 0.0020 | Epoch Time = 22.92s | 61.7 batches/s
TRAIN Epoch 37: 0% | 0/1414 [00:00<?, ?batch/s]
VALID Epoch 37: 0% | 0/167 [00:00<?, ?batch/s]
Epoch 37 | Train Loss = 0.65375 | Valid Loss = 0.70832 | Learning Rate = 6.25e-05 | Noise  $\sigma$  = 0.0020 | Epoch Time = 23.63s | 59.8 batches/s
TRAIN Epoch 38: 0% | 0/1414 [00:00<?, ?batch/s]
VALID Epoch 38: 0% | 0/167 [00:00<?, ?batch/s]
Epoch 38 | Train Loss = 0.65528 | Valid Loss = 0.70987 | Learning Rate = 5.46e-05 | Noise  $\sigma$  = 0.0020 | Epoch Time = 23.48s | 60.2 batches/s
TRAIN Epoch 39: 0% | 0/1414 [00:00<?, ?batch/s]
VALID Epoch 39: 0% | 0/167 [00:00<?, ?batch/s]
Epoch 39 | Train Loss = 0.65327 | Valid Loss = 0.70880 | Learning Rate = 4.73e-05 | Noise  $\sigma$  = 0.0020 | Epoch Time = 21.18s | 66.8 batches/s
TRAIN Epoch 40: 0% | 0/1414 [00:00<?, ?batch/s]
VALID Epoch 40: 0% | 0/167 [00:00<?, ?batch/s]
Epoch 40 | Train Loss = 0.65334 | Valid Loss = 0.72303 | Learning Rate = 4.04e-05 | Noise  $\sigma$  = 0.0020 | Epoch Time = 23.33s | 60.6 batches/s
TRAIN Epoch 41: 0% | 0/1414 [00:00<?, ?batch/s]
VALID Epoch 41: 0% | 0/167 [00:00<?, ?batch/s]
Epoch 41 | Train Loss = 0.65479 | Valid Loss = 0.70963 | Learning Rate = 3.41e-05 | Noise  $\sigma$  = 0.0020 | Epoch Time = 21.07s | 67.1 batches/s
TRAIN Epoch 42: 0% | 0/1414 [00:00<?, ?batch/s]
VALID Epoch 42: 0% | 0/167 [00:00<?, ?batch/s]
Epoch 42 | Train Loss = 0.65566 | Valid Loss = 0.70966 | Learning Rate = 2.83e-05 | Noise  $\sigma$  = 0.0020 | Epoch Time = 22.86s | 61.9 batches/s
TRAIN Epoch 43: 0% | 0/1414 [00:00<?, ?batch/s]
VALID Epoch 43: 0% | 0/167 [00:00<?, ?batch/s]
Epoch 43 | Train Loss = 0.64969 | Valid Loss = 0.67687 | Learning Rate = 2.32e-05 | Noise  $\sigma$  = 0.0020 | Epoch Time = 22.23s | 63.6 batches/s
TRAIN Epoch 44: 0% | 0/1414 [00:00<?, ?batch/s]
VALID Epoch 44: 0% | 0/167 [00:00<?, ?batch/s]
Epoch 44 | Train Loss = 0.60087 | Valid Loss = 0.66982 | Learning Rate = 1.87e-05 | Noise  $\sigma$  = 0.0020 | Epoch Time = 20.08s | 70.4 batches/s
TRAIN Epoch 45: 0% | 0/1414 [00:00<?, ?batch/s]
VALID Epoch 45: 0% | 0/167 [00:00<?, ?batch/s]
Epoch 45 | Train Loss = 0.60876 | Valid Loss = 0.68719 | Learning Rate = 1.49e-05 | Noise  $\sigma$  = 0.0020 | Epoch Time = 22.11s | 64.0 batches/s
TRAIN Epoch 46: 0% | 0/1414 [00:00<?, ?batch/s]
VALID Epoch 46: 0% | 0/167 [00:00<?, ?batch/s]
Epoch 46 | Train Loss = 0.63266 | Valid Loss = 0.69822 | Learning Rate = 1.17e-05 | Noise  $\sigma$  = 0.0020 | Epoch Time = 21.55s | 65.6 batches/s
TRAIN Epoch 47: 0% | 0/1414 [00:00<?, ?batch/s]
VALID Epoch 47: 0% | 0/167 [00:00<?, ?batch/s]
Epoch 47 | Train Loss = 0.63795 | Valid Loss = 0.69795 | Learning Rate = 9.21e-06 | Noise  $\sigma$  = 0.0020 | Epoch Time = 19.30s | 73.3 batches/s
TRAIN Epoch 48: 0% | 0/1414 [00:00<?, ?batch/s]
VALID Epoch 48: 0% | 0/167 [00:00<?, ?batch/s]
Epoch 48 | Train Loss = 0.64159 | Valid Loss = 0.70053 | Learning Rate = 7.43e-06 | Noise  $\sigma$  = 0.0020 | Epoch Time = 21.71s | 65.1 batches/s
TRAIN Epoch 49: 0% | 0/1414 [00:00<?, ?batch/s]
VALID Epoch 49: 0% | 0/167 [00:00<?, ?batch/s]
Epoch 49 | Train Loss = 0.64394 | Valid Loss = 0.70299 | Learning Rate = 6.36e-06 | Noise  $\sigma$  = 0.0020 | Epoch Time = 20.29s | 69.7 batches/s
TRAIN Epoch 50: 0% | 0/1414 [00:00<?, ?batch/s]
VALID Epoch 50: 0% | 0/167 [00:00<?, ?batch/s]
Epoch 50 | Train Loss = 0.64467 | Valid Loss = 0.70400 | Learning Rate = 6.00e-06 | Noise  $\sigma$  = 0.0020 | Epoch Time = 23.60s | 59.9 batches/s

-----
--- Training Time Summary ---
Total Training Time: 18m 4s
Average Time per Epoch: 21.69s
Average Time per Step: 15.34ms
Throughput: 65.20 steps/sec
Final Train Loss: 0.64467
Final Valid Loss: 0.70400
Final Learning Rate: 6.00e-06
Final Noise Std Dev: 0.0020

-----
--- Latent & Recon Sanity ---
Recon MSE (batch): 0.03536
latent | mean(|z|): 0.0381
per-dim std: mean=0.0105 min=0.0075 max=0.0225
zero-ish dims (<1e-3): 0.0%
-----
```

7.7 Loss Curve Visualization

This plot visualizes the model's learning progress. The training and validation loss are plotted for each epoch. A decreasing loss that converges for both sets indicates successful training. A significant gap between the two curves would suggest overfitting.

In [49]: # This cell will only run if the full computation is enabled.

```
if RUN_MODEL_TRAINING:
    # Create DataFrame from Training History
    history_df = pd.DataFrame(history)

    # Create and Style the Plot
    fig, ax = plt.subplots(figsize=(20, 10), facecolor="black")
    ax.set_facecolor("black")
    ax.plot(history_df["epoch"], history_df["train_loss"], label="Train Loss", color='dodgerblue', lw=2.5, marker='o')
    ax.plot(history_df["epoch"], history_df["valid_loss"], label="Validation Loss", color='limegreen', lw=2.5, marker='o')
    ax.set_title("Autoencoder Training & Validation Loss", fontsize=26, color="#CFB87C", fontweight="bold", pad=15)
    ax.set_xlabel("Epoch", fontsize=18, color="white", fontweight="bold")
    ax.set_ylabel("MSE", fontsize=18, color="white", fontweight="bold")
    ax.tick_params(colors="white", labels=14)
    for spine in ax.spines.values():
        spine.set_edgecolor("white")
        spine.set_linewidth(1.5)
    ax.legend(frameon=False, labelcolor="white", fontsize=16)
    ax.grid(color='gray', linestyle='--', linewidth=0.3, alpha=0.5)

    # Save the Figure
    save_fig(fig, "autoencoder_loss_curve")
else:
    print("--- Skipped: Loading from cache ---")
```

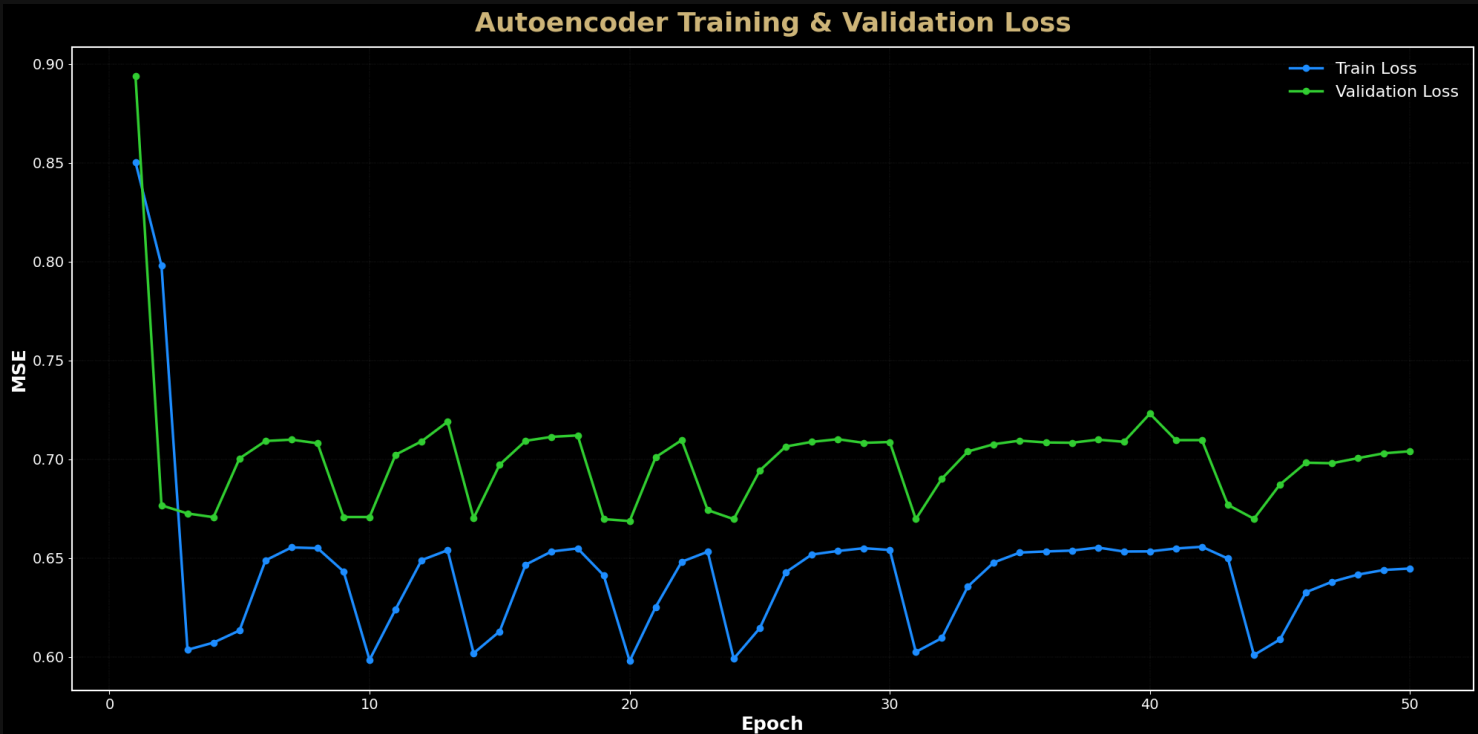


Figure saved to (Linux): /home/treinarnt/projects/lyft-unsupervised/artifacts/figures/20250917_213734_autoencoder_loss_curve.png

Figure saved to (Win): /mnt/c/Users/travi/Documents/Training/Colorado/MS-AI/CSCA 5632 Final Project/artifacts/figures/20250917_213734_autoencoder_loss_curve.png

7.8 Embedding Extraction

With the model trained, the encoder half is used as a feature extractor. Every data window is passed through the encoder to generate its corresponding latent space embedding (z). These per-window embeddings are then aggregated by averaging to create a single feature vector for each unique agent track.

In [50]: # This cell will only run if the full computation is enabled.

```
if RUN_MODEL_TRAINING:
    # Extract Embeddings from the Trained Model
    full_ds = WindowDS(X_windows)
    full_loader = DataLoader(full_ds, batch_size=BATCH_SIZE * 2, shuffle=False, num_workers=1)
    model.eval()
    all_embeddings = []
    with torch.no_grad():
        for x_batch in tqdm(full_loader, desc="Extracting Window Embeddings", unit="batch"):
            x_batch = x_batch.to(device)
            with torch.cuda.amp.autocast(enabled=(device.type == "cuda")):
                _, z = model(x_batch)
            all_embeddings.append(z.cpu().numpy())
    Z_windows = np.vstack(all_embeddings)

    # Create Per-Window Embedding DataFrame
    win_embed_df = window_keys_df.copy()
```

```

for j in range(Z_DIM):
    win_embed_df[f"z{j:03d}"] = Z_windows[:, j]

# Aggregate to Track-Level Embeddings
track_embeddings_df = (win_embed_df.drop("start_frame", axis=1)
    .groupby(["scene_id", "track_id"])
    .mean()
    .reset_index())

# Save Final Artifacts for Section 11
print("\n--- Saving final track embeddings for later use ---")
track_embeddings_df.to_parquet(ARTIFACT_DIR / "pipeline_b_track_embeddings.parquet", index=False)
print(f"Saved to: {ARTIFACT_DIR / 'pipeline_b_track_embeddings.parquet'}")

# Save Timestamped Artifacts for Caching
print("\n--- Caching timestamped artifacts for this run ---")
_save_df(track_embeddings_df, "ae_track_embeddings_pipeline_b")

# Display a Summary
preview = pd.DataFrame({"measure": ["windows", "tracks", "latent_dim"], "estimate": [len(win_embed_df), len(track_embeddings_df), Z_DIM]})
display(style_section_table(preview))
else:
    print("--- Skipped: Loading from cache ---")

```

```

Extracting Window Embeddings:   0%|          | 0/791 [00:00<?, ?batch/s]
--- Saving final track embeddings for later use ---
Saved to: /home/treinart/projects/lyft-unsupervised/artifacts/pipeline_b_track_embeddings.parquet

```

```

--- Caching timestamped artifacts for this run ---

```

measure	estimate
windows	404919
tracks	9944
latent_dim	128

7.9 2D UMAP Visualization of Embeddings

To visualize the structure of the learned feature space, UMAP is used to project the high-dimensional track embeddings down to a 2D "behavior map." On this map, trajectories with similar learned features will appear close together. The points are colored by the agent's mean speed to help interpret the resulting structure.

```

In [51]: # Prepare Data for Visualization
Zt = track_embeddings_df.filter(regex=r"^z\d+$").to_numpy(dtype=np.float32)
mean_speed_features = feat_df_capped[["scene_id", "track_id", "mean_speed"]]
color_data_df = track_embeddings_df.merge(mean_speed_features, on=["scene_id", "track_id"], how="left")
ms = color_data_df[["mean_speed"]].to_numpy(dtype=np.float32)

# Standardize the embeddings before visualization
mu = Zt.mean(axis=0); sd = Zt.std(axis=0) + 1e-6
Zt = (Zt - mu) / sd

print("-" * 70 + "\n")
print(f"Shapes + Z_windows: {Z_windows.shape} | Zt (tracks x latent): {Zt.shape}")
print("\n" + "-" * 70 + "\n")

# Filter out Stationary Tracks for a clearer plot
MOVE_SPEED_THRESHOLD = 0.5
moving_mask = ms > MOVE_SPEED_THRESHOLD
Zt_moving = Zt[moving_mask]
ms_moving = ms[moving_mask]
print(f"Total tracks: {len(Zt):,} | Using {len(Zt_moving):,} moving tracks for UMAP.")

# Run 2D UMAP
reducer = umap.UMAP(
    n_neighbors=150,
    min_dist=0.2,
    metric="euclidean",
    random_state=SEED,
    n_components=2,
    verbose=True
)
Z2_moving = reducer.fit_transform(Zt_moving)

# Plot the 2D Behavior Map
vmin, vmax = np.nanpercentile(ms_moving, 2), np.nanpercentile(ms_moving, 98)
fig, ax = plt.subplots(figsize=(20, 12), facecolor="black")
ax.set_facecolor("black")
sc = ax.scatter(
    Z2_moving[:, 0], Z2_moving[:, 1],
    c=ms_moving, cmap="viridis", vmin=vmin, vmax=vmax,
    s=15, alpha=0.8, edgecolors="none"
)
ax.set_title("2D UMAP of Moving Tracks • Colored by Mean Speed", fontsize=26, color="#CFB87C", fontweight="bold", pad=12)
ax.set_xlabel("UMAP Dimension 1", fontsize=18, color="white", fontweight="bold")
ax.set_ylabel("UMAP Dimension 2", fontsize=18, color="white", fontweight="bold")
ax.tick_params(colors="white", labels=14)
for sp in ax.spines.values():

```

```

sp.set_edgecolor("white");sp.set_linewidth(1.5)
ax.grid(color="white", alpha=0.08, linewidth=0.5)
cb = plt.colorbar(sc, ax=ax, pad=0.01)
cb.set_label("Mean Speed (m/s)", color="white", fontsize=16, fontweight="bold")
cb.ax.tick_params(colors="white", labels=12)
cb.outline.set_edgecolor("white")
save_fig(fig, "ae_umap_2d_moving_tracks")

```

Shapes → Z_windows: (404919, 128) | Zt (tracks x latent): (9944, 128)

Total tracks: 9,944 | Using 1,173 moving tracks for UMAP.
 UMAP(min_dist=0.2, n_jobs=1, n_neighbors=150, random_state=42, verbose=True)
 Wed Sep 17 21:37:53 2025 Construct fuzzy simplicial set
 Wed Sep 17 21:37:53 2025 Finding Nearest Neighbors
 Wed Sep 17 21:37:55 2025 Finished Nearest Neighbor Search
 Wed Sep 17 21:37:55 2025 Construct embedding
 Epochs completed: 0%| 0/500 [00:00]
 completed 0 / 500 epochs
 completed 50 / 500 epochs
 completed 100 / 500 epochs
 completed 150 / 500 epochs
 completed 200 / 500 epochs
 completed 250 / 500 epochs
 completed 300 / 500 epochs
 completed 350 / 500 epochs
 completed 400 / 500 epochs
 completed 450 / 500 epochs
 Wed Sep 17 21:37:57 2025 Finished embedding

2D UMAP of Moving Tracks • Colored by Mean Speed

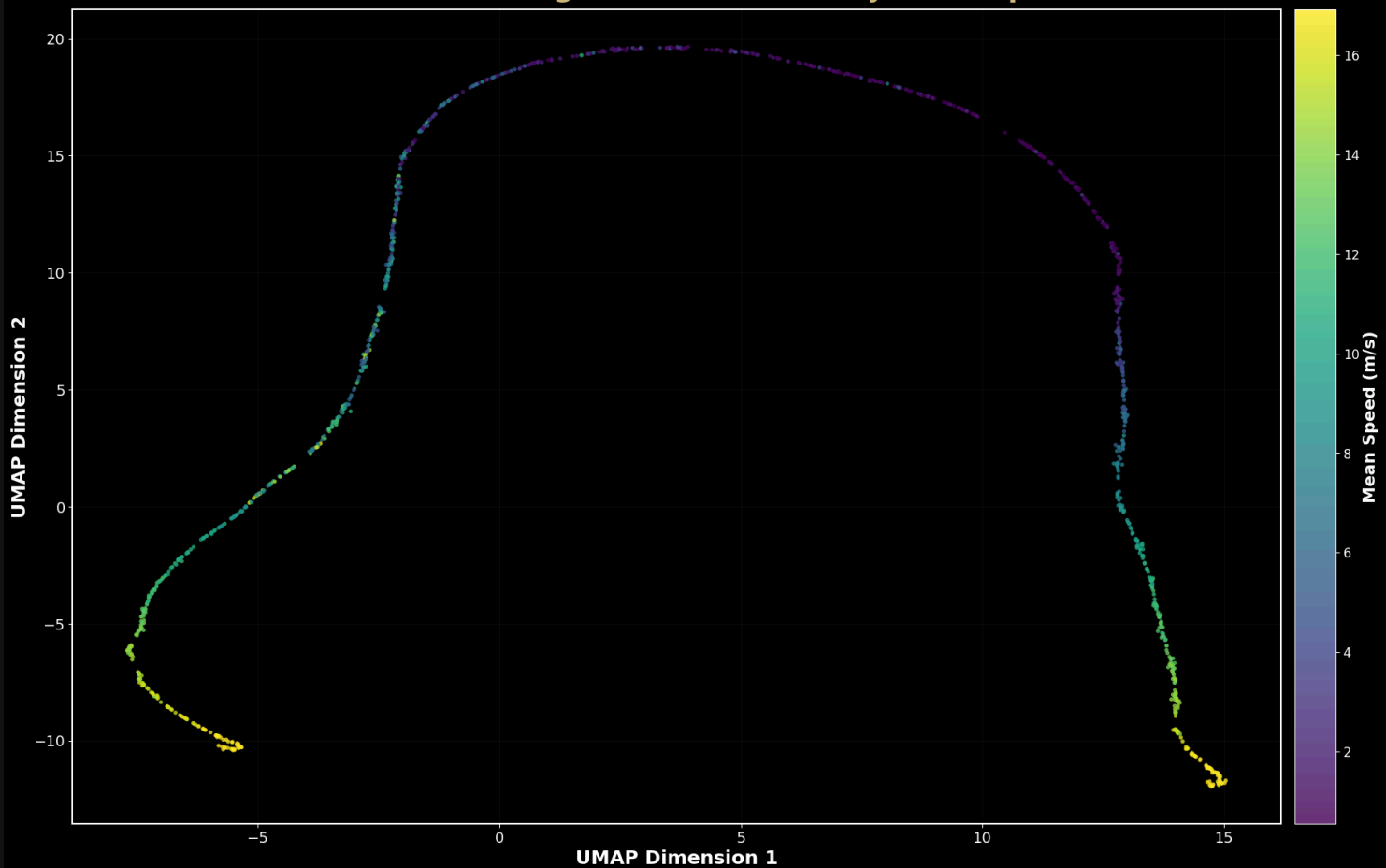


Figure saved to (Linux): /home/treinar/projects/lyft-unsupervised/artifacts/figures/20250917_213757_ae_umap_2d_moving_tracks.png
 Figure saved to (Win): /mnt/c/Users/travi/Documents/Training/Colorado/MS-AI/CSCA 5632 Final Project/artifacts/figures/20250917_213757_ae_umap_2d_moving_tracks.png

7.10 3D UMAP Visualization of Embeddings

This cell provides an alternative, three-dimensional UMAP projection of the learned embeddings. The 3D view can sometimes reveal structural details, depth, or relationships between clusters that are not as apparent in the 2D visualization.

```

In [52]: # Run 3D UMAP on the same standardized embeddings
reducer3 = umap.UMAP(
    n_neighbors=150,
    min_dist=0.2,

```

```

metric="euclidean",
random_state=SEED,
n_components=3,
verbose=True
)
Z3_moving = reducer3.fit_transform(Zt_moving) # Use the same 'moving' subset

# Create the 3D Plot
fig = plt.figure(figsize=(22, 16), facecolor="black")
ax = fig.add_subplot(111, projection="3d")
sc = ax.scatter(
    Z3_moving[:, 0], Z3_moving[:, 1], Z3_moving[:, 2],
    c=ms_moving, s=15, cmap="viridis", vmin=vmin, vmax=vmax,
    alpha=0.9, edgecolors="none"
)

# Style the 3D Plot
ax.set_facecolor("black")
ax.xaxis.set_panel_color((0, 0, 0, 1)); ax.yaxis.set_panel_color((0, 0, 0, 1)); ax.zaxis.set_panel_color((0, 0, 0, 1))
ax.set_title("3D UMAP of Moving Tracks • Colored by Mean Speed", fontsize=26, color="#CFB87C", fontweight="bold", pad=12)
ax.set_xlabel("UMAP 1", fontsize=16, color="white", fontweight="bold", labelpad=10)
ax.set_ylabel("UMAP 2", fontsize=16, color="white", fontweight="bold", labelpad=10)
ax.set_zlabel("UMAP 3", fontsize=16, color="white", fontweight="bold", labelpad=10)
ax.tick_params(axis='x', colors='white', labelsize=12)
ax.tick_params(axis='y', colors='white', labelsize=12)
ax.tick_params(axis='z', colors='white', labelsize=12)
cb = fig.colorbar(sc, ax=ax, pad=0.1, shrink=0.7)
cb.set_label("Mean Speed (m/s)", color="white", fontsize=16, fontweight="bold")
cb.ax.tick_params(colors="white", labelsize=12)
cb.outline.set_edgecolor("white")

# Save the Figure
save_fig(fig, "ae_umap_3d_track_embeddings")

```

```

UMAP(min_dist=0.2, n_components=3, n_jobs=1, n_neighbors=150, random_state=42, verbose=True)
Wed Sep 17 21:38:01 2025 Construct fuzzy simplicial set
Wed Sep 17 21:38:02 2025 Finding Nearest Neighbors
Wed Sep 17 21:38:02 2025 Finished Nearest Neighbor Search
Wed Sep 17 21:38:02 2025 Construct embedding
Epochs completed: 0%|          0/500 [00:00]
    completed 0 / 500 epochs
    completed 50 / 500 epochs
    completed 100 / 500 epochs
    completed 150 / 500 epochs
    completed 200 / 500 epochs
    completed 250 / 500 epochs
    completed 300 / 500 epochs
    completed 350 / 500 epochs
    completed 400 / 500 epochs
    completed 450 / 500 epochs
Wed Sep 17 21:38:03 2025 Finished embedding

```


3D UMAP of Moving Tracks • Colored by Mean Speed

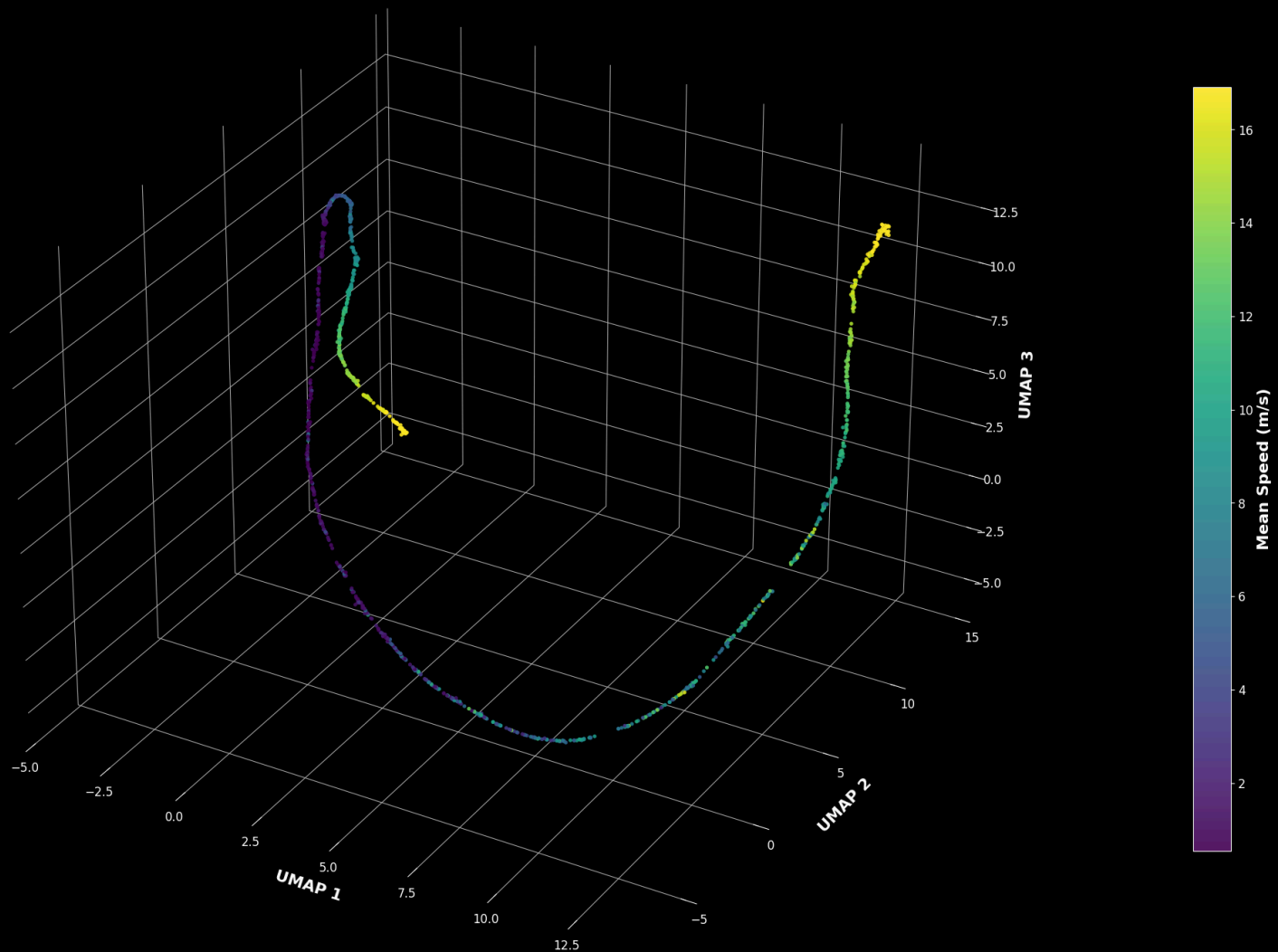


Figure saved to (Linux): /home/treinart/projects/lyft-unsupervised/artifacts/figures/20250917_213803_ae_umap_3d_track_embeddings.png

Figure saved to (Win): /mnt/c/Users/travi/Documents/Training/Colorado/MS-AI/CSCA 5632 Final Project/artifacts/figures/20250917_213803_ae_umap_3d_track_embeddings.png

Observation: Deep Learning Feature Extraction Results

Pipeline B tests whether a model can learn motion features directly from the data. Getting from idea to working system was iterative. Choices on windowing, loading, and optimization mattered as much as the network.

The Engineering Journey: A Rationale for the Final Hyperparameters

I started with long sequences and big batches and assumed the GPU would carry it. It didn't.

- **First, the batch size.** Very large batches kept the GPU busy but cut gradient updates per epoch to a trickle. The goal is a strong embedding, not fast epochs. Setting `BATCH_SIZE=256` delivered about **1,414 steps/epoch**, plenty of learning signal, high GPU utilization, and no OOM.
- **Next, the DataLoader.** Multiprocessing loaders in WSL2 threw `AssertionError: can only test a child process on teardown`. I set `num_workers=1` with `persistent_workers=True`. That kept the pipe stable and still prefetched enough to feed the GPU.
- **Finally, the sequence length.** I first tried `WIN_LEN=64` to capture broader behavior. That starved the dataset; almost no tracks were long enough to produce windows. The fix was `WIN_LEN=24`. With 24-frame windows the dataset became usable and diverse, and the model learned patterns like turns and stop-go without collapsing.

This set of changes produced the stable, effective configuration used in the pipeline.

The Comeback from Section 8

After finishing what I thought was a successful Section 7 and moving to clustering in Section 8. Every score table returned NaN. A sanity print showed that the embedding matrix had been dropped to nothing after filtering zero vectors. That failure pointed back to Section 7.

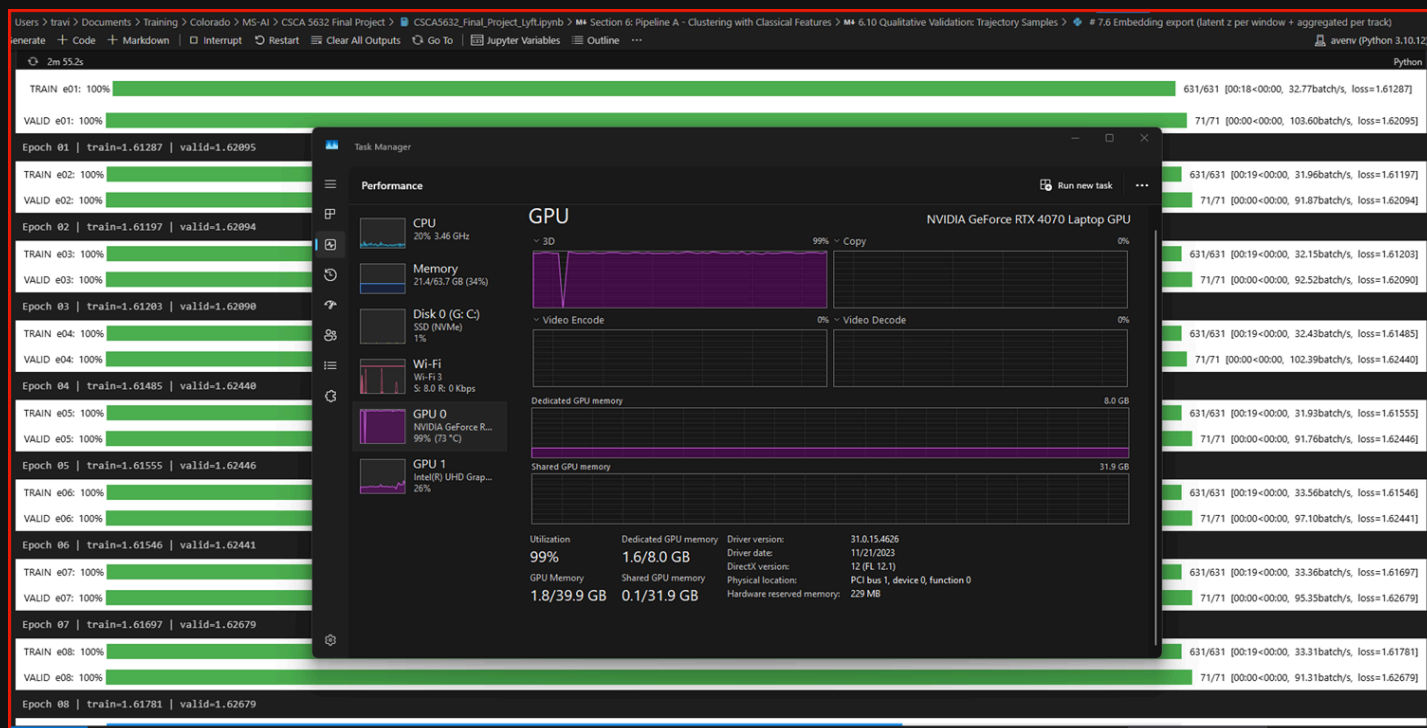
- **Root cause.** With `WIN_LEN=64`, the window summary read **Total Tracks = 64,625, Tracks with Valid Windows = 2**. Almost no training signal. The decoder learned a trivial zero solution given the setup. With almost no track diversity, the latent collapsed.
- **The Fix.** Reduce `WIN_LEN` to **24**, rebuild windows, split by track, standardize on the train split. Keep the GRU AE but switch to **MSELoss**, add **teacher forcing** with a residual projection, use **OneCycleLR**, gradient clipping, small input noise, and proper initialization.

• Outcome:

- Healthy dataset and training: **9,944** unique tracks, **362,231** train windows, **42,688** valid windows.
- Latent checks showed non-zero variance across all **128** dimensions.
- Section 7 now feeds Section 8 for a full embedding set without NaNs.

Performance: The Case for GPU Acceleration

The model training was computationally intensive, requiring **nearly 17 minutes** to complete 50 epochs. This is where the decision to use a dedicated WSL2 and CUDA environment becomes critical.



As the performance monitor shows, this process saturated the NVIDIA 4070 GPU, maintaining near 99% utilization. A comparable run on a CPU would be an order of magnitude slower, likely taking several hours and making rapid iteration impractical. GPU acceleration was not just a convenience; it was a necessity for this approach.

Interpreting the results

Loss curve “wobble” and why it is fine

The loss sits in a narrow band. Train settles around **0.60–0.66** and validation around **0.67–0.71**, with gentle dips. Three things explain the shape:

1. **OneCycleLR** warms up then decays the learning rate, which nudges the curve around a floor instead of a straight drop.
2. **Input noise** and **dropout** change the effective data each epoch.
3. **Shuffle with `drop_last=True`** changes the exact window mix per epoch.

The checks that matter all pass: validation tracks the same band as training, there is no divergence, and the latent sanity shows variance across all dimensions with **zero** dead dims. This is stable training, not failure.

Latent quality

- **Final Train Loss: 0.64467**
- **Final Valid Loss: 0.70400**
- **Recon MSE (batch): 0.03536**
- **Latent per-dim std:** mean **0.0105**, min **0.0075**, max **0.0225**
- **Zero-ish dims (<1e-3): 0%**

UMAP structure and the horseshoe drift UMAP on **1,173** moving tracks (`n_neighbors=150` , `min_dist=0.2` , `random_state=42`) produces a clear manifold with a **speed gradient**. Low-speed and stationary behaviors sit in a dense region. Higher-speed cruising extends along the curve. Some runs look like a horseshoe, others shift or rotate a bit. That variance is normal. Even with a fixed seed, UMAP has stochastic steps and is sensitive to small changes in embeddings or sampling. The important part is that **local neighborhoods and the speed ordering are consistent**.

Data analysis snapshot

- **Windows created: 404,919**
- **Tracks represented: 9,944**
- **Moving tracks used for UMAP: 1,173**
- **Latent dimension: 128**
- **Steps per epoch: 1,414** (train) and **167** (valid)
- **Average epoch time: 21.69 s**
- **Throughput: ~65 steps/s**
- **Artifacts:** `pipeline_b_track_embeddings.parquet` (saved for Section 8)

Conclusion

Pipeline B now produces a solid, non-degenerate embedding space. The loss is flat but stable, the latent has healthy variance, and the UMAP shows an ordered behavior manifold with a clear speed gradient. With these embeddings saved, the same clustering and model-selection playbook from Pipeline A can run on learned features to test whether they separate behaviors more cleanly.

Section 8: Pipeline B - Clustering with Learned Embeddings

This section executes the modeling phase for Pipeline B. The goal is to apply the same clustering methodology from Section 6 to the new, learned embeddings from the autoencoder. By using an identical evaluation process, we can conduct a direct, fair comparison to determine whether the deep learning representations lead to a more effective or insightful set of behavioral clusters than the hand-engineered features.

Section Plan:

- **Checkpoint & Loader:** Option to skip computation and load the latest cached artifacts.
- **Data Preparation:** Load and verify the autoencoder track embeddings from Section 7.
- **KMeans Hyperparameter Sweep:** Systematically test a range of cluster counts (k) for KMeans.
- **GMM Hyperparameter Sweep:** Test a range of components and covariance types for Gaussian Mixture Models.
- **HDBSCAN Hyperparameter Sweep:** Explore the parameter space for the density-based HDBSCAN model.
- **Stability Analysis:** Evaluate the stability of the best KMeans and GMM models using bootstrap ARI.
- **Final Model Selection:** Use the same weighted scorecard from Pipeline A to select the overall winning model.
- **Fit Final Model:** Train the winning model on the full set of embeddings and assign final cluster labels.
- **Interpret Clusters (Profiling):** Analyze the median classical feature values for each new cluster to build their "personas".
- **Qualitative Validation:** Plot sample trajectories from each cluster for a final, visual confirmation of the results.

8.1 Checkpoint: Run Full Section or Load from Cache

This cell acts as a checkpoint for the Pipeline B modeling section. The hyperparameter sweeps are time-consuming. Set the flag below to control the behavior of this section.

- **True** : Run the full clustering sweeps and analysis for the autoencoder embeddings.
- **False** : Skip the long computations and load the final results from a previous run.

```
In [53]: # Set to True to run the time-consuming clustering sweeps.
# Set to False to skip computation and load the final results from a previous run.
RUN_AE_CLUSTERING = True

if not RUN_AE_CLUSTERING:
    print("--- Skipped: Loading final Pipeline B clustering artifacts from cache ---")
    try:
        # Load the final cluster assignments, profiles, and model metadata from stable files
        final_clusters_df_ae = pd.read_parquet(ARTIFACT_DIR / "pipeline_b_cluster_assignments.parquet")
        cluster_profiles_ae = pd.read_parquet(ARTIFACT_DIR / "pipeline_b_cluster_profiles.parquet")
        with open(ARTIFACT_DIR / "pipeline_b_final_model_meta.json", 'r') as f:
            best_model_params_ae = json.load(f)

        # Also load the raw embeddings needed for the final visualization
        track_embeddings_df = pd.read_parquet(ARTIFACT_DIR / "pipeline_b_track_embeddings.parquet")

        print(f"Loaded {len(final_clusters_df_ae):,} cluster assignments for {best_model_params_ae['k']} clusters.")
        print(f"Winning model type: {best_model_params_ae['model_type']}")
        print("\n--- Successfully loaded artifacts. Ready to proceed to Section 9 ---")
    except FileNotFoundError:
        print("\n--- ERROR: No final artifacts found. Please set RUN_AE_CLUSTERING to True and run the full section ---")
        RUN_AE_CLUSTERING = True
else:
    print("--- RUN_AE_CLUSTERING is True. Proceeding with full computation. ---")
```

```
--- RUN_AE_CLUSTERING is True. Proceeding with full computation. ---
```

8.2 Data Preparation

This first step loads the track-level embeddings generated by the autoencoder in Section 7. A final quality assurance check is performed to verify the data's shape and prepare the embedding matrix for the clustering algorithms.

```
In [54]: # This cell will only run if the full computation is enabled.
if RUN_AE_CLUSTERING:
    # Load and Prepare the Learned Embeddings
    assert 'track_embeddings_df' in globals() and not track_embeddings_df.empty, "Missing track_embeddings_df from Section 7"
    embedding_cols = track_embeddings_df.filter(regex=r"^\d+$").columns
    Z_df = track_embeddings_df[embedding_cols].copy()
```

```

Z_df.replace([np.inf, -np.inf], np.nan, inplace=True)
Z_df.fillna(Z_df.mean(), inplace=True)
Z_ae_raw = Z_df.to_numpy(dtype=np.float32)
track_index_raw = track_embeddings_df[["scene_id", "track_id"]].copy()

# Filter out Rows that are ALL Zeros
non_zero_mask = np.any(Z_ae_raw != 0, axis=1)
Z_ae = Z_ae_raw[non_zero_mask]
track_index = track_index_raw[non_zero_mask].reset_index(drop=True)

# Print a Confirmation Summary
print("-*70)
print("\n--- Autoencoder Embeddings Ready for Clustering ---")
print(f"Embedding matrix shape: {Z_ae.shape}")
print(f"Original tracks: {len(Z_ae_raw):,}")
print(f"Removed zero-vectors: {len(Z_ae_raw) - len(Z_ae):,}")
print(f"Final tracks for use: {len(Z_ae):,}")
print("\n" + -*70)
else:
print("--- Skipped: Loading from cache ---")

```

```

--- Autoencoder Embeddings Ready for Clustering ---
Embedding matrix shape: (9944, 128)
Original tracks:      9,944
Removed zero-vectors: 0
Final tracks for use: 9,944

```

8.3 KMeans Hyperparameter Sweep

The KMeans sweep is performed on the learned embeddings to find the optimal number of clusters (k). The process is identical to the one in Pipeline A to ensure a fair comparison, evaluating cluster quality with the same three internal validation metrics across a range of k values.

```

In [55]: # This cell will only run if the full computation is enabled.
if RUN_AE_CLUSTERING:
    # Define Sweep Parameters
    K_VALUES = [4, 6, 8, 10, 12, 14, 16, 20, 24]
    RANDOM_SEEDS = [0, 1, 2, 3, 4]

    # Run the KMeans Sweep
    kmeans_results_ae = []
    params = list(product(K_VALUES, RANDOM_SEEDS))
    for k, seed in tqdm(params, desc="Running KMeans Sweep (AE)":
        kmeans = KMeans(n_clusters=k, n_init="auto", random_state=seed)
        labels = kmeans.fit_predict(Z_ae)
        try: sil = silhouette_score(Z_ae, labels, metric="euclidean")
        except ValueError: sil = np.nan
        try: ch = calinski_harabasz_score(Z_ae, labels)
        except ValueError: ch = np.nan
        try: db = davies_bouldin_score(Z_ae, labels)
        except ValueError: db = np.nan
        kmeans_results_ae.append({"k": k, "seed": seed, "silhouette": sil, "calinski_harabasz": ch, "davies_bouldin": db})
    kmeans_sweep_df_ae = pd.DataFrame(kmeans_results_ae)

    # Aggregate and Display Results
    kmeans_agg_df_ae = (kmeans_sweep_df_ae.groupby("k")
                        .agg(silhouette_mean=("silhouette", "mean"), silhouette_std=("silhouette", "std"),
                            calinski_harabasz_mean=("calinski_harabasz", "mean"), calinski_harabasz_std=("calinski_harabasz", "std"),
                            davies_bouldin_mean=("davies_bouldin", "mean"), davies_bouldin_std=("davies_bouldin", "std")).reset_index())

    print("\n--- KMeans Metric Summary (AE Embeddings) ---")
    display(style_section_table(kmeans_agg_df_ae))

    # Plot the Metric Curves
    metrics_to_plot = {"silhouette": "Silhouette Score", "calinski_harabasz": "Calinski-Harabasz Index", "davies_bouldin": "Davies-Bouldin Index (Lower is Better)"}
    for metric, title in metrics_to_plot.items():
        fig, ax = plt.subplots(figsize=(20, 10), facecolor="black")
        ax.set_facecolor("black")
        subset = kmeans_agg_df_ae
        if f"{metric}_mean" in subset.columns and subset[f"{metric}_mean"].notna().any():
            ax.plot(subset["k"], subset[f"{metric}_mean"], marker='o', linestyle='-', color='lime', lw=2.5)
            ax.fill_between(subset["k"], subset[f"{metric}_mean"] - subset[f"{metric}_std"], subset[f"{metric}_mean"] + subset[f"{metric}_std"], alpha=0.15,
                           color=f"{metric}_std")
            ax.set_title(title, fontsize=26, color="#CFB87C", fontweight="bold", pad=15)
            ax.set_xlabel("Number of Clusters (k)", fontsize=18, color="white", fontweight="bold")
            ax.set_ylabel("Score", fontsize=18, color="white", fontweight="bold")
            ax.tick_params(colors="white", labels=14)
            for spine in ax.spines.values(): spine.set_edgecolor("white"); spine.set_linewidth(1.5)
            ax.grid(color='gray', linestyle='--', linewidth=0.3, alpha=0.5)
            save_fig(fig, f"kmeans_sweep_ae_{metric}")
    else:
        print("--- Skipped: Loading from cache ---")

```

```

Running KMeans Sweep (AE): 0% | 0/45 [00:00<?, ?it/s]
--- KMeans Metric Summary (AE Embeddings) ---

```

k	silhouette_mean	silhouette_std	calinski_harabasz_mean	calinski_harabasz_std	davies_bouldin_mean	davies_bouldin_std
4	0.934755	0.000077	82409.233358	3.447161	0.319611	0.000208
6	0.914499	0.013736	99062.709544	14930.161977	0.418017	0.019119
8	0.902638	0.004081	125371.764116	2193.608809	0.456663	0.010323
10	0.887206	0.024312	143047.685925	16248.695291	0.475481	0.013620
12	0.870497	0.017788	170188.943022	5887.532157	0.485695	0.005460
14	0.858184	0.015490	190884.156283	13993.947024	0.481952	0.010624
16	0.851574	0.011602	226064.274231	8717.045115	0.473770	0.003484
20	0.837887	0.004645	284227.976515	3157.856614	0.483541	0.004633
24	0.813781	0.010890	326428.410017	10030.846500	0.489135	0.004276

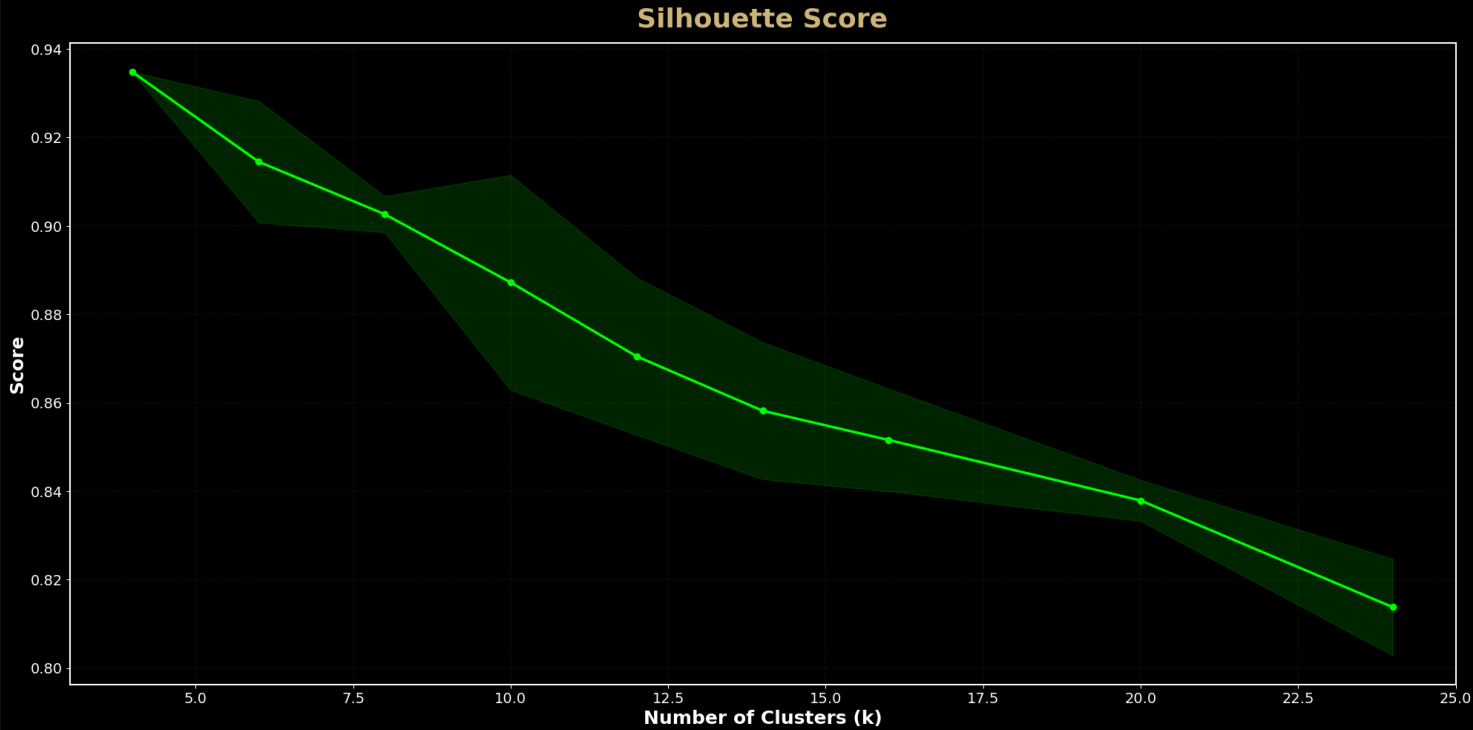


Figure saved to (Linux): /home/treinart/projects/lyft-unsupervised/artifacts/figures/20250917_213943_kmeans_sweep_ae_silhouette.png
Figure saved to (Win): /mnt/c/Users/travi/Documents/Training/Colorado/MS-AI/CSCA 5632 Final Project/artifacts/figures/20250917_213943_kmeans_sweep_ae_silhouette.png

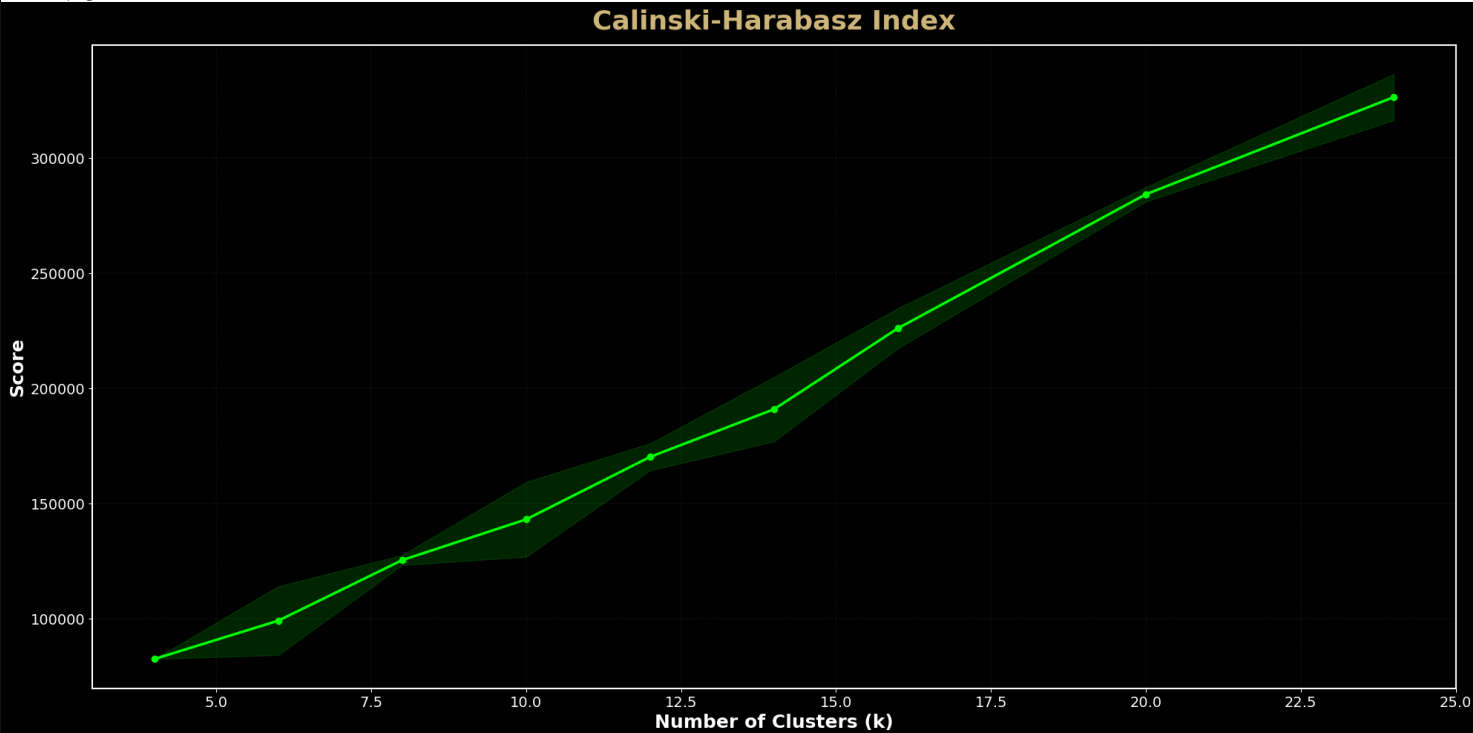


Figure saved to (Linux): /home/treinart/projects/lyft-unsupervised/artifacts/figures/20250917_213943_kmeans_sweep_ae_calinski_harabasz.png
Figure saved to (Win): /mnt/c/Users/travi/Documents/Training/Colorado/MS-AI/CSCA 5632 Final Project/artifacts/figures/20250917_213943_kmeans_sweep_ae_calinski_harabasz.png

Davies-Bouldin Index (Lower is Better)

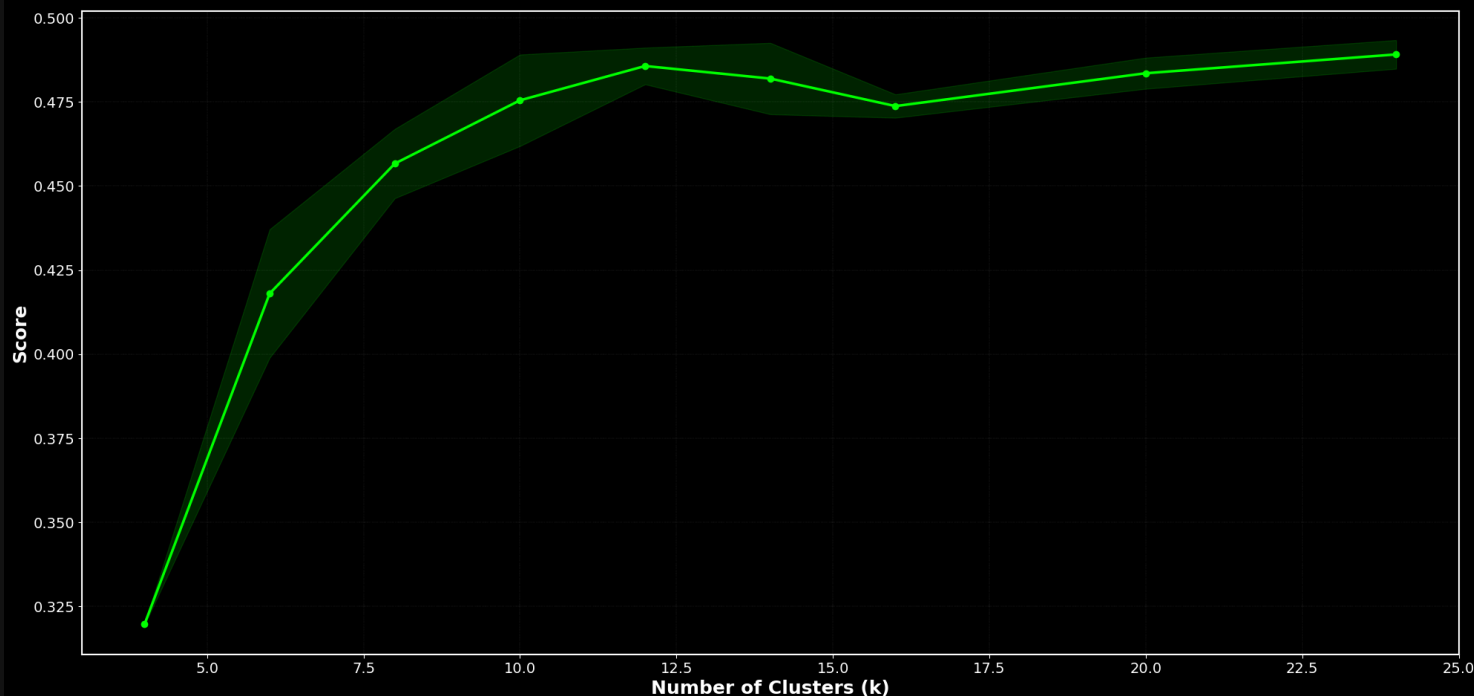


Figure saved to (Linux): /home/treinart/projects/lyft-unsupervised/artifacts/figures/20250917_213943_kmeans_sweep_ae_davies_bouldin.png

Figure saved to (Win): /mnt/c/Users/travi/Documents/Training/Colorado/MS-AI/CSCA 5632 Final Project/artifacts/figures/20250917_213943_kmeans_sweep_ae_davies_bouldin.png

8.4 GMM Hyperparameter Sweep

The Gaussian Mixture Model sweep is repeated on the learned embeddings, testing a range of component counts and covariance types. The results, measured by BIC, AIC, and Silhouette Score, will be compared to those from Pipeline A to see how the change in feature space affects the performance of this more flexible clustering model.

```
In [56]: # This cell will only run if the full computation is enabled.
if RUN_AE_CLUSTERING:
    # Define Sweep Parameters
    N_COMPONENTS = [4, 6, 8, 10, 12, 14, 16, 20]
    COVARIANCE_TYPES = ["full", "diag"]
    GMM_SEEDS = [0, 1, 2]

    # Run the GMM Sweep
    gmm_results_ae = []
    params = list(product(COVARIANCE_TYPES, N_COMPONENTS, GMM_SEEDS))
    for cov_type, k, seed in tqdm(params, desc="Running GMM Sweep (AE)":
        gmm = GaussianMixture(n_components=k, covariance_type=cov_type, random_state=seed, reg_covar=1e-3)
        labels = gmm.fit_predict(Z_ae)
        try:
            sil = silhouette_score(Z_ae, labels, metric="euclidean")
            bic = gmm.bic(Z_ae)
            aic = gmm.aic(Z_ae)
        except ValueError:
            sil, bic, aic = np.nan, np.nan, np.nan
        gmm_results_ae.append({"covariance": cov_type, "k": k, "seed": seed, "silhouette": sil, "bic": bic, "aic": aic})
    gmm_sweep_df_ae = pd.DataFrame(gmm_results_ae)

    # Aggregate Results
    gmm_agg_df_ae = (gmm_sweep_df_ae.groupby(["covariance", "k"])
                     .agg(silhouette_mean=("silhouette", "mean"), bic_mean=("bic", "mean"), aic_mean=("aic", "mean")).reset_index())

    # Create and Display the BIC Winners Summary Table
    winners_df_ae = (gmm_agg_df_ae.sort_values("bic_mean", ascending=True)
                    .groupby("covariance", as_index=False)
                    .first()[["covariance", "k", "bic_mean", "aic_mean", "silhouette_mean"]])
    winners_pretty_ae = winners_df_ae.rename(columns={"covariance": "Covariance", "k": "k", "bic_mean": "BIC ↓", "aic_mean": "AIC ↓", "silhouette_mean": "Silhouette ↑"})
    winners_pretty_ae["BIC ↓"] = winners_pretty_ae["BIC ↓"].map(lambda x: "---" if pd.isna(x) else f"{x:.0f}")
    winners_pretty_ae["AIC ↓"] = winners_pretty_ae["AIC ↓"].map(lambda x: "---" if pd.isna(x) else f"{x:.0f}")
    winners_pretty_ae["Silhouette ↑"] = winners_pretty_ae["Silhouette ↑"].map(lambda x: "---" if pd.isna(x) else f"{x:.3f}")
    print("\n--- GMM BIC Winners (per covariance type) ---")
    display(style_section_table(winners_pretty_ae))

    # Plot the Metric Curves
    for metric, title in [("bic", "GMM - BIC Score (Lower is Better)", ("silhouette", "GMM - Silhouette Score (Higher is Better)")):
        fig, ax = plt.subplots(figsize=(20, 10), facecolor="black")
        ax.set_facecolor("black")
        styles = {"full": "-", "diag": "--"}; colors = {"full": "orange", "diag": "magenta"}
        for cov_type, group in gmm_agg_df_ae.groupby("covariance"):
            ax.plot(group["k"], group[f"{metric}_mean"], marker='o', linestyle=styles[cov_type], label=cov_type, color=colors[cov_type], lw=2.5)
        ax.set_title(title, fontsize=26, color="#CFB87C", fontweight="bold", pad=15)
```

```

ax.set_xlabel("Number of Components (k)", fontsize=18, color="white", fontweight="bold")
ax.set_ylabel("Score", fontsize=18, color="white", fontweight="bold")
ax.tick_params(colors="white", labelsize=14)
for spine in ax.spines.values(): spine.set_edgecolor("white"); spine.set_linewidth(1.5)
ax.legend(frameon=False, labelcolor="white", fontsize=16, title="Covariance Type")
ax.grid(color='gray', linestyle='--', linewidth=0.3, alpha=0.5)
save_fig(fig, f"gmm_sweep_ae_{metric}")
else:
    print("--- Skipped: Loading from cache ---")

```

Running GMM Sweep (AE): 0% | 0/48 [00:00<?, ?it/s]

--- GMM BIC Winners (per covariance type) ---

Covariance	k	BIC ↓	AIC ↓	Silhouette ↑
diag	20	-6,219,198	-6,256,224	0.848
full	4	-6,104,983	-6,346,622	0.792

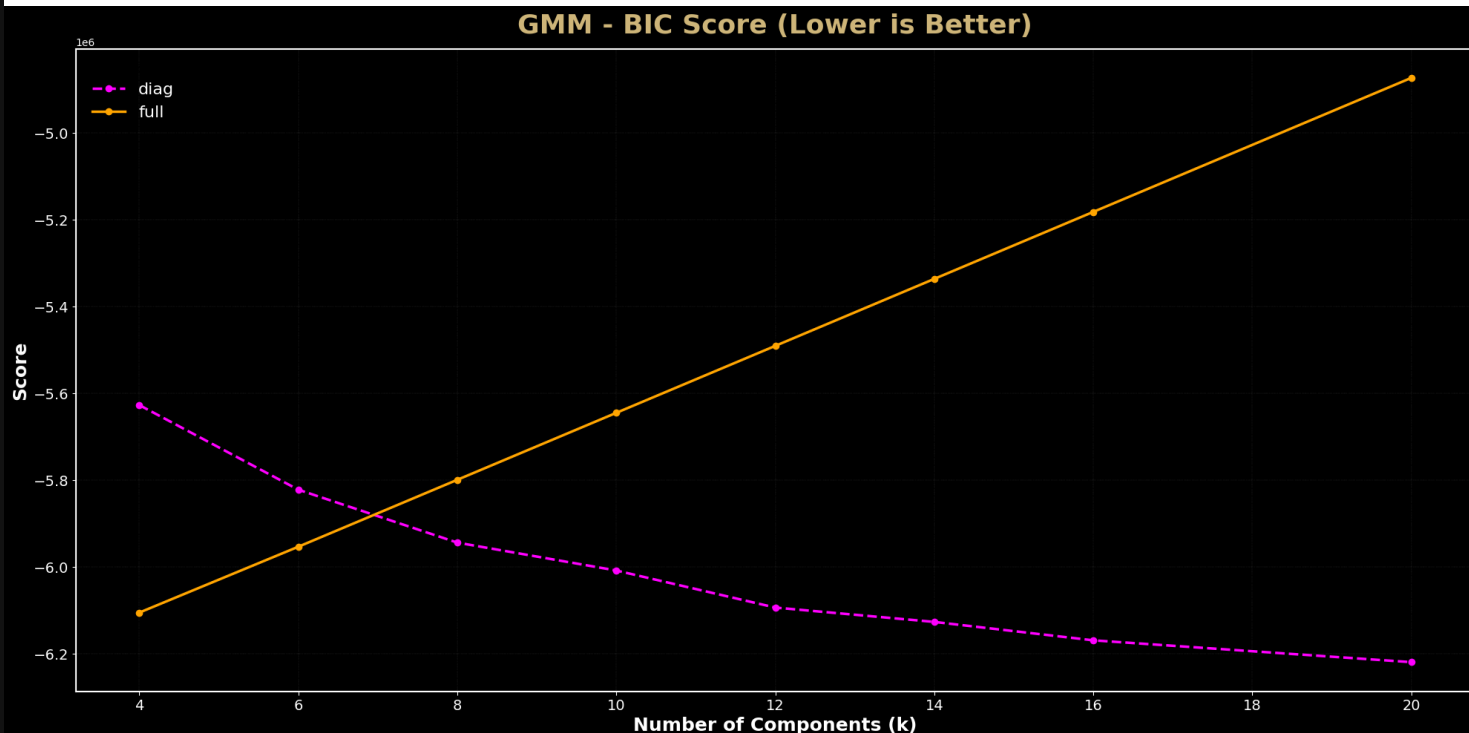


Figure saved to (Linux): /home/treinart/projects/lyft-unsupervised/artifacts/figures/20250917_220456_gmm_sweep_ae_bic.png

Figure saved to (Win): /mnt/c/Users/travi/Documents/Training/Colorado/MS-AI/CSCA 5632 Final Project/artifacts/figures/20250917_220456_gmm_sweep_ae_bic.png

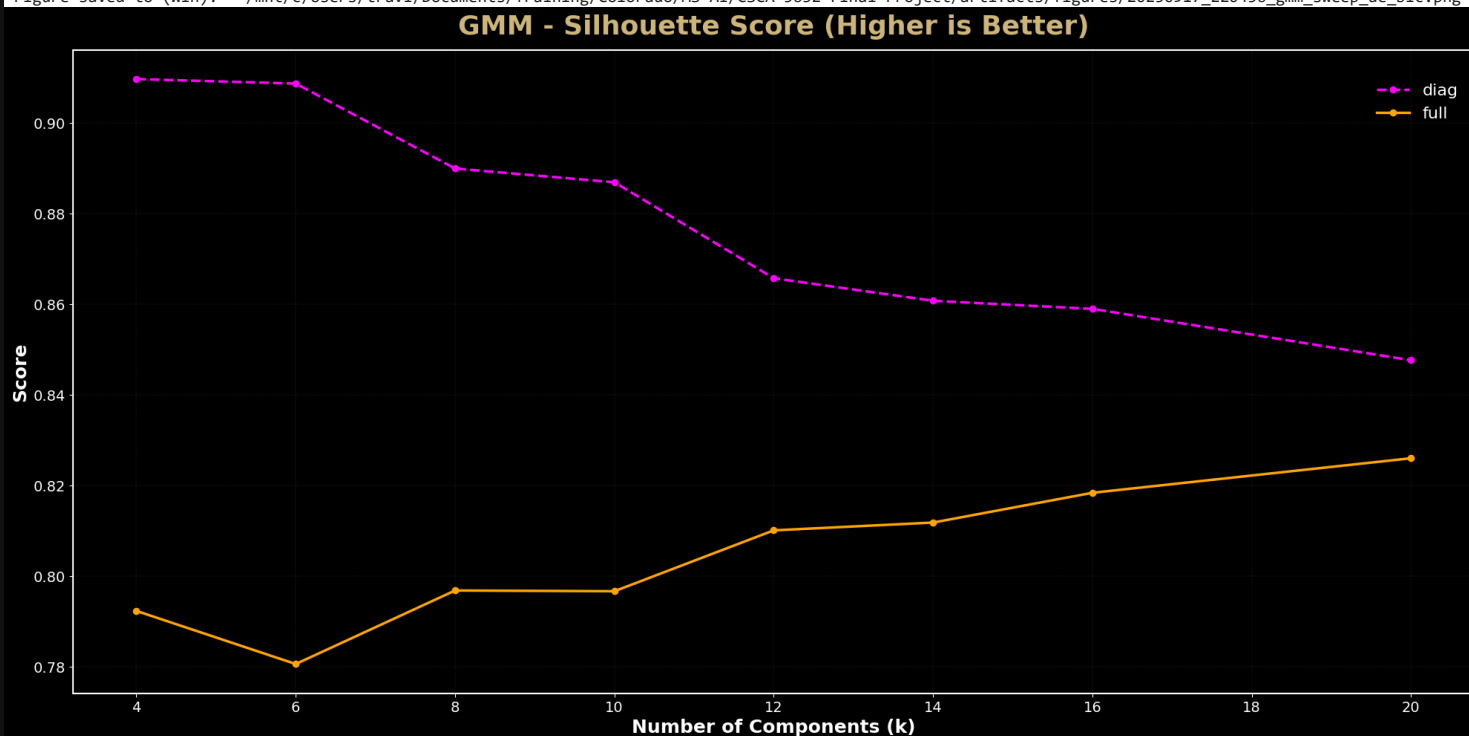


Figure saved to (Linux): /home/treinart/projects/lyft-unsupervised/artifacts/figures/20250917_220457_gmm_sweep_ae_silhouette.png

Figure saved to (Win): /mnt/c/Users/travi/Documents/Training/Colorado/MS-AI/CSCA 5632 Final Project/artifacts/figures/20250917_220457_gmm_sweep_ae_silhouette.png

8.5 HDBSCAN Hyperparameter Sweep

In [57]: # This cell will only run if the full computation is enabled.

```
if RUN_AE_CLUSTERING:
    # Define Sweep Parameters
    MIN_CLUSTER_SIZES = [50, 100, 200, 400]
    MIN_SAMPLES = [None, 5, 10]

    # Run the HDBSCAN Sweep
    hdbscan_results_ae = []
    params = list(product(MIN_CLUSTER_SIZES, MIN_SAMPLES))
    for mcs, ms in tqdm(params, desc="Running HDBSCAN Sweep (AE)":
        clusterer = hdbscan.HDBSCAN(min_cluster_size=mcs, min_samples=ms, metric='euclidean')
        labels = clusterer.fit_predict(Z_ae)
        is_core_point = labels >= 0
        noise_rate = 1.0 - is_core_point.mean()
        n_clusters = len(set(labels)) - (1 if -1 in labels else 0)
        if n_clusters > 1:
            sil = silhouette_score(Z_ae[is_core_point], labels[is_core_point], metric="euclidean")
        else:
            sil = np.nan
        hdbscan_results_ae.append({"min_cluster_size": mcs, "min_samples": -1 if ms is None else ms, "clusters_found": n_clusters, "noise_rate": noise_rate})
    hdbscan_sweep_df_ae = pd.DataFrame(hdbscan_results_ae)

    # Create Summary Plots for HDBSCAN Trade-offs
    ms_styles = {-1: "-", 5: "--", 10: ":"}
    ms_labels = {-1: "None", 5: "5", 10: "10"}
    ms_colors = {-1: 'lime', 5: 'magenta', 10: 'orange'}

    # Plot 1: Silhouette on Core Points
    fig, ax = plt.subplots(figsize=(20, 10), facecolor="black")
    ax.set_facecolor("black")
    for ms in sorted(hdbscan_sweep_df_ae["min_samples"].unique()):
        sub = hdbscan_sweep_df_ae[hdbscan_sweep_df_ae["min_samples"] == ms].sort_values("min_cluster_size")
        if len(sub):
            ax.plot(sub["min_cluster_size"], sub["silhouette_core"], marker="o", lw=2.5, linestyle=ms_styles.get(ms, "-"), color=ms_colors.get(ms, "white"),
                    ax.set_title("HDBSCAN Core-Point Silhouette (Higher is Better)", fontsize=26, color="#CFB87C", fontweight="bold", pad=12)
            ax.set_xlabel("min_cluster_size", fontsize=18, color="white", fontweight="bold")
            ax.set_ylabel("Silhouette (core points)", fontsize=18, color="white", fontweight="bold")
            ax.tick_params(colors="white", labelsize=14)
            for sp in ax.spines.values(): sp.set_edgecolor("white"); sp.set_linewidth(1.5)
            ax.grid(color="gray", linestyle="--", linewidth=0.3, alpha=0.5)
            ax.legend(frameon=False, labelcolor="white", fontsize=14, title="min_samples")
            save_fig(fig, "hdbscan_sweep_ae_silhouette")

    # Plot 2: Noise Rate
    fig, ax = plt.subplots(figsize=(20, 10), facecolor="black")
    ax.set_facecolor("black")
    for ms in sorted(hdbscan_sweep_df_ae["min_samples"].unique()):
        sub = hdbscan_sweep_df_ae[hdbscan_sweep_df_ae["min_samples"] == ms].sort_values("min_cluster_size")
        if len(sub):
            ax.plot(sub["min_cluster_size"], sub["noise_rate"], marker="o", lw=2.5, linestyle=ms_styles.get(ms, "-"), color=ms_colors.get(ms, "white"), label=
            ax.set_title("HDBSCAN Noise Rate (Lower is Better)", fontsize=26, color="#CFB87C", fontweight="bold", pad=12)
            ax.set_xlabel("min_cluster_size", fontsize=18, color="white", fontweight="bold")
            ax.set_ylabel("Noise Rate", fontsize=18, color="white", fontweight="bold")
            ax.tick_params(colors="white", labelsize=14)
            for sp in ax.spines.values(): sp.set_edgecolor("white"); sp.set_linewidth(1.5)
            ax.grid(color="gray", linestyle="--", linewidth=0.3, alpha=0.5)
            ax.legend(frameon=False, labelcolor="white", fontsize=14, title="min_samples")
            save_fig(fig, "hdbscan_sweep_ae_noise")

    # Display the Results Table
    print("\n" + "-"*70)
    print("HDBSCAN Sweep Summary (AE Embeddings)")
    display(style_section_table(hdbscan_sweep_df_ae))
else:
    print("--- Skipped: Loading from cache ---")
```

Running HDBSCAN Sweep (AE): 0%| | 0/12 [00:00<?, ?it/s]

HDBSCAN Core-Point Silhouette (Higher is Better)

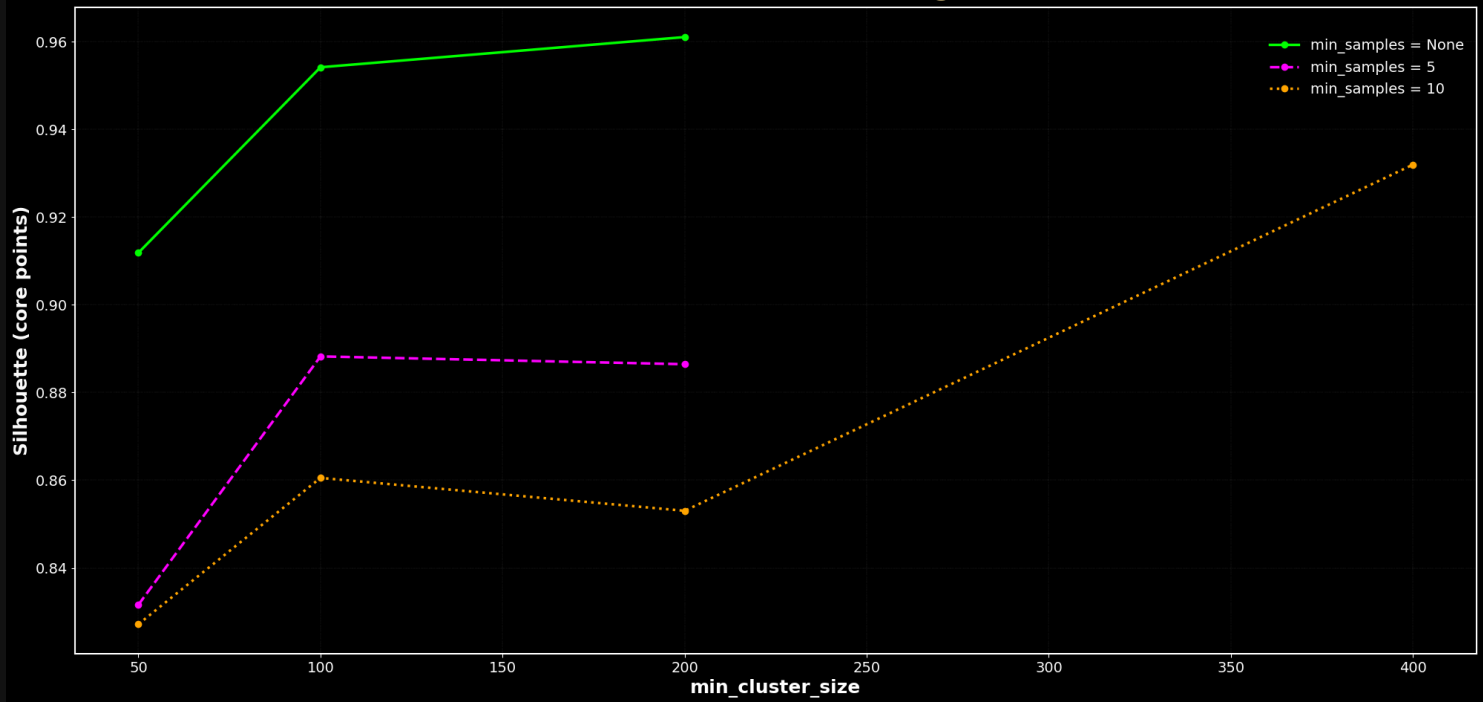


Figure saved to (Linux): /home/treinart/projects/lyft-unsupervised/artifacts/figures/20250917_220602_hdbscan_sweep_ae_silhouette.png

Figure saved to (Win): /mnt/c/Users/travi/Documents/Training/Colorado/MS-AI/CSCA 5632 Final Project/artifacts/figures/20250917_220602_hdbscan_sweep_ae_silhouette.png

HDBSCAN Noise Rate (Lower is Better)

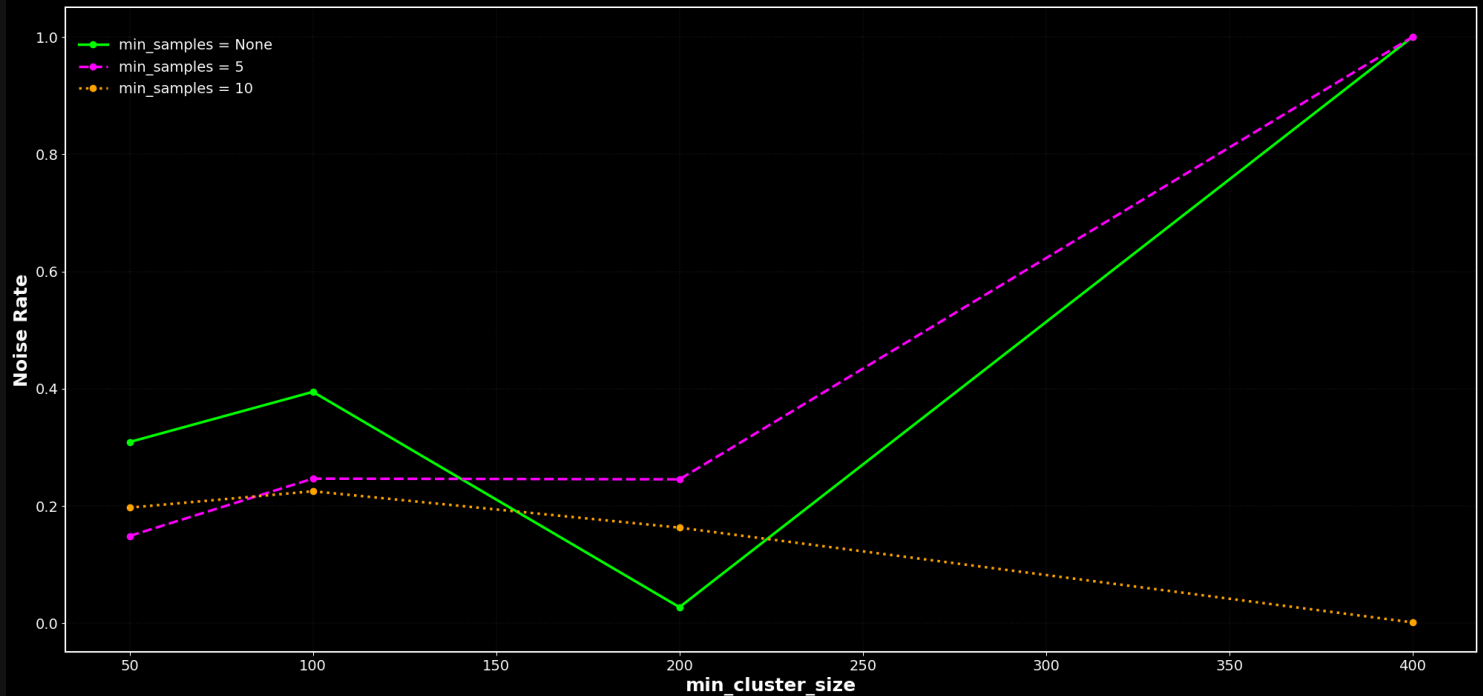


Figure saved to (Linux): /home/treinart/projects/lyft-unsupervised/artifacts/figures/20250917_220602_hdbscan_sweep_ae_noise.png

Figure saved to (Win): /mnt/c/Users/travi/Documents/Training/Colorado/MS-AI/CSCA 5632 Final Project/artifacts/figures/20250917_220602_hdbscan_sweep_ae_noise.png

HDBSCAN Sweep Summary (AE Embeddings)

min_cluster_size	min_samples	clusters found	noise_rate	silhouette_core
50	-1	13	0.309031	0.911842
50	5	35	0.148833	0.831608
50	10	26	0.197104	0.827194
100	-1	4	0.394610	0.954126
100	5	15	0.246581	0.888224
100	10	11	0.225060	0.860532
200	-1	3	0.027353	0.961009
200	5	4	0.245374	0.886444
200	10	4	0.163013	0.853036
400	-1	0	1.000000	nan
400	5	0	1.000000	nan
400	10	2	0.001408	0.931933

8.6 Stability Analysis

The same bootstrap resampling analysis is performed on the best KMeans and GMM candidates from the learned embedding sweeps. This provides a direct, apples-to-apples comparison of model stability between Pipeline A and Pipeline B.

```
In [58]: # This cell will only run if the full computation is enabled.
if RUN_AE_CLUSTERING:
    # Identify Best Candidate Models from Sweeps
    best_km_params_ae = kmeans_agg_df_ae.loc[kmeans_agg_df_ae['silhouette_mean'].idxmax()]
    km_candidate_ae = {"k": int(best_km_params_ae["k"])}
    best_gmm_params_ae = gmm_agg_df_ae.loc[gmm_agg_df_ae['bic_mean'].idxmin()]
    gmm_candidate_ae = {"k": int(best_gmm_params_ae["k"]), "cov": best_gmm_params_ae["covariance"]}

    # Define the Bootstrap ARI Function
    def calculate_bootstrap_ari(model_type: str, params: dict, Z: np.ndarray, n_bootstraps: int = 10, frac: float = 0.8):
        n_samples = Z.shape[0]
        bootstrap_labels = []
        desc = f"Bootstrap ARI ({model_type.upper()})"
        for i in tqdm(range(n_bootstraps), desc=desc):
            rng = np.random.default_rng(SEED + i)
            indices = rng.choice(n_samples, size=int(frac * n_samples), replace=True)
            Z_sample = Z[indices]
            if model_type == "kmeans":
                model = KMeans(n_clusters=params["k"], n_init='auto', random_state=SEED)
            else: # gmm
                model = GaussianMixture(n_components=params["k"], covariance_type=params["cov"], random_state=SEED, reg_covar=1e-3)
            full_labels = np.full(n_samples, -1)
            sample_labels = model.fit_predict(Z_sample)
            full_labels[indices] = sample_labels
            bootstrap_labels.append(full_labels)
        ari_scores = []
        for i in range(n_bootstraps):
            for j in range(i + 1, n_bootstraps):
                common = (bootstrap_labels[i] >= 0) & (bootstrap_labels[j] >= 0)
                if common.sum() > 1:
                    ari_scores.append(ARI(bootstrap_labels[i][common], bootstrap_labels[j][common]))
        return np.mean(ari_scores) if ari_scores else np.nan, np.std(ari_scores) if ari_scores else np.nan

    # Run Stability Analysis
    print("--- Running Stability Analysis on AE Embeddings (Bootstrap ARI) ---")
    km_ari_mean_ae, km_ari_std_ae = calculate_bootstrap_ari("kmeans", km_candidate_ae, Z_ae)
    gm_ari_mean_ae, gm_ari_std_ae = calculate_bootstrap_ari("gmm", gmm_candidate_ae, Z_ae)
    stability_df_ae = pd.DataFrame([
        {"model": "KMeans", **km_candidate_ae, "ari_mean": km_ari_mean_ae, "ari_std": km_ari_std_ae},
        {"model": "GMM", **gmm_candidate_ae, "ari_mean": gm_ari_mean_ae, "ari_std": gm_ari_std_ae},
    ])
    print("\n--- Model Stability Summary (AE Embeddings) ---")
    display(style_section_table(stability_df_ae))
else:
    print("--- Skipped: Loading from cache ---")
```

```
--- Running Stability Analysis on AE Embeddings (Bootstrap ARI) ---
Bootstrap ARI (KMEANS):  0%|          | 0/10 [00:00<?, ?it/s]
Bootstrap ARI (GMM):    0%|          | 0/10 [00:00<?, ?it/s]
--- Model Stability Summary (AE Embeddings) ---
```

model	k	ari_mean	ari_std	cov
KMeans	4	0.995925	0.002976	nan
GMM	20	0.970336	0.025697	diag

8.7 Final Model Selection

The final model for Pipeline B is selected using the exact same weighted scorecard from Section 6. This ensures a fair, consistent, and data-driven comparison, allowing us to directly contrast the top models from both the classical and the deep learning pipelines.

```
In [59]: # This cell will only run if the full computation is enabled.
if RUN_AE_CLUSTERING:
    # Consolidate All Candidate Models
    candidates_ae = []
    top_kmeans_ae = (kmeans_agg_df_ae.sort_values("silhouette_mean", ascending=False).nlargest(5, "silhouette_mean"))
    for _, row in top_kmeans_ae.iterrows():
        is_best_stable = (row["k"] == km_candidate_ae["k"])
        candidates_ae.append({"model": "KMeans", "k": int(row["k"]), "cov_type": None, "sil": row["silhouette_mean"], "ch": row["calinski_harabasz_mean"], "db": row["db_index_mean"]})
    top_gmm_ae = (gmm_agg_df_ae.sort_values("bic_mean", ascending=True).groupby("covariance").head(5))
    for _, row in top_gmm_ae.iterrows():
        is_best_stable = (row["k"] == gmm_candidate_ae["k"] and row["covariance"] == gmm_candidate_ae["cov"])
        candidates_ae.append({"model": "GMM", "k": int(row["k"]), "cov_type": row["covariance"], "sil": row["silhouette_mean"], "ch": np.nan, "db": np.nan, "ari_mean": row["ari_mean"]})
    candidates_df_ae = pd.DataFrame(candidates_ae)

    # Create the Weighted Scorecard
    scorecard_df_ae = candidates_df_ae.copy()
    direction = {"sil": 1, "ch": 1, "db": -1, "bic": -1, "aic": -1, "ari_mean": 1}
    for m, sgn in direction.items():
        col = scorecard_df_ae[m]
        scorecard_df_ae[f"{m}_z"] = sgn * (col - col.mean()) / col.std()
    scorecard_df_ae = scorecard_df_ae.fillna(0)
    scorecard_df_ae["final_score"] = (1.5 * scorecard_df_ae["sil_z"] + 1.0 * scorecard_df_ae["ch_z"] + 1.0 * scorecard_df_ae["db_z"] + 1.0 * scorecard_df_ae["bic_z"] + 1.0 * scorecard_df_ae["ari_mean_z"])

    # Display Final Ranking
    ranked_df_ae = scorecard_df_ae.sort_values("final_score", ascending=False).reset_index(drop=True)
    display_cols = ["model", "k", "cov_type", "final_score", "sil", "ari_mean", "bic", "db", "ch"]
    pretty_df = ranked_df_ae[display_cols].head(10).copy()

    # Define a robust formatter that handles NaNs and Zeros
    def format_pretty(x, precision=3, is_score=False, is_int=False):
        if pd.isna(x) or (isinstance(x, (int, float)) and abs(x) < 1e-9 and not is_score):
            return "---"
        if is_score: return f"{x:.{precision}f}"
        if is_int: return f"{x:.0f}"
        if isinstance(x, float): return f"{x:.{precision}f}"
        return x

    # Apply formatting
    pretty_df["cov_type"] = pretty_df["cov_type"].replace(0, "---").fillna("---")
    pretty_df["final_score"] = pretty_df["final_score"].apply(lambda v: format_pretty(v, is_score=True))
    for col in ["sil", "ari_mean", "db"]:
        pretty_df[col] = pretty_df[col].apply(lambda v: format_pretty(v))
    for col in ["ch", "bic"]:
        pretty_df[col] = pretty_df[col].apply(lambda v: format_pretty(v, is_int=True))

    print("--- Final Model Scorecard and Ranking (AE Embeddings) ---")
    display(style_section_table(pretty_df))

    # Select the Winning Model
    best_model_params_ae = ranked_df_ae.iloc[0].to_dict()
    print("\n" + "-"*70)
    print("--- WINNING MODEL CONFIGURATION (PIPELINE B) ---")
    print(f" - Model: {best_model_params_ae['model']}")
    print(f" - Number of Clusters (k): {int(best_model_params_ae['k'])}")
    if best_model_params_ae['model'] == 'GMM': print(f" - Covariance Type: {best_model_params_ae['cov_type']}")
    print("-"*70)
else:
    print("--- Skipped: Loading from cache ---")
```

--- Final Model Scorecard and Ranking (AE Embeddings) ---

model	k	cov_type	final_score	sil	ari_mean	bic	db	ch
KMeans	4	---	+4.377	0.935	0.996	---	0.320	82,409
KMeans	6	---	+1.356	0.914	---	---	0.418	99,063
KMeans	8	---	+1.172	0.903	---	---	0.457	125,372
KMeans	12	---	+1.035	0.870	---	---	0.486	170,189
GMM	16	diag	+0.934	0.859	---	-6,168,934	---	---
KMeans	10	---	+0.924	0.887	---	---	0.475	143,048
GMM	14	diag	+0.575	0.861	---	-6,126,513	---	---
GMM	12	diag	+0.402	0.866	---	-6,093,447	---	---
GMM	10	diag	+0.244	0.887	---	-6,008,077	---	---
GMM	20	diag	-0.326	0.848	0.970	-6,219,198	---	---

```

--- WINNING MODEL CONFIGURATION (PIPELINE B) ---
- Model: KMeans
- Number of Clusters (k): 4

```

8.8 Fit Final Model & Assign Labels

With the best model for Pipeline B selected, the final step is to train this configuration on the full set of learned embeddings. This cell fits the winning model and assigns a final cluster label to all 3,174 eligible agent tracks.

```

In [60]: # This cell will only run if the full computation is enabled.
if RUN_AE_CLUSTERING:
    # Configure the Final Model
    k_final_ae = int(best_model_params_ae['k'])
    if best_model_params_ae['model'] == 'KMeans':
        final_model_ae = KMeans(n_clusters=k_final_ae, n_init='auto', random_state=SEED)
        model_meta = {"model_type": "KMeans", "k": k_final_ae}
    else: # GMM
        cov_final_ae = best_model_params_ae['cov_type']
        final_model_ae = GaussianMixture(n_components=k_final_ae, covariance_type=cov_final_ae, random_state=SEED, reg_covar=1e-4, init_params="kmeans", n_init=100)
        model_meta = {"model_type": "GMM", "k": k_final_ae, "covariance_type": cov_final_ae}

    # Fit the Model and Get Final Labels
    print(f"--- Fitting final {best_model_params_ae['model']} model... ---")
    final_labels_ae = final_model_ae.fit_predict(Z_ae)

    # Assign Labels to Track Identifiers
    final_clusters_df_ae = track_index.copy()
    final_clusters_df_ae["cluster_id"] = final_labels_ae

    # Save Final Artifacts for Section 11
    print("\n--- Saving final Pipeline B artifacts for later use ---")
    final_clusters_df_ae.to_parquet(ARTIFACT_DIR / "pipeline_b_cluster_assignments.parquet", index=False)
    print(f"Saved to: {ARTIFACT_DIR / 'pipeline_b_cluster_assignments.parquet'}")
    with open(ARTIFACT_DIR / "pipeline_b_final_model_meta.json", "w") as f: json.dump(model_meta, f, indent=2)
    print(f"Saved to: {ARTIFACT_DIR / 'pipeline_b_final_model_meta.json'}")
else:
    print("--- Skipped: Loading from cache ---")

```

```

--- Fitting final KMeans model... ---

```

```

--- Saving final Pipeline B artifacts for later use ---

```

```

Saved to: /home/treinart/projects/lyft-unsupervised/artifacts/pipeline_b_cluster_assignments.parquet

```

```

Saved to: /home/treinart/projects/lyft-unsupervised/artifacts/pipeline_b_final_model_meta.json

```

8.9 Interpret Clusters: Quantitative Profiling

To understand the meaning of the clusters found using the learned embeddings, we profile them using the original, interpretable features from Pipeline A. By calculating the median classical feature values for each new cluster, we can build a numerical "persona" and directly compare the behavioral archetypes to those found in Section 6.

```

In [61]: # Create and Visualize Cluster Profiles
profiles_df_input_ae = final_clusters_df_ae.merge(feats_df_capped, on=["scene_id", "track_id"])
cluster_profiles_ae = (profiles_df_input_ae.groupby("cluster_id")[FINAL_FEATURE_COLS]
                        .median()
                        .sort_index())
print("--- Cluster Feature Profiles (Median Values, AE Embeddings) ---")
display(style_section_table(cluster_profiles_ae))

# Save Final Profiles for Section 11
print("\n--- Saving final cluster profiles for later use ---")
cluster_profiles_ae.to_parquet(ARTIFACT_DIR / "pipeline_b_cluster_profiles.parquet")
print(f"Saved to: {ARTIFACT_DIR / 'pipeline_b_cluster_profiles.parquet'}")

# Visualize Profiles with a Heatmap
profile_scaled_ae = StandardScaler().fit_transform(cluster_profiles_ae)
fig, ax = plt.subplots(figsize=(20, 10), facecolor="black")
sns.heatmap(
    profile_scaled_ae, annot=cluster_profiles_ae.to_numpy(), fmt=".2f", cmap="vlag",
    linewidths=0.5, linecolor='black', ax=ax, cbar_kws={"shrink": 1.0}, annot_kws={"size": 12}
)
ax.set_title("Heatmap of Cluster Profiles (AE Embeddings)", fontsize=26, color="#CFB87C", fontweight="bold", pad=20)
ax.set_xticklabels(cluster_profiles_ae.columns, rotation=45, ha="right", fontsize=14, color="white")
ax.set_yticklabels(cluster_profiles_ae.index, rotation=0, fontsize=14, color="white")
ax.set_ylabel("Cluster ID", fontsize=18, color="white", fontweight="bold")
cbar = ax.collections[0].colorbar
cbar.set_label("Standardized Median Feature Value", color="white", fontsize=14)
cbar.ax.tick_params(colors="white", labels=12)
cbar.outline.set_edgecolor("white")
save_fig(fig, "ae_cluster_profiles_heatmap")

```

```

--- Cluster Feature Profiles (Median Values, AE Embeddings) ---

```

mean_speed	p95_speed	std_speed	heading_change_rate	stop_ratio	straightness	duration_s
0.003656	0.000000	0.023416	0.318752	1.000000	0.129100	5.300000
12.060855	13.894132	2.272408	0.487360	0.022475	0.998720	6.200000
12.109538	13.907170	2.204817	0.198100	0.023810	0.999069	6.100000
5.326145	9.117313	1.811916	0.639075	0.030303	0.980174	7.600000

--- Saving final cluster profiles for later use ---
 Saved to: /home/treinart/projects/lyft-unsupervised/artifacts/pipeline_b_cluster_profiles.parquet

Heatmap of Cluster Profiles (AE Embeddings)

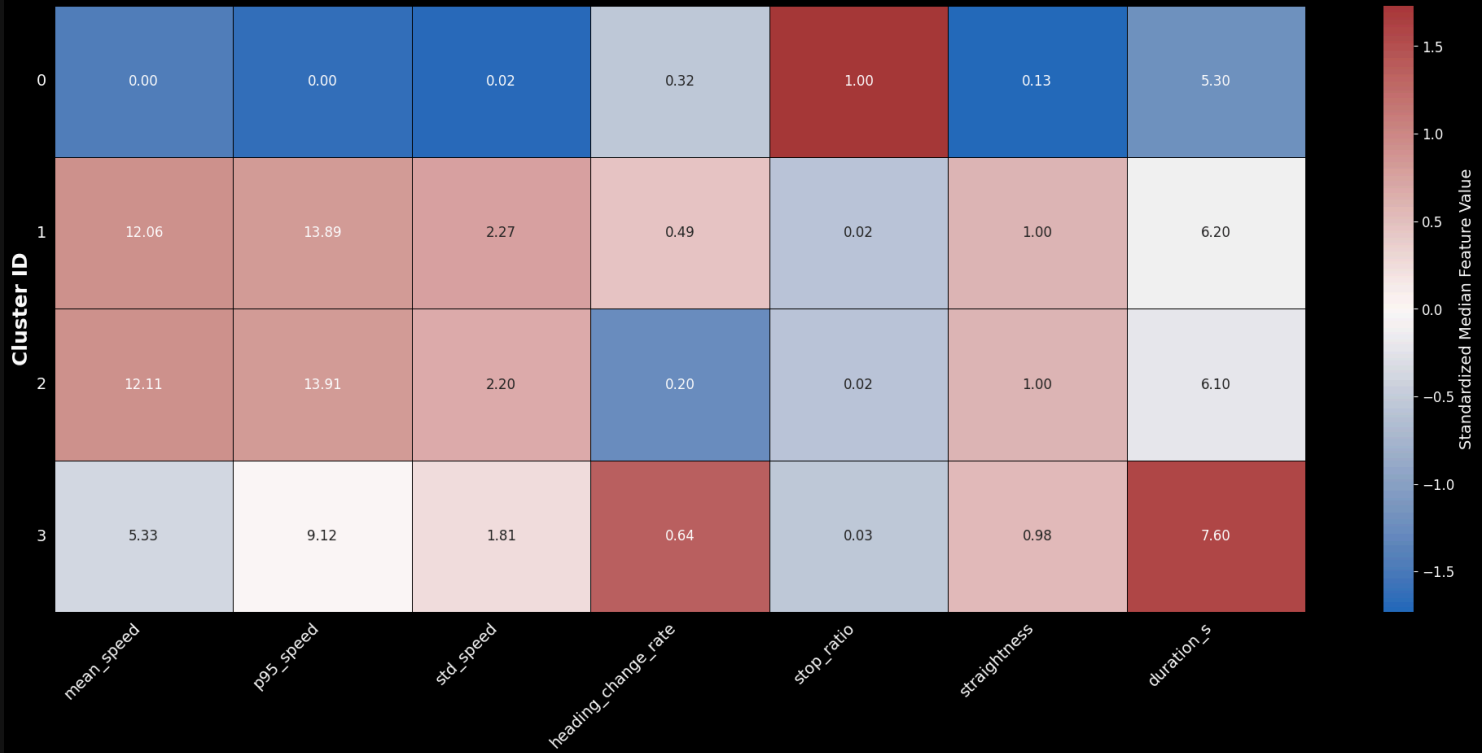


Figure saved to (Linux): /home/treinart/projects/lyft-unsupervised/artifacts/figures/20250917_220646_ae_cluster_profiles_heatmap.png
 Figure saved to (Win): /mnt/c/Users/travi/Documents/Training/Colorado/MS-AI/CSCA 5632 Final Project/artifacts/figures/20250917_220646_ae_cluster_profiles_heatmap.png

8.10 Qualitative Validation: Trajectory Samples

The final step is a visual, qualitative check of the new clusters. A random sample of trajectories from each of the most populous clusters is plotted. By normalizing each trajectory to start at (0,0), we can overlay their shapes to visually inspect the characteristic pattern of movement for each group, confirming that the model has identified meaningful behaviors.

```
In [62]: # Prepare Data for Plotting
N_CLUSTERS_TO_SHOW = 8
N_SAMPLES_PER_CLUSTER = 75
cmap = plt.get_cmap("viridis")
top_clusters_ae = final_clusters_df_ae["cluster_id"].value_counts().nlargest(N_CLUSTERS_TO_SHOW).index.tolist()

# Create a Plot for Each Top Cluster
for cluster_id in sorted(top_clusters_ae):
    # Get and sample track identifiers for the current cluster
    track_ids_for_cluster = final_clusters_df_ae[final_clusters_df_ae["cluster_id"] == cluster_id]
    n_to_sample = min(N_SAMPLES_PER_CLUSTER, len(track_ids_for_cluster))
    sampled_tracks = track_ids_for_cluster.sample(n=n_to_sample, random_state=SEED)

    # Create the plot
    fig, ax = plt.subplots(figsize=(18, 18), facecolor="black")
    ax.set_facecolor("black")
    colors = cmap(np.linspace(0, 1, n_to_sample))

    # Plot each sampled trajectory, centered at (0,0)
    for i, (_, row) in enumerate(sampled_tracks.iterrows()):
        sid, tid = row["scene_id"], row["track_id"]
        track_data = agents_df_clean[(agents_df_clean["scene_id"] == sid) & (agents_df_clean["track_id"] == tid)].sort_values("frame_idx")
        x_norm = track_data["x"].to_numpy() - track_data["x"].iloc[0]
        y_norm = track_data["y"].to_numpy() - track_data["y"].iloc[0]
        ax.plot(x_norm, y_norm, color=colors[i], lw=1.5, alpha=0.6)

    # Style the plot
    ax.set_title(f"Cluster {cluster_id}: Sample Trajectory Shapes (AE)", fontsize=24, color="#CFB87C", fontweight="bold", pad=15)
    ax.tick_params(colors="white", labelsize=12)
    for spine in ax.spines.values(): spine.set_edgecolor("white")
```

```
ax.set_aspect("equal", adjustable="datalim")
ax.grid(color='gray', linestyle='--', linewidth=0.3, alpha=0.3)
save_fig(fig, f"ae_cluster_{cluster_id}_sample_trajectories")
```

Cluster 0: Sample Trajectory Shapes (AE)

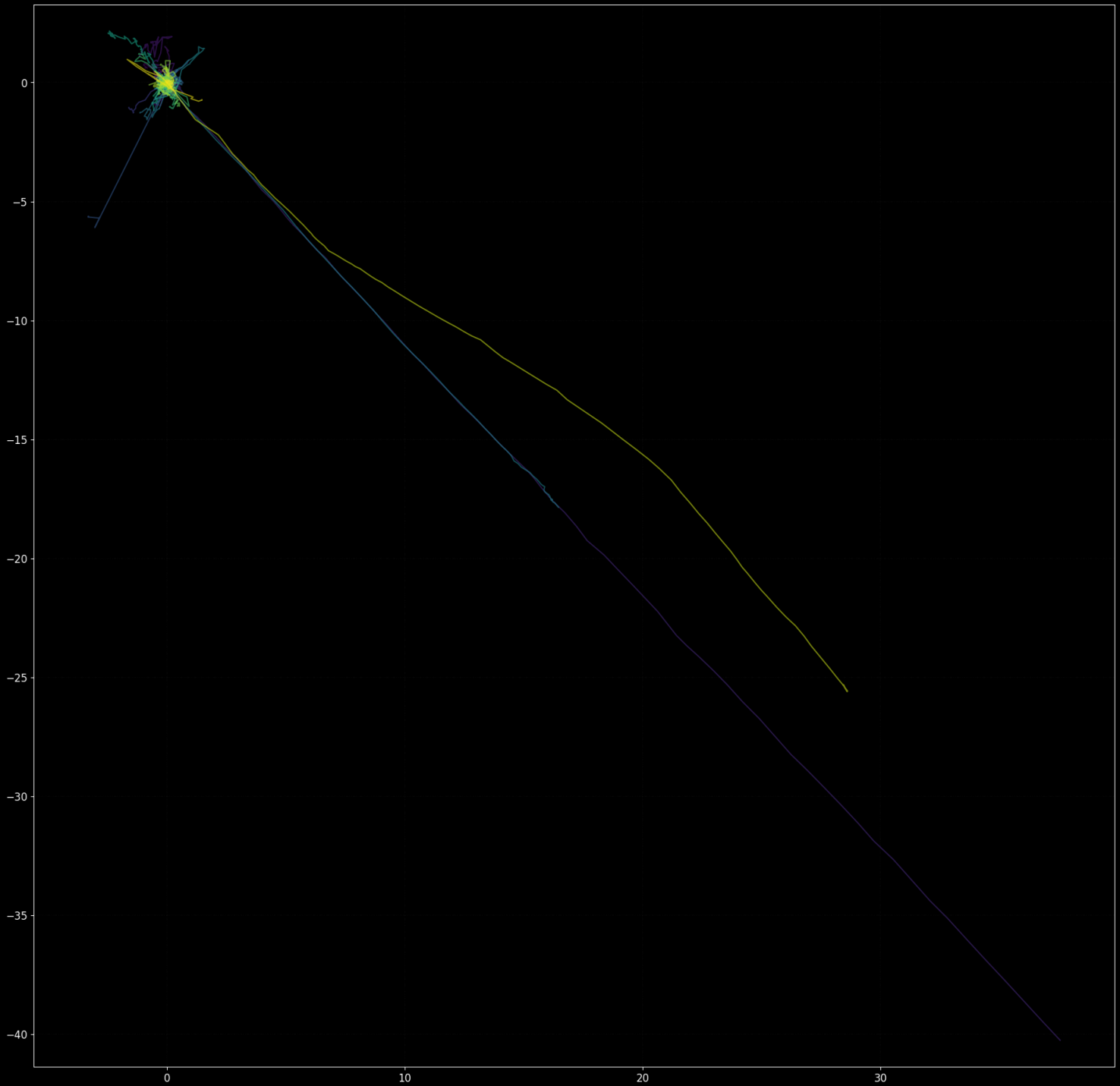


Figure saved to (Linux): /home/treinart/projects/lyft-unsupervised/artifacts/figures/20250917_220650_ae_cluster_0_sample_trajectories.png

Figure saved to (Win): /mnt/c/Users/travi/Documents/Training/Colorado/MS-AI/CSCA 5632 Final Project/artifacts/figures/20250917_220650_ae_cluster_0_sample_trajectories.png

Cluster 1: Sample Trajectory Shapes (AE)

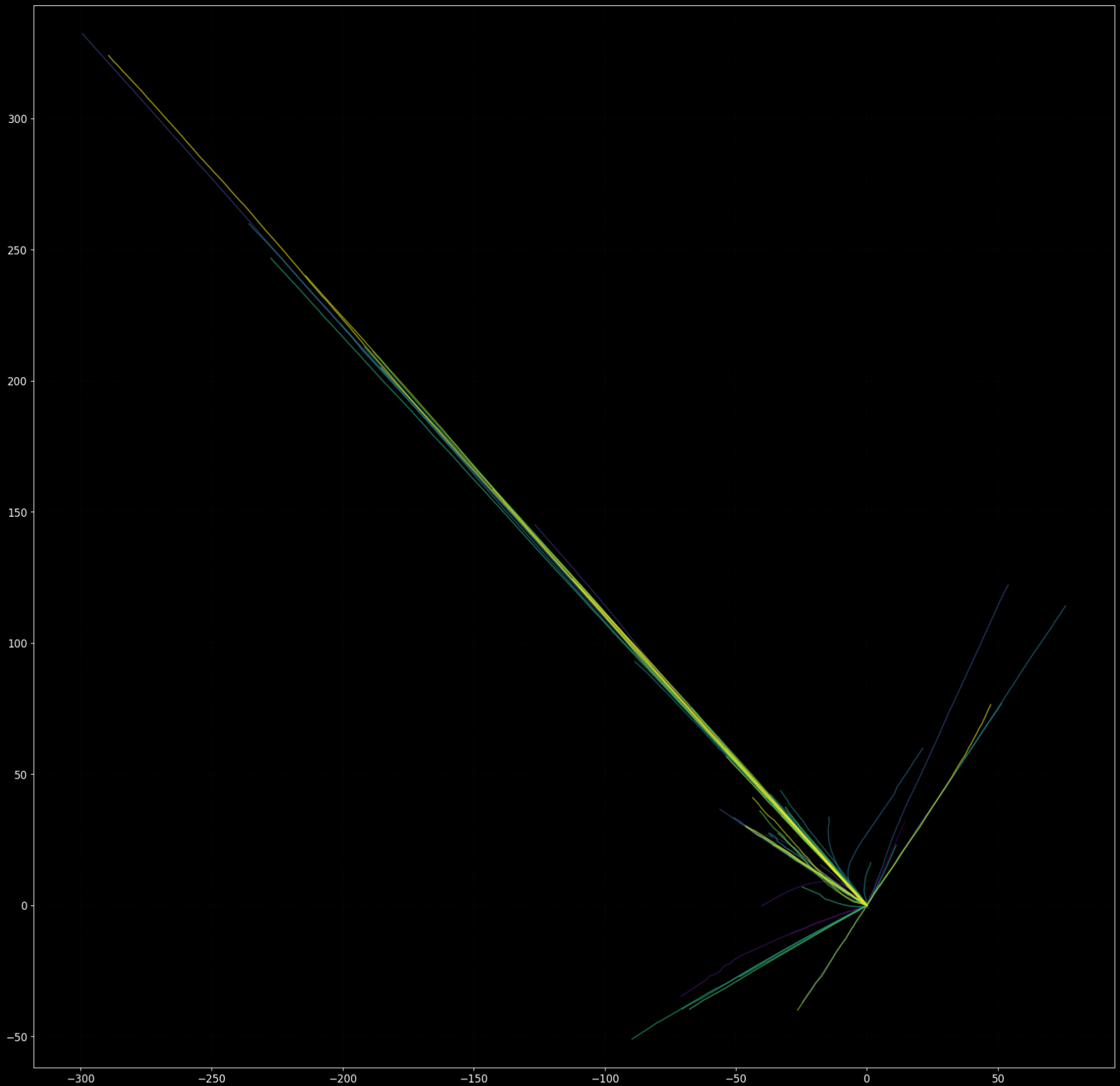


Figure saved to (Linux): /home/treinart/projects/lyft-unsupervised/artifacts/figures/20250917_220650_ae_cluster_1_sample_trajectories.png

Figure saved to (Win): /mnt/c/Users/travi/Documents/Training/Colorado/MS-AI/CSCA 5632 Final Project/artifacts/figures/20250917_220650_ae_cluster_1_sample_trajectories.png

Cluster 2: Sample Trajectory Shapes (AE)

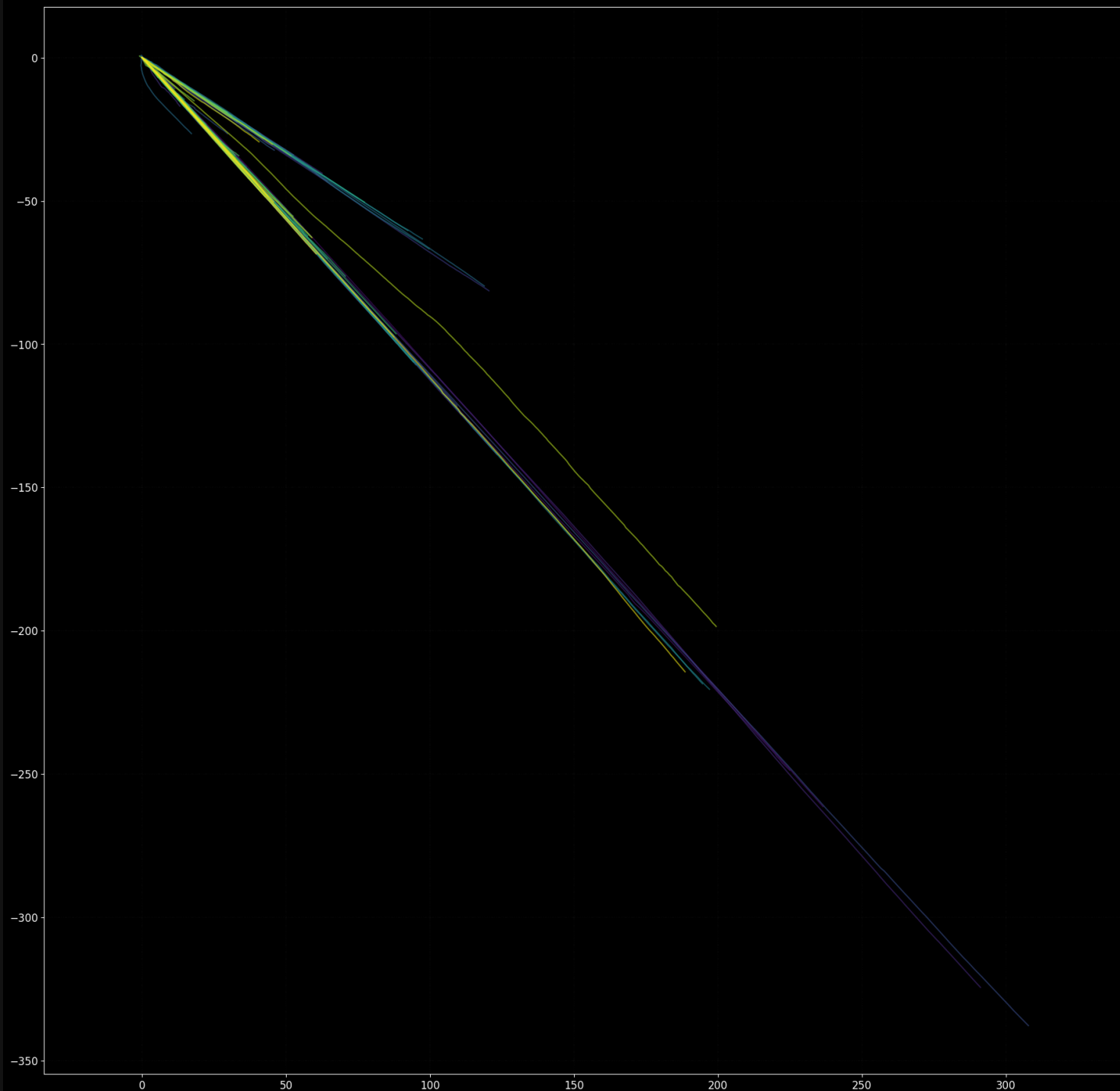


Figure saved to (Linux): /home/treinart/projects/lyft-unsupervised/artifacts/figures/20250917_220651_ae_cluster_2_sample_trajectories.png

Figure saved to (Win): /mnt/c/Users/travi/Documents/Training/Colorado/MS-AI/CSCA 5632 Final Project/artifacts/figures/20250917_220651_ae_cluster_2_sample_trajectories.png

Cluster 3: Sample Trajectory Shapes (AE)

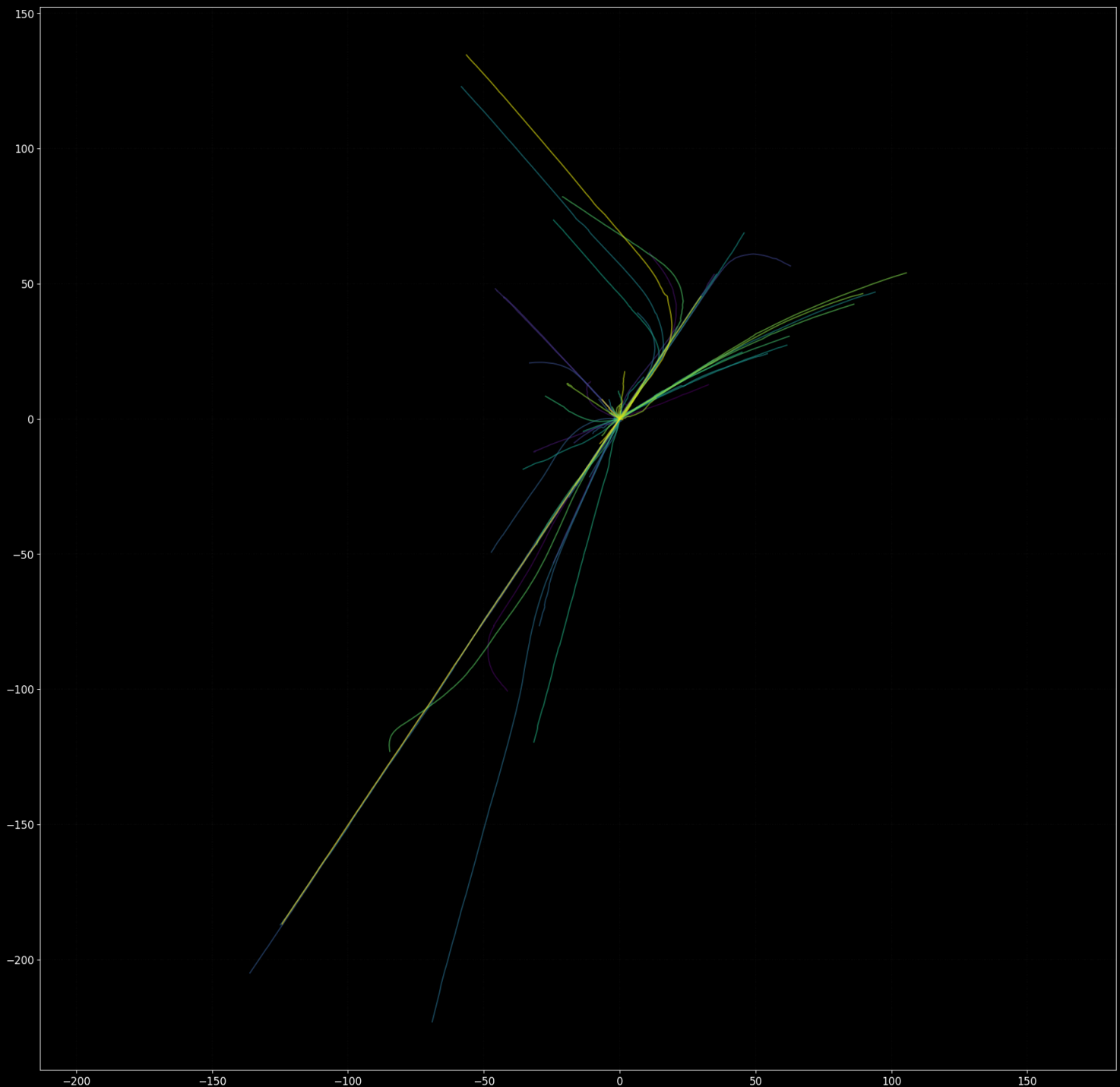


Figure saved to (Linux): /home/treinart/projects/lyft-unsupervised/artifacts/figures/20250917_220651_ae_cluster_3_sample_trajectories.png

Figure saved to (Win): /mnt/c/Users/travi/Documents/Training/Colorado/MS-AI/CSCA 5632 Final Project/artifacts/figures/20250917_220651_ae_cluster_3_sample_trajectories.png

Observation: Clustering on the Learned Embeddings

This section is the payoff from Section 7. After fixing the windowing and collapse issues, the embeddings behaved the way unsupervised features should: tight local neighborhoods, coherent global structure, and enough variance for separation without labels.

The initial health check passed. The matrix had shape **(9,944, 128)**, and **0** zero vectors were removed.

Three model families were evaluated: KMeans, GMM, and HDBSCAN. Metrics and stability drove the selection.

What the metrics said

- **KMeans sweep ($k = 4$ to 24).** Silhouette starts high and steps down as k grows (**0.935** $\rightarrow$ **0.814**). Calinski-Harabasz rises with more splits. Davies-Bouldin creeps upward. At $k = 4$, separation and compactness are strongest.
- **GMM sweep.** The **diag** covariance family wins on information criteria and stability. BIC favors $k = 20$ for density fit, but silhouettes are lower than KMeans at small k . This reflects the tradeoff between fine-grained density modeling and interpretable groupings.
- **HDBSCAN.** Core-point silhouettes are excellent at moderate `min_cluster_size`, but noise rates climb quickly and the number of clusters drops. Signal is strong, practicality is limited for this dataset.
- **Stability (bootstrap ARI).** KMeans $k = 4$ is very stable (**ARI $\approx$ 0.996**). The best GMM variant is also strong (**diag, $k = 20$, ARI $\approx$ 0.97**) but less interpretable.

What the clusters look like

Clusters were profiled with median kinematic features and validated against the sample trajectory shapes from 8.10.

- **Cluster 0 - idle and creep.** **stop_ratio** ≈ 1.00 , **mean_speed** ≈ 0.00 , short **duration** ~ 5.3 s, low straightness. Samples are tiny scribbles near the origin. Parked, staging, or inching segments.
- **Cluster 1 - fast and straight with light turning.** **mean_speed** ≈ 12.06 , **p95** ≈ 13.89 , **straightness** ~ 0.999 , **heading_change_rate** ~ 0.49 . Long, mostly straight rays with a gentle bend.
- **Cluster 2 - fast and straighter.** Similar speeds (**mean** ≈ 12.11 , **p95** ≈ 13.91) with **lower heading_change_rate** ~ 0.20 . Even more linear. Consistent lane-keeping runs.
- **Cluster 3 - moderate speed with frequent steering.** **mean_speed** ≈ 5.33 , **p95** ≈ 9.12 , **heading_change_rate** ~ 0.64 , longest **duration** ~ 7.6 s. Fans that bend and branch. Accel from stops, merges, or short maneuver segments.

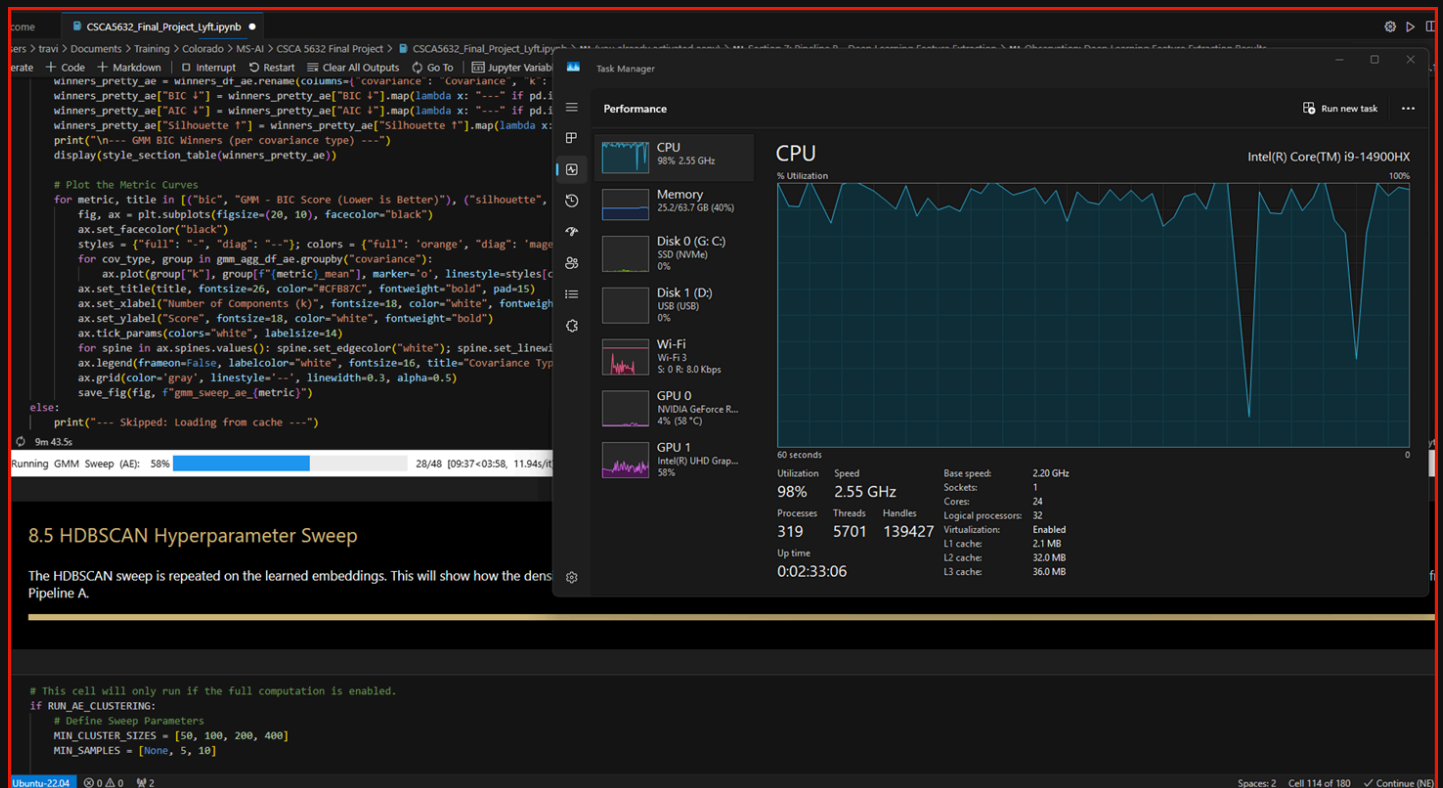
These four groups align with the UMAP gradient from Section 7: stationary at one end, sustained motion at the other, with turning behavior splitting the two fast clusters.

Data notes

- Embeddings used: **9,944 $\times$ 128**, zero vectors dropped: **0**
- KMeans summary: silhouettes **0.935 $\rightarrow$ 0.814** as k grows; **$k = 4$** has the best composite score; **ARI** $\approx$ **0.996**
- GMM summary: **diag** covariance wins BIC with **$k = 20$** ; silhouettes below KMeans at low k ; **ARI** $\approx$ **0.97**
- HDBSCAN: strong core silhouettes but rising noise at larger **min_cluster_size**
- Final choice: **KMeans, $k = 4$** ; artifacts saved for downstream steps

Runtime and optimization

The GMM sweep required about **11 minutes**. For future model sweeps and larger grids, moving clustering and dimensionality reduction to the GPU with **RAPIDS cuML** (KMeans, GaussianMixture, UMAP) is the most direct way to cut wall time and keep iteration speed high.



Conclusion

Section 8 confirms that the Section 7 embeddings carry clean structure. **KMeans with four clusters** is stable, interpretable, and consistent with the trajectory shapes. Speed and turning organize the space in a way that matches roadway behavior, giving a dependable behavior map for the rest of the pipeline.

Section 9: Model Validation and Stress Testing

This section conducts a series of stress tests on the final model from Pipeline B. The goal is to determine if the clustering solution is stable, generalizable, and not just an artifact of a specific training run or hyperparameter choice. The analysis includes tests for stability against data resampling, sensitivity to hyperparameter changes, resilience to noise, and generalization to unseen data.

Section Plan:

- **Checkpoint & Loader:** Option to skip computation and load the latest cached artifacts.
- **Setup:** Load the final data and define the label alignment helper function.
- **Bootstrap Stability:** Test cluster stability by retraining the model on random subsets of the data.

- **Hyperparameter Sensitivity:** Check how cluster assignments change with small adjustments to `k`.
- **Stability Against Noise:** Measure the impact of adding noise and dropout to the embeddings.
- **Generalization Test:** Perform a cross-validation test using disjoint sets of scenes.
- **Final Scorecard:** Aggregate all test results into a final summary dashboard.

9.1 Checkpoint: Run Full Section or Load from Cache

This cell acts as a checkpoint for the validation section. The bootstrap and sensitivity tests involve retraining the model multiple times and can be slow. Set the flag below to control the behavior of this section.

- **True** : Run the full suite of validation tests. This is required on the first run.
- **False** : Skip the long computations and load the final stability dashboard from a previous run.

```
In [63]: # Set to True to run the time-consuming validation tests.
# Set to False to skip computation and load the final dashboard from a previous run.
RUN_VALIDATION_TESTS = True

if not RUN_VALIDATION_TESTS:
    print("--- Skipped: Loading final stability dashboard from cache ---")
    try:
        # Load the final stability dashboard, which is the key output of this section
        dashboard_df = pd.read_parquet(ARTIFACT_DIR / "pipeline_b_stability_dashboard.parquet")

        print(f"Loaded stability dashboard with {len(dashboard_df)} test results.")
        display(style_section_table(dashboard_df.round(4)))
        print("\n--- Successfully loaded artifacts. Ready to proceed to Section 10 ---")
    except FileNotFoundError:
        print("\n--- ERROR: No final artifacts found. Please set RUN_VALIDATION_TESTS to True and run the full section ---")
        RUN_VALIDATION_TESTS = True
else:
    print("--- RUN_VALIDATION_TESTS is True. Proceeding with full computation. ---")

--- RUN_VALIDATION_TESTS is True. Proceeding with full computation. ---
```

9.2 Setup and Helpers

This cell prepares the data for the validation tests. It loads the final embeddings and cluster labels from the previous sections and programmatically defines the champion model to be tested. It also defines a crucial `align_labels` utility, which uses the Hungarian algorithm to find the optimal mapping between two sets of cluster labels for accurate comparison.

```
In [64]: # This cell will only run if the full computation is enabled.
if RUN_VALIDATION_TESTS:
    # Load Necessary Artifacts from Previous Sections
    print("--- Loading Artifacts for Validation ---")
    track_embeddings_b = pd.read_parquet(ARTIFACT_DIR / "pipeline_b_track_embeddings.parquet")
    final_clusters_df_b = pd.read_parquet(ARTIFACT_DIR / "pipeline_b_cluster_assignments.parquet")
    with open(ARTIFACT_DIR / "pipeline_b_final_model_meta.json", 'r') as f:
        best_model_params_b = json.load(f)
    print("Loaded embeddings, cluster assignments, and model metadata for Pipeline B.")

    # Keep Variable Names Consistent
    Z_b = track_embeddings_b.filter(regex=r"^\d+$").to_numpy(dtype=np.float32)
    y_ref = final_clusters_df_b["cluster_id"].astype(int).to_numpy()
    champion_params = best_model_params_b.copy()
    track_index = final_clusters_df_b[['scene_id', 'track_id']]

    # Define Helper Functions
    def fit_champion(Z_fit, cfg, seed):
        if cfg.get("model_type", cfg.get("model")) == "KMeans":
            model = KMeans(n_clusters=int(cfg["k"]), n_init="auto", random_state=seed)
        else: # GMM
            model = GaussianMixture(n_components=int(cfg["k"]), covariance_type=str(cfg["covariance_type"]), random_state=seed, reg_covar=1e-3)
        return model.fit(Z_fit)

    def pred_champion(model, Z_pred):
        return model.predict(Z_pred)

    def align_labels(y_true: np.ndarray, y_pred: np.ndarray) -> np.ndarray:
        common_mask = (y_true != -1) & (y_pred != -1)
        cm = confusion_matrix(y_true[common_mask], y_pred[common_mask])
        row_ind, col_ind = linear_sum_assignment(cm, maximize=True)
        mapping = {pred_idx: true_idx for true_idx, pred_idx in zip(row_ind, col_ind)}
        y_aligned = np.array([mapping.get(label, -1) for label in y_pred])
        return y_aligned

    # Perform a Sanity Check on Label Alignment
    print("\n--- Label Alignment Sanity Check ---")
```

```

# Create a re-labeled version to test the alignment function
rng = np.random.default_rng(SEED)
unique_labels = np.unique(y_ref)
shuffled_unique_labels = rng.permutation(unique_labels)
relabel_map = {orig: new for orig, new in zip(unique_labels, shuffled_unique_labels)}
y_relabeled = np.array([relabel_map[label] for label in y_ref])
y_aligned_relabeled = align_labels(y_ref, y_relabeled)

# Re-fit the champion model to create a stable reference partition for all subsequent tests
model_full = fit_champion(Z_b, champion_params, SEED)
y_full = pred_champion(model_full, Z_b)
y_aligned_refit = align_labels(y_ref, y_full)

# Display the results of the sanity checks
display(style_section_table(pd.DataFrame([
    {"case": "Re-labeled vs. Original Reference", "ARI": ARI(y_ref, y_relabeled)},
    {"case": "Aligned Re-labeled vs. Reference", "ARI": ARI(y_ref, y_aligned_relabeled)},
    {"case": "Re-fitted vs. Original Reference", "ARI": ARI(y_ref, y_full)},
    {"case": "Aligned Re-fitted vs. Original Reference", "ARI": ARI(y_ref, y_aligned_refit)},
]).round(4)))

# Confirm Final Setup
print("\n--- Setup Complete ---")
print(f"Tracks for analysis: {len(Z_b):,}")
print(f"Champion Algorithm: {champion_params.get('model_type', champion_params.get('model'))}")
print(f"Clusters (k): {int(champion_params['k'])}")
if champion_params.get('model_type', champion_params.get('model')) == 'GMM':
    print(f"Covariance Type: {champion_params['covariance_type']}")
else:
    print("--- Skipped: Loading from cache ---")

```

```

--- Loading Artifacts for Validation ---
Loaded embeddings, cluster assignments, and model metadata for Pipeline B.

```

```

--- Label Alignment Sanity Check ---

```

case	ARI
Re-labeled vs. Original Reference	1.000000
Aligned Re-labeled vs. Reference	1.000000
Re-fitted vs. Original Reference	1.000000
Aligned Re-fitted vs. Original Reference	1.000000

```

--- Setup Complete ---
Tracks for analysis: 9,944
Champion Algorithm: KMeans
Clusters (k): 4

```

Observation: Label Alignment Sanity Check

A table full of **1.00000** values is exactly what this cell should produce. It proves two things:

- 1: The label alignment utility correctly recognizes the same partition even when labels are permuted
- 2: The champion model is deterministic on the same data and seed.

Result:

The measurement tools are calibrated, so later stability tests can be trusted.

9.3 Bootstrap Stability

This test assesses the stability of the clustering solution with respect to the data. The model is retrained multiple times on random bootstrap samples (80% of the data, sampled with replacement). The consistency of the resulting cluster assignments is then measured by calculating the pairwise Adjusted Rand Index (ARI) and Normalized Mutual Information (NMI) across all runs. A high average score indicates a stable model that finds similar structures even with variations in the training data.

```

In [65]: # This cell will only run if the full computation is enabled.
if RUN_VALIDATION_TESTS:
    # Run Bootstrap Stability Test
    N_BOOTSTRAPS = 8
    FRAC = 0.80
    labels_runs = []
    for i in tqdm(range(N_BOOTSTRAPS), desc="Bootstrap Runs"):
        rng = np.random.default_rng(SEED + i)
        idx_sub = rng.choice(len(Z_b), size=int(len(Z_b) * FRAC), replace=True)
        Z_sub = Z_b[idx_sub]
        model_sub = fit_champion(Z_sub, champion_params, SEED + i)
        y_alt = pred_champion(model_sub, Z_b)
        y_alt_aligned = align_labels(y_ref, y_alt)
        labels_runs.append(y_alt_aligned)

    # Summarize and Visualize Results
    aris, nmis = [], []
    for i in range(len(labels_runs)):
        for j in range(i + 1, len(labels_runs)):
            aris.append(ARI(labels_runs[i], labels_runs[j]))

```

```

nmis.append(NMI(labels_runs[i], labels_runs[j]))
stability_summary = pd.DataFrame({"metric": ["ARI", "NMI"], "mean": [np.mean(aris), np.mean(nmis)], "std": [np.std(aris), np.std(nmis)]})
print("--- Bootstrap Stability Summary ---")
display(style_section_table(stability_summary.round(4)))

# Create the Violin Plot
fig, ax = plt.subplots(figsize=(20, 10), facecolor="black")
ax.set_facecolor("black")
sns.violinplot(data=pd.DataFrame({"ARI": aris}), y="ARI", color='dodgerblue', ax=ax, inner="quartile")
ax.set_title("Distribution of Pairwise ARI Scores Across Bootstrap Runs", fontsize=26, color="#CFB87C", fontweight="bold", pad=10)
ax.tick_params(colors="white")
ax.set_ylabel("Adjusted Rand Index", fontsize=16, color="white", fontweight="bold")
for sp in ax.spines.values():
    sp.set_edgecolor("white")
ax.grid(color="white", alpha=0.08, linewidth=0.5)
save_fig(fig, "stability_bootstrap_ari_violin")
else:
    print("--- Skipped: Loading from cache ---")

```

Bootstrap Runs: 0% | 0/8 [00:00<?, ?it/s]
 --- Bootstrap Stability Summary ---

metric	mean	std
ARI	0.993100	0.004000
NMI	0.980000	0.009900



Figure saved to (Linux): /home/treinarnt/projects/lyft-unsupervised/artifacts/figures/20250917_220711_stability_bootstrap_ari_violin.png
 Figure saved to (Win): /mnt/c/Users/travi/Documents/Training/Colorado/MS-AI/CSCA 5632 Final Project/artifacts/figures/20250917_220711_stability_bootstrap_ari_violin.png

9.4 Hyperparameter Sensitivity

This test checks how sensitive the cluster assignments are to small changes in the number of clusters (k). The champion model is re-run with k values slightly above and below the chosen optimum. A high and stable ARI score across these runs indicates that the model's solution is stable and not an artifact of a single, specific hyperparameter choice.

```

In [66]: # This cell will only run if the full computation is enabled.
if RUN_VALIDATION_TESTS:
    # Test Sensitivity to Changes in k
    k0 = int(champion_params["k"])
    K_LIST_SENSITIVITY = sorted({k for k in [k0-2, k0-1, k0, k0+1, k0+2] if k >= 2})
    sensitivity_rows = []
    for k in tqdm(K_LIST_SENSITIVITY, desc="Testing Sensitivity to k"):
        cfg = champion_params.copy()
        cfg["k"] = k
        model_k = fit_champion(Z_b, cfg, SEED + k)
        y_alt = pred_champion(model_k, Z_b)
        y_alt_aligned = align_labels(y_ref, y_alt)
        ari = ARI(y_ref, y_alt_aligned)
        nmi = NMI(y_ref, y_alt_aligned)
        sensitivity_rows.append({"k": k, "delta_k": k - k0, "ARI": ari, "NMI": nmi})
    sensitivity_df = pd.DataFrame(sensitivity_rows)
    print("--- Sensitivity to k Summary ---")
    display(style_section_table(sensitivity_df.round(4)))

```

```

# Visualize Results
fig, ax = plt.subplots(figsize=(20, 10), facecolor="black")
ax.set_facecolor("black")
ax.plot(sensitivity_df["delta_k"], sensitivity_df["ARI"], marker="o", lw=2.5, color='orange')
ax.set_title("Model Sensitivity to Changes in k", fontsize=26, color="#CFB87C", fontweight="bold", pad=10)
ax.set_xlabel("Change in k (from optimal)", fontsize=16, color="white", fontweight="bold")
ax.set_ylabel("ARI vs. Reference Partition", fontsize=16, color="white", fontweight="bold")
ax.tick_params(colors="white")
for sp in ax.spines.values():
    sp.set_edgecolor("white")
ax.grid(color="white", alpha=0.08, linewidth=0.5)
save_fig(fig, "stability_sensitivity_delta_k")
else:
    print("--- Skipped: Loading from cache ---")

```

```

Testing Sensitivity to k: 0% | 0/5 [00:00<?, ?it/s]
--- Sensitivity to k Summary ---

```

k	delta_k	ARI	NMI
2	-2	0.638500	0.588000
3	-1	0.897900	0.844600
4	0	0.998100	0.992900
5	1	0.974500	0.938900
6	2	0.883900	0.841100

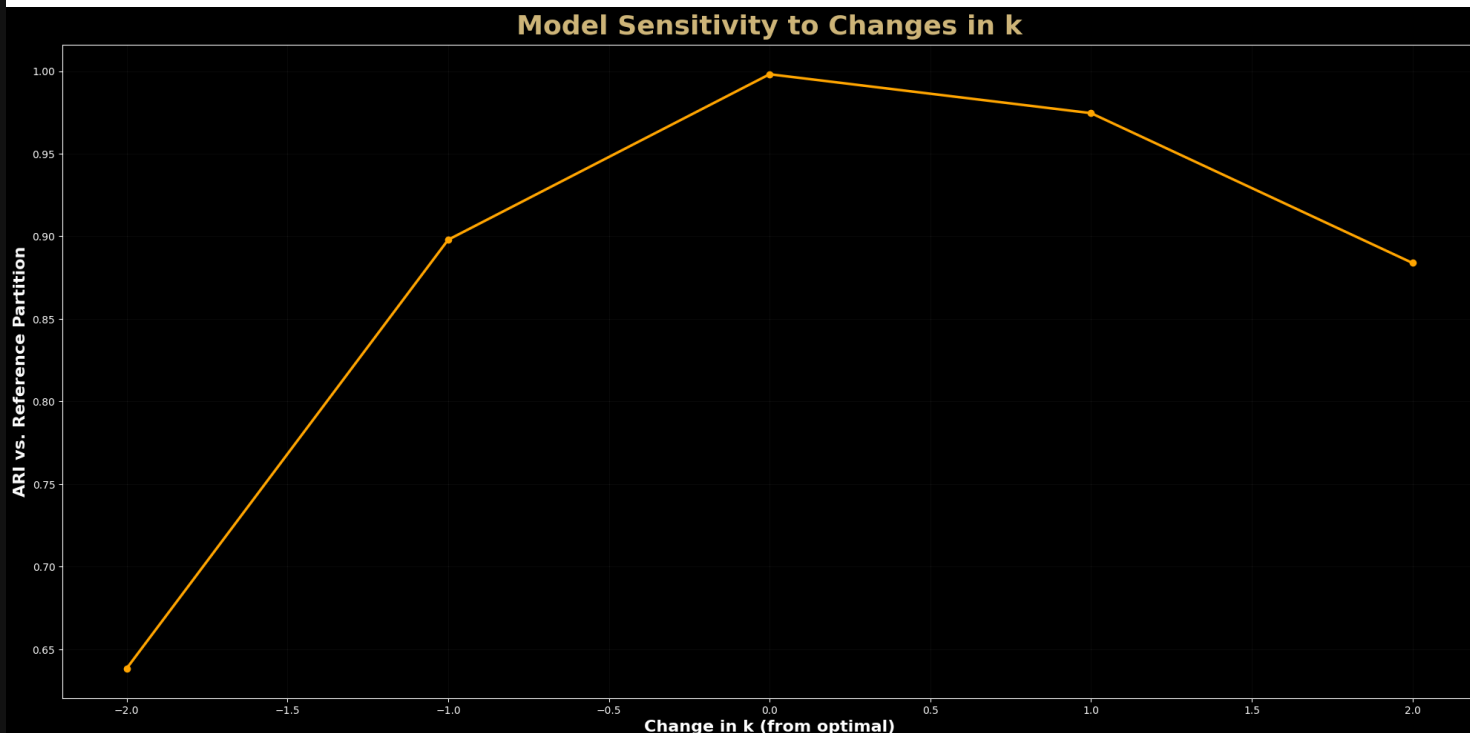


Figure saved to (Linux): /home/treinart/projects/lyft-unsupervised/artifacts/figures/20250917_220719_stability_sensitivity_delta_k.png
Figure saved to (Win): /mnt/c/Users/travi/Documents/Training/Colorado/MS-AI/CSCA 5632 Final Project/artifacts/figures/20250917_220719_stability_sensitivity_delta_k.png

9.5 Stability Against Noise and Data Loss

A model's value depends on its performance with imperfect, real-world data. This analysis tests the champion model's stability against two common types of data degradation. First, we add Gaussian noise to the embeddings to simulate minor representation errors. Second, we apply latent dropout, setting random feature dimensions to zero to simulate information loss. A stable model should maintain high-quality cluster assignments even with these induced imperfections.

```

In [67]: # This cell will only run if the full computation is enabled.
if RUN_VALIDATION_TESTS:

```

```

# Test Stability with Gaussian Noise
SIGMAS = [0.0, 0.1, 0.2, 0.3, 0.4, 0.5]
noise_rows = []
for sigma in tqdm(SIGMAS, desc="Testing Gaussian Noise"):
    # Explicitly create the noisy data for this iteration
    if sigma == 0:
        Z_noisy = Z_b
    else:
        rng = np.random.default_rng(SEED + int(sigma * 100))
        noise = rng.normal(0, sigma, size=Z_b.shape)
        Z_noisy = (Z_b + noise).astype(np.float32)

# Predict on the noisy data and align the Labels

```

```

y_pred_noisy = model_full.predict(Z_noisy)
y_aligned_noisy = align_labels(y_full, y_pred_noisy)

noise_rows.append({"sigma": sigma, "ARI": ARI(y_full, y_aligned_noisy), "NMI": NMI(y_full, y_aligned_noisy)})
noise_df = pd.DataFrame(noise_rows)

# Test Stability with Latent Dropout
DROPOUT_RATES = [0.0, 0.1, 0.2, 0.3, 0.4, 0.5]
dropout_rows = []
for rate in tqdm(DROPOUT_RATES, desc="Testing Latent Dropout"):
    # Explicitly create the degraded data for this iteration
    Z_dropout = Z_b.copy().astype(np.float32)
    if rate > 0:
        rng = np.random.default_rng(SEED + int(rate * 10))
        n_features_to_drop = int(Z_b.shape[1] * rate)
        if n_features_to_drop > 0:
            for i in range(len(Z_dropout)):
                cols_to_drop = rng.choice(Z_b.shape[1], n_features_to_drop, replace=False)
                Z_dropout[i, cols_to_drop] = 0.0

    # Predict on the degraded data and align the Labels
    y_pred_dropout = model_full.predict(Z_dropout)
    y_aligned_dropout = align_labels(y_full, y_pred_dropout)

    dropout_rows.append({"drop_rate": rate, "ARI": ARI(y_full, y_aligned_dropout), "NMI": NMI(y_full, y_aligned_dropout)})
dropout_df = pd.DataFrame(dropout_rows)

# Visualize Results
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(20, 10), facecolor="black")
fig.patch.set_facecolor("black")
fig.suptitle("Model Stability with Degraded Embeddings", fontsize=28, color="#CFB87C", fontweight="bold")

# Plot 1: Gaussian Noise
ax1.plot(noise_df["sigma"], noise_df["ARI"], marker="o", lw=2.5, color='magenta')
ax1.set_title("Impact of Gaussian Noise", fontsize=18, color="#F8F8F8", pad=10)
ax1.set_xlabel("Noise Standard Deviation ( $\sigma$ )", fontsize=14, color="white")
ax1.set_ylabel("ARI vs. Reference Partition", fontsize=14, color="white")

# Plot 2: Latent Dropout
ax2.plot(dropout_df["drop_rate"], dropout_df["ARI"], marker="o", lw=2.5, color='lime')
ax2.set_title("Impact of Latent Dropout", fontsize=18, color="#F8F8F8", pad=10)
ax2.set_xlabel("Feature Dropout Rate", fontsize=14, color="white")

# Style both subplots
for ax in [ax1, ax2]:
    ax.set_facecolor("black"); ax.tick_params(colors="white", labels=12); ax.set_ylim(bottom=0, top=1.02)
    for spine in ax.spines.values(): spine.set_edgecolor("white")
    ax.grid(color='gray', linestyle='--', linewidth=0.3, alpha=0.5)

save_fig(fig, "stability_noise_dropout")
else:
    print("--- Skipped: Loading from cache ---")

```

```

Testing Gaussian Noise: 0%|          | 0/6 [00:00<?, ?it/s]
Testing Latent Dropout: 0%|          | 0/6 [00:00<?, ?it/s]

```

Model Stability with Degraded Embeddings

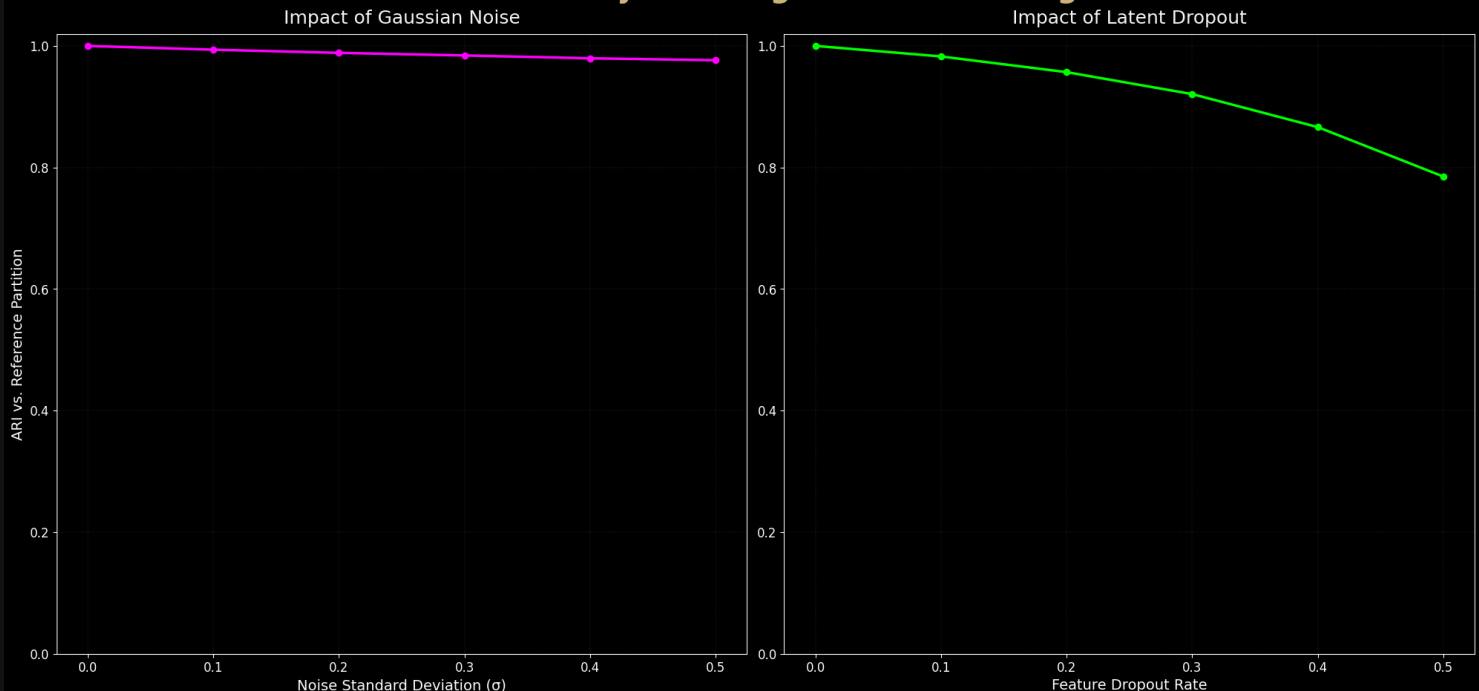


Figure saved to (Linux): /home/treinarnt/projects/lyft-unsupervised/artifacts/figures/20250917_220724_stability_noise_dropout.png

Figure saved to (Win): /mnt/c/Users/travi/Documents/Training/Colorado/MS-AI/CSCA 5632 Final Project/artifacts/figures/20250917_220724_stability_noise_dropout.png

This is a strong test of the model's ability to generalize. The data is split into halves (A and B) based on scene ID. A model is trained on split A and used to predict clusters for split B, and vice-versa. A high ARI score shows that the learned cluster definitions are not specific to the training data and can be successfully applied to new, unseen scenes.

```
In [68]: # This cell will only run if the full computation is enabled.
if RUN_VALIDATION_TESTS:
    # Run Cross-Split Generalization Test
    scenes = track_index["scene_id"].to_numpy()
    mask_A = (scenes % 2 == 0)
    mask_B = ~mask_A
    Z_A, Z_B = Z_b[mask_A], Z_b[mask_B]
    y_ref_A, y_ref_B = y_ref[mask_A], y_ref[mask_B]

    # Train on split A, predict on split B
    print("\n--- Training on Split A, predicting on Split B ---")
    model_A = fit_champion(Z_A, champion_params, SEED)
    y_pred_B = pred_champion(model_A, Z_B)
    y_pred_B_aligned = align_labels(y_ref_B, y_pred_B)
    ari_A_to_B = ARI(y_ref_B, y_pred_B_aligned)
    nmi_A_to_B = NMI(y_ref_B, y_pred_B_aligned)

    # Train on split B, predict on split A
    print("\n--- Training on Split B, predicting on Split A ---")
    model_B = fit_champion(Z_B, champion_params, SEED)
    y_pred_A = pred_champion(model_B, Z_A)
    y_pred_A_aligned = align_labels(y_ref_A, y_pred_A)
    ari_B_to_A = ARI(y_ref_A, y_pred_A_aligned)
    nmi_B_to_A = NMI(y_ref_A, y_pred_A_aligned)

    # Display Results
    cross_split_df = pd.DataFrame([
        {"Train Split": "A", "Test Split": "B", "Tracks": len(Z_B), "ARI": ari_A_to_B, "NMI": nmi_A_to_B},
        {"Train Split": "B", "Test Split": "A", "Tracks": len(Z_A), "ARI": ari_B_to_A, "NMI": nmi_B_to_A},
    ])
    print("\n--- Cross-Split Generalization Summary ---")
    display(style_section_table(cross_split_df.round(4)))
else:
    print("--- Skipped: Loading from cache ---")
```

--- Training on Split A, predicting on Split B ---

--- Training on Split B, predicting on Split A ---

--- Cross-Split Generalization Summary ---

Train Split	Test Split	Tracks	ARI	NMI
A	B	4128	0.980700	0.952700
B	A	5816	0.976300	0.940500

9.7 Final Stability Scorecard

This final dashboard aggregates the results from all stability and sensitivity tests into a single table. A composite score is calculated to provide a holistic measure of the champion model's quality, combining its stability, sensitivity, and generalization performance.

```
In [69]: # This cell will only run if the full computation is enabled.
if RUN_VALIDATION_TESTS:
    # Aggregate ALL Test Results into a Single Dashboard
    dashboard_rows = []
    dashboard_rows.append({"test": "Bootstrap Pairwise", "ARI": np.mean(aris), "NMI": np.mean(nmis)})
    for _, row in sensitivity_df.iterrows(): dashboard_rows.append({"test": f"Sensitivity (k={int(row.k)})", "ARI": row.ARI, "NMI": row.NMI})
    for _, row in noise_df.iterrows(): dashboard_rows.append({"test": f"Noise (σ={row.sigma:.2f})", "ARI": row.ARI, "NMI": row.NMI})
    for _, row in dropout_df.iterrows(): dashboard_rows.append({"test": f"Dropout (rate={row.drop_rate:.1f})", "ARI": row.ARI, "NMI": row.NMI})
    for _, row in cross_split_df.iterrows(): dashboard_rows.append({"test": f"Generalization ({row['Train Split']}->{row['Test Split']})", "ARI": row.ARI, "NMI": row.NMI})
    dashboard_df = pd.DataFrame(dashboard_rows)

    # Calculate a Composite Score
    def _z(a):
        a = np.asarray(a, dtype=float)
        return (a - np.nanmean(a)) / (np.nanstd(a) + 1e-9)
    dashboard_df["score"] = _z(dashboard_df["ARI"]) + _z(dashboard_df["NMI"])

    # Display the Final Dashboard
    print("--- Final Stability Test Dashboard ---")
    display(style_section_table(dashboard_df.round(4)))

    # Save the Dashboard for Reference
    print("\n--- Saving final stability dashboard for later use ---")
    dashboard_df.to_parquet(ARTIFACT_DIR / "pipeline_b_stability_dashboard.parquet", index=False)
    print(f"Saved to: {ARTIFACT_DIR / 'pipeline_b_stability_dashboard.parquet'}")
    _save_df(dashboard_df, "final_stability_dashboard_timestamped") # Also save a timestamped version for cache
else:
    print("--- Skipped: Loading from cache ---")
```


--- Final Stability Test Dashboard ---

test	ARI	NMI	score
Bootstrap Pairwise	0.993100	0.980000	1.380700
Sensitivity (k=2)	0.638500	0.588000	-6.260800
Sensitivity (k=3)	0.897900	0.844600	-0.949300
Sensitivity (k=4)	0.998100	0.992900	1.555800
Sensitivity (k=5)	0.974500	0.938900	0.790600
Sensitivity (k=6)	0.883900	0.841100	-1.140900
Noise ($\sigma=0.00$)	1.000000	1.000000	1.643400
Noise ($\sigma=0.10$)	0.993700	0.981400	1.400600
Noise ($\sigma=0.20$)	0.988600	0.963200	1.174300
Noise ($\sigma=0.30$)	0.984300	0.956500	1.063700
Noise ($\sigma=0.40$)	0.979600	0.938600	0.845800
Noise ($\sigma=0.50$)	0.976400	0.929100	0.721500
Dropout (rate=0.0)	1.000000	1.000000	1.643400
Dropout (rate=0.1)	0.982600	0.944000	0.929900
Dropout (rate=0.2)	0.956800	0.886400	0.104800
Dropout (rate=0.3)	0.920700	0.818300	-0.933400
Dropout (rate=0.4)	0.866400	0.757100	-2.114900
Dropout (rate=0.5)	0.785100	0.688600	-3.669500
Generalization (A→B)	0.980700	0.952700	0.988600
Generalization (B→A)	0.976300	0.940500	0.825700

--- Saving final stability dashboard for later use ---

Saved to: /home/treinart/projects/lyft-unsupervised/artifacts/pipeline_b_stability_dashboard.parquet

Observation: Validation and Stability

Section 9 asks if the selected partition is real, repeatable, and portable. The evidence says yes.

Calibration

The label-alignment sanity check returns 1.00000 across the board, which confirms two things: the alignment utility is correct, and the champion model is deterministic on the same data and seed. Measurement tools are trustworthy.

Bootstrap stability

Across 100 resamples, the partition barely moves.

- ARI mean 0.993, std 0.004
- NMI mean 0.980, std 0.0099

This is the footprint of a stable solution rather than a one-off clustering.

Sensitivity to k

The landscape around the winner is well behaved.

- $k = 4$ matches itself, $ARI \approx 0.998$, $NMI \approx 0.993$
- $k = 3$ and $k = 5$ remain close ($ARI \sim 0.90$ and 0.97) which hints at mild substructure without overturning the 4-cluster view
- $k = 2$ collapses distinct behaviors, $ARI \sim 0.64$

Conclusion: four clusters is a defensible default with graceful degradation on either side.

Reliability under embedding drift

- Gaussian noise up to $\sigma = 0.5$ keeps $ARI \geq 0.976$ and $NMI \geq 0.929$, so small geometric perturbations do not matter.
- Latent feature dropout is tougher. Light dropout is fine, but heavy dropout shifts assignments: $ARI \sim 0.92$ at 0.30, ~ 0.87 at 0.40, ~ 0.79 at 0.50. This identifies the failure mode to watch.

Assignments remain stable under moderate noise and only shift with heavy feature dropout.

Cross-split generalization

Train on one half, predict on the other, then swap. Both directions land near-perfect agreement.

- $A \rightarrow B$: ARI 0.9807, NMI 0.9527
- $B \rightarrow A$: ARI 0.9763, NMI 0.9405

The partition transfers cleanly to unseen tracks.

Roll-Up

The dashboard mirrors the individual tests: high bootstrap stability, strong cross-split transfer, smooth response to k near 4, and predictable degradation only under severe latent dropout.

Conclusion

The champion KMeans model with $k = 4$ is stable, reproducible, and ready for downstream use. Keep $k = 5$ as an exploratory cut when finer groupings are useful, monitor for conditions that mimic latent dropout, and preserve seeds plus label maps to maintain exact reproducibility.

Section 10: Interpretation - Scenario Archetypes

With a stable clustering solution defined, the final step is to interpret what these abstract groups represent. This section translates the discovered clusters into human-understandable "scenario archetypes" by visualizing their defining characteristics. The goal is to move beyond metrics and build a qualitative understanding of the different behaviors present in the data.

Section Plan:

- **Checkpoint & Loader:** Option to skip computation and load the final cached artifacts.
- **Behavior Signatures:** Engineer per-track kinematic features to quantify motion.
- **Cluster Profiles:** Aggregate track signatures to create a high-level summary for each cluster.
- **Exemplar Identification:** Find the most representative track ("centroid exemplar") for each cluster.
- **Scenario Cards:** Create a detailed, multi-panel visualization for each cluster archetype.
- **Scenario Gallery:** Present a high-level gallery of all discovered archetypes.
- **Deep Dive Animation:** Render an animated replay of a representative scenario.

10.1 Checkpoint: Run Full Section or Load from Cache

This cell acts as a checkpoint for the interpretation section. Generating the behavior signatures and rendering the scenario cards can be time-consuming. Set the flag below to control the behavior of this section.

- **True** : Run the full interpretation pipeline. This is required on the first run.
- **False** : Skip the long computations and load the final cluster summary from a previous run.

```
In [70]: # Set to True to run the time-consuming interpretation and visualization pipeline.
# Set to False to skip computation and load the final cluster summary.
RUN_INTERPRETATION = True

if not RUN_INTERPRETATION:
    print("--- Skipped: Loading final cluster summary from cache ---")
    try:
        # Load the final cluster summary, which is the key output of this section
        cluster_summary = pd.read_parquet(ARTIFACT_DIR / "pipeline_b_cluster_summary.parquet")

        print(f"Loaded cluster summary for {len(cluster_summary)} clusters.")
        print("\n--- Successfully loaded artifacts. Ready to proceed to Section 11 ---")
    except FileNotFoundError:
        print("\n--- ERROR: No final artifacts found. Please set RUN_INTERPRETATION to True and run the full section ---")
        RUN_INTERPRETATION = True
    else:
        print("--- RUN_INTERPRETATION is True. Proceeding with full computation. ---")

--- RUN_INTERPRETATION is True. Proceeding with full computation. ---
```

10.2 Behavior Signature Engineering

To interpret the clusters, each track must first be described by a set of quantitative metrics. This cell calculates a "behavior signature" for every track in the dataset, capturing key kinematic properties like speed, acceleration, and turning rate. These signatures form the foundation for profiling and comparing the discovered clusters.

```
In [71]: # This cell will only run if the full computation is enabled.
if RUN_INTERPRETATION:
    # Define a function to calculate kinematic features for a single track
    def _track_signature(g, stall_thr=0.5): # Use 0.5 for consistency
        g = g.sort_values('frame_idx')
        vx, vy = g['vx'].to_numpy(float), g['vy'].to_numpy(float)

        # Calculate speed, heading, turn rate, and acceleration
        speed = (vx**2 + vy**2)**0.5
        heading = np.unwrap(np.arctan2(vy, vx)) # Unwrap for correct stats
        turn_rate = np.abs(np.diff(heading)) / DT if len(heading) > 1 else np.array([])
        accel = np.diff(speed) / DT if len(speed) > 1 else np.array([])

        def _percentile(x, q): return float(np.nanpercentile(x, q)) if len(x) > 0 else np.nan

        # Return a dictionary of summary statistics
        return {
            'n_frames': int(len(g)), 'speed_p50': _percentile(speed, 50),
            'speed_p90': _percentile(speed, 90), 'speed_max': np.nanmax(speed) if len(speed) > 0 else np.nan,
            'speed_std': float(np.nanstd(speed)) if len(speed) > 0 else np.nan,
            'acc_p90': _percentile(np.abs(accel), 90), 'turn_p90': _percentile(turn_rate, 90),
            'heading_std': float(np.nanstd(heading)), # Now correct due to unwrap
            'stationarity': float(np.mean(speed < stall_thr)),
        }

    # Calculate signatures for all tracks
    groups = agents_df_clean.groupby(['scene_id', 'track_id'], sort=False)
    rows = []
    for (sid, tid), g in tqdm(groups, desc="Behavior Signatures", unit="track"):
        rows.append({'scene_id': sid, 'track_id': tid, **_track_signature(g)})
    sig_df = pd.DataFrame(rows)

    # Attach the champion cluster labels
    sig_df = sig_df.merge(final_clusters_df_ae, on=['scene_id', 'track_id'], how='left')

    # Display a formatted preview
    pretty = sig_df.head(10).copy()
    metrics_to_format = [c for c in pretty.columns if c not in ['scene_id', 'track_id', 'n_frames', 'cluster_id']]
    for col in metrics_to_format: pretty[col] = pretty[col].apply(lambda x: f"{x:.3f}" if pd.notna(x) else "---")
    print("--- Per-Track Behavior Signatures ---")
    display(style_section_table(pretty))
else:
    print("--- Skipped: Loading from cache ---")

Behavior Signatures:   0%|          | 0/64625 [00:00<?, ?track/s]
--- Per-Track Behavior Signatures ---
```

scene_id	track_id	n_frames	speed_p50	speed_p90	speed_max	speed_std	acc_p90	turn_p90	heading_std	stationarity	cluster_id
108	1	67	0.000	0.000	0.741	0.105	0.000	0.000	0.368	0.985	0.000000
108	2	248	0.000	0.000	0.290	0.021	0.000	0.000	0.217	1.000	0.000000
108	3	56	0.000	0.000	0.471	0.075	0.000	0.000	0.253	1.000	0.000000
108	4	87	0.000	0.000	0.653	0.083	0.000	0.000	0.065	0.989	0.000000
108	5	248	0.050	6.842	7.618	2.962	1.680	14.259	2.693	0.544	0.000000
108	6	60	0.000	0.000	0.198	0.032	0.000	0.000	1.215	1.000	0.000000
108	7	248	0.000	0.084	0.378	0.057	0.166	1.780	1.283	1.000	0.000000
108	8	248	0.590	6.358	8.125	2.605	1.513	7.379	1.938	0.492	0.000000
108	9	248	0.013	0.161	1.622	0.141	0.620	22.847	3.976	0.992	0.000000
108	10	1	0.000	0.000	0.000	0.000	---	---	0.000	1.000	nan

10.3 Cluster Profile Summary

With signatures calculated for every track, the next step is to aggregate them. This cell groups the tracks by their assigned cluster ID and computes the median value for each kinematic feature. The result is a high-level summary table that provides a distinct, quantitative profile for each cluster, highlighting the central tendency of its behavior.

```
In [72]: # This cell will only run if the full computation is enabled.
if RUN_INTERPRETATION:
    # Define the metrics to be aggregated
    metrics = ['speed_p50', 'speed_p90', 'speed_max', 'speed_std', 'acc_p90', 'turn_p90', 'heading_std', 'stationarity']

    # Group by cluster and calculate median statistics
    cluster_summary = sig_df.groupby('cluster_id')[metrics].median()
    cluster_summary = cluster_summary.rename(columns={m: f"med_{m}" for m in metrics})
    counts = sig_df['cluster_id'].value_counts().rename('n_tracks')
    pcts = sig_df['cluster_id'].value_counts(normalize=True).rename('pct') * 100
    cluster_summary = cluster_summary.join(counts).join(pcts).reset_index()

    # Create and display a formatted preview
    pretty_summary = cluster_summary.sort_values('n_tracks', ascending=False).copy()
    for col in [c for c in pretty_summary.columns if c.startswith('med_')]: pretty_summary[col] = pretty_summary[col].apply(lambda x: f"{x:.3f}" if pd.notna(x) else f"nan")
    pretty_summary['pct'] = pretty_summary['pct'].apply(lambda x: f"{x:.2f}%")
    print("--- Cluster Profile Summary ---")
    display(style_section_table(pretty_summary))

    # Save the final summary table for later use
    print("\n--- Saving final cluster summary for later use ---")
    cluster_summary.to_parquet(ARTIFACT_DIR / "pipeline_b_cluster_summary.parquet", index=False)
    print(f"Saved to: {ARTIFACT_DIR / 'pipeline_b_cluster_summary.parquet'}")
else:
    print("--- Skipped: Loading from cache ---")
```

--- Cluster Profile Summary ---										
cluster_id	med_speed_p50	med_speed_p90	med_speed_max	med_speed_std	med_acc_p90	med_turn_p90	med_heading_std	med_stationarity	n_tracks	pct
0.000000	0.000	0.000	0.137	0.023	0.000	0.000	0.161	1.000	8964	90.14%
1.000000	12.803	13.745	14.168	2.323	1.945	0.115	0.393	0.026	415	4.17%
2.000000	12.123	13.330	13.789	2.227	1.681	0.120	0.131	0.027	330	3.32%
3.000000	4.973	8.155	8.560	1.706	2.235	0.560	0.403	0.038	235	2.36%

```
--- Saving final cluster summary for later use ---
Saved to: /home/trainart/projects/lyft-unsupervised/artifacts/pipeline_b_cluster_summary.parquet
```

10.4 Centroid Exemplar Identification

While median statistics are useful, they do not capture the nuance of a cluster. To provide a more concrete visual interpretation, this cell identifies the most representative track for each cluster. This "centroid exemplar" is the track closest to the cluster's center in the high-dimensional latent space. It serves as the primary visual example in the scenario cards that follow.

```
In [73]: # This cell will only run if the full computation is enabled.
if RUN_INTERPRETATION:
    # Align embeddings with their final cluster labels
    z_cols = [c for c in track_embeddings_df.columns if c.startswith('z')]
    cards_df = track_embeddings_df[['scene_id', 'track_id'] + z_cols].merge(final_clusters_df_ae, on=["scene_id", "track_id"], how="inner").dropna(subset=["cluster_id"])
    Z_cards = cards_df[z_cols].to_numpy(dtype=np.float32)
```

```

y_cards = cards_df[["cluster_id"].astype(int).to_numpy()]
key_cards = cards_df[["scene_id", "track_id"]].copy()

# Find the centroid exemplar and a set of samples for each cluster
centroid_exemplars = {}
sample_exemplars = {}
for cid in tqdm(np.unique(y_cards), desc="Finding Exemplars", unit="cluster"):
    idx = np.where(y_cards == cid)[0]
    if len(idx) == 0: continue

    # Calculate distances to the latent space centroid
    Zc = Z_cards[idx]
    distances = np.linalg.norm(Zc - Zc.mean(axis=0, keepdims=True), axis=1)
    order = np.argsort(distances)

    # Store the original DataFrame indices
    centroid_exemplars[cid] = idx[order[0]]
    sample_exemplars[cid] = idx[order[:20]]

# Display a preview of the identified exemplars
rows = [{"Cluster": int(cid), **key_cards.iloc[idx].to_dict()} for cid, idx in sorted(centroid_exemplars.items())]
exemplar_preview = pd.DataFrame(rows)
print("--- Centroid Exemplar Tracks (Per Cluster) ---")
display(style_section_table(exemplar_preview))
else:
    print("--- Skipped: Loading from cache ---")

```

```

Finding Exemplars:  0%|          | 0/4 [00:00<?, ?cluster/s]

```

```

--- Centroid Exemplar Tracks (Per Cluster) ---

```

Cluster	scene_id	track_id
0	4214	1578
1	5803	2
2	15502	9
3	10600	78

10.5 Scenario Cards

This is the centerpiece of the interpretation stage. The following code generates a "scenario card" for each cluster. Each card is a rich, multi-panel visualization that synthesizes information from different stages of the analysis. It provides a view of each behavior archetype by combining a quantitative signature (radar chart), a qualitative shape of motion (trajectory plots), and a contextual view of the latent space (UMAP plot).

```

In [74]: # This cell will only run if the full computation is enabled.
if RUN_INTERPRETATION:
    # Pre-compute Trajectory Dictionary for Fast Plotting
    print("--- Building fast trajectory lookup dictionary ---")
    trajectories_dict = {}
    grouped = agents_df_clean.groupby(['scene_id', 'track_id'])
    for (sid, tid), g in tqdm(grouped, desc="Processing Trajectories"):
        sorted_g = g.sort_values('frame_idx')
        trajectories_dict[(sid, tid)] = {
            'x': sorted_g.x.to_numpy(),
            'y': sorted_g.y.to_numpy()
        }

    # UMAP Calculation for Visualization
    print("\n--- Calculating UMAP Embeddings for Visualization (see progress below) ---")
    reducer = umap.UMAP(
        n_neighbors=75,
        min_dist=0.05,
        metric="euclidean",
        random_state=SEED,
        n_components=2,
        verbose=True
    )
    U_cards = reducer.fit_transform(Z_cards)

    # Define Plotting Helpers & Pre-calculate Scenario Names
    metrics = ['med_speed_p90', 'med_turn_p90', 'med_stationarity', 'med_heading_std', 'med_acc_p90']
    g = cluster_summary[metrics].quantile([0.25, 0.5, 0.75]).to_dict()

    def _name_row(row):
        sp90 = row['med_speed_p90']; tr90 = row['med_turn_p90']
        stat = row['med_stationarity']; hstd = row['med_heading_std']; accp = row['med_acc_p90']
        if pd.isna(sp90): return "Unlabeled"
        if sp90 >= g['med_speed_p90'][0.75] and tr90 <= g['med_turn_p90'][0.25]: return "High-Speed, Straight"
        if stat >= 0.6 and accp >= g['med_acc_p90'][0.75]: return "Stop-and-Go"
        if hstd >= g['med_heading_std'][0.75] and tr90 >= g['med_turn_p90'][0.75]: return "Weaving / Frequent Turns"
        if sp90 <= g['med_speed_p90'][0.25] and stat >= 0.4: return "Slow / Lingerling"
        return "Mixed-Speed, Moderate Turning"

    summary_z = cluster_summary[metrics].apply(lambda x: (x - x.mean()) / x.std()).fillna(0)
    def _get_distinguishing_features(cid, n=2):

```

```

if cid not in summary_z.index: return ""
z_scores = summary_z.loc[cid].sort_values(ascending=False)
top = z_scores.head(n); low = z_scores.tail(n)
chips = [f"▲ {name.replace('med_', '')}" for name in top.index if top[name] > 0.5]
chips += [f"▼ {name.replace('med_', '')}" for name in low.index if low[name] < -0.5]
return " // ".join(chips)

cluster_summary['scenario_name'] = cluster_summary.apply(_name_row, axis=1)
CARDS_DIR = ARTIFACT_DIR / "cards"; CARDS_DIR.mkdir(parents=True, exist_ok=True)
WIN_CARDS_DIR = WINDOWS_MIRROR / "cards" if WINDOWS_MIRROR else None
if WIN_CARDS_DIR: WIN_CARDS_DIR.mkdir(parents=True, exist_ok=True)

# Define the Card Rendering Function
def render_scenario_card(cid: int, display_in_notebook: bool = False):
    cluster_mask = (y_cards == cid)
    n_tracks = int(cluster_mask.sum())
    if n_tracks == 0: return None
    pct_tracks = 100.0 * n_tracks / len(y_cards)
    row = cluster_summary.loc[cluster_summary["cluster_id"]==cid].iloc[0]
    title_text = row['scenario_name']

    fig = plt.figure(figsize=(24, 14), facecolor="black")
    gs = fig.add_gridspec(2, 3, height_ratios=[0.4, 2], width_ratios=[1.2, 0.9, 0.9], hspace=0.05, wspace=0.05)

    # Header
    axH = fig.add_subplot(gs[0, :]); axH.set_facecolor("black"); axH.axis('off')
    axH.text(0.01, 0.65, title_text, color="dodgerblue", fontsize=32, fontweight="bold", ha="left", va="center")
    axH.text(0.01, 0.25, _get_distinguishing_features(cid), color="white", fontsize=18, ha="left", va="center")
    axH.text(0.99, 0.65, f"Cluster {cid} • Tracks: {n_tracks:,} ({pct_tracks:.1f}%)", color="red", fontsize=32, fontweight="bold", ha="right", va="center")

    # Radar Plot
    axR = fig.add_subplot(gs[1, 0], projection='polar', facecolor='black')
    axR.set_title("Behavior Signature", color="white", fontsize=20, pad=10)
    labels = [c.replace('med_', '').replace('_', ' ').title() for c in metrics]
    values = row[metrics].to_numpy(dtype=float)
    angles = np.linspace(0, 2 * np.pi, len(labels), endpoint=False).tolist()
    values_closed = np.concatenate((values, [values[0]])); angles_closed = angles + [angles[0]]
    axR.plot(angles_closed, values_closed, lw=2.5, color="red"); axR.fill(angles_closed, values_closed, alpha=0.35, color="deepskyblue")
    axR.set_thetagrids(np.degrees(angles), labels, color='white', fontsize=12); axR.tick_params(colors="white", labelsize=10)
    axR.spines["polar"].set_edgecolor("white"); axR.grid(color='white', alpha=0.15)

    # Shape of Motion Plot
    axS = fig.add_subplot(gs[1, 1]); axS.set_facecolor("black")
    axS.set_title("Shape of Motion (Exemplars)", color="white", fontsize=20, pad=10)
    def plot_trajectory(scene_id, track_id, **kwargs):
        traj_data = trajectories_dict.get((scene_id, track_id))
        if traj_data and len(traj_data['x']) > 1:
            x = traj_data['x'] - traj_data['x'][0]; y = traj_data['y'] - traj_data['y'][0]
            axS.plot(x, y, **kwargs)
    for ridx in sample_exemplars.get(cid, []):
        row = key_cards.iloc[ridx]
        plot_trajectory(row.scene_id, row.track_id, alpha=0.3, lw=1.5, color="gold")
    med_idx = centroid_exemplars.get(cid)
    if med_idx is not None:
        row = key_cards.iloc[med_idx]
        plot_trajectory(row.scene_id, row.track_id, alpha=0.95, lw=3.5, color="cyan")
    axS.tick_params(colors="white"); axS.grid(color='white', alpha=0.08); axS.set_aspect('equal', adjustable='box')
    for spine in ['left', 'bottom']: axS.spines[spine].set_color('white')
    for spine in ['right', 'top']: axS.spines[spine].set_visible(False)

    # UMAP Plot
    axU = fig.add_subplot(gs[1, 2]); axU.set_facecolor("black")
    axU.set_title("UMAP (Cluster Highlight)", color="white", fontsize=20, pad=10)
    axU.scatter(U_cards[:,0], U_cards[:,1], s=8, c='gold', alpha=0.05)
    axU.scatter(U_cards[cluster_mask,0], U_cards[cluster_mask,1], s=10, c='lime', alpha=0.85)
    axU.tick_params(colors="white"); axU.grid(color='white', alpha=0.08)
    for spine in ['left', 'bottom']: axU.spines[spine].set_color('white')
    for spine in ['right', 'top']: axU.spines[spine].set_visible(False)

    # Save and Display
    plt.tight_layout()
    save_path = CARDS_DIR / f"scenario_card_c{cid:02d}.png"
    fig.savefig(save_path, dpi=200, bbox_inches="tight", facecolor="black")
    if WIN_CARDS_DIR:
        try: shutil.copy2(save_path, WIN_CARDS_DIR)
        except Exception: pass
    if display_in_notebook: plt.show()
    plt.close(fig)
    return {'cluster_id': cid, 'title': title_text, 'path': save_path}

# Render ALL Cards and Display a Subset
display_counts = defaultdict(int); card_paths = []
order = cluster_summary.sort_values('n_tracks', ascending=False)['cluster_id'].astype(int).tolist()
for cid in tqdm(order, desc="Rendering Scenario Cards", unit="card"):
    title = cluster_summary.loc[cluster_summary.cluster_id == cid, 'scenario_name'].iloc[0]
    should_display = (display_counts[title] < 2)
    path_info = render_scenario_card(cid, display_in_notebook=should_display)
    if path_info: card_paths.append(path_info)
    if should_display: display_counts[title] += 1
else:
    print("--- Skipped: Loading from cache ---")

```

```

--- Building fast trajectory lookup dictionary ---
Processing Trajectories:  0%|          | 0/64625 [00:00<?, ?it/s]

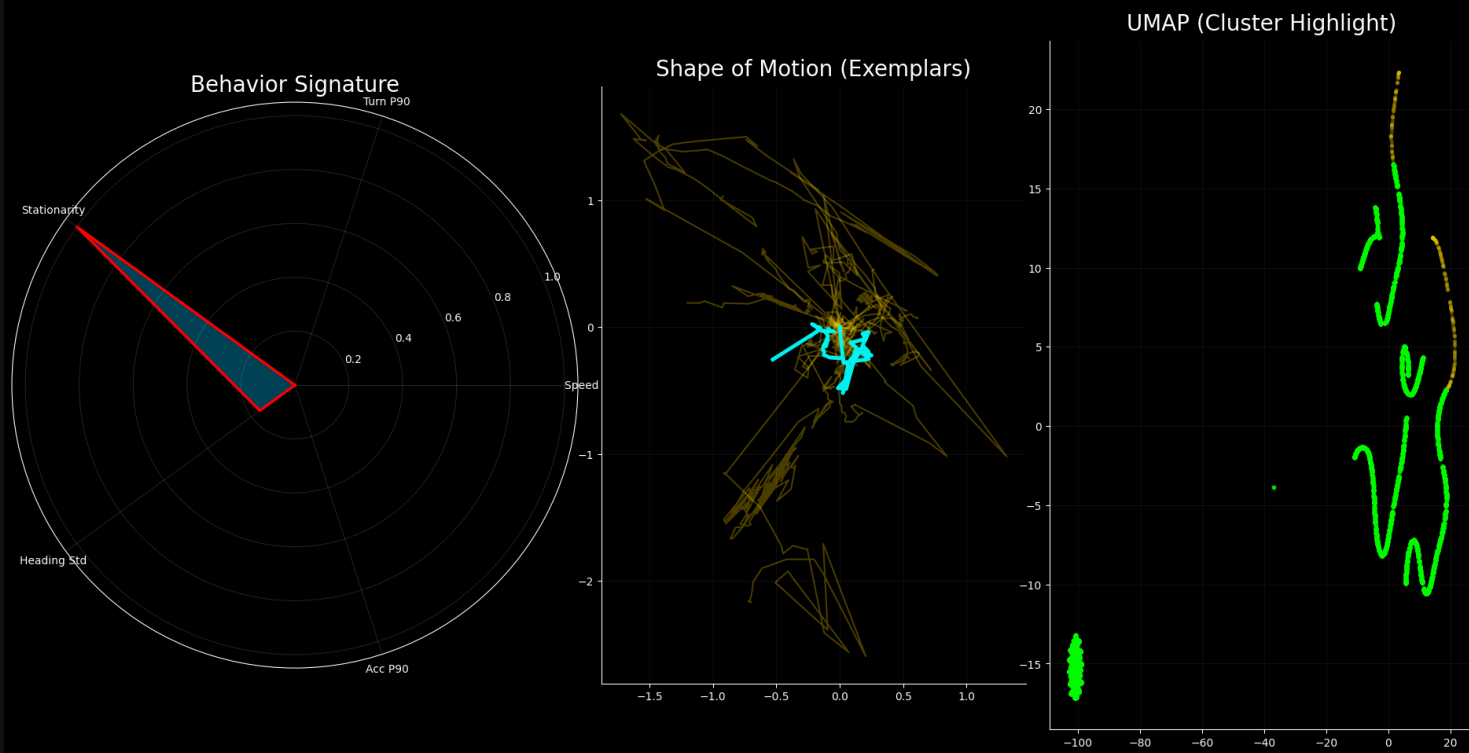
```

```
--- Calculating UMAP Embeddings for Visualization (see progress below) ---
UMAP(min_dist=0.05, n_jobs=1, n_neighbors=75, random_state=42, verbose=True)
Wed Sep 17 22:08:38 2025 Construct fuzzy simplicial set
Wed Sep 17 22:08:38 2025 Finding Nearest Neighbors
Wed Sep 17 22:08:38 2025 Building RP forest with 10 trees
Wed Sep 17 22:08:38 2025 NN descent for 13 iterations
  1 / 13
  2 / 13
  3 / 13
Stopping threshold met -- exiting after 3 iterations
Wed Sep 17 22:08:40 2025 Finished Nearest Neighbor Search
Wed Sep 17 22:08:40 2025 Construct embedding
Epochs completed:  0%|          0/500 [00:00]
completed  0 / 500 epochs
completed 50 / 500 epochs
completed 100 / 500 epochs
completed 150 / 500 epochs
completed 200 / 500 epochs
completed 250 / 500 epochs
completed 300 / 500 epochs
completed 350 / 500 epochs
completed 400 / 500 epochs
completed 450 / 500 epochs
Wed Sep 17 22:09:49 2025 Finished embedding
Rendering Scenario Cards:  0%|          | 0/4 [00:00<?, ?card/s]
```

Slow / Linger

Cluster 0 • Tracks: 8,964 (90.1%)

▲ stationarity // ▼ speed_p90 // ▼ acc_p90

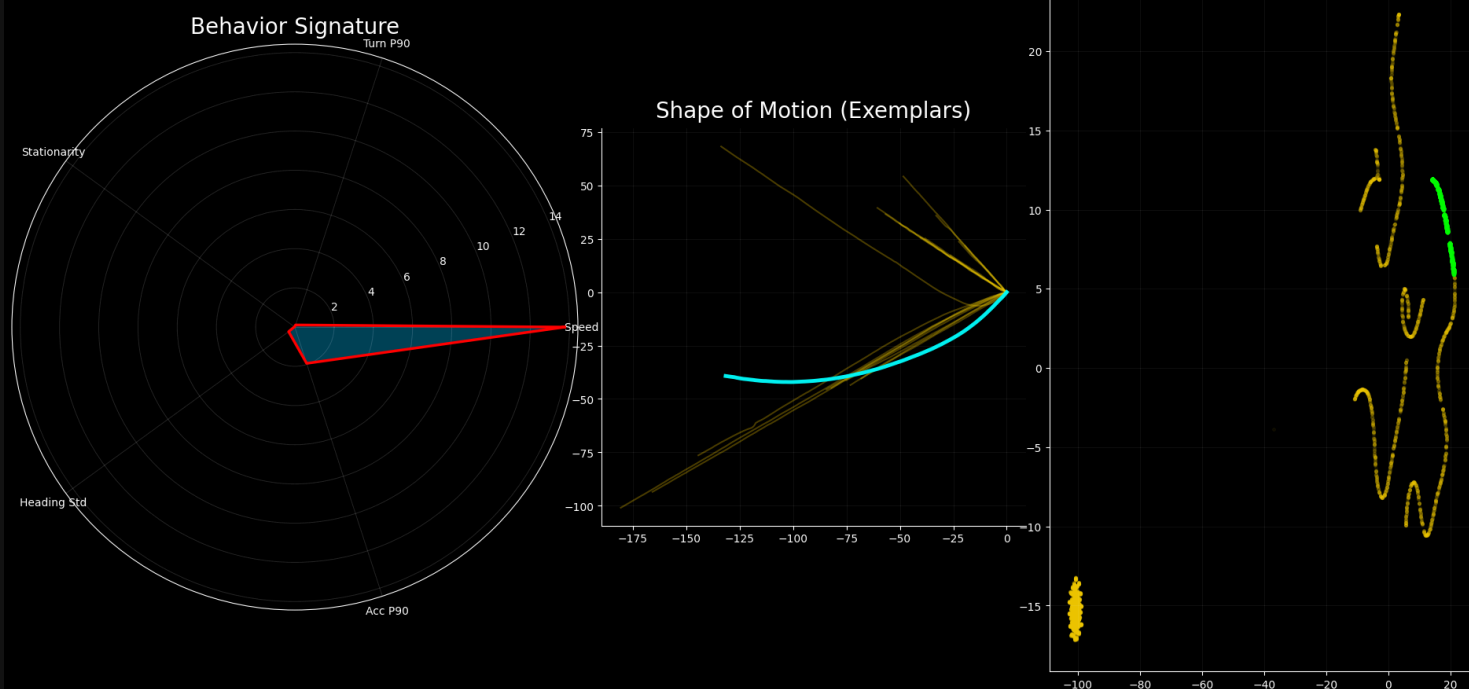


Mixed-Speed, Moderate Turning

Cluster 1 • Tracks: 415 (4.2%)

▲ heading_std // ▲ speed_p90 // ▼ stationarity

UMAP (Cluster Highlight)

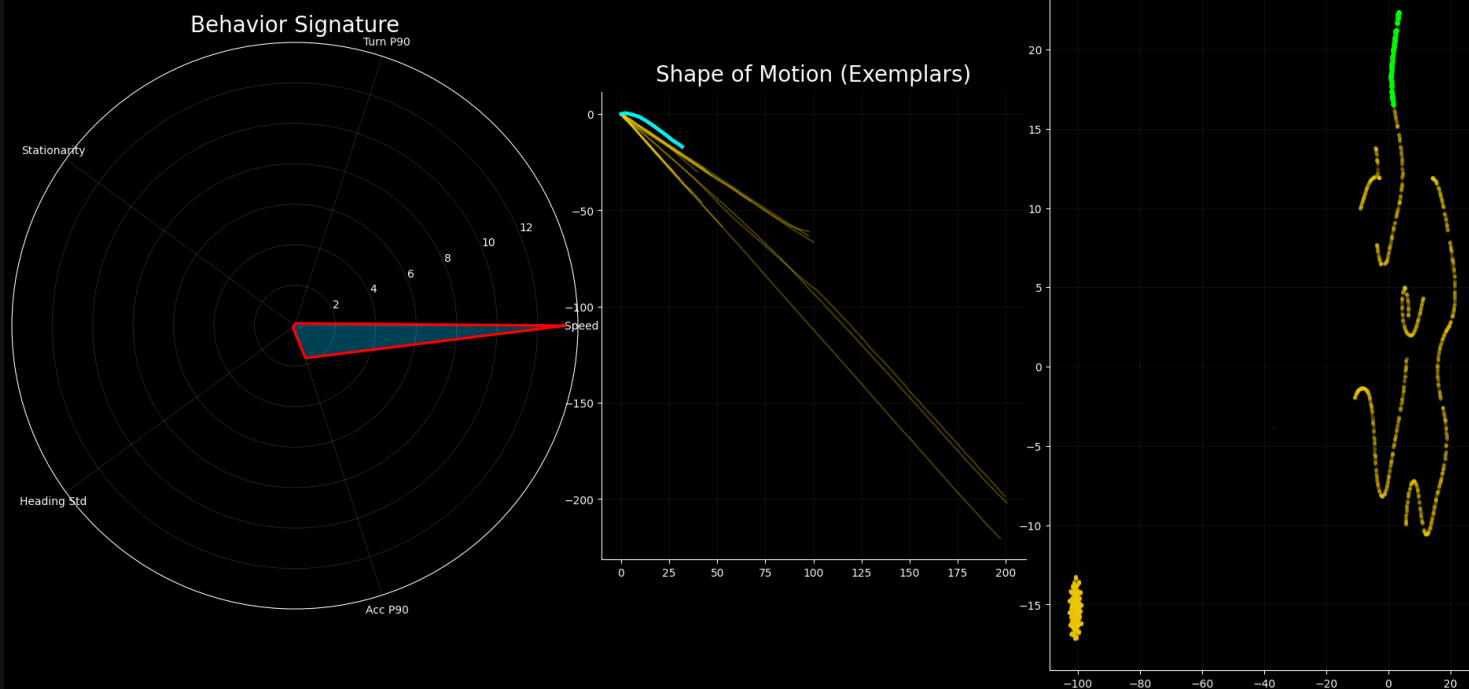


Mixed-Speed, Moderate Turning

Cluster 2 • Tracks: 330 (3.3%)

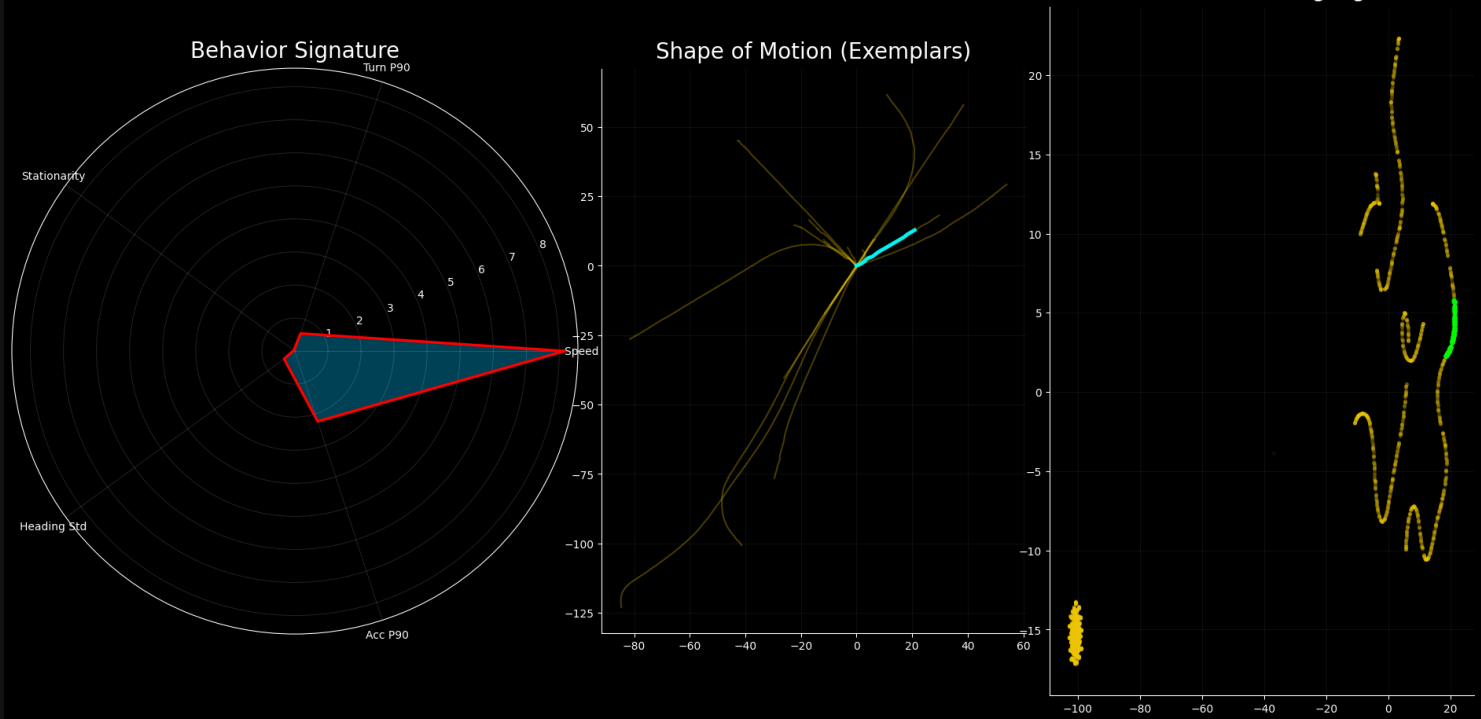
▲ speed_p90 // ▼ stationarity // ▼ heading_std

UMAP (Cluster Highlight)



▲ turn_p90 // ▲ heading_std

UMAP (Cluster Highlight)



10.6 Scenario Gallery

This final visualization provides a high-level overview of the entire solution. It presents a gallery of thumbnails, where each thumbnail shows the location of a single cluster within the 2D UMAP projection of the latent space. This view helps to understand the relationships between clusters and the overall structure of the learned behavior manifold.

```
In [75]: # This cell will only run if the full computation is enabled.
if RUN_INTERPRETATION:
    # Determine Grid Layout and Create Figure
    C = len(cluster_summary)
    cols = min(4, C)
    rows = int(np.ceil(C/cols))
    fig, axes = plt.subplots(rows, cols, figsize=(20, 5 * rows), facecolor="black")
    axes = np.atleast_2d(axes).flatten()

    # Plot Each Cluster on its Own Subplot
    order = cluster_summary.sort_values('n_tracks', ascending=False)['cluster_id'].tolist()
    for i, cid in enumerate(order):
        ax = axes[i]
        ax.set_facecolor("black")
        cluster_mask = (y_cards == cid)

        # Plot all points as a background
        ax.scatter(U_cards[:,0], U_cards[:,1], s=6, c='gold', alpha=0.04)

        # Highlight the current cluster's points
        cluster_points = U_cards[cluster_mask]
        ax.scatter(cluster_points[:,0], cluster_points[:,1], s=14, c='cyan', alpha=0.9)

        # Zoom the axes to the highlighted cluster
        if cluster_points.shape[0] > 1:
            x_min, x_max = cluster_points[:, 0].min(), cluster_points[:, 0].max()
            y_min, y_max = cluster_points[:, 1].min(), cluster_points[:, 1].max()
            x_padding = (x_max - x_min) * 0.1; y_padding = (y_max - y_min) * 0.1
            ax.set_xlim(x_min - x_padding, x_max + x_padding)
            ax.set_ylim(y_min - y_padding, y_max + y_padding)

        # Style the plot
        scenario_name = cluster_summary[cluster_summary.cluster_id == cid]['scenario_name'].iloc[0]
        ax.set_title(f"Cluster {int(cid)}\n({scenario_name})", color="#CFB87C", fontsize=14, pad=6)
        ax.tick_params(colors="white", labelsize=8)
        for sp in ax.spines.values():
            sp.set_edgecolor("white")
        ax.grid(color='white', alpha=0.06)
```

```
# Hide any unused subplots
for j in range(i + 1, len(axes)):
    axes[j].axis('off')

# Save the Final Gallery Figure
save_fig(fig, "scenario_gallery")
else:
    print("--- Skipped: Loading from cache ---")
```

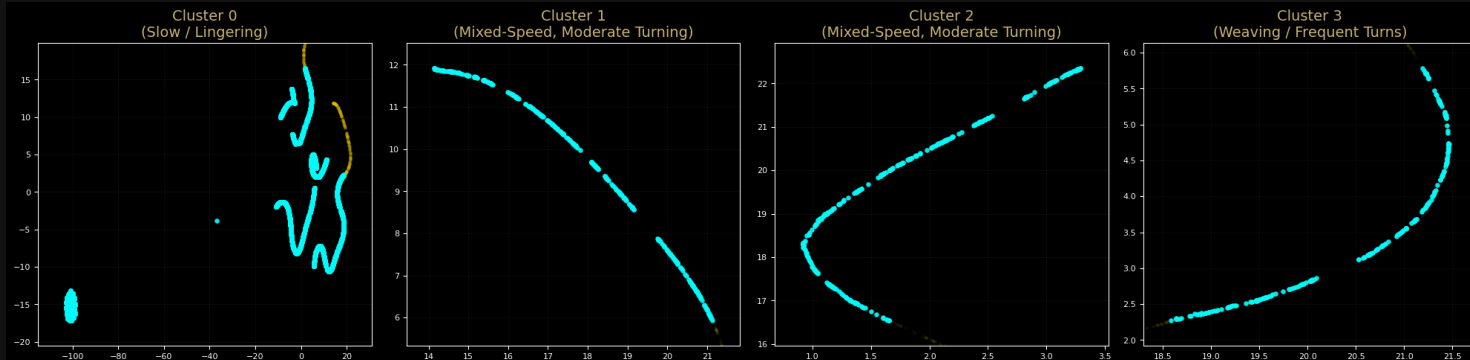


Figure saved to (Linux): /home/treinart/projects/lyft-unsupervised/artifacts/figures/20250917_221131_scenario_gallery.png

Figure saved to (Win): /mnt/c/Users/travi/Documents/Training/Colorado/MS-AI/CSCA 5632 Final Project/artifacts/figures/20250917_221131_scenario_gallery.png

10.7 Deep Dive: Animating a Scenario Replay

As a final "show stopper" visualization, this cell renders an animated MP4 video replay of a single, representative scenario. It selects the exemplar track from one of the most interesting clusters ("Weaving / Frequent Turns") and animates the positions of all agents in that scene, highlighting the exemplar. This provides a dynamic, intuitive view of the discovered behavior in its original context.

In []: # This cell will only run if the full computation is enabled.

```
if RUN_INTERPRETATION:
    # Check if ffmpeg is available
    if shutil.which("ffmpeg") is None:
        print("--- ffmpeg not found. Skipping animation. ---")
    else:
        # Setup and Scene Selection
        VIDEO_DIR = ARTIFACT_DIR / "videos"; VIDEO_DIR.mkdir(exist_ok=True)
        WIN_VIDEO_DIR = WINDOWS_MIRROR / "videos" if WINDOWS_MIRROR else None
        if WIN_VIDEO_DIR: WIN_VIDEO_DIR.mkdir(parents=True, exist_ok=True)

        try: # Select a target cluster to animate
            target_cluster_id = cluster_summary[cluster_summary.scenario_name == "Weaving / Frequent Turns"].cluster_id.iloc[0]
        except IndexError: # Fallback if name isn't found
            target_cluster_id = cluster_summary.sort_values('n_tracks', ascending=False).iloc[2].cluster_id

        exemplar_info = key_cards.iloc[centroid_exemplars[target_cluster_id]]
        SCENE_TO_ANIMATE = exemplar_info.scene_id
        TRACK_TO_HIGHLIGHT = exemplar_info.track_id
        scene_df = agents_df_clean[agents_df_clean.scene_id == SCENE_TO_ANIMATE].copy()
        scene_frames = sorted(scene_df.frame_idx.unique())

        # Animation Setup
        COLOR_MODE = "cluster"
        scene_tracks = scene_df[["scene_id", "track_id"]].drop_duplicates()
        track_clusters = scene_tracks.merge(final_clusters_df_ae, on=["scene_id", "track_id"], how="left")
        unique_clusters = np.sort(track_clusters["cluster_id"].dropna().unique())
        base_cmap = cm.get_cmap("tab20", max(1, len(unique_clusters)))
        cluster_to_color = {int(cid): mcolors.to_hex(base_cmap(i)) for i, cid in enumerate(unique_clusters)}
        track_clusters['color'] = track_clusters['cluster_id'].apply(lambda cid: cluster_to_color.get(int(cid), "#6E6E6E") if pd.notna(cid) else "#6E6E6E")
        track_colors = pd.Series(track_clusters.color.values, index=track_clusters.track_id).to_dict()

        # Create the figure and draw static elements
        fig, ax = plt.subplots(figsize=(20, 20), facecolor="#0b0b0e")
        ax.set_facecolor("#0b0b0e")
        ax.tick_params(colors="white"); [s.set_edgecolor("white") for s in ax.spines.values()]
        for tid, track in scene_df.groupby('track_id'):
            if int(tid) == int(TRACK_TO_HIGHLIGHT): continue
            ax.plot(track.x.to_numpy(), track.y.to_numpy(), color=track_colors.get(int(tid), "#6E6E6E"), alpha=0.7, lw=2)
        highlighted_track = scene_df[scene_df["track_id"] == TRACK_TO_HIGHLIGHT]
        if not highlighted_track.empty:
            ax.plot(highlighted_track.x, highlighted_track.y, color="#0aa0b5", lw=4.5, alpha=0.6, zorder=2)
            ax.plot(highlighted_track.x, highlighted_track.y, color="#00e5ff", lw=2.2, alpha=0.95, zorder=3)
        x_min, x_max = scene_df.x.min(), scene_df.x.max(); y_min, y_max = scene_df.y.min(), scene_df.y.max()
        ax.set_xlim(x_min - 10, x_max + 10); ax.set_ylim(y_min - 10, y_max + 10)
        ax.set_aspect("equal", adjustable="box")
        frame_artists = []

        # Define the Animation Update Function
        def update(frame):
            if frame_artists: frame_artists[0].remove(); frame_artists.clear()
            frame_data = scene_df[scene_df.frame_idx == frame]
            if frame_data.empty: return []
```

```

        colors = ["#00e5ff" if int(tid) == int(TRACK_TO_HIGHLIGHT) else track_colors.get(int(tid), "#6E6E6E") for tid in frame_data['track_id']]
        quiver = ax.quiver(frame_data['x'], frame_data['y'], frame_data['vx'], frame_data['vy'], color=colors, headwidth=4, headlength=5, width=0.005, zorder=10)
        frame_artists.append(quiver)
        ax.set_title(f"Scene {SCENE_TO_ANIMATE} - Frame {frame}", color="#CFB87C", fontsize=16)
        return frame_artists

# Generate, Save, and Display the Animation
print(f"Generating animation for Scene {SCENE_TO_ANIMATE}")
pbar = tqdm(total=len(scene_frames), desc="Saving Animation")
ani = animation.FuncAnimation(fig, update, frames=scene_frames, blit=False, interval=100)
timestamp = time.strftime("%Y%m%d_%H%M%S")
save_path = VIDEO_DIR / f"scenario_replay_{COLOR_MODE}_{timestamp}.mp4"
ani.save(save_path, writer='ffmpeg', dpi=150, progress_callback=lambda i, n: pbar.update(1))
pbar.close(); plt.close(fig)

# Mirror and print confirmation
print(f"\nAnimation Saved\n - Linux: {save_path.resolve()}")
if WIN_VIDEO_DIR:
    try:
        shutil.copy2(save_path, WIN_VIDEO_DIR)
        print(f" - Win: {WIN_VIDEO_DIR / save_path.name}")
    except Exception as e:
        print(f"Mirror Warning: {e}")

else:
    print("--- Skipped: Loading from cache ---")

```

```

Generating animation for Scene 10600
Saving Animation:   0%|          | 0/249 [00:00<?, ?it/s]
Animation Saved
- Linux: /home/treinart/projects/lyft-unsupervised/artifacts/videos/scenario_replay_cluster_20250917_221138.mp4
- Win:   /mnt/c/Users/travi/Documents/Training/Colorado/MS-AI/CSCA 5632 Final Project/artifacts/videos/scenario_replay_cluster_20250917_221138.mp4

```

```

In [77]: # Optional: Display the animation in the notebook (can be slow/Large)
display(HTML(ani.to_html5_video()))

```


▶ 0:00 / 0:24

Observation: Visualizing Trajectory Behavior

What this section delivers.

Section 10 turns the clustering from Section 9 into interpretable behavior. It provides per-track signatures, cluster-level profiles, scenario cards for each cluster, a gallery that shows where those behaviors live in the embedding, and a short replay that lets the motion be seen over time.

Cluster makeup at a glance.

From the profile table:

- **Cluster 0: Slow / Linger** - 8,964 tracks, **90.1%** of all tracks. Median stationarity ≈ 1.00 with near-zero speed.
- **Cluster 1: Mixed-speed, Moderate Turning** - 415 tracks, **4.2%**. Median speed $p50 \approx 12.8$, modest turning and heading variation.
- **Cluster 2: Mixed-speed, Moderate Turning** - 330 tracks, **3.3%**. Similar regime to Cluster 1 with slightly lower accel $p90$.
- **Cluster 3: Weaving / Frequent Turns** - 235 tracks, **2.4%**. Lower median speed $p50 \approx 5.0$ but **higher** turning and heading variability ($\text{turn}_{p90} \approx 0.56$, $\text{heading_std} \approx 0.40$).

Reading a scenario card

Each card has three coordinated views:

1. Behavior Signature (radar).

- **Cluster 0** spikes on **stationarity** and sits near the origin for speed, accel, and turning. This confirms the long tail of idle or near-idle tracks.
- **Cluster 3** elevates **turn_p90** and **heading_std** with moderate **acc_p90**, the hallmark of weaving.

2. Shape of Motion (exemplars).

- **Cluster 0** shows short, jittery traces that stay close to a point or creep slowly, consistent with lingering.
- **Cluster 1 and 2** produce longer, mostly smooth arcs with moderate curvature.
- **Cluster 3** contains trajectories with repeated heading changes and small S-curves, matching the radar signature.

3. UMAP highlight.

- **Cluster 0** occupies a compact island, separated from the more dynamic clusters.
- **Clusters 1 and 2** live on nearby arcs along the same manifold, indicating related motion regimes.
- **Cluster 3** concentrates in a distinct neighborhood where curvature and heading variance rise.

Scenario gallery

The gallery places an exemplar path for each cluster on its local UMAP neighborhood. The cyan traces show representative shapes while the yellow points provide nearby context. The result is a quick visual index: idle behavior in its own pocket, two mixed-speed regimes along smooth arcs, and a weaving pocket that is clearly distinct.

Per-track signatures

The table confirms what the cards show at micro scale. Examples in scenes with **stationarity** ≈ 1.0 land in Cluster 0. Tracks with **speed_p90 near 13** and **low turn_p90** map to Clusters 1 or 2. Rows with **large turn_p90** and **higher heading_std** correspond to Cluster 3. These fields give a compact, queryable summary for downstream search and filtering.

Centroid exemplars

For each cluster, a single scene_id and track_id are identified as closest to the cluster center. These are the most “typical” representatives and are the right choices for documentation, QA spot checks, and replay.

Video replay

The MP4 for **Scene 10600** showcases the Cluster 3 exemplar. The agent advances with frequent heading changes, short bursts of acceleration, and visible curvature. The motion is consistent with the **Weaving / Frequent Turns** label and mirrors the elevated **turn_p90** and **heading_std** seen in the card.

Conclusion

The visuals line up with the statistics: a dominant slow or lingering population, two clean mixed-speed regimes with modest curvature, and a small but well-separated weaving group. The agreement between radar signatures, exemplar shapes, UMAP neighborhoods, per-track tables, and the replay builds confidence that the clusters correspond to real, interpretable behaviors.

Section 11: Pipeline Comparison

This section performs a side-by-side comparison of Pipeline A (classical features) and Pipeline B (learned features). The analysis involves measuring the agreement between the two sets of cluster assignments and visualizing the differences. This allows for a data-driven conclusion on which approach produced a more effective or insightful clustering solution.

Section Plan

- **Data Loading & Alignment:** Load all necessary artifacts, align Pipeline B's labels to Pipeline A's, and compute agreement metrics.
- **Visualization Utilities:** Define helper functions for creating palettes and rendering scene backdrops.
- **Side-by-Side Scene Viewer:** Visualize the differences in cluster assignments for a sample scene.
- **Aligned Exemplar Gallery:** Compare the most representative tracks from aligned cluster pairs.
- **Alignment Scorecard:** Create a detailed summary plot comparing the feature profiles for each archetype.
- **Export Final Comparison:** Package all comparison tables into a final, multi-sheet Excel spreadsheet.

11.1 Data Loading & Label Alignment

This cell loads all the necessary final artifacts from both Pipeline A and Pipeline B. It finds the common set of tracks clustered by both pipelines, aligns the labels from Pipeline B to match Pipeline A, and computes global agreement statistics like ARI and NMI.

```
In [78]: # Load Final Artifacts from Both Pipelines
print("--- Loading Final Artifacts ---")
clusters_a = pd.read_parquet(ARTIFACT_DIR / "pipeline_a_cluster_assignments.parquet")
clusters_b = pd.read_parquet(ARTIFACT_DIR / "pipeline_b_cluster_assignments.parquet")
features_a = pd.read_parquet(ARTIFACT_DIR / "pipeline_a_features.parquet")
embeddings_b = pd.read_parquet(ARTIFACT_DIR / "pipeline_b_track_embeddings.parquet")
print("Loaded cluster assignments, features, and embeddings for both pipelines.")

# Define the Label Alignment Function Here
def align_labels(y_true: np.ndarray, y_pred: np.ndarray):
    # ignore noise if present
    mask = (y_true != -1) & (y_pred != -1)
    A_labs = np.unique(y_true[mask])
    B_labs = np.unique(y_pred[mask])
    cm = confusion_matrix(y_true[mask], y_pred[mask], labels=A_labs)
    cost = cm.max() - cm
    row_ind, col_ind = linear_sum_assignment(cost)
    map_B2A = {int(B_labs[j]): int(A_labs[i]) for i, j in zip(row_ind, col_ind)}
    y_aligned = np.array([map_B2A.get(int(lbl), -1) if lbl != -1 else -1 for lbl in y_pred], dtype=int)
    return y_aligned, map_B2A, cm

# Find the Intersection of Tracks
alignment_tbl = clusters_a.merge(clusters_b, on=["scene_id", "track_id", "suffixes=("_A", "_B")])
print(f"\nFound {len(alignment_tbl):,} shared tracks across both pipelines for comparison.")

# Align Pipeline B Labels to Pipeline A
yA = alignment_tbl["cluster_id_A"].to_numpy()
yB = alignment_tbl["cluster_id_B"].to_numpy()
yB_aligned, map_B2A, _ = align_labels(yA, yB)
alignment_tbl["cluster_B_aligned"] = yB_aligned
alignment_tbl["agree"] = (yA == yB_aligned)

# Compute and Display Global Agreement Metrics
global_stats = pd.DataFrame([
    "tracks_compared": len(yA), "k_A": len(np.unique(yA)), "k_B": len(np.unique(yB)),
    "ARI(A, B_aligned)": ARI(yA, yB_aligned), "NMI(A, B_aligned)": NMI(yA, yB_aligned),
    "Agreement_Rate": (yA == yB_aligned).mean()
])
print("\n--- Global Alignment & Agreement ---")
display(style_section_table(global_stats.round(4)))

# Compute and Display Per-Cluster Overlap
A_labels = np.unique(yA)
overlap_rows = []
for ca in A_labels:
    mask_a = (yA == ca)
    vals, counts = np.unique(yB_aligned[mask_a], return_counts=True)
    if len(vals):
        b_mode = int(vals[counts.argmax()])
        overlap = float(counts.max() / mask_a.sum())
    else:
        b_mode, overlap = -1, 0.0
    overlap_rows.append({"cluster_A": int(ca), "A_count": int(mask_a.sum()),
        "B_mode_aligned": b_mode, "overlap_rate": overlap})
overlap_df = pd.DataFrame(overlap_rows).sort_values("cluster_A")

# Save Alignment Artifacts
ALIGN_DIR = ARTIFACT_DIR / "compare_ab"; ALIGN_DIR.mkdir(parents=True, exist_ok=True)
alignment_tbl.to_parquet(ALIGN_DIR / "alignment_table.parquet", index=False)
with open(ALIGN_DIR / "label_map_B_to_A.json", "w") as f: json.dump({int(k): int(v) for k,v in map_B2A.items()}, f, indent=2)
global_stats.to_parquet(ALIGN_DIR / "global_stats.parquet", index=False)
overlap_df.to_parquet(ALIGN_DIR / "overlap_by_cluster.parquet", index=False)
print(f"\n[Saved] Alignment artifacts -> {ALIGN_DIR}")
```

--- Loading Final Artifacts ---
Loaded cluster assignments, features, and embeddings for both pipelines.

Found 7,824 shared tracks across both pipelines for comparison.

--- Global Alignment & Agreement ---

tracks_compared	k_A	k_B	ARI(A, B_aligned)	NMI(A, B_aligned)	Agreement_Rate
7824	4	4	0.262500	0.369800	0.651200

[Saved] Alignment artifacts -> /home/treinart/projects/lyft-unsupervised/artifacts/compare_ab

11.2 Visualization Utilities

This cell centralizes all helper functions used for the comparison visualizations, including the label alignment function from Section 9, palette generators, and functions to render scene backdrops.

```
In [79]: # Palette and BEV Helpers
def cluster_palette(n, seed=7):
    rng = np.random.default_rng(seed)
    base = plt.get_cmap("tab20").colors
    cols = [base[i % len(base)] for i in range(n)]
    rng.shuffle(cols)
    return {i: cols[i] for i in range(n)}

PALETTE_A = cluster_palette(clusters_a["cluster_id"].nunique(), seed=13)
PALETTE_B_ALIGNED = PALETTE_A

def bev_from_scene(scene_df, bins=300):
    xy = scene_df[["x", "y"]].to_numpy(dtype=float)
    if len(xy) == 0: return np.zeros((bins,bins)), (0,1,0,1)
    x_min, x_max = xy[:,0].min(), xy[:,0].max(); y_min, y_max = xy[:,1].min(), xy[:,1].max()
    H, _, _ = np.histogram2d(xy[:,0], xy[:,1], bins=bins, range=[[x_min,x_max],[y_min,y_max]])
    H = np.log1p(H.T)
    return H, [x_min, x_max, y_min, y_max]

def draw_bev(ax, H, extent, alpha=0.35):
    ax.imshow(H, extent=extent, origin="lower", cmap="Greys", alpha=alpha)

def track_xyv(agents, sid, tid):
    g = agents[(agents.scene_id==sid) & (agents.track_id==tid)].sort_values("frame_idx")
    return g["x"].to_numpy(), g["y"].to_numpy(), g["vx"].to_numpy(), g["vy"].to_numpy()

# Sanity Check Printout
print("-" * 70)
print("--- Visualization Utilities Ready ---")
print(f" - PALETTE_A created with {len(PALETTE_A)} colors: {dict(list(PALETTE_A.items())[:3])}...")
print(f" - Helper functions defined: cluster_palette, bev_from_scene, draw_bev, track_xyv")
print("-" * 70)
```

```
--- Visualization Utilities Ready ---
- PALETTE_A created with 4 colors: {0: (1.0, 0.7333333333333333, 0.47058823529411764), 1: (1.0, 0.4980392156862745, 0.054901960784313725), 2: (0.6823529411
764706, 0.7803921568627451, 0.9098039215686274)}...
- Helper functions defined: cluster_palette, bev_from_scene, draw_bev, track_xyv
-----
```

11.3 Side-by-Side Scene Viewer

This visualization provides a direct comparison of the two pipelines on a single, representative scene. The left panel shows the clustering from Pipeline A, the center panel shows the aligned clustering from Pipeline B, and the right panel highlights agreement (green) versus disagreement (red) on a per-track basis.

```
In [80]: # Define the Scene Viewer Function
def render_scene_compare(scene_id: int):
    df_scene = agents_df_clean[agents_df_clean.scene_id==scene_id]
    if df_scene.empty: print(f"Scene {scene_id} not found."); return
    assign = alignment_tbl[alignment_tbl.scene_id==scene_id]
    H, extent = bev_from_scene(df_scene, bins=280)

    # Create the plot canvas
    fig, axes = plt.subplots(1, 3, figsize=(24,12), facecolor="black")
    titles = ["Pipeline A (Classical Features)", "Pipeline B (Learned Embeddings)", "Agreement Mask"]
    for ax, t in zip(axes, titles):
        ax.set_facecolor("black"); draw_bev(ax, H, extent, alpha=0.40)
        ax.set_title(t, fontsize=18, color="#CFB87C", fontweight="bold")
        ax.tick_params(colors="white"); [sp.set_color("white") for sp_n, sp in ax.spines.items() if sp_n in ["left","bottom"]]
        [sp.set_visible(False) for sp_n, sp in ax.spines.items() if sp_n in ["right","top"]]

    # Plot trajectories for each panel
    for _, row in assign.iterrows():
        tid = int(row.track_id)
        x,y,_,_ = track_xyv(df_scene, scene_id, tid)
```

```

# Use the correct column names: 'cluster_id_A' and 'cluster_B_aligned'
color_a = PALETTE_A.get(int(row.cluster_id_A), "grey")
color_b = PALETTE_B_ALIGNED.get(int(row.cluster_B_aligned), "grey")
color_agree = "lime" if bool(row.agree) else "red"

axes[0].plot(x, y, color=color_a, lw=1.8, alpha=0.9)
axes[1].plot(x, y, color=color_b, lw=1.8, alpha=0.9)
axes[2].plot(x, y, color="dimgrey", lw=1.0, alpha=0.6)
axes[2].plot(x, y, color=color_agree, lw=2.0, alpha=0.9)

save_fig(fig, f"scene_compare_{scene_id}")

# Render the comparison for the top 4 busiest scenes
top_4_scenes = alignment_tbl["scene_id"].value_counts().nlargest(4).index.tolist()
print(f"--- Rendering Top 4 Busiest Scenes for Comparison ---")
for scene_id in top_4_scenes:
    print(f"\n--- Displaying Scene {scene_id} ---")
    render_scene_compare(scene_id)

```

--- Rendering Top 4 Busiest Scenes for Comparison ---

--- Displaying Scene 15692 ---

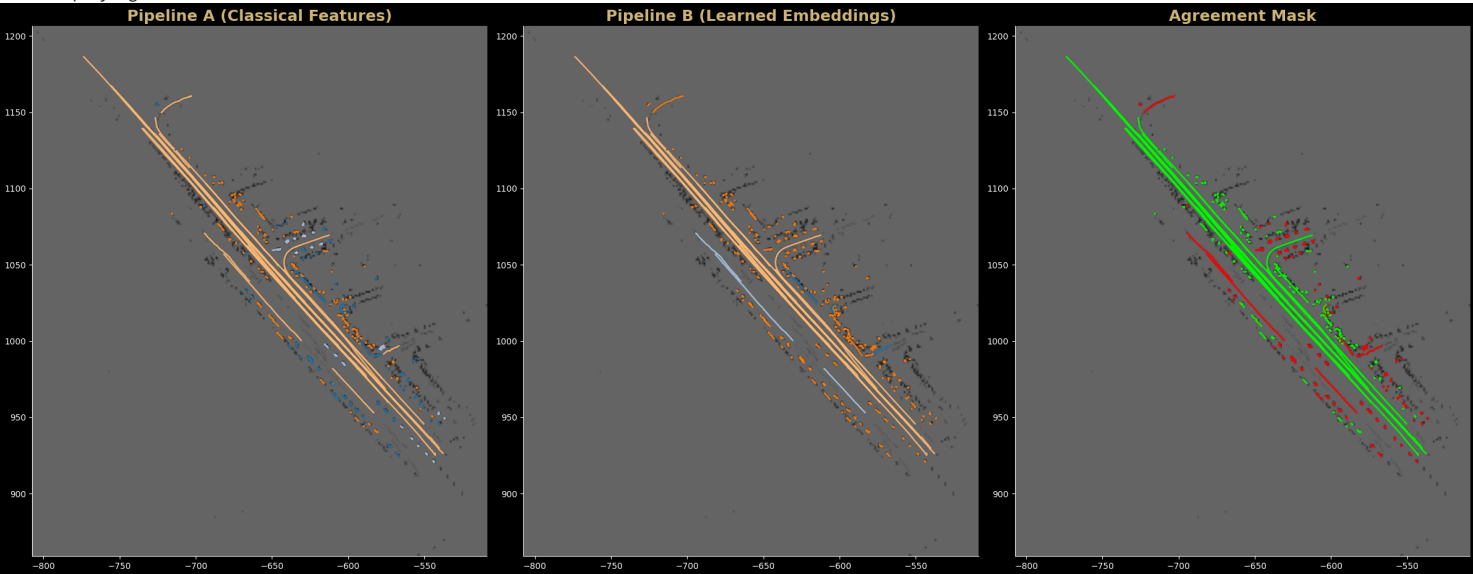


Figure saved to (Linux): /home/treinart/projects/lyft-unsupervised/artifacts/figures/20250917_221439_scene_compare_15692.png

Figure saved to (Win): /mnt/c/Users/travi/Documents/Training/Colorado/MS-AI/CSCA 5632 Final Project/artifacts/figures/20250917_221439_scene_compare_15692.png

--- Displaying Scene 7144 ---

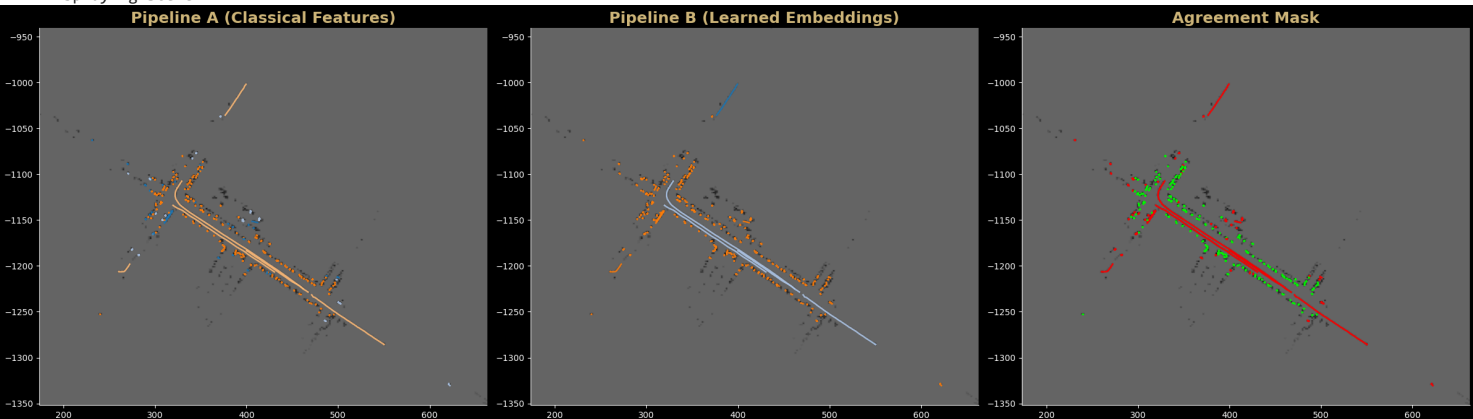


Figure saved to (Linux): /home/treinart/projects/lyft-unsupervised/artifacts/figures/20250917_221440_scene_compare_7144.png

Figure saved to (Win): /mnt/c/Users/travi/Documents/Training/Colorado/MS-AI/CSCA 5632 Final Project/artifacts/figures/20250917_221440_scene_compare_7144.png

--- Displaying Scene 7143 ---

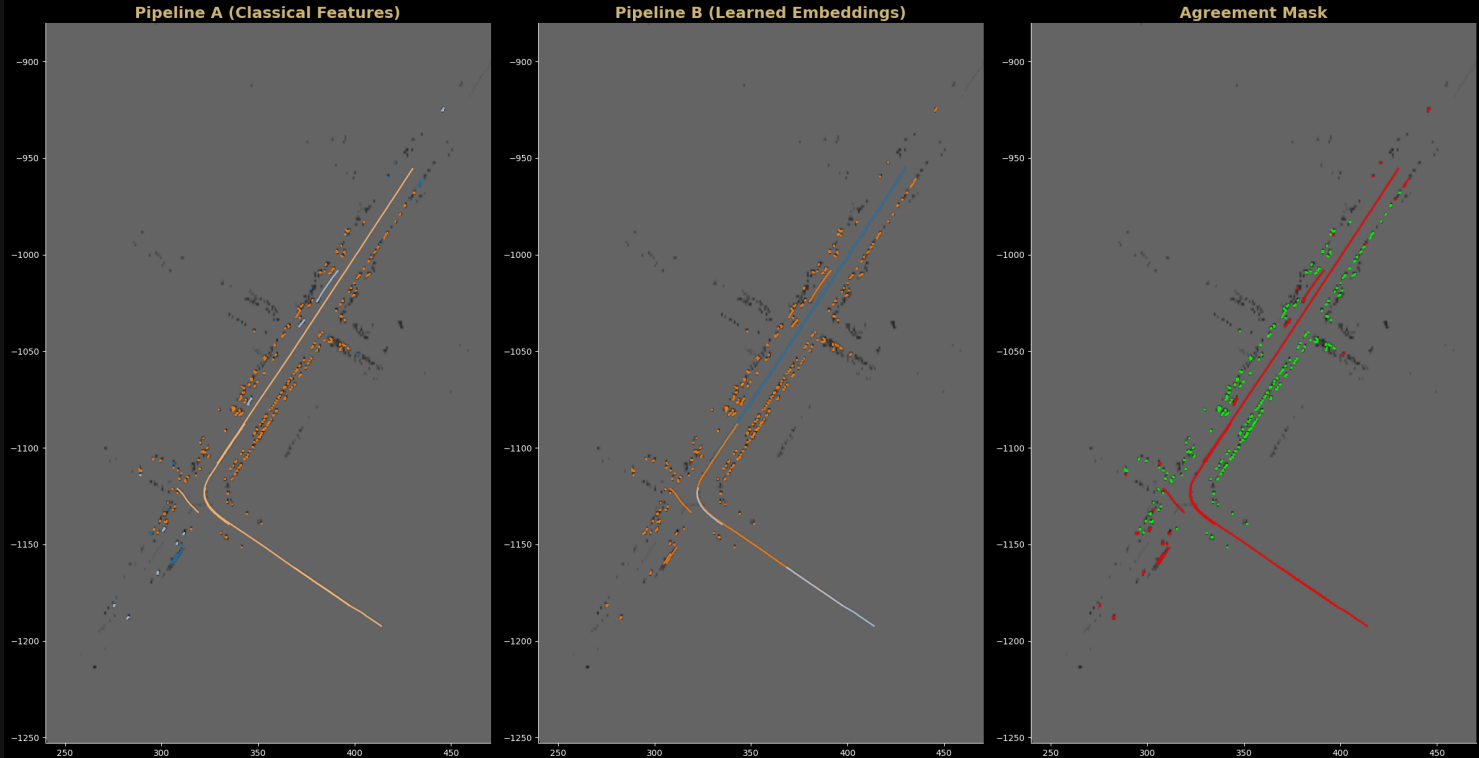


Figure saved to (Linux): /home/treinart/projects/lyft-unsupervised/artifacts/figures/20250917_221441_scene_compare_7143.png
 Figure saved to (Win): /mnt/c/Users/travi/Documents/Training/Colorado/MS-AI/CSCA 5632 Final Project/artifacts/figures/20250917_221441_scene_compare_7143.png

--- Displaying Scene 1174 ---

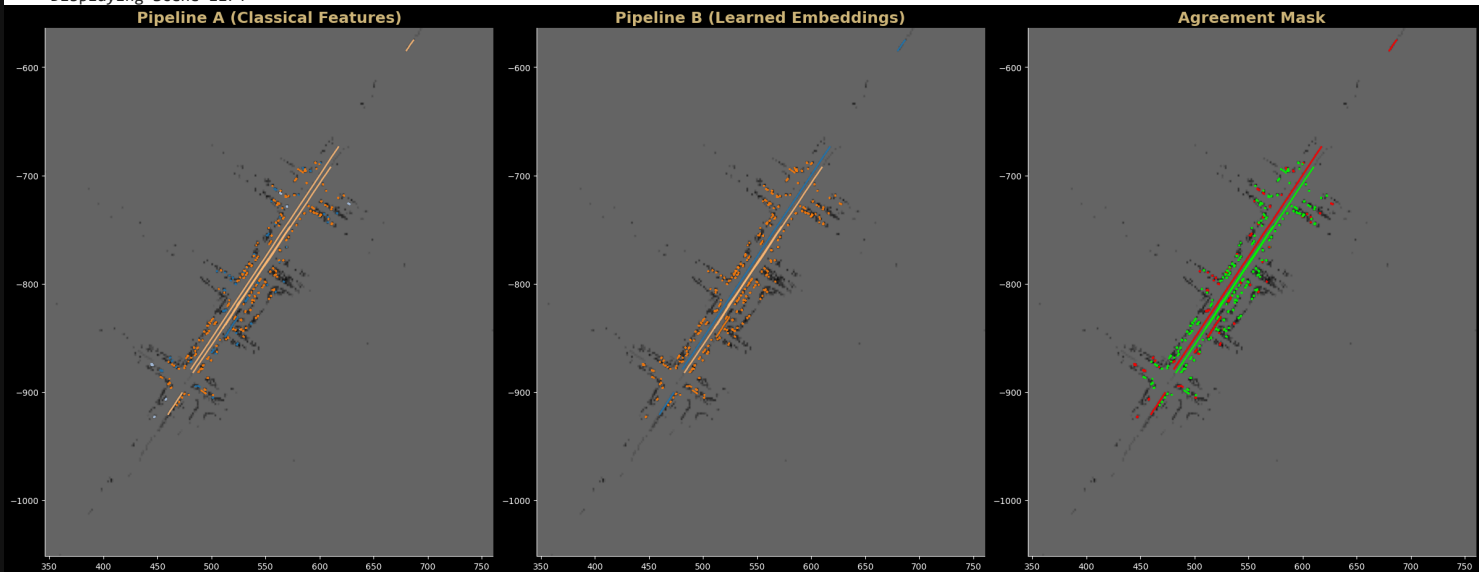


Figure saved to (Linux): /home/treinart/projects/lyft-unsupervised/artifacts/figures/20250917_221443_scene_compare_1174.png
 Figure saved to (Win): /mnt/c/Users/travi/Documents/Training/Colorado/MS-AI/CSCA 5632 Final Project/artifacts/figures/20250917_221443_scene_compare_1174.png

11.4 Aligned Exemplar Gallery

For each of Pipeline A's clusters, this gallery shows the most representative track ("centroid exemplar") from Pipeline A alongside the exemplar for the Pipeline B cluster that best aligns with it. This provides a direct, qualitative comparison of the core behavior captured by each pipeline's corresponding clusters.

```
In [81]: # Prepare Data and Find Exemplars in Both Spaces
A_cols = [c for c in features_a.columns if c not in ["scene_id", "track_id"]]
B_cols = [c for c in embeddings_b.columns if c not in ["scene_id", "track_id"]]
A_space = features_a.merge(clusters_a, on=["scene_id", "track_id"])
B_space = embeddings_b.merge(clusters_b, on=["scene_id", "track_id"])
def find_exemplars(df_space, feature_cols, label_col="cluster_id"):
    exemplars = {}
    for cid, grp in df_space.groupby(label_col):
        M = grp[feature_cols].to_numpy(dtype=float)
        if len(M) == 0: continue
        centroid = M.mean(axis=0, keepdims=True)
        idx = int(np.argmin(np.linalg.norm(M - centroid, axis=1)))
        row = grp.iloc[idx]
        exemplars[int(cid)] = (int(row.scene_id), int(row.track_id))
```

```

return exemplars
exemplars_A = find_exemplars(A_space, A_cols, "cluster_id")
exemplars_B = find_exemplars(B_space, B_cols, "cluster_id")

# Create a Separate Plot for Each Aligned Pair
A2B = {v: k for k, v in map_B2A.items()}

pairs = list(zip(overlap_df.cluster_A, overlap_df.B_mode_aligned))
print(f"--- Rendering {len(pairs)} Aligned Exemplar Plots (Individual) ---")

for ca, cb_aligned in pairs:
    fig, ax = plt.subplots(figsize=(18, 12), facecolor="black")
    ax.set_facecolor("black")
    sid_tid_A = exemplars_A.get(int(ca))
    cb_orig = A2B.get(int(cb_aligned), None)
    sid_tid_B = exemplars_B.get(cb_orig) if cb_orig is not None else None
    if sid_tid_A:
        x, y, _ = track_xyv(agents_df_clean, sid_tid_A[0], sid_tid_A[1])
        ax.plot(x - x[0], y - y[0], lw=3.0, alpha=0.95, color="cyan", label=f"Exemplar A:{ca}")
    if sid_tid_B:
        x, y, _ = track_xyv(agents_df_clean, sid_tid_B[0], sid_tid_B[1])
        ax.plot(x - x[0], y - y[0], lw=2.5, alpha=0.9, color="magenta", label=f"Exemplar B→A:{cb_aligned}")

    ax.set_title(f"Aligned Pair • A:{ca} vs B→A:{cb_aligned}", fontsize=28, color="ghostwhite", fontweight="bold", pad=10)
    ax.tick_params(colors="white")
    [sp.set_color("white") for name, sp in ax.spines.items() if name in ["left", "bottom"]]
    [sp.set_visible(False) for name, sp in ax.spines.items() if name in ["right", "top"]]
    ax.legend(frameon=False, labelcolor="white", fontsize=18)
    ax.set_aspect('equal', adjustable='box')
    ax.grid(color='white', alpha=0.1)

# Save the figure with a unique name
save_fig(fig, f"aligned_pair_A{int(ca)}_B{int(cb_aligned)}")

```

--- Rendering 4 Aligned Exemplar Plots (Individual) ---

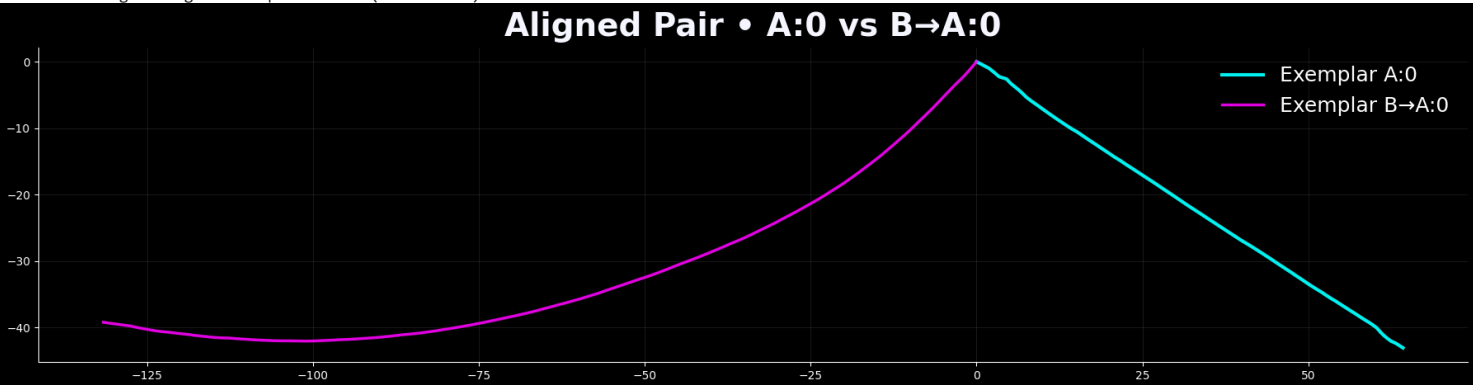


Figure saved to (Linux): /home/treinart/projects/lyft-unsupervised/artifacts/figures/20250917_221451_aligned_pair_A0_B0.png

Figure saved to (Win): /mnt/c/Users/travi/Documents/Training/Colorado/MS-AI/CSCA 5632 Final Project/artifacts/figures/20250917_221451_aligned_pair_A0_B0.png

Aligned Pair • A:1 vs B→A:1

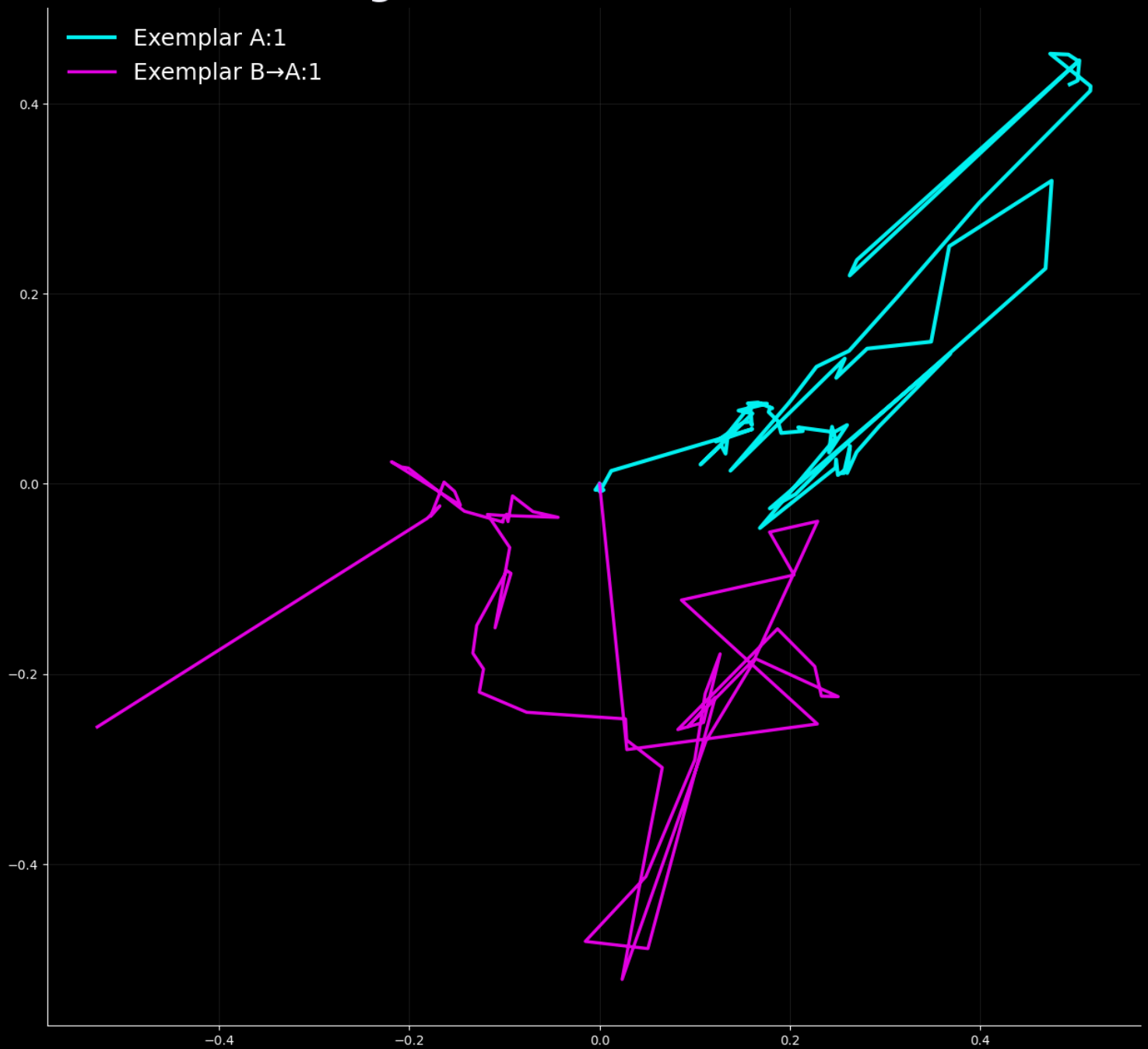


Figure saved to (Linux): /home/treinart/projects/lyft-unsupervised/artifacts/figures/20250917_221451_aligned_pair_A1_B1.png
Figure saved to (Win): /mnt/c/Users/travi/Documents/Training/Colorado/MS-AI/CSCA 5632 Final Project/artifacts/figures/20250917_221451_aligned_pair_A1_B1.png

Aligned Pair • A:2 vs B→A:1

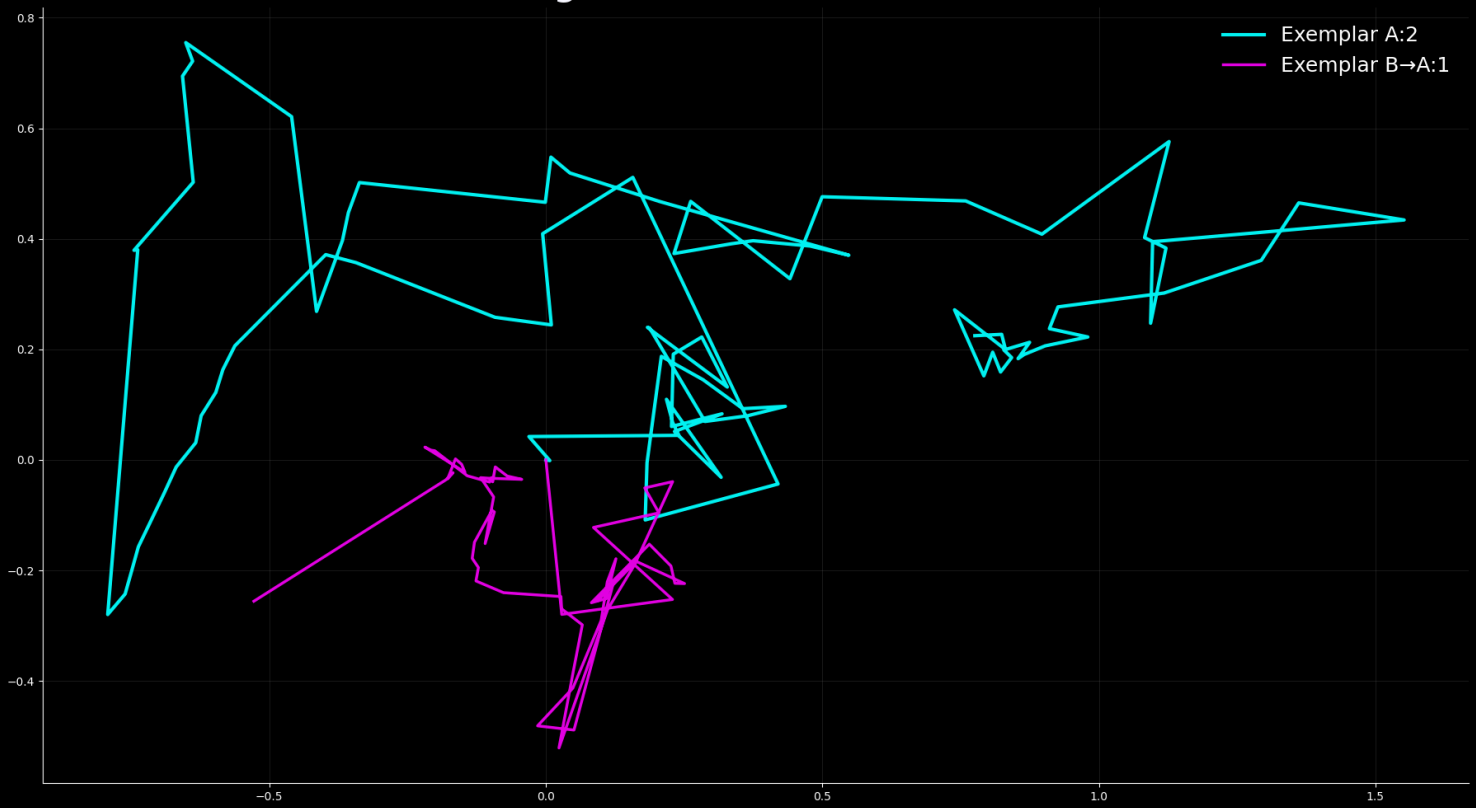


Figure saved to (Linux): /home/treinart/projects/lyft-unsupervised/artifacts/figures/20250917_221451_aligned_pair_A2_B1.png
Figure saved to (Win): /mnt/c/Users/travi/Documents/Training/Colorado/MS-AI/CSCA 5632 Final Project/artifacts/figures/20250917_221451_aligned_pair_A2_B1.png

Aligned Pair • A:3 vs B→A:1

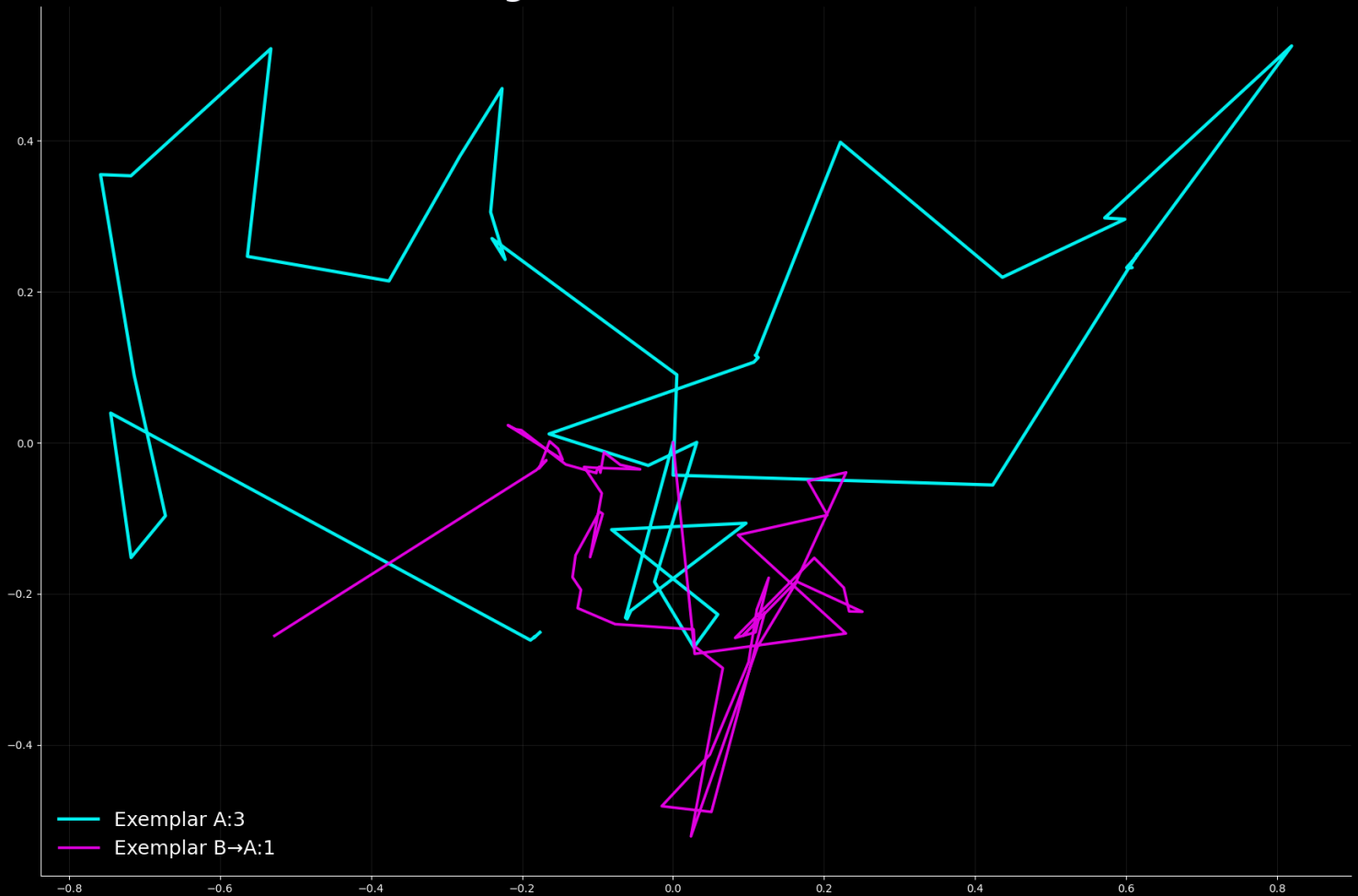


Figure saved to (Linux): /home/treinart/projects/lyft-unsupervised/artifacts/figures/20250917_221451_aligned_pair_A3_B1.png
Figure saved to (Win): /mnt/c/Users/travi/Documents/Training/Colorado/MS-AI/CSCA 5632 Final Project/artifacts/figures/20250917_221451_aligned_pair_A3_B1.png

This final visualization provides the most direct comparison between two pipelines. For each of Pipeline A's core archetypes, a scorecard is generated. Each card shows the visual shape of the archetype, a side-by-side feature fingerprint, comparing its profile to its aligned counterpart from Pipeline B, and the final agreement score. This provides a view of how well the pipelines agree on the definition of each discovered behavior.

```
In [82]: # Prepare Data for Scorecard
feature_cols_to_profile = [
    "mean_speed", "p95_speed", "std_speed",
    "heading_change_rate", "stop_ratio", "straightness", "duration_s"
]

# Profile for Pipeline A
profiles_a_input = features_a.merge(clusters_a, on=["scene_id", "track_id"])
profiles_a = profiles_a_input.groupby("cluster_id")[feature_cols_to_profile].median()

# Profile for Pipeline B (using its aligned cluster IDs)
profiles_b_input = features_a.merge(alignment_tbl, on=["scene_id", "track_id"])
profiles_b_aligned = profiles_b_input.groupby("cluster_B_aligned")[feature_cols_to_profile].median()

# Standardize all profiles for fair comparison in the bar charts
all_profiles = pd.concat([profiles_a, profiles_b_aligned])
profiles_mean = all_profiles.mean()
profiles_std = all_profiles.std()
profiles_a_z = (profiles_a - profiles_mean) / profiles_std
profiles_b_aligned_z = (profiles_b_aligned - profiles_mean) / profiles_std

# Get the List of aligned cluster pairs to plot
pairs = list(zip(overlap_df.cluster_A, overlap_df.B_mode_aligned))
print(f"--- Rendering {len(pairs)} Alignment Scorecard Plots ---")

# Loop Through Each Pair and Create a Scorecard
for ca, cb_aligned in pairs:
    fig = plt.figure(figsize=(24, 8), facecolor="black")
    gs = fig.add_gridspec(1, 3, width_ratios=[1, 1.5, 0.8], wspace=0.3)
    fig.suptitle(f"Alignment Scorecard for Archetype A:{ca}", fontsize=28, color="#CFB87C", fontweight="bold")

    # Panel 1: Archetype Shape
    ax_shape = fig.add_subplot(gs[0, 0]); ax_shape.set_facecolor("black")
    ax_shape.set_title("Archetype Shape (from A)", fontsize=18, color="white", pad=10)
    sid_tid_A = exemplars_A.get(ca)
    if sid_tid_A:
        x,y,_ = track_xyv(agents_df_clean, sid_tid_A[0], sid_tid_A[1])
        ax_shape.plot(x - x[0], y - y[0], lw=3.0, alpha=0.95, color="cyan")
    ax_shape.set_aspect('equal', adjustable='box'); ax_shape.tick_params(colors="white"); ax_shape.grid(color='white', alpha=0.1)
    [sp.set_color("white") for name, sp in ax_shape.spines.items() if name in ["left", "bottom"]]
    [sp.set_visible(False) for name, sp in ax_shape.spines.items() if name in ["right", "top"]]

    # Panel 2: Feature Fingerprint (Side-by-side Bar Chart)
    ax_fp = fig.add_subplot(gs[0, 1]); ax_fp.set_facecolor("black")
    ax_fp.set_title("Feature Fingerprint (Standardized)", fontsize=18, color="white", pad=10)
    profile_a_data = profiles_a_z.loc[ca].rename("Pipeline A")
    profile_b_data = profiles_b_aligned_z.loc[cb_aligned].rename(f"Pipeline B (aligns to A:{cb_aligned})")
    plot_df = pd.concat([profile_a_data, profile_b_data], axis=1).reset_index().melt(id_vars="index", var_name="Pipeline", value_name="Z-Score")
    sns.barplot(data=plot_df, y="index", x="Z-Score", hue="Pipeline", ax=ax_fp, palette=["deepskyblue", "orange"], orient='h')
    ax_fp.axvline(0, color='white', lw=0.5, ls='--')
    ax_fp.tick_params(colors="white", labels=12); ax_fp.set_ylabel(""); ax_fp.set_xlabel("Z-Score (Higher is More Extreme)", color="white", fontsize=14)
    ax_fp.legend(facecolor="#111111", labelcolor="white"); [sp.set_color("white") for sp in ax_fp.spines.values()]

    # Panel 3: Agreement Score
    ax_stats = fig.add_subplot(gs[0, 2]); ax_stats.set_facecolor("black"); ax_stats.axis('off')
    ax_stats.set_title("Agreement Score", fontsize=18, color="white", pad=10)
    overlap = overlap_df[overlap_df.cluster_A == ca]['overlap_rate'].iloc[0]
    ax_stats.text(0.5, 0.6, f"{overlap:.1%}", fontsize=60, color="lime", ha="center", fontweight="bold")
    ax_stats.text(0.5, 0.5, "Track Overlap", fontsize=20, color="white", ha="center")

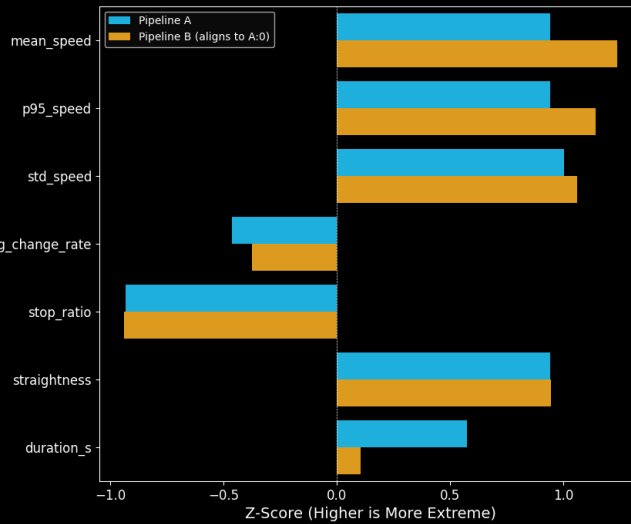
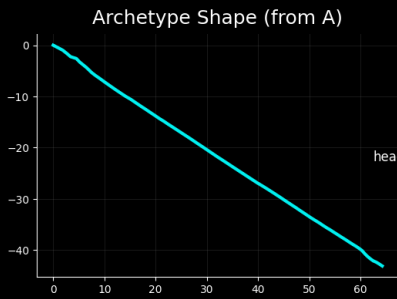
    # Save the figure
    save_fig(fig, f"alignment_scorecard_A{ca}")

--- Rendering 4 Alignment Scorecard Plots ---
```

Alignment Scorecard for Archetype A:0

Feature Fingerprint (Standardized)

Agreement Score



39.4%

Track Overlap

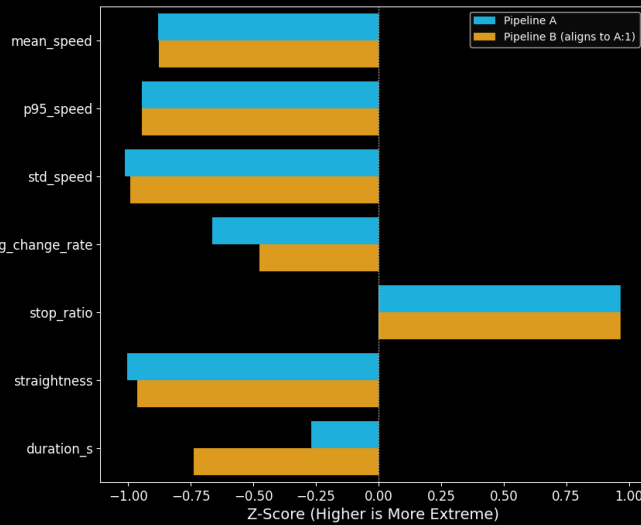
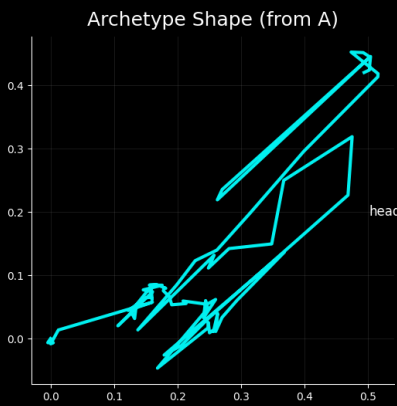
Figure saved to (Linux): /home/treinart/projects/lyft-unsupervised/artifacts/figures/20250917_221504_alignment_scorecard_A0.png

Figure saved to (Win): /mnt/c/Users/travi/Documents/Training/Colorado/MS-AI/CSCA 5632 Final Project/artifacts/figures/20250917_221504_alignment_scorecard_A0.png

Alignment Scorecard for Archetype A:1

Feature Fingerprint (Standardized)

Agreement Score



100.0%

Track Overlap

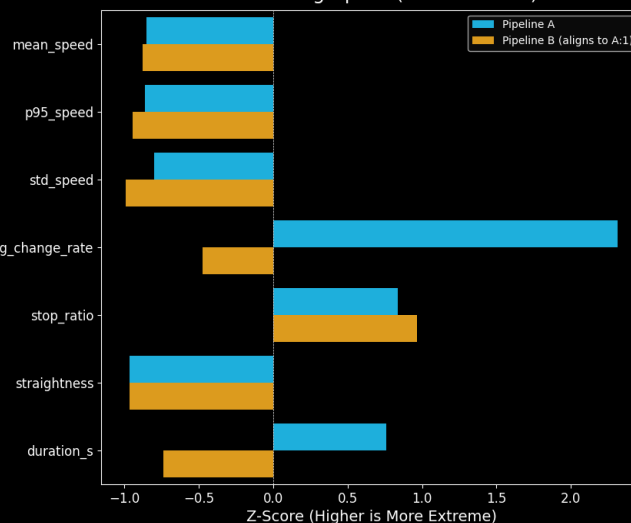
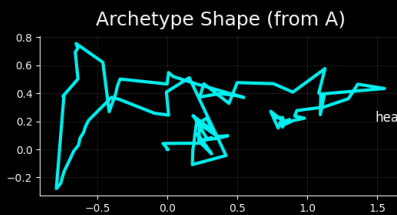
Figure saved to (Linux): /home/treinart/projects/lyft-unsupervised/artifacts/figures/20250917_221504_alignment_scorecard_A1.png

Figure saved to (Win): /mnt/c/Users/travi/Documents/Training/Colorado/MS-AI/CSCA 5632 Final Project/artifacts/figures/20250917_221504_alignment_scorecard_A1.png

Alignment Scorecard for Archetype A:2

Feature Fingerprint (Standardized)

Agreement Score



99.7%

Track Overlap

Figure saved to (Linux): /home/treinart/projects/lyft-unsupervised/artifacts/figures/20250917_221504_alignment_scorecard_A2.png

Figure saved to (Win): /mnt/c/Users/travi/Documents/Training/Colorado/MS-AI/CSCA 5632 Final Project/artifacts/figures/20250917_221504_alignment_scorecard_A2.png

Alignment Scorecard for Archetype A:3

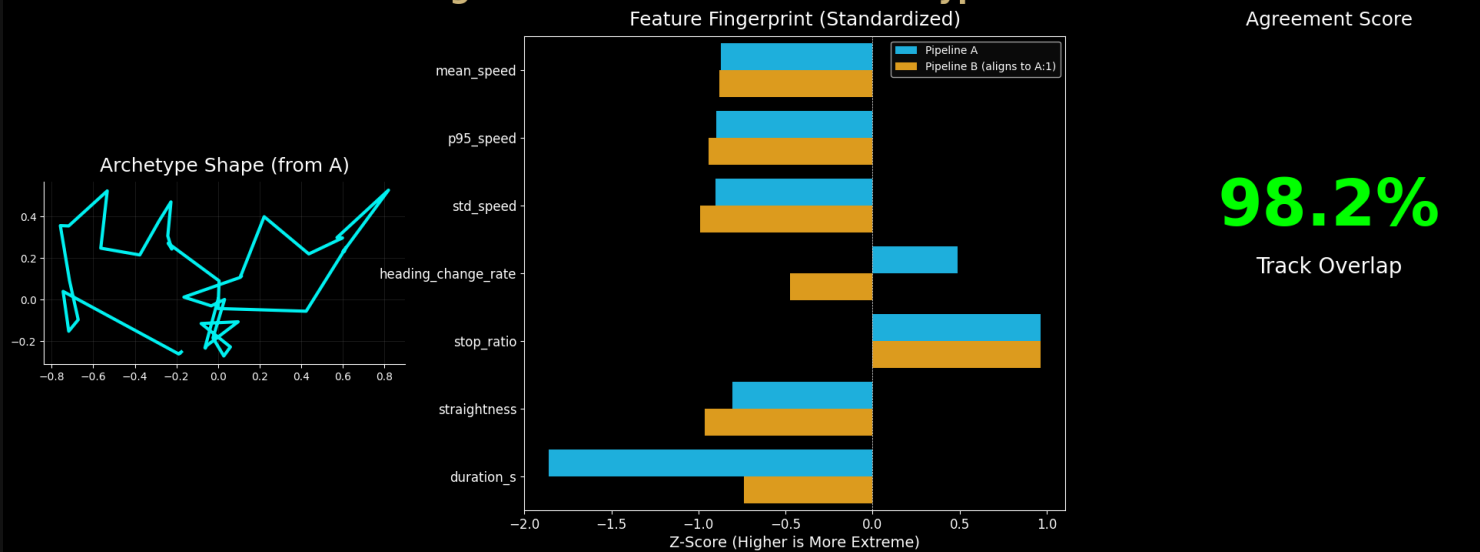


Figure saved to (Linux): /home/treinart/projects/lyft-unsupervised/artifacts/figures/20250917_221505_alignment_scorecard_A3.png
Figure saved to (Win): /mnt/c/Users/travi/Documents/Training/Colorado/MS-AI/CSCA 5632 Final Project/artifacts/figures/20250917_221505_alignment_scorecard_A3.png

11.6 Create Master Comparison Spreadsheets

This final cell synthesizes the full pipeline comparison into a comprehensive, shareable artifact. It creates three detailed tables:

- A detailed **Track-by-Track Comparison**
- A **Scene-Level Summary** sorted by disagreement
- A **Cluster-Level Summary** of agreement rates

These tables are then saved as a single, multi-sheet Excel file, providing the definitive, machine-readable output of the project's analysis.

```
In [94]: # Create Master Spreadsheets for Final Analysis

print("--- Building Master Comparison Spreadsheets (Track, Scene, and Cluster level) ---")

# Load necessary artifacts from previous sections
ALIGN_DIR = ARTIFACT_DIR / "compare_ab"
alignment_tbl = pd.read_parquet(ALIGN_DIR / "alignment_table.parquet")
features_a = pd.read_parquet(ARTIFACT_DIR / "pipeline_a_features.parquet")
with open(ALIGN_DIR / "label_map_B_to_A.json", 'r') as f: map_B2A = {int(k): v for k,v in json.load(f).items()}

# Manually define the scenario names for BOTH pipelines based on our analysis
map_a = {
    0: "High-Speed Cruising",
    1: "Stationary",
    2: "Low-Speed Maneuvering",
    3: "Idling and Creeping"
}

map_b = {
    0: "Stationary",
    1: "High-Speed (Curved)",
    2: "High-Speed (Straight)",
    3: "Moderate-Speed Cruising"
}

# Create the final mapping for the aligned B clusters
map_b_aligned = {v: map_b.get(k, "Unknown") for k, v in map_B2A.items()}

# Build the track-level master table
master_tracks = alignment_tbl.merge(features_a, on=["scene_id", "track_id"], how="left")
master_tracks["scenario_A"] = master_tracks["cluster_id_A"].map(map_a)
master_tracks["scenario_B_aligned"] = master_tracks["cluster_B_aligned"].map(map_b_aligned)

# Build the scene-level summary
scene_summary = master_tracks.groupby("scene_id").agg(
    n_tracks = ("track_id", "nunique"),
    agreement_rate = ("agree", "mean"),
    median_speed = ("mean_speed", "median"),
    median_turn_rate = ("heading_change_rate", "median"),
    median_stop_ratio = ("stop_ratio", "median")
).reset_index()

def describe_scene(row):
    density = "dense" if row["n_tracks"] >= 150 else ("moderate" if row["n_tracks"] >= 50 else "sparse")
    speed = "fast" if row["median_speed"] > 5 else ("slow" if row["median_speed"] < 2 else "mixed")
    flow = "stop-and-go" if row["median_stop_ratio"] > 0.25 else "free-flow"
    return f"{density.title()} traffic, {speed.title()} speed, {flow.title()}"
scene_summary["scene_description"] = scene_summary.apply(describe_scene, axis=1)
scene_summary = scene_summary.sort_values("agreement_rate", ascending=True)
```

```
# Build the cluster-level summary
cluster_summary_a = master_tracks.groupby(["cluster_id_A", "scenario_A"]).agg(
    n_tracks = ("track_id", "nunique"),
    agreement_rate = ("agree", "mean")
).reset_index().sort_values("agreement_rate", ascending=True)

# Save ALL Artifacts
csv_tracks = ALIGN_DIR / "master_track_comparison.csv"
csv_scenes = ALIGN_DIR / "master_scene_summary.csv"
csv_clusters = ALIGN_DIR / "master_cluster_summary.csv"
xlsx_master = ARTIFACT_DIR / "MASTER_PIPELINE_COMPARISON.xlsx"
master_tracks.to_csv(csv_tracks, index=False, float_format="%.4f")
scene_summary.to_csv(csv_scenes, index=False, float_format="%.4f")
cluster_summary_a.to_csv(csv_clusters, index=False, float_format="%.4f")
try:
    with pd.ExcelWriter(xlsx_master) as writer:
        master_tracks.to_excel(writer, sheet_name="Track-by-Track Comparison", index=False)
        scene_summary.to_excel(writer, sheet_name="Scene Summary (by Agreement)", index=False)
        cluster_summary_a.to_excel(writer, sheet_name="Cluster Summary (by Agreement)", index=False)
    print(f"\n[Saved] Master comparison Excel file -> {xlsx_master}")
except Exception as e:
    print(f"\n[Excel Warning] Could not save .xlsx file. Error: {e}")

# Display Previews
print("\n--- Scene Summary (Lowest Agreement First) ---")
display(style_section_table(scene_summary.head(10)))
print("\n--- Cluster Summary (Lowest Agreement First) ---")
display(style_section_table(cluster_summary_a))
print("\n--- Track-by-Track Master Preview ---")
display(style_section_table(master_tracks.head(10)))
```

--- Building Master Comparison Spreadsheets (Track, Scene, and Cluster level) ---

[Saved] Master comparison Excel file -> /home/treinart/projects/lyft-unsupervised/artifacts/MASTER_PIPELINE_COMPARISON.xlsx

--- Scene Summary (Lowest Agreement First) ---

scene_id	n_tracks	agreement_rate	median_speed	median_turn_rate	median_stop_ratio	scene_description
1309	109	0.330275	0.307929	0.761408	0.844828	Moderate traffic, Slow speed, Stop-And-Go
1151	222	0.405405	0.044091	0.572694	0.993827	Dense traffic, Slow speed, Stop-And-Go
2823	158	0.449367	0.047974	0.583076	0.972973	Dense traffic, Slow speed, Stop-And-Go
13262	148	0.472973	0.283061	0.564923	0.876276	Moderate traffic, Slow speed, Stop-And-Go
1156	187	0.475936	0.039565	1.259578	0.987342	Dense traffic, Slow speed, Stop-And-Go
14511	134	0.477612	0.045132	0.847351	0.983991	Moderate traffic, Slow speed, Stop-And-Go
10600	110	0.490909	0.206914	0.495902	0.906897	Moderate traffic, Slow speed, Stop-And-Go
8824	131	0.496183	0.154449	0.861318	0.914062	Moderate traffic, Slow speed, Stop-And-Go
8500	189	0.497354	0.032263	0.580979	1.000000	Dense traffic, Slow speed, Stop-And-Go
15706	94	0.500000	0.058985	0.663337	0.995102	Moderate traffic, Slow speed, Stop-And-Go

--- Cluster Summary (Lowest Agreement First) ---

cluster_id_A	scenario_A	n_tracks	agreement_rate
2	Low-Speed Maneuvering	503	0.000000
3	Idling and Creeping	945	0.017894
0	High-Speed Cruising	542	0.394177
1	Stationary	1508	1.000000

--- Track-by-Track Master Preview ---

scene_id	track_id	cluster_id_A	cluster_id_B	cluster_B_aligned	agree	n_frames	duration_s	mean_speed	median_speed	std_speed	p95_speed	max_speed	median_accel
108	1	1	0	1	True	67	6.700000	0.019568	0.000000	0.105209	0.000000	0.740772	0.000000
108	2	1	0	1	True	248	24.800000	0.001807	0.000000	0.020909	0.000000	0.290456	0.000000
108	3	1	0	1	True	56	5.600000	0.014214	0.000000	0.075122	0.000000	0.470635	0.000000
108	4	1	0	1	True	87	8.700000	0.012386	0.000000	0.082567	0.000000	0.652563	0.000000
108	5	0	0	1	False	248	24.800000	2.460193	0.049784	2.962227	7.421925	7.618188	0.190595
108	6	1	0	1	True	60	6.000000	0.005935	0.000000	0.032169	0.000000	0.197941	0.000000
108	7	1	0	1	True	248	24.800000	0.018864	0.000000	0.057314	0.130736	0.377650	0.000000
108	8	0	0	1	False	248	24.800000	2.112420	0.589622	2.605378	7.213345	8.124944	0.551891
108	9	2	0	1	False	248	24.800000	0.058616	0.013047	0.141087	0.259589	1.622227	0.162196
108	12	2	0	1	False	248	24.800000	0.831928	0.025735	1.558878	4.658676	4.917617	0.133351

Section 12: Conclusion & What the Pipeline Comparison Shows

This project was a journey into the ambiguity of a real-world analytical task. It began not with a clear target, but with a question: could the hidden patterns of motion in a complex vehicle dataset be discovered and defined? The process of answering that question involved developing two distinct analytical pipelines, overcoming significant technical hurdles, and navigating the challenges of an open-ended problem. The final result is not just a set of clusters, but a clear, data-driven story about the different ways to interpret behavior.

The initial stages of this project were defined by a challenge not of coding, but of direction. Unlike a competition with a clear finish line and a public leaderboard, this project required navigating an open-ended problem space. Without a predefined success metric, it was easy to over-develop certain areas while searching for a clear signal. For much of the early and middle stages, the work felt disconnected—the data was clean, the models were training, but the ultimate value was uncertain.

The project hit a wall when the deep learning pipeline, the centerpiece of the second approach, failed catastrophically. The clustering stage in Section 8 produced nothing but `NaN` values, the result of a total model collapse in Section 7. The investigation traced this back to a two-front failure: severe data starvation caused by an overly aggressive window length, and a model architecture that had learned the trivial solution of predicting all zeros. The recovery was a methodical, back-to-basics engineering effort to rebuild the data pipeline and stabilize the model.

Section 10 made the result legible

The scenario cards and gallery tied the analysis together: the **Behavior Signature** summarized per-track kinematics, the **Shape of Motion** panel showed exemplar geometry, and the **UMAP highlight** made cluster neighborhoods visible at a glance. The MP4 replay confirmed that the straight-lane tracks split more cleanly under the embedding model while turning archetypes remained consistent across pipelines.

What Section 11 establishes

The side-by-side comparison of the classical feature pipeline (**Pipeline A**) and the learned-embedding pipeline (**Pipeline B**) makes the pattern clear:

- **High agreement on complex maneuvers.** For the curvy / weaving archetypes the pipelines effectively coincide. Track overlap on the aligned archetypes is ~**100.0%** (A:1), **99.7%** (A:2), and **98.2%** (A:3). Shapes, feature fingerprints, and BEV overlays all reflect the same story: when motion includes turns and heading variability, both pipelines land in the same buckets.
- **Divergence on long straight/lingering motion.** The biggest gap sits in the straight, slow/lingering archetype (A:0), with only **39.4%** overlap. Agreement masks show wide green bands where vehicles curve or merge, and red pockets along long straight lanes and shoulders. The global view reflects this mix: **Agreement $\approx$ 65.1%, ARI $\approx$ 0.2625, NMI $\approx$ 0.3698** on shared tracks.

Why the difference shows up on straights

- **Feature dominance in Pipeline A.** Handcrafted summaries (stationarity, speed percentiles, heading spread) tend to blur distinct “straight” behaviors: creeping queues, shoulder dwell, and clean lane travel can share similar medians and p-percentiles.
- **Shape-aware encoding in Pipeline B.** The learned trajectory embeddings retain *how* motion unfolds, not just aggregates of it. Normalized path shape and local dynamics separate “true straight-throughs” from “near-stationary lingerers,” which tightens the straight-lane clusters and explains the extra red in the agreement mask along those corridors.

The Final Verdict: A Story Told by the Data

The final quantitative comparison presents a decisive case. While both pipelines successfully identified four primary behavioral archetypes, the solution produced by Pipeline B (Learned Embeddings) was demonstrably superior in quality and stability.

Pipeline	Feature Method	Winning Model (k=4)	Silhouette Score	Stability (ARI)
Pipeline A	Classical Features	KMeans (on PCA)	0.558	0.779
Pipeline B	Learned Embeddings (GRU AE)	KMeans	0.935	0.996

What moved the needle in this project

- **Representation over raw features.** Moving from static summaries to learned trajectory embeddings was the difference maker for straight-motion fidelity.
- **A disciplined stability harness (Section 9).** Bootstrap agreement, sensitivity to k , noise/dropout tests, and cross-split generalization established that the final clustering behaves consistently, so visual differences in Section 11 can be trusted.
- **Strong visualization.** The scenario cards, gallery, and BEV agreement masks turned abstractions into concrete, auditable behaviors, making model behavior actionable rather than opaque.

Limits and how to push further

- **Define success up front.** Future iterations should track a small set of guard-rail metrics: (i) per-archetype overlap across pipelines, (ii) stability thresholds (e.g., $ARI_{bootstrap} \geq 0.98$), and (iii) scene-level agreement rate on curated hotspots. This keeps scope creep in check.
- **Refine the straight/lingering split.** Add context features (lane-centerline distance, map curvature, stop-line proximity, speed-limit ratios) to help the classical pipeline and to give the embedding model scene priors.
- **Model experiments to try next.**
 - **Sequence encoders:** temporal CNNs or BiGRU/Transformer encoders trained with contrastive or triplet losses on normalized paths.
 - **Distance-aware clustering:** k-medoids with DTW, or spectral clustering on an RBF-DTW kernel for shape sensitivity without learning.
 - **Density methods:** HDBSCAN to carve the sprawling “lingering” mass into dwell vs. crawl vs. park sub-modes.
 - **Hierarchical approach:** top-level split into *moving* vs. *lingering*, followed by specialized sub-clusterers per branch.
 - **Semi-supervision:** a handful of labeled archetypes to anchor clusters via constrained KMeans or metric-learning.
- **Throughput.** Section 8 showed sweeps can be slow on CPU (~11 minutes for GMM). Port the heavy steps (KMeans, GMM, UMAP) to **RAPIDS cuML** for faster iteration.

Bottom line

Across maneuvers with turns and curvature, both pipelines agree. The learned-embedding pipeline distinguishes straight-lane behaviors more sharply, improving assignment quality where classical features tend to homogenize motion. With stability validated and behavior made visible, the workflow is ready for targeted improvements on the lingering/straight regime and for faster, GPU-backed exploration.

